

研究生教学用书

公共基础课系列

数值分析

(第2版)

Numerical Analysis

李 红



华中科技大学出版社

<http://www.hustp.com>



李红，博士，华中科技大学教授，
博士生导师。国家精品课程《复变函
数与积分变换》负责人。研究方向：
逼近与计算、小波分析与图像处理、
机器学习与模式识别。

研究生教学用书 公共基础课系列

ISBN 978-7-5609-3062-6



01>

9 787560 930626

定价：29.80元

研究生教学用书
公共基础课系列

数值分析

(第2版)

李 红

华中科技大学出版社
中国·武汉

图书在版编目(CIP)数据

数值分析(第2版)/李红. —武汉:华中科技大学出版社,2010年5月
ISBN 978-7-5609-3062-6

I. 数… II. 李… III. 数值计算 IV. O241

中国版本图书馆 CIP 数据核字(2010)第 084617 号

数值分析(第2版)

李红

策划编辑:李德

责任编辑:李德

责任校对:张琳

封面设计:潘群

责任监印:熊庆玉

出版发行:华中科技大学出版社(中国·武汉)

武昌喻家山 邮编:430074 电话:(027)87557437

录排:武汉众欣图文激光照排中心

印刷:华中科技大学印刷厂

开本:787mm×960mm 1/16

印张:18.5

字数:350 000

版次:2010年5月第2版

印次:2010年5月第3次印刷

定价:29.80元

ISBN 978-7-5609-3062-6/O·296

(本书若有印装质量问题,请向出版社发行部调换)

目 录

第 1 章 绪 论	(1)
1.1 课程的意义、内容和特点	(1)
1.2 误差及有关概念	(4)
1.3 数值稳定性和病态问题	(7)
1.4 数值运算中的一些原则	(9)
1.5 几个算例	(10)
1.6 算法的实现	(12)
习题 1	(13)
数值实验题 1	(14)
第 2 章 插 值 法	(17)
2.1 问题的提法	(17)
2.2 Lagrange(拉格朗日)插值	(18)
2.3 差商与 Newton(牛顿)插值	(24)
2.4 差分与等距节点的 Newton 插值	(27)
2.5 Hermite(埃尔米特)插值	(32)
2.6 分段插值法	(36)
2.7 3 次样条(spline)插值	(40)
习题 2	(49)
数值实验题 2	(51)
第 3 章 函数逼近与曲线拟合	(53)
3.1 内积空间	(53)
3.2 函数的最佳平方逼近	(56)
3.3 正交多项式	(59)
3.4 用正交函数系作最佳平方逼近	(64)
3.5 曲线拟合的最小二乘法	(67)
3.6 最佳一致逼近多项式及其求法	(75)

习题 3	(85)
数值实验题 3	(86)
第 4 章 数值积分	(88)
4.1 数值求积公式的基本概念	(88)
4.2 Newton-Cotes(牛顿-柯特斯)公式	(92)
4.3 复化求积公式及其收敛性	(97)
4.4 Romberg(龙贝格)算法	(102)
4.5 Gauss(高斯)型求积公式	(106)
4.6 数值微分	(116)
习题 4	(121)
数值实验题 4	(123)
第 5 章 常微分方程的数值方法	(125)
5.1 建立常微分方程数值方法的基本思想与途径	(125)
5.2 Euler(欧拉)方法及其截断误差和阶	(126)
5.3 Runge-Kutta(龙格-库塔)方法	(130)
5.4 单步法收敛性与稳定性	(136)
5.5 线性多步法	(142)
5.6 预测-校正技术和外推技巧	(149)
习题 5	(152)
数值实验题 5	(154)
第 6 章 线性代数方程组的解法	(156)
6.1 引言及预备知识	(156)
6.2 Gauss(高斯)消去法	(161)
6.3 Gauss 主元素消去法	(165)
6.4 矩阵分解及其在解方程组中的应用	(168)
6.5 误差分析	(184)
6.6 线性代数方程组的迭代解法	(187)
习题 6	(202)
数值实验题 6	(205)
第 7 章 非线性方程和方程组的解法	(208)
7.1 二分法	(208)

7.2 简单迭代法	(209)
7.3 迭代过程的加速	(217)
7.4 Newton 迭代法	(219)
7.5 弦截法与抛物线法	(225)
7.6 解非线性方程组的 Newton 迭代法	(227)
习题 7	(229)
数值实验题 7	(230)
第 8 章 矩阵特征值与特征向量的计算	(232)
8.1 幂法和反幂法	(232)
8.2 Jacobi(雅可比)方法	(239)
8.3 QR 方法	(242)
习题 8	(248)
数值实验题 8	(249)
答案与提示	(251)
附录 数值实验程序	(255)
参考文献	(287)

第 2 版 前 言

本书第 2 版是应新世纪对理工科学生的科学计算能力的要求,在《数值分析》(第 1 版)(华中科技大学出版社出版)的基础上,广泛吸取校内外教师的意见后修订而成的.从这几年使用第 1 版教材的高校教师的反馈情况来看,普遍认为本教材取材合理、叙述清楚、简明精要、易于教学.因此,新版在主要内容和结构框架上未作大的改动,但从教学角度出发对语句进行了仔细的推敲,改写了一些陈述,调整了例、习题的配置.

对曾使用过第 1 版教材的各位教师和读者表示衷心的感谢,正是依据你们使用后的意见,作者在第 2 版中修正了不少错漏,使得本书更趋完善.

新版中的附录包含了各章实验题的 Matlab 程序源代码,所有的程序均在 Matlab R 2008a 中通过测试.肖光润博士完成了教材中整个数值实验的程序设计工作.在此,谨致以诚挚的谢意.

限于水平,书中一定有不少缺点和错漏之处,恳望得到批评指正.

作 者

2010 年 4 月于华中科技大学

前 言

21 世纪是数字化、信息化、智能化技术超常规发展的年代,计算机作为其基本的载体,其技术理论的研究和应用直接影响到当今科学技术发展的速度和质量。

随着计算机的迅速发展,熟练地运用计算机进行科学计算已成为科技工作者不可缺少的技能.应用计算机求解各类问题更成为各行各业知识更新的必要环节,培养学生和应用者的科学计算能力,数值分析课程具有不可替代的作用。

作者经过十多年的教学实践积累和不断的探索改进,重新编著了《数值分析》教材.本书以提高学生的数学素质,培养学生科学计算的实践能力为要旨,强调重概念、重方法、重应用及重能力的培养,其特点是:(1)内容新,删除了以前教材中的陈旧内容,增加科学计算中一些新的使用方法和计算方法;(2)在着重数值计算基本原理和各种方法的基本思想阐述时,注重强化数学概念的严密性和准确性,使学生的数学逻辑思维能力得到训练;(3)加强数值实验,强化实践能力的培养.本书各章都给出相应的数值实验习题,以帮助学生掌握各种数值计算方法,从而提高学生应用数值计算方法解决实际问题的能力。

在编写本教材的过程中,得到了我校研究生院及数学系的大力支持,很多老师给予了作者关心和指教.徐长发教授、贺云峰老师在试用教材期间提出了宝贵意见,徐永兵老师、郑成勇硕士完成了教材中整个数值实验的程序设计工作.在此,谨致以诚挚的谢意。

限于水平,书中一定有不少缺点和错漏之处,愿望得到批评指正。

作 者

2003.8

第 1 章 绪 论

1.1 课程的意义、内容和特点

计算机技术是对当今科学、工程技术和人类社会生活影响最深刻的最新技术之一,它对科学技术最深刻的影响,莫过于使科学计算平行于理论分析和实验研究,成为人类探索未知科学领域和进行大型工程设计的方法和手段.在独创性工作的先行性研究过程中,科学计算更具有突出的作用.科学计算能力是跨世纪人才不可或缺的.科学计算是数学与计算机有机结合的结果,其核心内容是以现代化的计算机及数学软件为工具,以数学模型为基础进行模拟研究.近年来,它也成为数学科学本身发展的源泉和途径之一,高等教育中如何培养学生科学计算的能力的问题正日益受到关注,成为当前教育改革的核心和焦点之一.

数值分析及有关的数学基础教学环节,在培养学生科学计算能力上具有不可替代的作用.目前,国内许多高等学校已将数值分析列入自然科学、工程技术乃至社会科学本科的教学计划之中.

本课程的内容可概括为用计算机求解数学问题的数值方法和理论,简称数值分析或计算方法.数学学科十分广泛,这里只涉及工程和科学实验中常见的数学问题,如线性方程组、函数的插值、离散数据的拟合、微积分、微分方程、非线性方程等,它们是其他数学学科的基础.

电子计算机实质上只会做有限次的加、减、乘、除等基本运算,而研究怎样通过计算机所能执行的基本运算,来求得各类问题的数值解或近似数值解,就是数值分析所要研究的根本课题.由基本运算及运算顺序的规定所构成的完整的解题步骤,称为算法.数值分析的根本任务,也可说是研究算法.

有许多数学问题的解,不可能经过有限次算术运算计算出来.例如,计算任意角的三角函数值、求一般方程的根、计算任意函数的积分、求一般微分方程的解.对于这类问题,计算方法常常采用近似替代的办法.

例如,已知角的弧度 x 在 0 与 $\pi/4$ 之间,计算 $\sin x$ 的值.

根据微分学的 Taylor(泰勒)公式,可得

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots + (-1)^n \frac{x^{2n+1}}{(2n+1)!} + R_{2n+1}(x).$$

由于计算多项式的值只用到算术运算,而且当 n 充分大时,余项 $R_{2n+1}(x)$ 的数值很

小,便可用上式右端前 $(2n+1)$ 次多项式来近似替代 $\sin x$,把这多项式的值当作 $\sin x$ 的值,得到计算 $\sin x$ 的近似公式

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots + (-1)^n \frac{x^{2n+1}}{(2n+1)!}.$$

这个公式在 $x=0$ 的邻近近似程度较高,但在其他地方,这个公式误差较大. 近似值与真正值之差,称为该近似值的误差. 理想的公式,应使误差在整个区间 $[0, \pi/4]$ 上都很小. 计算机常用的标准函数,就是按这种“理想”公式设计的.

又例如,求非线性方程 $f(x)=0$ 的根. 一般说方程很难求解,但如已知根的粗略近似值为 x_0 ,则根据 Taylor 级数

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \cdots$$

取等式右边前两项近似替代 $f(x)$,就会得到很容易求解的线性方程

$$f(x_0) + f'(x_0)(x - x_0) = 0.$$

把解出的 x 记为 x_1 ,则有

$$x_1 = x_0 - f(x_0)/f'(x_0).$$

x_1 虽然不一定是根,但往往比 x_0 更接近于根. 用 x_1 代替上面的 x_0 ,进行类似计算,即可得 x_2 . 这样继续下去,往往可得一系列越来越接近根的近似值 $x_1, x_2, \cdots$, 其中

$$x_{n+1} = x_n - f(x_n)/f'(x_n), \quad (1.1)$$

这种求根法称为牛顿迭代法. 以具体方程 $x^2 - 2 = 0$ 为例,式(1.1)成为

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n} = \frac{1}{2} \left(x_n + \frac{2}{x_n} \right).$$

取 $x_0 = 1.4$,按上式计算得 $x_1 = 1.414285714$, $x_2 = 1.414213564$,可见越来越接近根 $\sqrt{2}$ ($=1.41421356237\cdots$).

再如求解常微分方程的初值问题

$$\begin{cases} y' = f(x, y), & a \leq x \leq b, \\ y(a) = y_0, \end{cases} \quad (1.2)$$

通常无法求出区间 $[a, b]$ 上的解 $y(x)$ 的解析表达式. 但实际问题往往只需算出它在某些点处的近似值,如在点 $x_n = a + nh$ ($n = 0, 1, 2, \cdots, N; h = (b - a)/N$) 处的近似值 $y_n \approx y(x_n)$. 由式(1.2),得

$$y'(x_n) = f(x_n, y(x_n)), \quad n = 0, 1, 2, \cdots, N. \quad (1.3)$$

又由于

$$y'(x_n) = \lim_{h \rightarrow 0} \frac{y(x_{n+1}) - y(x_n)}{h},$$

用差商 $[y(x_{n+1}) - y(x_n)]/h$ 近似替代式(1.3)中的微商(导数) $y'(x_n)$,用 y_n 近似替代 $y(x_n)$,得到

$$\frac{y_{n+1} - y_n}{h} = f(x_n, y_n), \quad n = 0, 1, 2, \cdots, N-1,$$

从而有

$$y_{n+1} = y_n + hf(x_n, y_n), \quad n = 0, 1, 2, \dots, N-1. \quad (1.4)$$

这样,由已知值 y_0 出发,便可逐步算出 $y_1, y_2, \dots, y_N$, 这种求初值问题(1.2)解近似值的方法,称为 Euler(欧拉)折线法. 以具体问题

$$\begin{cases} y' = y - \frac{2x}{y}, & 0 \leq x \leq 1, \\ y(0) = 1 \end{cases}$$

为例,则 Euler 法计算公式(1.4)成为 $y_{n+1} = y_n + h(y_n - 2x_n/y_n)$. 取 $h=0.1$ 及 $h=0.01$, 由 $y_0=1$ 出发,按此公式可算得表 1.1 中的第二、三行;第四行是解的真正值 $y(x_n) = \sqrt{1+2x_n}$. 计算时用 5 位小数,但表中只记录了 4 位小数. 由表 1.1 可见, h 越小,计算结果越准确.

表 1.1

x_n	0.1	0.2	0.3	0.4	0.5	1
y_n	1.1	1.1918	1.2774	1.3582	1.4351	1.7848
y_n	1.0959	1.1841	1.2662	1.3434	1.4164	1.7380
$y(x_n)$	1.0954	1.1832	1.2649	1.3416	1.4142	1.7321

上述三例都是利用近似替代,把不能用有限次运算求解的问题,转化为比较简单的,可用有限次运算求解的问题,从而用简化问题的解作为原来问题的近似解. 由此,人们自然会问,这种近似解的误差有多大? 能否通过增大 n (如前两例)或缩小 h (如第三例)使误差无限减小? 数值分析在采用近似替代的同时,还需研究这两个问题,即误差估计和收敛性问题.

随着电子计算机的发展,计算机运算速度越来越快,存储数据的容量越来越大. 但无论多快多大,毕竟有一定限度. 所以,对于可用有限次运算求解的问题,还需考虑先算什么量,后算什么量,才能最快算出结果,并使占有的内存最省.

例如,计算多项式

$$0.0625x^4 + 0.425x^3 + 1.215x^2 + 1.912x + 2.1296$$

的值. 如果先算各项然后相加,需做 10 次乘法和 4 次加法. 但如改为下式

$$(((0.0625x + 0.425)x + 1.215)x + 1.912)x + 2.1296$$

计算,则只需做 4 次乘法和 4 次加法(此算法称为秦九韶算法). 进而改为下式

$$[(0.5x + 0.6)^2 + 0.5x + 0.7][(0.5x + 0.6)^2 + 0.8] + 0.9$$

计算,则可减少到 3 次乘法和 5 次加法.

又例如,要求解 $n=20$ 的线性代数方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2, \\ \cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n, \end{cases}$$

如果按 Cramer(克莱姆)法则求解,即按下面公式计算:

$$x_k = D_k/D, \quad k = 1, 2, \cdots, n,$$

其中, $D = \begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix}$, D_k 是把 D 中第 k 列 $(a_{1k}, a_{2k}, \cdots, a_{nk})^T$ 换为 $(b_1, b_2, \cdots, b_n)^T$ 后得到的行列式.

那么,这要计算 $(n+1)$ 个行列式,并做 n 次除法,而每个行列式包含 $n!$ 个乘积,计算每个乘积需做 $(n-1)$ 次乘法,这样共需要 $[(n+1)!(n-1)+n]$ 次乘除法. 当 $n=20$ 时,则要做 9.7×10^{20} 次乘除法;即使在每秒做 10^8 次乘除法的计算机上计算,也需 30 多万年! 可见理论上很“漂亮”的 Cramer 法则在计算机上并不适用.

因此,在构造算法时,还应考虑如何计算,才能既快又省.

1.2 误差及有关概念

前述已提及过误差,这里对误差及有关概念作进一步的讨论.

1.2.1 误差的来源

利用计算机解决科学计算问题的过程大致可以分为两个环节. 首先,将实际问题归结为数学问题,建立起比较适合的、具体的数学模型,如微分、积分、方程、级数求和等;然后,选择适当的解题方法,编制好程序,上机算出结果. 在这两个环节中,每一步都可能产生误差.

一个物理量的真实值和计算出的值往往存在差异,它们之差称为误差.

误差大致可分为以下四类.

(1) 模型误差. 通过对实际问题进行抽象,简化得到的数学模型与实际现象之间必然存在误差,这种误差称之为模型误差.

(2) 观测误差(参量误差、数据误差). 一般数学问题包含若干参量,它们的值往往通过观测得到,而观测难免带来误差,这种误差称之为观测误差.

(3) 截断误差(方法误差). 一般数学问题难以求解,往往要通过近似替代,简化为较易求解的问题,由简化引起的误差称之为截断误差.

许多数学运算(如微分、积分及无穷级数求和等)是通过极限过程来定义的,然而计

计算机上只能完成有限次的算术运算和逻辑运算,因此需要将解题方案加工成算术运算与逻辑运算的有限序列,这样的加工常常表现为无穷过程的截断,由此产生的误差通常称作截断误差.如 e^x 可展开为级数形式

$$e^x = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!} + \cdots.$$

但用计算机求值时,不能直接得出等号右端无穷多项的和,而只能截取有限项求值,即

$$S_n(x) = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!},$$

用部分和 $S_n(x)$ 作为 e^x 的值必然会有误差,由 Taylor 余项定理,其截断误差为

$$e^x - S_n(x) = \frac{x^{n+1}}{(n+1)!} e^{\theta x}, \quad 0 < \theta < 1.$$

(4)舍入误差(计算误差).由于实际计算是按有限位数进行的,数值解的每一步都可能误差,这种误差称之为舍入误差.

在计算过程中所用数据可能位数很多,然而受机器字长的限制,用机器代码表示的数据必须舍入成一定的位数,这也会引起舍入误差.

例如,在实际计算时用 0.33333 代替了 $\frac{1}{3}$,则舍入误差为

$$R = \frac{1}{3} - 0.33333 = 0.00000333\cdots.$$

数值分析研究数学问题的数值解法,所以不讨论模型误差,只讨论截断误差与舍入误差.

1.2.2 误差限与有效数字

准确数 x 与其近似数 x^* 之差,即 $e = x - x^*$ 称为绝对误差.

误差虽然不可避免,但人们总是希望计算结果能足够准确,这就需要估计误差,误差 $(x - x^*)$ 的具体数值通常是无法确定的,只能根据测量工具或计算过程,设法估算出它的取值范围,即误差绝对值的一个上界

$$|e| = |x - x^*| \leq \epsilon.$$

这种上界 ϵ 称作近似值 x^* 的绝对误差限,简称误差限,或称精度(误差限一般都取到某位的半个单位).精度为 ϵ 的近似值 x^* ,可以认为和真值 x 关于允许误差 ϵ 是“重合”的,或者说,结果关于精度 ϵ 是“准确”的.

写出一个近似值后,当然希望指明它的准确程度,为此需要引进有效数字的概念.

众所周知,当准确数 x 有很多位数时,常常按“四舍五入”原则去得到 x 的近似数,例如, $\pi = 3.14159265\cdots$ 按舍入原则若取四位小数,得 $\pi = 3.1416$;取五位小数,得 $\pi = 3.14159$,它们的绝对误差不超过末位数的半个单位,即

$$|\pi - 3.1416| \leq \frac{1}{2} \times 10^{-4}, \quad |\pi - 3.14159| \leq \frac{1}{2} \times 10^{-5}.$$

四以下舍,五以上入这些都无问题,若刚好是五,则作如下规定:若前面是偶数,则将五舍去;若前面是奇数,则将五进一.实践证明,当进行大量运算时,按上述原则进行舍入,整个运算的误差积累较小.

如果近似值 x^* 的误差限是它的某一位的半个单位,就说它“准确”到这一位,并且从这一位一直到前面第一个非零数字为止的所有数字均称有效数字,有效数字实际上是将四舍五入抽象成数学语言而引入的一个新名词.

定义 1.1 若近似值 x^* 的误差限是某一位的半个单位,该位到 x^* 的第一位非零数字有 n 位,则称 x^* 有 n 位有效数字.

定义 1.1 也可如下表述.

定义 1.1' 对于 x 的近似值(规格化形式)

$x^* = \pm 0.a_1a_2\cdots a_n \times 10^m$ (其中, $a_1, a_2, \cdots, a_n$ 是 0 到 9 之间的自然数, $a_1 \neq 0$),

如果误差 $|x - x^*| \leq \frac{1}{2} \times 10^{m-l}$, $1 \leq l \leq n$,

则称近似值 x^* 有 l 位有效数字.

按照这种说法, π 的近似值 3.1416, 3.14159 分别有 5 位和 6 位有效数字.

1.2.3 相对误差限与有效数字的联系

用绝对误差来刻画近似数的精确程度是有局限性的,因为它没有反映出它在原数中所占的比例.于是,记

$$e_r = \frac{x - x^*}{x},$$

e_r 称为近似值 x^* 的相对误差,由于准确值 x 未知,实际上总把 $\frac{x - x^*}{x^*}$ 作为 x^* 的相对误差,相对误差一般用百分比表示.

相对误差绝对值的一个上限 ϵ_r^* 称作近似值的相对误差限,即有

$$\frac{|x - x^*|}{|x^*|} \leq \epsilon_r^*.$$

下面分析相对误差限与有效数字的联系.

定理 1.1 设近似值 $x^* = \pm 0.a_1a_2\cdots a_n \times 10^m$ 有 n 位有效数字,则其相对误差限为 $\frac{1}{2a_1} \times 10^{-n+1}$.

证 由于 x^* 有 n 位有效数字,则

$$|x - x^*| \leq \frac{1}{2} \times 10^{m-n},$$

而

$$|x^*| \geq a_1 \times 10^{m-1},$$

所以

$$\frac{|x - x^*|}{|x^*|} \leq \frac{\frac{1}{2} \times 10^{m-n}}{a_1 \times 10^{m-1}} = \frac{1}{2a_1} \times 10^{-n+1}. \quad (1.5)$$

定理 1.2 设近似值 $x^* = \pm 0.a_1a_2\cdots a_n \times 10^m$ 的相对误差限为 $\frac{1}{2(a_1+1)} \times 10^{-n+1}$, 则它至少有 n 位有效数字.

证 由于 $|x^*| \leq (a_1+1) \times 10^{m-1}$,
故由题设, 有

$$\begin{aligned} |x - x^*| &= \frac{|x - x^*|}{|x^*|} \cdot |x^*| \leq \frac{1}{2(a_1+1)} \times 10^{-n+1} \times (a_1+1) \times 10^{m-1} \\ &= \frac{1}{2} \times 10^{m-n}. \end{aligned} \quad (1.6)$$

所以 x^* 至少有 n 位有效数字.

1.2.4 数值运算的误差估计

设近似数 x_1^* 与 x_2^* 的误差限分别为 $\epsilon(x_1^*)$ 及 $\epsilon(x_2^*)$, 则它们进行加、减、乘、除运算得到的误差限分别为

$$\begin{cases} \epsilon(x_1^* \pm x_2^*) \approx \epsilon(x_1^*) + \epsilon(x_2^*), \\ \epsilon(x_1^* \cdot x_2^*) \approx |x_1^*| \epsilon(x_2^*) + |x_2^*| \epsilon(x_1^*), \quad (x_2^* \neq 0), \\ \epsilon(x_1^* / x_2^*) \approx \frac{|x_1^*| \epsilon(x_2^*) + |x_2^*| \epsilon(x_1^*)}{|x_2^*|^2} \end{cases} \quad (1.7)$$

设函数为 $f(x_1, x_2, \cdots, x_n)$, 若需要计算 $A = f(x_1, x_2, \cdots, x_n)$, 而 $x_1, x_2, \cdots, x_n$ 的近似值为 $x_1^*, x_2^*, \cdots, x_n^*$, A 的近似值为 $A^* = f(x_1^*, x_2^*, \cdots, x_n^*)$, 于是由 Taylor 展开得近似值 A 的误差 $e(A^*)$ 为

$$\begin{aligned} e(A^*) &= A - A^* = f(x_1, x_2, \cdots, x_n) - f(x_1^*, x_2^*, \cdots, x_n^*) \\ &\approx \sum_{k=1}^n \left(\frac{\partial f(x_1^*, x_2^*, \cdots, x_n^*)}{\partial x_k} \right) (x_k^* - x_k) \\ &= \sum_{k=1}^n \left(\frac{\partial f}{\partial x_k} \right)^* e_k^*, \end{aligned} \quad (1.8)$$

$$\text{误差限为} \quad \epsilon(A^*) \approx \sum_{k=1}^n \left| \left(\frac{\partial f}{\partial x_k} \right)^* \right| \epsilon(x_k^*), \quad (1.9)$$

而 A^* 的相对误差限为

$$\epsilon_r^* = \epsilon_r(A^*) = \frac{\epsilon(A^*)}{|A^*|} \approx \sum_{k=1}^n \left| \left(\frac{\partial f}{\partial x_k} \right)^* \right| \frac{\epsilon(x_k^*)}{|A^*|}. \quad (1.10)$$

1.3 数值稳定性和病态问题

算法的数值稳定性是指, 用一个算法进行计算, 由于初始数据误差在计算中传播, 使计算结果误差增长很快, 就说该算法是数值不稳定的, 否则是数值稳定的.

例 1.1 求积分 $I_n = \int_0^1 \frac{x^n}{x+5} dx$, $n = 0, 1, \dots, 8$ 的值.

解 由

$$I_n + 5I_{n-1} = \int_0^1 \frac{x^n + 5x^{n-1}}{x+5} dx = \int_0^1 x^{n-1} dx = \frac{1}{n},$$

可得两个递推算法.

算法一
$$I_n = \frac{1}{n} - 5I_{n-1}, \quad n=1, 2, \dots, 8,$$

算法二
$$I_{n-1} = \frac{1}{5} \left(\frac{1}{n} - I_n \right), \quad n=8, 7, \dots, 1.$$

算法一的初始值

$$I_0 = \int_0^1 \frac{1}{x+5} dx = \ln(1.2) = 0.1823215\dots$$

算法二的初始值用 Matlab 的库函数 quadl 计算的结果是

$$I_8 = \int_0^1 \frac{x^8}{x+5} dx = 0.01883692\dots$$

现在假设计算机的十进制尾数是 4 位, 即取十进制 4 位有效数字进行计算. 记算法一的计算值为 $\tilde{I}_n^{(1)}$, 算法二的计算值为 $\tilde{I}_n^{(2)}$. 运算结果见表 1.2, 其中 $\tilde{I}_n$ 是由库函数 quadl 求出的 I_n 的近似值, 取 6 位有效数字. 易知, 对于任何自然数 n , 都有 $0 < I_n < 1$, 并且 I_n 单调减. 可见, 算法一是不稳定的, 算法二是稳定的.

表 1.2

n	$\tilde{I}_n^{(1)}$	$\tilde{I}_n^{(2)}$	$\tilde{I}_n$
0	0.1823	0.1823	0.182322
1	0.08850	0.08839	0.0883922
2	0.05750	0.05804	0.0580389
3	0.04583	0.04314	0.0431387
4	0.02085	0.03431	0.0343063
5	0.09575	0.02847	0.0284684
6	-0.3121	0.02433	0.0243249
7	1.703	0.02123	0.0212326
8	-8.390	0.01884	0.0188369

由于串行运算的舍入误差是逐步传递的, 所以可用“向后误差”分析方法, 即把舍入误差的影响看成是初值误差即输入数据误差的传播.

定义 1.2 对于某个算法, 若输入数据的误差在计算过程中迅速增长而得不到控制, 则称该算法是数值不稳定的, 否则是数值稳定的.

在例 1.1 中, 当仅考虑初始值有误差时, 对于算法一, 由

$$I_n = \frac{1}{n} - 5I_{n-1}, \quad \tilde{I}_n^{(1)} = \frac{1}{n} - 5\tilde{I}_{n-1}^{(1)},$$

得知误差 $e_n^{(1)} = I_n - \tilde{I}_n^{(1)}$ 满足

$$e_n^{(1)} = -5e_{n-1}^{(1)} = (-5)^n e_0^{(1)}, \quad n = 1, 2, \dots,$$

因此算法一是不稳定的. 对于算法二, 同理可知误差 $e_i^{(2)} = I_i - \tilde{I}_i^{(2)}$ 满足

$$e_{i-1}^{(2)} = -\frac{1}{5}e_i^{(2)}, \quad i = n, n-1, \dots, 1,$$

从而 $e_0^{(2)} = \left(-\frac{1}{5}\right)^n e_n^{(2)}$, 因此算法二是稳定的.

例 1.2 求实系数二次代数方程 $ax^2 + bx + c = 0$ 的根, 其中 $b^2 - 4ac > 0, a \neq 0$.

解 考虑下列两种算法.

算法一
$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

算法二
$$x_1 = \frac{-b - \text{sign}(b)\sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{c}{ax_1},$$

其中 sign 表示取数的符号, 即

$$\text{sign}(b) = \begin{cases} 1, & b \geq 0, \\ -1, & b < 0. \end{cases}$$

对于算法一, 若 $b^2 \gg 4ac$, 则算法一是不稳定的, 否则是稳定的. 这是因为前一种情况的分子有一个相近数相减, 会大量损失有效数字, 从而必有一个结果的误差很大. 算法二不存在这个问题, 因此在任何情况下都是稳定的. 称算法一是条件稳定的, 算法二是无条件稳定的.

与例 1.1 一样用 4 位有效数字计算方程

$$x^2 + 62.10x + 1.000 = 0,$$

结果是

算法一
$$x_1 \approx -62.08, \quad x_2 \approx -0.02000.$$

算法二
$$x_1 \approx -62.08, \quad x_2 \approx -0.01611.$$

准确解是 $x_1 = -62.083892\dots, x_2 = -0.016107237\dots$, 这里 $b^2 \gg 4ac$, 所以算法一不稳定, 舍入误差对 x_2 的影响很大.

1.4 数值运算中的一些原则

数值运算总是在一个预先设计好的算法中进行的. 所谓算法, 就是一个有限的基本运算序列, 这个序列规定了怎样从输入数据去计算出问题的解. 由于运算是在计算机上进行的, 而计算机的字长有限, 因而产生舍入误差. 为减小舍入误差的影响, 设计算法时应遵循以下一些原则.

(1) 避免除数绝对值远远小于被除数绝对值的除法(见式(1.7)中的除法运算).

(2) 避免两相近数相减.

例如, $a_1=0.12345$, $a_2=0.12346$, 各有 5 位有效数字. 而 $a_2-a_1=0.00001$, 只剩下 1 位有效数字.

几种经验性避免方法:

$$\sqrt{x+\epsilon}-\sqrt{x}=\frac{\epsilon}{\sqrt{x+\epsilon}+\sqrt{x}}; \quad \ln(x+\epsilon)-\ln x=\ln\left(1+\frac{\epsilon}{x}\right);$$

当 $|x|\ll 1$ 时,

$$1-\cos x=2\sin^2 \frac{x}{2},$$

$$e^x-1=x\left[1+\frac{1}{2}x+\frac{1}{6}x^2+\cdots\right].$$

(3) 防止大数“吃掉”小数.

例如, 在 5 位十进制计算机上, 计算

$$s=52492+\sum_{i=1}^{1000}a_i, \quad 0.1\leq a_i\leq 0.9,$$

由于浮点运算要对阶, 在计算机中 52492 表示为 0.52492×10^5 , 若取 $a_i=0.9$, 对阶时 $a_i=0.000009\times 10^5$, 它在 5 位的计算机中表示为机器 0, 所以将 52492 与每个 a_i 逐一相加, 其结果仍为 52492, 出现了大数 52492“吃掉”小数 a_i , 这时, 应先将数量级相同的 1000 个 a_i 相加, 最后再加 52492, 才能得出精确的结果.

注: 求和时从小到大相加, 可使和的误差减小.

(4) 注意简化计算步骤, 减少运算次数(见计算多项式的秦九韶算法), 避免误差积累.

(5) 要有数值稳定性, 即能控制舍入误差的传播(见例 1.1).

1.5 几个算例

例 1.3 求 $\sqrt{3}$ 的近似值, 使其绝对误差限精确到 $\frac{1}{2}\times 10^{-1}$, $\frac{1}{2}\times 10^{-2}$, $\frac{1}{2}\times 10^{-3}$.

解 因为 $\sqrt{3}=1.73205\cdots$, 又由于

$$\epsilon^*(1.7)=|\sqrt{3}-1.7|=0.03205\cdots<0.05,$$

$$\epsilon^*(1.73)=|\sqrt{3}-1.73|=0.00205\cdots<0.005,$$

$$\epsilon^*(1.732)=|\sqrt{3}-1.732|=0.00005\cdots<0.00005,$$

所以

$$x_1^*=1.7, \quad x_2^*=1.73, \quad x_3^*=1.732.$$

例 1.4 测量一木板长是 954 cm, 问测量的相对误差是多大?

解 因为实际问题所截取的近似数, 其绝对误差限一般不超过最小刻度的半个单位, 所以当 $x=954$ cm 时, 有 $\epsilon^*=0.5$ cm, 则

$$e_r(954) = \frac{0.5}{954} = 0.0005241\cdots < 0.000525 = 0.0525\% < 0.053\%,$$

故其相对误差限可取为 $\epsilon_r^* = 0.053\%$.

例 1.5 已知近似数 x^* 有两位有效数字, 试求其相对误差限.

解 利用式(1.5), 已知 $n=2$, 但近似数 x^* 的第一位有效数字 a_1 未知, 遇到这种情况时, 可按第一位有效数字出现的最不利的情况估计, 即令 $a_1=1$, 则

$$\epsilon_r^* \leq \frac{1}{2a_1} \times 10^{-(n-1)} = \frac{1}{2 \times 1} \times 10^{-(2-1)} = 5\%,$$

故 x^* 的相对误差限为 5% .

例 1.6 为使 $\sqrt{70}$ 的近似数的相对误差限小于 0.1% , 问查开方表时, 要取几位有效数字?

解 设查表时取 n 位有效数字, 那么由式(1.5), 并注意到 $8 \leq \sqrt{70} \leq 9$, 可取 $a_1=8$. 因此为使 $\sqrt{70}$ 的近似数相对误差限 $\epsilon_r^* < 0.1\%$, 只需取 n 使得

$$\frac{1}{2a_1} \times 10^{-(n-1)} = \frac{1}{2 \times 8} \times 10^{-(n-1)} < 0.1\%,$$

得 $n=3$ 即可. 查开方表取 $\sqrt{70} \approx 8.37$.

例 1.7 设有 3 个近似数

$$a = 2.31, \quad b = 1.93, \quad c = 2.24,$$

它们都有 3 位有效数字, 试计算 $p=a+bc$, $\epsilon(p)$ 和 $\epsilon_r(p)$; 并问: p 的计算结果能有几位有效数字?

解

$$p = 2.31 + 1.93 \times 2.24 = 6.6332,$$

$$\begin{aligned} \epsilon(p) &= \epsilon(a) + \epsilon(bc) \approx \epsilon(a) + |b| \epsilon(c) + |c| \epsilon(b) \\ &= 0.005 + 0.005(1.93 + 2.24) = 0.02585, \end{aligned}$$

$$\epsilon_r(p) = \frac{\epsilon(p)}{|p|} \approx \frac{0.02585}{6.6332} \approx 0.39\%.$$

因为 $\epsilon(p) \approx 0.02585 < 0.05$, 所以, $p=6.6332$ 中能有 2 位有效数字.

例 1.8 设 $f(x, y) = \frac{\cos y}{x}$, $x = 1.30 \pm 0.005$, $y = 0.871 \pm 0.0005$. 如果用 $\tilde{u} = f(1.30, 0.871)$ 作为 $f(x, y)$ 的近似值, 问 $\tilde{u}$ 能有几位有效数字?

解

$$\tilde{u} = \frac{\cos 0.871}{1.30} \approx 0.49543,$$

由于

$$\frac{\partial f}{\partial x} = -\frac{\cos y}{x^2}, \quad \frac{\partial f}{\partial y} = -\frac{\sin y}{x},$$

所以

$$\epsilon(\tilde{u}) \approx \left| \frac{\cos 0.871}{1.30^2} \right| \times 0.005 + \left| \frac{\sin 0.871}{1.30} \right| \times 0.0005 \approx 0.0022 < 0.005.$$

因而 $\tilde{u}=0.49543$ 能有 2 位有效数字.

例 1.9 求二次方程 $x^2-16x+1=0$ 的较小正根, 要求有 3 位有效数字.

解 求解方程 $x^2-16x+1=0$, 得两根, $x_1=8-\sqrt{63}$, $x_2=8+\sqrt{63}$. 此方程最小正根为 $x_1=8-\sqrt{63}$. 若取 $\sqrt{63}\approx 7.94$, 具有 3 位有效数字, 则 $x_1=8-\sqrt{63}\approx 8-7.94=0.06=x_1^*$, 只有 1 位有效数字. 若改用

$$x_1=8-\sqrt{63}=\frac{1}{8+\sqrt{63}}\approx\frac{1}{15.94}\approx 0.0627,$$

则此方程最小正根具有 3 位有效数字.

例 1.10 如果利用 4 位函数表计算 $(1-\cos 2^\circ)$, 试用不同方法计算并比较结果的误差.

解 用 4 位函数表直接计算.

$1-\cos 2^\circ\approx 1-0.9994=0.0006$, 只有 1 位有效数字.

$1-\cos 2^\circ=\frac{\sin^2 2^\circ}{1+\cos 2^\circ}\approx\frac{(0.03490)^2}{1.9994}\approx 6.092\times 10^{-4}$, 具有 4 位有效数字.

$1-\cos 2^\circ=2\sin^2 1^\circ\approx 6.09\times 10^{-4}$, 具有 3 位有效数字.

准确值 $1-\cos 2^\circ=6.0917\cdots\times 10^{-4}$, 故以上 3 种算法误差限分别为 0.1×10^{-4} , 0.0003×10^{-4} , 0.002×10^{-4} .

例 1.11 已测得某场地长 l 的值为 $l^*=110$ m, 宽 d 的值为 $d^*=80$ m, 已知 $|l-l^*|\leq 0.2$ m, $|d-d^*|\leq 0.1$ m. 试求面积 $S=ld$ 的绝对误差限与相对误差限.

解 因 $S=ld$, $\frac{\partial S}{\partial l}=d$, $\frac{\partial S}{\partial d}=l$, 由式(1.8)知

$$\epsilon(S^*)\approx\left|\left(\frac{\partial S}{\partial l}\right)^*\right|\epsilon(l^*)+\left|\left(\frac{\partial S}{\partial d}\right)^*\right|\epsilon(d^*),$$

其中 $\left(\frac{\partial S}{\partial l}\right)^*=d^*=80$ m, $\left(\frac{\partial S}{\partial d}\right)^*=l^*=110$ m,

而 $\epsilon(l^*)=0.2$ m, $\epsilon(d^*)=0.1$ m,

于是绝对误差限

$$\epsilon(S^*)\approx(80\times 0.2+110\times 0.1)\text{ m}^2=27\text{ m}^2.$$

相对误差限

$$\epsilon_r(S^*)=\frac{\epsilon(S^*)}{|S^*|}=\frac{\epsilon(S^*)}{l^*d^*}\approx\frac{27}{8800}\approx 0.31\%.$$

1.6 算法的实现

掌握数值计算方法的根本目的是解决所遇到的各种数学问题, 为此必须在计算机上实现算法. 这包含使用软件工具和自编程序两方面的含义.

本课程涉及的大多是数值分析的基本问题,有很多现成的通用或专用数学软件包含有实现这些算法的子程序,如 Lapack, YSMP, Odepack, Minpack, Opt, 等等. 高级计算机语言,如 Fortran, C, Matlab, 以及数学工具软件 Mathematica, 都含有丰富的库函数. 对于比较简单的数学问题,可以直接调用这些子程序或库函数求出数值解. 其中, Mathematica 还可对某些数学问题求出解析解.

尽管如此,自编程序仍是不可缺少的. 一则,子程序和库函数都是孤立的功能块,而实际问题往往是错综复杂的,需要综合使用多种数学方法才能解决;二则,一个数学问题可能有多种算法,它们各自有优缺点,为了选择合适的算法,并对可能出现的问题采取解决的对策,必须对算法的性质有深刻的了解,而这只有通过理论和实践两方面才能得到解决;三则,数学软件和函数库并非包罗万象,有的问题需要自己构造算法并编制软件.

为了提高实现算法的能力,本课程对主要算法都有数值计算的例题及配备有数值试验题.

习 题 1

1. 设 $x > 0$, x 的相对误差为 δ , 求 $\ln x$ 的误差.
2. 设 x 的相对误差为 2% , 求 x^n 的相对误差.
3. 下列各数都是经过四舍五入得到的近似数,即误差限不超过最后一位的半个单位,试指出它们是几位有效数字.

$$x_1^* = 1.1021, x_2^* = 0.031, x_3^* = 385.6, x_4^* = 56.430, x_5^* = 7 \times 1.0.$$

4. 利用式(1.6)求下列各近似值的误差限.

$$(1) x_1^* + x_2^* + x_4^*; (2) x_1^* \cdot x_2^* \cdot x_3^*; (3) x_2^* / x_4^*.$$

其中 $x_1^*, x_2^*, x_3^*, x_4^*$ 均为第3题所给的数.

5. 计算球体积,要使相对误差限为 1% ,问度量半径 R 时允许的相对误差限是多少?

6. 设 $Y_0 = 28$, 按递推公式

$$Y_n = Y_{n-1} - \frac{1}{100} \sqrt{783}, \quad n = 1, 2, \dots.$$

计算到 Y_{100} . 若取 $\sqrt{783} \approx 27.982$ (5位有效数字), 试问计算 Y_{100} 将有多大误差?

7. 求方程 $x^2 - 56x + 1 = 0$ 的两个根,使它至少具有4位有效数字 ($\sqrt{783} \approx 27.982$).

8. 当 N 充分大时, 怎样求 $\int_N^{N+1} \frac{1}{1+x^2} dx$?
9. 正方形的边长大约为 100 cm, 应怎样测量才能使其面积误差不超过 1 cm^2 ?
10. 设 $s = \frac{1}{2}gt^2$, 假定 g 是准确的, 而对 t 的测量有 $\pm 0.1 \text{ s}$ 的误差, 证明: 当 t 增加时, s 的绝对误差增加, 而相对误差却减少.
11. 已测得某住房长 $l^* = 4.32 \text{ m}$, 宽 $d^* = 3.12 \text{ m}$, 已知 $|l - l^*| \leq 0.01 \text{ m}$, $|d - d^*| \leq 0.01 \text{ m}$, 试求住房面积 $S = ld$ 的绝对误差限与相对误差限.
12. 序列 $\{y_n\}$ 满足递推关系

$$y_n = 10y_{n-1} - 1, \quad n = 1, 2, \dots$$

若 $y_0 = \sqrt{2} \approx 1.41$ (3 位有效数字), 问计算到 y_{10} 时误差有多大? 这个计算过程稳定吗?

13. 计算 $f = (\sqrt{2} - 1)^6$, 取 $\sqrt{2} \approx 1.4$. 利用下列算式计算, 哪一个得到的结果最好?

(1) $\frac{1}{(\sqrt{2}+1)^6}$; (2) $(3-2\sqrt{2})^3$; (3) $\frac{1}{(3+2\sqrt{2})^3}$; (4) $99-70\sqrt{2}$.

14. $f(x) = \ln(x - \sqrt{x^2 - 1})$, 求 $f(30)$ 的值, 若开平方用 6 位函数表, 问求对数时误差有多大? 若改用另一等价公式

$$\ln(x - \sqrt{x^2 - 1}) = -\ln(x + \sqrt{x^2 - 1})$$

计算, 求对数时误差有多大?

15. 若用下列两种方法计算 e^{-5} 的近似值, 问下列哪种方法能提供较好的近似值?

(1) $e^{-5} \approx \sum_{i=0}^9 (-1)^i \frac{5^i}{i!} = x_1^*$; (2) $e^{-5} \approx \left(\sum_{i=0}^9 \frac{5^i}{i!} \right)^{-1} = x_2^*$.

16. 设 $I_n = \int_0^1 x^n e^{x-1} dx, n = 0, 1, 2, \dots$, 验证 $I_0 = 1 - e^{-1}, I_n = 1 - nI_{n-1}$. 若 $e^{-1} \approx 0.3679$, 依次计算 $I_0, I_1, \dots, I_9$, 并证明: 误差是逐次递增的, 且 I_9 严重失真.

数值实验题 1

实验 1.1 病态问题

实验目的: 算法有“优”与“劣”之分, 问题也有“好”和“坏”之别. 对数值方法的研究而言, 所谓坏问题就是问题本身对扰动敏感, 反之属于好问题. 希望读者通过本实验对此有一个初步的体会.

数值分析的大部分研究课题中, 如线性代数方程组、矩阵特征值、非线性方程及方

程组等都存在病态的问题. 病态问题要通过研究和构造特殊的算法来解决, 当然一般要付出一些代价, 如耗用更多的机器时间、占用更多的存储空间等.

问题提出: 考虑一个高次的代数多项式

$$p(x) = (x-1)(x-2)\cdots(x-20) = \prod_{k=1}^{20} (x-k), \quad (\text{E. 1. 1})$$

显然该多项式的全部根为 $1, 2, \dots, 20$, 共计 20 个, 且每个根都是单重的 (也称为简单的). 现考虑该多项式的一个扰动

$$p(x) + \varepsilon x^{19} = 0, \quad (\text{E. 1. 2})$$

其中, ε 是一个非常小的数. 这相当于是对方程 (E. 1. 1) 中 x^{19} 的系数作一个小的扰动. 比较方程 (E. 1. 1) 和方程 (E. 1. 2) 根的差别, 从而分析方程 (E. 1. 1) 的解对扰动的敏感性.

实验内容: 为了实验方便, 我们先介绍两个 Matlab 函数: “roots” 和 “poly”.

$$u = \text{roots}(a),$$

其中, 若变量 a 为存储的 $(n+1)$ 维的向量, 则该函数的输出 u 为一个 n 维的向量. 设 a 的元素依次为 $a_1, a_2, \dots, a_{n+1}$, 则输出 u 的各分量是多项式方程

$$a_1 x^n + a_2 x^{n-1} + \cdots + a_n x + a_{n+1} = 0$$

的全部根. 而函数

$$b = \text{poly}(v)$$

的输出 b 是一个 $(n+1)$ 维变量, 它是以 n 维变量 v 的各分量为根的多项式的系数. 可见 “roots” 和 “poly” 是两个互逆的运算函数.

$$ve = \text{zeros}(1, 21);$$

$$ve(2) = \text{ess};$$

$$\text{roots}(\text{poly}(1:20) + ve).$$

上述简单的 Matlab 程序便得到方程 (E. 1. 2) 的全部根, 程序中 “ess” 即是方程 (E. 1. 2) 中的 ε .

实验要求:

(1) 选择充分小的 ess 反复进行上述实验, 记录结果的变化并进行分析. 如果扰动项的系数 ε 很小, 自然感觉方程 (E. 1. 1) 和方程 (E. 1. 2) 的解应当相差很小. 计算中你有什么出乎意料的发现? 表明有些解关于如此的扰动敏感性如何?

(2) 将方程 (E. 1. 2) 中的扰动项改成 εx^{18} 或其他形式, 实验中又有怎样的现象出现?

实验 1.2 误差传播与算法稳定性

实验目的: 体会稳定性在选择算法中的地位. 误差扩张的算法是不稳定的, 是我们

所不期望的;误差衰竭的算法是稳定的,是我们努力寻求的,这是贯穿本课程的目标.

问题提出:考虑一个简单的由积分定义的序列

$$E_n = \int_0^1 x^n e^{x-1} dx, \quad n = 1, 2, \dots, \quad (\text{E. 1. 3})$$

显然 $E_n > 0 (n=1, 2, \dots)$, 当 $n=1$ 时, 利用部分积分易得

$$E_1 = \int_0^1 x e^{x-1} dx = 1/e.$$

而对 $n \geq 2$, 经分部积分可得

$$E_n = \int_0^1 x^n e^{x-1} dx = x^n e^{x-1} \Big|_0^1 - \int_0^1 n x^{n-1} e^{x-1} dx, \quad n = 1, 2, \dots.$$

也就是

$$E_n = 1 - n E_{n-1}, \quad n = 2, 3, \dots, \quad (\text{E. 1. 4})$$

而再由式(E. 1. 3), 可得

$$E_n = \int_0^1 x^n e^{x-1} dx \leq \int_0^1 x^n dx = 1/(n+1). \quad (\text{E. 1. 5})$$

实验内容:由递推关系式(E. 1. 4),可以得到计算式(E. 1. 4)积分序列 $\{E_n\}$ 的两种算法. 其一为式(E. 1. 4)的直接应用,即

$$E_1 = 1/e, \quad E_n = 1 - n E_{n-1}, \quad n = 2, 3, \dots. \quad (\text{E. 1. 6})$$

另一种算法则是利用式(E. 1. 4)变形得到 $E_N = 0$,

$$E_{n-1} = \frac{1 - E_n}{n}, \quad n = N-1, N-2, \dots, 3, 2. \quad (\text{E. 1. 7})$$

实验要求:

(1) 分别用算法(E. 1. 6)、算法(E. 1. 7)并在计算中分别采用5位、6位和7位有效数字,请判断哪种算法能给出精确的结果?

(2) 两种算法的优劣,与你的第一感觉是否吻合? 请从理论上证明你实验得出的结果,解释实验的结果. 设算法(E. 1. 6)中的 E_1 的计算误差为 e_1 , 由 E_1 递推计算 $E_n (n < N)$ 的误差为 e_n . 算法(E. 1. 7)中 E_N 的误差为 ϵ_N , 由 E_N 向前递推计算 $e_n (n < N)$ 的误差为 ϵ_n , 如果在上述两算法中都假定后面的计算不再引入其他误差,试给出 e_n 与 e_1 的关系以及 ϵ_n 与 ϵ_N 的关系.

(3) 算法(E. 1. 6)中,通常 e_1 会很小,当 n 增大时, e_n 的变化趋势如何? 算法(E. 1. 7)中的 ϵ_N 通常相对比较大,当 n 减小时,误差 ϵ_n 又是如何传播的? 也就是说比较一下上述两个算法,在某一步产生误差后,该误差对后面的影响是衰减还是扩张的?

(4) 通过理论分析与计算实验,针对算法(E. 1. 6)和算法(E. 1. 7)的稳定性,给出你的结论.

第2章 插值法

实际问题中的函数是多种多样的,有的表达式很复杂,有的甚至给不出数学式子,只提供了一些离散数据,譬如某些点上的函数值与导数值.由于问题的复杂性,直接研究函数 $f(x)$ 可能很困难,面对这种情况,一个很自然的想法是设法将考察的函数 $f(x)$ “简单化”.就是说,构造某个简单函数 $P(x)$ 作为 $f(x)$ 的近似,然后通过处理 $P(x)$ 获得关于 $f(x)$ 的结果,如果要求近似函数 $P(x)$ 取给定的离散数据,则称之为 $f(x)$ 的插值函数.

2.1 问题的提法

已知数据表如表 2.1 所示. 其中, $f(x)$ 定义在区间 $[a, b]$ 上, $x_0, x_1, \dots, x_n$ 为区间 $[a, b]$ 上 $(n+1)$ 个互不相同的点,要求在一个性质优良、便于计算的函数类 $\{P(x)\}$ 中,选出一个满足

表 2.1

x	x_0	x_1	x_2	$\dots$	x_n
$y=f(x)$	y_0	y_1	y_2	$\dots$	y_n

$$P(x_i) = y_i, \quad i=0, 1, 2, \dots, n \quad (2.1)$$

的函数 $P(x)$ 作为 $f(x)$ 的近似. 这就是最基本的插值问题,为了便于叙述,通常称区间 $[a, b]$ 为插值区间,点 $x_0, x_1, \dots, x_n$ 为插值节点,函数类 $\{P(x)\}$ 为插值函数类,条件 (2.1) 为插值条件, $P(x)$ 为插值函数,求插值函数 $P(x)$ 的方法称为插值法.

插值函数类的取法很多,可以是代数多项式、三角多项式或有理函数,也可以是区间 $[a, b]$ 上任意光滑函数或分段光滑函数,等等. 当然对于同一问题来讲,在不同插值函数类中求的插值函数 $P(x)$,逼近被插值函数 $f(x)$ 的效果是不同的,本章选择一种最基本、最常用的函数类,即次数不超过 n 的代数多项式

$$P_n(x) = a_0 + a_1x + \dots + a_nx^n, \quad (2.2)$$

其中, $a_0, a_1, \dots, a_n$ 为实数. 形如式 (2.2) 的插值函数称为 n 次代数插值多项式,相应的插值问题称为代数多项式插值.

多项式插值的几何意义表述为,通过给定的 $(n+1)$ 个点 (x_i, y_i) ($i=0, 1, 2, \dots, n$) 作一条 n 次代数曲线 $y=P_n(x)$,用以近似地代替曲线 $y=f(x)$ (见图 2.1).

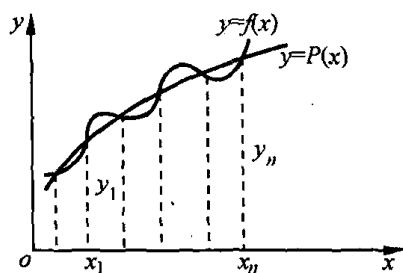


图 2.1

针对条件(2.1)来求它的插值多项式,即归结为求插值多项式 $P_n(x)=a_0+a_1x+\cdots+a_nx^n$ 的系数 a_i ($i=0, 1, 2, \dots, n$). 由插值条件(2.1)知,系数 a_i 应满足线性方程组

$$\begin{cases} a_0 + a_1x_0 + \cdots + a_nx_0^n = y_0, \\ a_0 + a_1x_1 + \cdots + a_nx_1^n = y_1, \\ \cdots \\ a_0 + a_1x_n + \cdots + a_nx_n^n = y_n. \end{cases} \quad (2.3)$$

方程组(2.3)的系数行列式为

$$V = \begin{vmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^n \end{vmatrix},$$

这是一个 $(n+1)$ 阶 Vandermonde(范德蒙)行列式. 由行列式的性质可得

$$V = \prod_{\substack{n \geq i > j \geq 0}} (x_i - x_j)$$

或

$$V = \prod_{i=1}^n \prod_{j=0}^{i-1} (x_i - x_j),$$

因为 $x_0, x_1, \dots, x_n$ 互不相同,故上式右端所有因子 $x_i - x_j \neq 0$, 于是 $V \neq 0$, 这表明方程组(2.3)存在唯一的解 $a_0, a_1, \dots, a_n$, 即得出如下结论.

定理 2.1 满足条件 $P_n(x_i)=y_i$ ($i=0, 1, 2, \dots, n$) 的 n 次插值多项式 $P_n(x)$ 是存在且唯一的.

必须指出,虽然利用线性代数知识证明了插值多项式的存在唯一性,但并没有真正解决插值多项式 $P_n(x)$ 的求法,因为利用方程组(2.3)求系数 a_i ($i=0, 1, 2, \dots, n$), 计算可能会很复杂. 在下面几节里将提供比较简便的求法.

注:若不将多项式次数限制为 n , 则插值多项式不唯一,例如, $P(x) = P_n(x) + q(x) \prod_{i=0}^n (x - x_i)$ 也是满足条件(2.1)的插值多项式,其中 $q(x)$ 可以是任意多项式.

2.2 Lagrange(拉格朗日)插值

2.2.1 Lagrange 插值多项式的基函数构造法

先从最简单的一次插值($n=1$)开始,即作一次式 $L_1(x)$,使之满足条件

$$L_1(x_0) = y_0, \quad L_1(x_1) = y_1.$$

从几何上看, $y=L_1(x)$ 即是过点 (x_0, y_0) 和 (x_1, y_1) 的直线, 由解析几何知道, 这条直线可用点斜式表示为

$$L_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0). \quad (2.4)$$

线性插值公式(2.4)亦可表为下列两点式

$$L_1(x) = \frac{x - x_1}{x_0 - x_1}y_0 + \frac{x - x_0}{x_1 - x_0}y_1. \quad (2.5)$$

若记

$$l_0(x) = \frac{x - x_1}{x_0 - x_1}, \quad l_1(x) = \frac{x - x_0}{x_1 - x_0},$$

则式(2.5)还可写为

$$L_1(x) = y_0 l_0(x) + y_1 l_1(x) = \sum_{i=0}^1 y_i l_i(x),$$

其中, $l_i(x)$ 称为插值基函数, 它们的图形见图 2.2, 且有如下特点:

$$\begin{cases} l_0(x) + l_1(x) = 1, \\ l_0(x_0) = 1, l_0(x_1) = 0, \\ l_1(x_0) = 0, l_1(x_1) = 1, \end{cases} \quad (2.6)$$

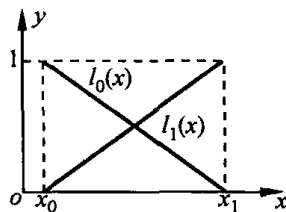


图 2.2

即

$$l_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j, \end{cases} \quad i, j = 0, 1.$$

由上可见, $l_0(x) = \frac{x - x_1}{x_0 - x_1}$ 与 $l_1(x) = \frac{x - x_0}{x_1 - x_0}$ 满足条件(2.6)且都是线性函数. 反过

来, 如果一次函数 $l_0(x)$ 与 $l_1(x)$ 满足条件(2.6), 那么可以证明, 它们只能是 $\frac{x - x_1}{x_0 - x_1}$ 与

$$\frac{x - x_0}{x_1 - x_0}.$$

事实上, 由代数多项式的性质知, 如果 x_0 是一个 n 次多项式 $L_n(x)$ 的零点, 则多项式 $L_n(x)$ 就一定含有一因子 $(x - x_0)$, 这时

$$L_n(x) = (x - x_0)L_{n-1}(x),$$

其中, $L_{n-1}(x)$ 为 $n-1$ 次多项式.

因此, 对于一次函数 $l_0(x)$, 性质 $l_0(x_1) = 0$ 说明 x_1 是 $l_0(x)$ 的零点, 这时 $l_0(x)$ 含有因子 $(x - x_1)$; 由于 $l_0(x)$ 是一次多项式, 所以

$$l_0(x) = c(x - x_1), \quad (2.7)$$

其中, c 是常数. 又由 $l_0(x_0) = 1$, 将其代入式(2.7), 得 $c = \frac{1}{x_0 - x_1}$, 于是得

$$l_0(x) = \frac{x - x_1}{x_0 - x_1}.$$

同理可得

$$l_1(x) = \frac{x - x_0}{x_1 - x_0}.$$

函数 $l_0(x), l_1(x)$ 也常称一次 Lagrange 基函数.

一般情形, 求作 n 次式 $L_n(x)$, 使之满足

$$L_n(x_i) = y_i = f(x_i), \quad i = 0, 1, 2, \dots, n. \quad (2.8)$$

从几何上看, 就是求作 n 次曲线 $y = L_n(x)$, 使之通过 $(n+1)$ 个点 $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. 设

$$L_n(x) = \sum_{i=0}^n y_i l_i(x),$$

也就是仍从构造所谓插值基函数 $l_i(x)$ 入手, 由插值条件 (2.8) 知 $l_i(x)$ 应满足条件 $l_i(x_j) = \delta_{ij} (i, j = 0, 1, 2, \dots, n)$, 即 n 次多项式 $l_i(x) (i = 0, 1, 2, \dots, n)$ 满足条件

$$\begin{cases} l_0(x_0) = 1, l_0(x_1) = 0, \dots, l_0(x_n) = 0, \\ l_1(x_0) = 0, l_1(x_1) = 1, \dots, l_1(x_n) = 0, \\ \dots \\ l_n(x_0) = 0, l_n(x_1) = 0, \dots, l_n(x_n) = 1. \end{cases} \quad (2.9)$$

由条件 (2.9) 知, n 次多项式 $l_0(x)$ 有 n 个零点, 它们为 $x_1, x_2, \dots, x_n$, 所以

$$l_0(x) = c_0(x - x_1)(x - x_2) \cdots (x - x_n), \quad (2.10)$$

其中, c_0 为待定常数; 把 $x = x_0$ 代入式 (2.10), 并注意到 $l_0(x_0) = 1$, 得

$$c_0 = \frac{1}{(x_0 - x_1)(x_0 - x_2) \cdots (x_0 - x_n)},$$

则

$$l_0(x) = \frac{(x - x_1)(x - x_2) \cdots (x - x_n)}{(x_0 - x_1)(x_0 - x_2) \cdots (x_0 - x_n)}.$$

一般地, 因 n 次多项式 $l_i(x)$ 有 n 个零点 $x_0, x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n$, 且 $l_i(x_i) = 1$, 可推得

$$l_i(x) = \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)} = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (2.11)$$

于是 $y = f(x)$ 的 n 次插值多项式可写为

$$L_n(x) = \sum_{i=0}^n y_i l_i(x) = \sum_{i=0}^n \left(\prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \right) y_i. \quad (2.12)$$

易验证 $L_n(x_i) = \sum_{i=0}^n y_i l_i(x_i) = y_i$, 即 $L_n(x)$ 满足插值条件 (2.8). 形如式 (2.12) 的插值多项式称为 Lagrange 插值多项式, 由式 (2.11) 所表示的 n 次代数多项式 $l_i(x) (i = 0, 1, 2, \dots, n)$ 称为以 $x_i (i = 0, 1, 2, \dots, n)$ 为节点的 Lagrange 插值基函数. 上述构造插值多项式的方法叫做基函数法.

特别地, 1 点 0 次插值多项式为

$$L_0(x) = y_0,$$

2 点 1 次插值(线性插值)多项式为

$$L_1(x) = \frac{x-x_1}{x_0-x_1}y_0 + \frac{x-x_0}{x_1-x_0}y_1,$$

3 点 2 次插值(抛物插值)多项式为

$$L_2(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}y_0 + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}y_1 + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}y_2. \quad (2.13)$$

例 2.1 给定函数表如表 2.2 所示. 试分别用线性插值与 2 次插值求 $\ln 11.75$ 的近似值.

表 2.2

x	...	10	11	12	13	14	...
$y = \ln x$	...	2.3026	2.3979	2.4849	2.5649	2.6391	...

解 选择与 $x=11.75$ 靠近的两个节点 $x_0=11, x_1=12$, 代入线性插值公式(2.5), 得

$$\ln 11.75 \approx y_0 \frac{11.75-x_1}{x_0-x_1} + y_1 \frac{11.75-x_0}{x_1-x_0} \approx 2.4632.$$

用 2 次插值计算 $\ln 11.75$ 的近似值时, 选取节点 $x_0=11, x_1=12, x_2=13$, 由插值公式(2.13), 得

$$\begin{aligned} \ln 11.75 &\approx 2.3979 \times \frac{(11.75-12)(11.75-13)}{(11-12)(11-13)} \\ &\quad + 2.4849 \times \frac{(11.75-11)(11.75-13)}{(12-11)(12-13)} \\ &\quad + 2.5649 \times \frac{(11.75-11)(11.75-12)}{(13-11)(13-12)} \approx 2.4638. \end{aligned}$$

这个结果与 $\ln 11.75$ 的真值 2.463853... 相比较, 其准确程度是相当高的.

2.2.2 Lagrange 插值多项式的截断误差公式

若在 $[a, b]$ 上用 $L_n(x)$ 近似 $f(x)$, 则其截断误差为 $R_n(x) = f(x) - L_n(x)$, 也称为插值多项式的余项, 关于插值余项有以下定理.

定理 2.2 设 $f^{(n)}(x)$ 在 $[a, b]$ 上连续, $f^{(n+1)}(x)$ 在 (a, b) 内存在, 节点 $a \leq x_0 < x_1 < \dots < x_n \leq b$, $L_n(x)$ 是满足条件(2.8)的插值多项式, 则对任何 $x \in [a, b]$, 插值余项

$$R_n(x) = f(x) - L_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x), \quad (2.14)$$

这里 $\xi \in (a, b)$, 且依赖于 x , $\omega_{n+1}(x) = \prod_{i=0}^n (x-x_i)$.

证 由给定条件知 $R_n(x)$ 在节点 $x_i (i=0, 1, 2, \dots, n)$ 上为零, 即 $R_n(x_i)=0 (i=0, 1, 2, \dots, n)$, 于是有

$$R_n(x) = K(x)(x-x_0)(x-x_1)\cdots(x-x_n) = K(x)\omega_{n+1}(x), \quad (2.15)$$

其中, $K(x)$ 是与 x 有关的待定函数.

现在把 x 看成 $[a, b]$ 上一个固定点, 作函数

$$\varphi(t) = f(t) - L_n(t) - K(x)(t-x_0)(t-x_1)\cdots(t-x_n),$$

根据插值条件及余项定义, 可知 $\varphi(t)$ 在点 $x_0, x_1, \dots, x_n$ 及 x 处均为零, 故 $\varphi(t)$ 在 $[a, b]$ 上有 $(n+2)$ 个零点, 根据 Rolle(罗尔)定理, $\varphi'(t)$ 在 $\varphi(t)$ 的两个零点间至少有一个零点, 故 $\varphi'(t)$ 在 (a, b) 内至少有 $(n+1)$ 个零点, 对 $\varphi'(t)$ 再应用 Rolle 定理, 可知 $\varphi''(t)$ 在 (a, b) 内至少有 n 个零点. 依此类推, $\varphi^{(n+1)}(t)$ 在 (a, b) 内至少有一个零点, 记为 $\xi \in (a, b)$, 使

$$\varphi^{(n+1)}(\xi) = f^{(n+1)}(\xi) - (n+1)!K(x) = 0,$$

于是

$$K(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}, \quad \xi \in (a, b), \text{ 且依赖于 } x.$$

将它代入式(2.15), 就得到余项表达式(2.14). 证毕.

应当指出, 余项表达式只有在 $f(x)$ 的高阶导数存在时才能应用, ξ 在 (a, b) 内的具体位置通常不可能给出, 如果可以求出 $\max_{a \leq x \leq b} |f^{(n+1)}(x)| = M_{n+1}$, 那么插值多项式 $L_n(x)$ 逼近 $f(x)$ 的截断误差限是

$$|R_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega_{n+1}(x)|. \quad (2.16)$$

当 $n=1$ 时, 线性插值余项为

$$R_1(x) = \frac{1}{2} f''(\xi) \omega_2(x) = \frac{1}{2} f''(\xi) (x-x_0)(x-x_1), \quad \xi \in (x_0, x_1); \quad (2.17)$$

当 $n=2$ 时, 抛物插值余项为

$$R_2(x) = \frac{1}{6} f'''(\xi) (x-x_0)(x-x_1)(x-x_2), \quad \xi \in (x_0, x_2). \quad (2.18)$$

例 2.2 已知 e^{-x} 在 $x=1, 2, 3$ 点的值, 试对分别用线性插值、二次插值计算 $e^{-2.1}$ 的近似值所产生的误差进行估计.

解
$$R_1(x) = e^{-x} - L_1(x) = \frac{e^{-\xi_1}}{2!} (x-2)(x-3), \quad \xi_1 \in (2, 3).$$

因 e^{-x} 是递减的, 故有

$$|R_1(x)| \leq \frac{e^{-2}}{2} |(x-2)(x-3)|,$$

即

$$|R_1(2.1)| \leq \frac{e^{-2}}{2} |(2.1-2)(2.1-3)| \approx 0.006090,$$

$$R_2(x) = e^{-x} - L_2(x) = \frac{-e^{-\xi_2}}{3!}(x-1)(x-2)(x-3), \quad \xi_2 \in (1,3).$$

$$\text{而} \quad |e^{-x}| \leq e^{-1}, \quad x \in [1,3].$$

$$\text{所以} \quad |R_2(x)| \leq \frac{e^{-1}}{6} |(x-1)(x-2)(x-3)|,$$

$$\text{即} \quad |R_2(2.1)| < 0.006070.$$

在实际计算中,不仅仅要考虑截断误差,还应考虑舍入误差.下面用一个例子说明线性插值时舍入误差的影响.

例 2.3 考察数学手册里 4 位对数表中线性插值的误差.

解 设 $f(x) = \lg x$, 则 $f''(x) = -\frac{\lg e}{x^2}$, $\lg e < 0.4343$. 若 x 介于 $x_0 (\geq 1)$ 与 x_1 之间, 即 $1 \leq x_0 < x < x_1$, 那么

$$R_1(x) = -\frac{(x-x_0)(x-x_1)}{2} \frac{\lg e}{\xi^2}, \quad x_0 < \xi < x_1,$$

由此得出 $R_1(x)$ 的上界与下界, 即

$$\frac{\lg e}{2x_1^2} |(x-x_0)(x-x_1)| \leq |R_1(x)| \leq \frac{\lg e}{2x_0^2} |(x-x_0)(x-x_1)|.$$

记表距 $h = x_1 - x_0$, 则有

$$\max_{x_0 \leq x \leq x_1} |(x-x_0)(x-x_1)| = \frac{h^2}{4},$$

$$|\lg x - L_1(x)| = |R_1(x)| \leq \frac{h^2}{8} \frac{0.4343}{x_0^2} \leq 0.05429h^2.$$

对于一张 4 位对数表, $h = 0.01$, 则

$$|\lg x - L_1(x)| \leq 5.429 \times 10^{-6}, \quad 1 \leq x_0 < x_1 \leq 10,$$

因而表中所给出的 4 位数字是足够精确的. 但由于舍入误差的影响, 实际误差比上述的截断误差要大. 事实上, 如果考虑因数据的舍入误差使

$$f(x_0) = f_0 + \epsilon_0, \quad f(x_1) = f_1 + \epsilon_1,$$

其中, f_0 和 f_1 是表值, 而 ϵ_0, ϵ_1 是舍入误差, 则有

$$|f(x_0) - f_0| = |\epsilon_0| \leq \epsilon, \quad |f(x_1) - f_1| = |\epsilon_1| \leq \epsilon,$$

这里 ϵ 是已知的. 对于 4 位对数表, $\epsilon = \frac{1}{2} \times 10^{-4}$.

若不考虑计算 $L_1(x)$ 中引入新的舍入误差, 用表值 f_0, f_1 作线性插值的整个误差是

$$\begin{aligned} e(x) &= f(x) - \frac{(x_1-x)f_0 + (x-x_0)f_1}{x_1-x_0} \\ &= f(x) - \frac{(x_1-x)f(x_0) + (x-x_0)f(x_1)}{x_1-x_0} \end{aligned}$$

$$+ \frac{(x_1 - x)\epsilon_0 + (x - x_0)\epsilon_1}{x_1 - x_0} = R_1(x) + \delta_1(x), \quad x_0 \leq x \leq x_1,$$

其中, $R_1(x)$ 是插值余项, 即截断误差. 而 $\delta_1(x)$ 是依赖于 ϵ_0, ϵ_1 的线性插值的舍入误差, 显然,

$$|\delta_1(x)| = \frac{|(x_1 - x)\epsilon_0 + (x - x_0)\epsilon_1|}{|x_1 - x_0|} \leq \max\{|\epsilon_0|, |\epsilon_1|\} \leq \epsilon.$$

由于 $\epsilon = \frac{1}{2} \times 10^{-4} = 5 \times 10^{-5}$, 所以舍入误差比截断误差 5.429×10^{-6} 约大一个量级.

而线性插值的整个误差为

$$|e(x)| \leq |R_1(x)| + |\delta_1(x)| \leq 5.429 \times 10^{-6} + 5 \times 10^{-5} \approx 5.55 \times 10^{-5}.$$

关于舍入误差的论述可以推广到高次多项式插值, 只不过 $\max|\delta(x)|$ 的估计较为复杂, 这里 $\delta(x)$ 是插值的舍入误差.

2.3 差商与 Newton(牛顿)插值

Lagrange 插值多项式, 其形式具有对称性, 便于编程序计算, 但如果要增加一个节点, Lagrange 插值多项式除了要增加一项之外, 原来的每一项都需要改变. 能否构造另一种形式的插值多项式, 它们仍然是满足插值条件(2.1)的次数不高于 n 的代数多项式, 但当增加一个节点时, 只需在原来的插值多项式中增加一项, 而原有的项不需变动呢? 答案是可以的. 为此, 先建立差商的概念.

2.3.1 差商的定义及其性质

定义 2.1 称 $f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$ ($i \neq j$) 为函数 $f(x)$ 关于点 x_i, x_j 的一阶差商; 称 $f[x_i, x_j, x_k] = \frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$ 为函数 $f(x)$ 在 x_i, x_j, x_k 处的二阶差商, 一般地称

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_0, x_1, \dots, x_{k-2}, x_k] - f[x_0, x_1, \dots, x_{k-1}]}{x_k - x_{k-1}}$$

$$\text{或} \quad f[x_0, x_1, \dots, x_k] = \frac{f[x_0, x_1, \dots, x_{k-1}] - f[x_1, x_2, \dots, x_k]}{x_0 - x_k} \quad (2.19)$$

为函数 $f(x)$ 在点 x_i ($i=0, 1, 2, \dots, k$) 处的 k 阶差商.

注 (1) k 阶差商必须由 $k+1$ 个节点构成, k 个节点是不能构造出 k 阶差商的.

(2) 为统一起见, 补充定义函数值 $f(x_0)$ 为零阶差商.

性质 1 k 阶差商可表为函数值 $f(x_0), \dots, f(x_k)$ 的线性组合, 即

$$f[x_0, \dots, x_k] = \sum_{j=0}^k \frac{f(x_j)}{(x_j - x_0) \cdots (x_j - x_{j-1})(x_j - x_{j+1}) \cdots (x_j - x_k)} = \sum_{j=0}^k \frac{f(x_j)}{\omega'_{k+1}(x_j)}.$$

性质 2 差商具有对称性, 即

$$f[x_0, x_1, \dots, x_k] = f[x_1, x_0, x_2, \dots, x_k] = \cdots = f[x_1, \dots, x_k, x_0].$$

性质3 若 $f(x)$ 在 $[a, b]$ 上有 n 阶导数, 且节点 $x_i \in [a, b] (i=0, 1, 2, \dots, n)$, 则 n 阶差商与 n 阶导数有关系式

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}, \quad \xi \in (a, b). \quad (2.20)$$

性质4 如果 $f(x)$ 是 n 次多项式, 则其 k 阶差商 $f[x_0, x_1, \dots, x_{k-1}, x]$, 当 $k \leq n$ 时是一个 $(n-k)$ 次多项式, 而当 $k > n$ 时恒为 0.

差商计算可列差商表如表 2.3 所示.

表 2.3

x_i	$f(x_i)$	一阶差商	二阶差商	三阶差商
x_0	$f(x_0)$			
x_1	$f(x_1)$	$f[x_0, x_1]$		
x_2	$f(x_2)$	$f[x_1, x_2]$	$f[x_0, x_1, x_2]$	
x_3	$f(x_3)$	$f[x_2, x_3]$	$f[x_1, x_2, x_3]$	$f[x_0, x_1, x_2, x_3]$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

2.3.2 Newton 插值多项式

Newton 插值是通过选取特殊的基函数来实现的, 这时, 取

$$\begin{cases} \varphi_0(x) = 1, \\ \varphi_{i+1}(x) = (x - x_i)\varphi_i(x), \quad i = 0, 1, 2, \dots, n-1 \end{cases}$$

作为 Newton 插值的以 $x_0, x_1, \dots, x_n$ 为节点的基函数, 而次数不超过 n 的插值多项式 $N_n(x)$ 可表示为

$$\begin{aligned} N_n(x) = & c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) \\ & + \dots + c_n(x - x_0)(x - x_1)\dots(x - x_{n-1}). \end{aligned} \quad (2.21)$$

它也可以由下列递推公式得到

$$\begin{cases} N_0(x) \equiv c_0, \\ N_k(x) = N_{k-1}(x) + c_k(x - x_0)(x - x_1)\dots(x - x_{k-1}), \quad k = 1, 2, \dots, n. \end{cases} \quad (2.22)$$

其中, $c_0, c_1, \dots, c_n$ 是待定系数, 将由插值条件(2.1)确定.

根据插值条件(2.1), 可得

$$\begin{cases} y_0 = c_0, \\ y_1 = c_0 + c_1(x_1 - x_0), \\ y_2 = c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1), \\ \dots \\ y_n = c_0 + c_1(x_n - x_0) + c_2(x_n - x_0)(x_n - x_1) \\ \quad + \dots + c_n(x_n - x_0)(x_n - x_1)\dots(x_n - x_{n-1}). \end{cases}$$

利用递推公式(2.22),上面 $(n+1)$ 个等式可表示为

$$\begin{cases} y_0 = N_0(x) \equiv c_0, \\ y_k = N_k(x_k) = N_{k-1}(x_k) + c_k(x_k - x_0) \cdots (x_k - x_{k-1}), \quad k=1, 2, \dots, n. \end{cases} \quad (2.23)$$

由式(2.23)以及差商定义,可知

$$\begin{cases} c_0 = y_0 = f(x_0), \\ c_1 = \frac{y_1 - N_0(x_1)}{x_1 - x_0} = \frac{y_1 - y_0}{x_1 - x_0} = f[x_0, x_1], \\ c_2 = \frac{y_2 - N_1(x_2)}{(x_2 - x_0)(x_2 - x_1)} = \frac{y_2 - N_0(x_2) - c_1(x_2 - x_0)}{(x_2 - x_0)(x_2 - x_1)} \\ = \frac{y_2 - y_0 - f[x_0, x_1](x_2 - x_0)}{(x_2 - x_0)(x_2 - x_1)} = f[x_0, x_1, x_2]. \end{cases}$$

今设

$$c_{k-1} = f[x_0, x_1, \dots, x_{k-1}] \quad (2.24)$$

成立,则据式(2.23),有

$$\frac{y_{k-1} - N_{k-2}(x_{k-1})}{(x_{k-1} - x_0) \cdots (x_{k-1} - x_{k-2})} = f[x_0, x_1, \dots, x_{k-1}].$$

因而又有

$$\frac{y_k - N_{k-2}(x_k)}{(x_k - x_0) \cdots (x_k - x_{k-2})} = f[x_0, x_1, \dots, x_{k-2}, x_k], \quad (2.25)$$

于是,由式(2.23),式(2.24),式(2.25)以及差商定义,可得

$$\begin{aligned} c_k &= \frac{y_k - N_{k-1}(x_k)}{(x_k - x_0) \cdots (x_k - x_{k-1})} = \frac{y_k - N_{k-2}(x_k) - c_{k-1}(x_k - x_0) \cdots (x_k - x_{k-2})}{(x_k - x_0) \cdots (x_k - x_{k-1})} \\ &= \frac{f[x_0, x_1, \dots, x_{k-2}, x_k] - f[x_0, x_1, \dots, x_{k-1}]}{x_k - x_{k-1}} = f[x_0, x_1, \dots, x_k]. \end{aligned} \quad (2.26)$$

由归纳法原理,可知上式对于 $k=1, 2, \dots, n$ 都成立.把系数(2.26)以及 $c_0=f(x_0)$ 代入多项式(2.21)就得到下列的满足插值条件(2.1)的 n 次插值多项式

$$\begin{aligned} N_n(x) &= f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots \\ &\quad + f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \cdots (x - x_{n-1}). \end{aligned} \quad (2.27)$$

形如式(2.27)的插值多项式称为 Newton 插值多项式.

例 2.4 已知函数 $y=f(x)$ 的数据如表 2.4 所示.试作一个 3 次插值多项式 $P_3(x)$, 利用 $P_3(x)$ 计算 $\sqrt{3}$.

表 2.4

i	0	1	2	3
x_i	0	1	2	3
$y_i=f(x_i)$	1	3	9	27

解 利用 Newton 插值公式

$$N_3(x) = f(x_0) + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2),$$

为此先作差商表(见表 2.5), 故

表 2.5

k	x_k	$f(x_k)$	差 商		
			$f[x_k, x_{k+1}]$	$f[x_k, x_{k+1}, x_{k+2}]$	$f[x_k, x_{k+1}, x_{k+2}, x_{k+3}]$
0	0	1			
1	1	3	2		
2	2	9	6	2	
3	3	27	18	6	4/3

$$P_3(x) = N_3(x) = 1 + 2x + 2x(x-1) + \frac{4}{3}x(x-1)(x-2) \\ = 1 + 2x + 2x^2 - 2x + \frac{4}{3}x^3 - 4x^2 + \frac{8}{3}x \\ = \frac{4}{3}x^3 - 2x^2 + \frac{8}{3}x + 1.$$

由差商表知, $f(x) = 3^x$, 而 $\sqrt{3} = 3^{1/2}$, 故令 $x = \frac{1}{2}$, 即得

$$\sqrt{3} \approx P_3\left(\frac{1}{2}\right) = \frac{4}{3} \times \left(\frac{1}{2}\right)^3 - 2 \times \left(\frac{1}{2}\right)^2 + \frac{8}{3} \times \frac{1}{2} + 1 = 2.$$

2.4 差分与等距节点的 Newton 插值

用差商表示的 Newton 插值多项式, 其节点 $x_0, x_1, \dots, x_n$ 可以是不等距的. 当节点以等距分布时, 即

$$x_k = a + kh, \quad k = 0, 1, 2, \dots, n; h = \frac{b-a}{n}, \quad (2.28)$$

其中, h 称为步长, 其各阶差商可以改写为用差分表示, 从而插值多项式也可以大为简化. 为此, 首先介绍差分的概念及其性质. 记 $y_k = f(x_k)$.

2.4.1 差分的定义

定义 2.2 函数 $y = f(x)$ 在小区间 $[x_k, x_{k+1}]$ 上的改变量 $(y_{k+1} - y_k)$ 称为 $f(x)$ 在 x_k 上以 $h = x_{k+1} - x_k$ 为步长的一阶向前差分, 记为 $\Delta y_k = y_{k+1} - y_k$. 于是 $f(x)$ 在各等距节点 x_k 的一阶向前差分为

$$\begin{cases} \Delta y_0 = y_1 - y_0, \\ \Delta y_1 = y_2 - y_1, \dots, \\ \Delta y_k = y_{k+1} - y_k, \dots, \end{cases} \quad (2.29)$$

一阶差分的差分,称为二阶(向前)差分,记为

$$\Delta^2 y_k = \Delta y_{k+1} - \Delta y_k = y_{k+2} - 2y_{k+1} + y_k, \quad k = 0, 1, 2, \dots, n-2. \quad (2.30)$$

一般地,函数 $f(x)$ 在节点 x_k 的 $(m-1)$ 阶(向前)差分的差分,称为在该节点的 m 阶(向前)差分,记为

$$\Delta^m y_k = \Delta^{m-1} y_{k+1} - \Delta^{m-1} y_k. \quad (2.31)$$

类似地,还可以如下定义向后差分和中心差分.

$$\nabla y_k = y_k - y_{k-1},$$

称为 $y=f(x)$ 在点 x_k 的一阶向后差分;

$$\nabla^m y_k = \nabla^{m-1} y_k - \nabla^{m-1} y_{k-1}, \quad (2.32)$$

称为 $y=f(x)$ 在点 x_k 的 m 阶向后差分;

$$\delta y_k = f\left(x_k + \frac{h}{2}\right) - f\left(x_k - \frac{h}{2}\right),$$

称为 $y=f(x)$ 在点 x_k 的一阶中心差分;

$$\delta^m y_k = \delta^{m-1} y_{k+1/2} - \delta^{m-1} y_{k-1/2}, \quad (2.33)$$

称为 $y=f(x)$ 在点 x_k 的 m 阶中心差分.

2.4.2 差分的性质

为了演算简便,引入下面几个算符:

符号 Δ, ∇, δ 分别表示向前、向后和中心差分算子;

符号 I 称为不变算子,有 $If(x_i) = f(x_i)$;

符号 E 称为移位算子,有 $Ef(x_i) = f(x_i + h)$;

符号 D 称为微分算子,有 $Df(x_i) = f'(x_i)$.

规定不变算子和移位算子的幂次,其含义如下:

$$I^n = I \quad (n \text{ 为整数}),$$

$$E^a f(x_i) = f(x_i + ah) \quad (a \text{ 为实数}),$$

利用上述算符可以简明地推证差分的重要性质.

性质 1 各类算子之间具有下列关系:

$$\Delta = E - I,$$

$$\nabla = I - E^{-1} = E^{-1} \cdot \Delta, \quad (2.34)$$

$$\delta = E^{1/2} - E^{-1/2} = E^{1/2}(I - E^{-1}) = E^{1/2} \nabla.$$

例如,

$$\begin{aligned} \Delta f(x_i) &= f(x_i + h) - f(x_i) \\ &= Ef(x_i) - If(x_i) = (E - I)f(x_i), \end{aligned}$$

即得

$$\Delta = E - I.$$

类似可验证式(2.34)的其他关系成立.

性质2 如果 $f(x)$ 是 x 的 n 次多项式, 记 $\Delta f(x) = f(x+h) - f(x)$, 则其 k 阶差分 $\Delta^k f(x)$ ($0 \leq k \leq n$) 是 x 的 $(n-k)$ 次多项式, 高于 n 阶的差分均为零.

事实上, 由泰勒公式, 有

$$\begin{aligned}\Delta f(x) &= f(x+h) - f(x) \\ &= hf'(x) + \frac{h^2}{2}f''(x) + \cdots + \frac{h^n}{n!}f^{(n)}(x).\end{aligned}$$

上式右端是 $(n-1)$ 次多项式. 对任意 k , 根据上述做法运用归纳法即可证明.

性质3 差分与函数值可互相线性表出, 记 $f_k = f(x_k)$, 差分

$$\begin{aligned}\Delta f_k &= (E - I)f_k = f_{k+1} - f_k, \\ \Delta^2 f_k &= (E - I)^2 f_k = (E^2 - 2E + I)f_k = f_{k+2} - 2f_{k+1} + f_k, \\ &\dots \\ \Delta^m f_k &= (E - I)^m f_k \\ &= \sum_{j=0}^m (-1)^j \begin{bmatrix} m \\ j \end{bmatrix} E^{m-j} f_k = \sum_{j=0}^m (-1)^j \begin{bmatrix} m \\ j \end{bmatrix} f_{k+m-j}.\end{aligned}\quad (2.35)$$

另一方面

$$f_{k+m} = f(x_k + mh) = E^m f_k = (I + \Delta)^m f_k = \sum_{j=0}^m \begin{bmatrix} m \\ j \end{bmatrix} \Delta^j f_k. \quad (2.36)$$

利用性质1, 对于向后差分也可得出类似的结论.

性质4 当节点等距时, 差商与差分存在着关系

$$f[x_0, x_1, \dots, x_k] = \frac{\Delta^k y_0}{k! h^k}, \quad k = 1, 2, \dots. \quad (2.37)$$

证 利用归纳法证明.

当 $k=1$ 时, $f[x_0, x_1] = \frac{\Delta y_0}{h}$, 结论成立.

设 $k=m-1$ 时, 有

$$\begin{aligned}f[x_0, x_1, \dots, x_{m-1}] &= \frac{\Delta^{m-1} y_0}{(m-1)! h^{m-1}}, \\ f[x_1, x_2, \dots, x_m] &= \frac{\Delta^{m-1} y_1}{(m-1)! h^{m-1}},\end{aligned}$$

于是

$$\begin{aligned}f[x_0, x_1, \dots, x_m] &= \frac{f[x_1, x_2, \dots, x_m] - f[x_0, x_1, \dots, x_{m-1}]}{x_m - x_0} \\ &= \frac{\Delta^{m-1} y_1 - \Delta^{m-1} y_0}{(m-1)! h^{m-1} \cdot mh} = \frac{\Delta^m y_0}{m! h^m}.\end{aligned}$$

对 $m=2, 3, \dots$, 关系式(2.37)均成立, 故性质4得证.

一般地有

$$f[x_k, x_{k+1}, \dots, x_{k+m}] = \frac{1}{m!} \frac{1}{h^m} \Delta^m f_k. \quad (2.38)$$

同理, 对向后差分有

$$f[x_k, x_{k-1}, \dots, x_{k-m}] = \frac{1}{m!} \frac{1}{h^m} \nabla^m f_k. \quad (2.39)$$

利用式(2.38)及式(2.20)又可得到

$$\Delta^n f_k = h^n f^{(n)}(\xi), \quad \xi \in (x_k, x_{k+n}). \quad (2.40)$$

2.4.3 Newton 向前、向后插值公式

将 Newton 插值多项式中各阶差商用相应差分代替, 就可得到各种形式的等距节点插值公式, 这里只推导常用的前插与后插公式.

如果节点 $x_k = x_0 + kh (k=0, 1, 2, \dots, n)$, 要计算 x_0 附近点 x 的函数的值 $f(x)$, 可令 $x = x_0 + th, 0 \leq t \leq 1$, 于是

$$\varphi_{k+1}(x) = \prod_{j=0}^k (x - x_j) = t(t-1)\cdots(t-k)h^{k+1}.$$

将此式及 $f[x_k, \dots, x_{k+m}] = \frac{1}{m!} \frac{1}{h^m} \Delta^m f_k (m=1, 2, \dots, n)$ 代入 Newton 插值公式, 得

$$N_n(x_0 + th) = f_0 + t\Delta f_0 + \frac{t(t-1)}{2!} \Delta^2 f_0 + \cdots + \frac{t(t-1)\cdots(t-n+1)}{n!} \Delta^n f_0, \quad (2.41)$$

称为 Newton 前插公式, 其余项为

$$R_n(x) = f(x) - N_n(x_0 + th) = \frac{t(t-1)\cdots(t-n)}{(n+1)!} h^{n+1} f^{(n+1)}(\xi), \quad \xi \in (x_0, x_n). \quad (2.42)$$

如果要求表示 x_n 附近的函数值 $f(x)$, 此时应用 Newton 插值公式, 插值点应按 $x_n, x_{n-1}, \dots, x_0$ 的次序排列, 有

$$N_n(x) = f(x_n) + f[x_n, x_{n-1}](x - x_n) + f[x_n, x_{n-1}, x_{n-2}](x - x_n)(x - x_{n-1}) \\ + \cdots + f[x_n, x_{n-1}, \dots, x_0](x - x_n)\cdots(x - x_1).$$

作变换 $x = x_n + th (-1 \leq t \leq 0)$, 并将 $f[x_k, x_{k-1}, \dots, x_{k-m}] = \frac{1}{m!} \frac{1}{h^m} \nabla^m f_k$ 代入上式得

$$N_n(x_n + th) = f_n + t\nabla f_n + \frac{t(t+1)}{2!} \nabla^2 f_n + \cdots \\ + \frac{t(t+1)\cdots(t+n-1)}{n!} \nabla^n f_n, \quad (2.43)$$

称为 Newton 后插公式, 其余项为

$$R_n(x) = f(x) - N_n(x_n + th) = \frac{t(t+1)\cdots(t+n)}{(n+1)!} h^{n+1} f^{(n+1)}(\xi), \quad \xi \in (x_0, x_n). \quad (2.44)$$

使用 Newton 前插或后插公式时,先造如表 2.6 所示差分表.

表 2.6

x_k	$f(x_k)$	Δy_k	$\Delta^2 y_k$	$\Delta^3 y_k$	$\Delta^4 y_k$
x_0	y_0	Δy_0	$\Delta^2 y_0$	$\Delta^3 y_0$	$\dots$
x_1	y_1	Δy_1	$\Delta^2 y_1$		
x_2	y_2	Δy_2			
x_3	y_3				

例 2.5 已知 $f(x) = \sin x$ 数值表如表 2.7 所示,分别用向前、向后 Newton 插值公式求 $\sin(0.57891)$ 的近似值.

表 2.7

x	0.4	0.5	0.6	0.7
$\sin x$	0.38942	0.47943	0.56464	0.64422

解 作差分表如表 2.8 所示. 使用前插公式(2.41),取 $x_0 = 0.5, x_1 = 0.6, x_2 = 0.7, x = x_0 + th, h = 0.1, t = (x - x_0)/h = 0.7891$,于是

表 2.8

x	$\sin x$	$\Delta f(x)$	$\Delta^2 f(x)$	$\Delta^3 f(x)$
0.4	0.38942	0.09001	-0.00480	-0.00083
0.5	0.47943	0.08521	-0.00563	
0.6	0.56464	0.07958		
0.7	0.64422			

$$\begin{aligned}
 N_2(0.57891) &= f_0 + t\Delta f_0 + t(t-1)\frac{\Delta^2 f_0}{2} \\
 &= 0.47943 + 0.7891 \times 0.08521 + 0.7891 \\
 &\quad \times (0.7891 - 1) \times \frac{-0.00563}{2} \\
 &\approx 0.54714.
 \end{aligned}$$

由式(2.42),知截断误差

$$R_2(x) = \frac{0.1^3}{3!} 0.7891(0.7891-1)(0.7891-2)(-\cos\xi) \quad (0.5 < \xi < 0.7),$$

故 $|R_2(x)| \leq 3.36 \times 10^{-5} |\cos(0.5)| = 2.95 \times 10^{-5}.$

若用后插公式(2.43),则可取 $x_0 = 0.6, x_{-1} = 0.5, x_{-2} = 0.4, x = x_0 + th, t = -0.2109$,于是

$$N_2(0.57891) = f_0 + t\nabla f_0 + \frac{t(t+1)}{2} \nabla^2 f_0$$

$$\begin{aligned}
&= 0.56464 + (-0.2109) \times 0.08521 \\
&\quad + \frac{(-0.2109)(-0.2109+1)}{2} \times (-0.00480) \\
&\approx 0.54707,
\end{aligned}$$

由式(2.44)知,截断误差

$$R_2(x) = \frac{0.1^3}{3!} (-0.2109)(-0.2109+1)(-0.2109+2)(-\cos\xi) \quad (0.4 < \xi < 0.6),$$

故 $|R_2(x)| \leq 4.57 \times 10^{-5}.$

2.5 Hermite(埃尔米特)插值

在实际中遇到的一些插值问题,不但要求插值多项式 $H(x)$ 与被插值函数 $f(x)$ 在节点处的函数值相等,而且还要求其导数值相等,即满足

$$\begin{cases} H(x_i) = f(x_i) = y_i, \\ H'(x_i) = f'(x_i) = m_i, \end{cases} \quad i = 0, 1, 2, \dots, n.$$

这类插值称为带导数的插值,又称为 Hermite 插值.

2.5.1 低次 Hermite 插值多项式

1. 2 点 3 次 Hermite 插值多项式

设给定区间 $[x_0, x_1]$ 两端点处的函数值与导数值如表 2.9 所示. 求插值多项式 $H_3(x)$, 使之满足条件

表 2.9

x	x_0	x_1
$f(x)$	y_0	y_1
$f'(x)$	m_0	m_1

$$\begin{cases} H_3(x_0) = y_0, H_3(x_1) = y_1, \\ H'_3(x_0) = m_0, H'_3(x_1) = m_1. \end{cases} \quad (2.45)$$

由于给出了 4 个条件,故可以唯一确定出一个 3 次多项式,不妨设

$$H_3(x) = \alpha_0(x)y_0 + \alpha_1(x)y_1 + \beta_0(x)m_0 + \beta_1(x)m_1, \quad (2.46)$$

因此问题归结于构造 $\alpha_0(x)$ 与 $\beta_0(x)$, $\alpha_1(x)$ 与 $\beta_1(x)$.

首先, $\alpha_0(x)$, $\beta_0(x)$, $\alpha_1(x)$, $\beta_1(x)$ 应是 3 次多项式;

再者,由插值条件易知 $\alpha_i(x)$, $\beta_i(x)$ ($i=0,1$) 应满足条件

$$\begin{cases} \alpha_i(x_j) = \delta_{ij} = \begin{cases} 1, & i=j, \\ 0, & i \neq j, \end{cases} & \alpha'_i(x_j) = 0, \\ \beta_i(x_j) = 0, \beta'_i(x_j) = \delta_{ij} = \begin{cases} 1, & i=j, \\ 0, & i \neq j \end{cases} & (i, j = 0, 1). \end{cases} \quad (2.47)$$

为了构造 $\alpha_i(x), \beta_i(x)$, 取

$$l_i^2(x) = \left(\frac{x-x_j}{x_i-x_j} \right)^2 \quad (i, j = 0, 1; i \neq j),$$

上式显然是 2 次多项式, 由条件 (2.47) 知 x_j 是 $\alpha_i(x), \beta_i(x) (j \neq i)$ 的二重零点, x_i 是 $\beta_i(x)$ 的一重零点, 故可令

$$\alpha_i(x) = (a_i x + b_i) l_i^2(x),$$

$$\beta_i(x) = c_i (x - x_i) l_i^2(x),$$

其中, $a_i, b_i, c_i (i=0, 1)$ 是待定常数. 由条件 (2.47) 可确定这些常数. 例如, 由 $\alpha_0(x_0) = 1, \alpha'_0(x_0) = 0$, 知 a_0, b_0 满足

$$\begin{cases} a_0 x_0 + b_0 = 1, \\ a_0 + 2(a_0 x_0 + b_0) l'_0(x_0) = 0, \end{cases}$$

解得

$$\begin{cases} a_0 = \frac{-2}{x_0 - x_1}, \\ b_0 = 1 + \frac{2x_0}{x_0 - x_1}. \end{cases}$$

从而有

$$\alpha_0(x) = \left(1 - 2 \frac{x-x_0}{x_0-x_1} \right) \left(\frac{x-x_1}{x_0-x_1} \right)^2.$$

类似地, 可以求得

$$\alpha_1(x) = \left(1 - 2 \frac{x-x_1}{x_1-x_0} \right) \left(\frac{x-x_0}{x_1-x_0} \right)^2,$$

再由 $\beta'_0(x_0) = 1$, 可得出 $c_0 = 1$, 从而有

$$\beta_0(x) = (x - x_0) \left(\frac{x-x_1}{x_0-x_1} \right)^2.$$

类似可求得

$$\beta_1(x) = (x - x_1) \left(\frac{x-x_0}{x_1-x_0} \right)^2.$$

于是

$$\begin{aligned} H_3(x) &= \sum_{i=0}^1 \alpha_i(x) y_i + \sum_{i=0}^1 \beta_i(x) m_i \\ &= y_0 \left(1 - 2 \frac{x-x_0}{x_0-x_1} \right) l_0^2(x) + y_1 \left(1 - 2 \frac{x-x_1}{x_1-x_0} \right) l_1^2(x) \\ &\quad + m_0 (x - x_0) l_0^2(x) + m_1 (x - x_1) l_1^2(x). \end{aligned} \quad (2.48)$$

2.3 点3次带一个导数值的插值多项式

假设给定函数表如表 2.10 所示,要求 3 次多项式 $H_3(x)$,使之满足

表 2.10

x	x_0	x_1	x_2
$f(x)$	y_0	y_1	y_2
$f'(x)$		m_1	

$$\begin{cases} H_3(x_i) = y_i, \\ H'_3(x_1) = m_1 \end{cases} \quad (i = 0, 1, 2).$$

利用满足前 3 个条件的 Newton 插值多项式,设

$$H_3(x) = y_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + k(x - x_0)(x - x_1)(x - x_2), \quad (2.49)$$

其中, k 为待定常数. 显然 $H_3(x)$ 仍满足前 3 个插值条件, 下面利用第 4 个条件来确定常数 k , 于是有

$$\begin{aligned} H'_3(x_1) &= f[x_0, x_1] + f[x_0, x_1, x_2](x_1 - x_0) + k(x_1 - x_0)(x_1 - x_2) = m_1, \\ k &= \frac{m_1 - f[x_0, x_1] - f[x_0, x_1, x_2](x_1 - x_0)}{(x_1 - x_0)(x_1 - x_2)}, \end{aligned}$$

将其代入式(2.49), 即可得出 $H_3(x)$ 的表达式.

2.5.2 一般 Hermite 插值多项式

当函数 $f(x)$ 在区间 $[a, b]$ 上的 $(n+1)$ 个节点 x_i 处函数值 $f(x_i) = y_i$ 及导数值 $f'(x_i) = m_i (i=0, 1, 2, \dots, n)$ 给定时, 要求一个次数不超过 $2n+1$ 的多项式 $H_{2n+1}(x)$ 满足

$$\begin{cases} H_{2n+1}(x_i) = y_i, \\ H'_{2n+1}(x_i) = m_i, \end{cases} \quad i = 0, 1, 2, \dots, n. \quad (2.50)$$

仿照式(2.46), 可设

$$H_{2n+1}(x) = \sum_{i=0}^n \alpha_i(x) y_i + \sum_{i=0}^n \beta_i(x) m_i, \quad (2.51)$$

其中, $\alpha_i(x), \beta_i(x)$ 都是次数不超过 $2n+1$ 的多项式, 且满足

$$\begin{cases} \alpha_i(x_j) = \delta_{ij}, \alpha'_i(x_j) = 0, \\ \beta_i(x_j) = 0, \beta'_i(x_j) = \delta_{ij} \end{cases} \quad (i, j = 0, 1, 2, \dots, n). \quad (2.52)$$

先确定 $\alpha_i(x)$, 可取

$$\alpha_i(x) = (a_i x + b_i) l_i^2(x), \quad l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

其中, a_i, b_i 为待定常数. 由条件(2.52), 在 $x=x_i$ 处有

$$\begin{cases} a_i x_i + b_i = 1, \\ a_i + 2(a_i x_i + b_i) l'_i(x_i) = 0, \end{cases}$$

解之得

$$\begin{cases} a_i = -2l'_i(x_i) = -2 \sum_{\substack{k=0 \\ k \neq i}}^n \frac{1}{x_i - x_k}, \\ b_i = 1 + 2x_i l'_i(x_i) = 1 + 2x_i \sum_{\substack{k=0 \\ k \neq i}}^n \frac{1}{x_i - x_k}, \end{cases}$$

于是
$$\alpha_i(x) = \left[1 - 2(x - x_i) \sum_{\substack{k=0 \\ k \neq i}}^n \frac{1}{x_i - x_k} \right] l_i^2(x), \quad i = 0, 1, 2, \dots, n. \quad (2.53)$$

再确定 $\beta_i(x)$, 由于 x_i 是 $\beta_i(x)$ 的零点, $x_j (i \neq j)$ 是二重零点, 故可设

$$\beta_i(x) = k_i (x - x_i) l_i^2(x),$$

其中, k_i 为待定常数. 由 $\beta'_i(x_i) = 1$, 可得出 $k_i = 1$, 从而

$$\beta_i(x) = (x - x_i) l_i^2(x), \quad i = 0, 1, 2, \dots, n. \quad (2.54)$$

将 $\alpha_i(x), \beta_i(x)$ 代入式(2.51), 得

$$H_{2n+1}(x) = \sum_{i=0}^n \left[1 - 2(x - x_i) \sum_{\substack{k=0 \\ k \neq i}}^n \frac{1}{x_i - x_k} \right] l_i^2(x) y_i + \sum_{i=0}^n (x - x_i) l_i^2(x) m_i.$$

下面讨论 Hermite 插值多项式的唯一性. 假设满足式(2.50)的次数不高于 $(2n+1)$ 的另一多项式为 $P_{2n+1}(x)$, 则

$$\varphi(x) = H_{2n+1}(x) - P_{2n+1}(x)$$

是次数不高于 $(2n+1)$ 的多项式, 且节点 $x_i (i=0, 1, 2, \dots, n)$ 均为 $\varphi(x)$ 的二重零点, 即至少有 $(2n+2)$ 个零点(二重零点算两个零点), 从而必有 $\varphi(x) \equiv 0$, 即 $H_{2n+1}(x) = P_{2n+1}(x)$.

仿照 Lagrange 插值的讨论方法, 可以得出 Hermite 插值的余项表达式. 设 $f(x)$ 在 $[a, b]$ 上存在 $2n+2$ 阶导数, 则有

$$R_{2n+1}(x) = f(x) - H_{2n+1}(x) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \omega_{n+1}^2(x), \quad \xi \in (a, b). \quad (2.55)$$

2.5.3 利用重节点的差商表构造 Hermite 插值多项式

由差商与微商的关系式(2.20), 即

$$f[x_0, x_1, \dots, x_{n-1}, x] = \frac{f^{(n)}(\xi)}{n!}, \quad \xi \text{ 介于 } x_i (i=0, 1, 2, \dots, n) \text{ 之间.}$$

易知, 对任意的正整数 k , 有重节点差商公式

$$\underbrace{f[x, x, \dots, x]}_{k+1} = \lim_{x_0, x_1, \dots, x_{k-1} \rightarrow x} f[x_0, x_1, \dots, x_{k-1}, x] = \frac{f^{(k)}(x)}{k!}. \quad (2.56)$$

利用式(2.56),可以构造 Hermite 插值多项式,这类插值问题是,求一个最低次的多项式,使它与被插函数在某些或所有节点处不仅有相同的函数值,而且有相同的直到某阶的导数值.

例 2.6 求一个 4 次插值多项式 $H(x)$,使 $x=0$ 时, $H(0)=-1, H'(0)=-2$;而 $x=1$ 时, $H(1)=0, H'(1)=10, H''(1)=40$,并写出插值余项的表达式.

解 由于在 $x=0$ 处有一阶导数值的插值条件,所以它是“二重节点”;而在 $x=1$ 处有直到二阶导数值的插值条件,所以 $x=1$ 是“三重节点”.因此,利用重节点差商公式可以做出差商表,如表 2.11 所示.

表 2.11

x_i	f_i	一阶差商	二阶差商	三阶差商	四阶差商
0	-1				
0	-1	-2			
1	0	1	3		
1	0	10	9	6	
1	0	10	$40/2! = 20$	11	5

根据 Newton 插值公式,插值多项式为

$$H(x) = -1 - 2x + 3x^2 + 6x^2(x-1) + 5x^2(x-1)^2,$$

且插值余项为

$$R(x) = \frac{1}{5!} f^{(5)}(\xi) x^2 (x-1)^3, \quad 0 < \xi < 1,$$

其中, $f(x)$ 是被插函数.

2.6 分段插值法

2.6.1 高次插值的 Runge(龙格)现象

在区间 $[a, b]$ 上用插值多项式 $P_n(x)$ 近似函数 $f(x)$, 是否 $P_n(x)$ 的次数越高逼近 $f(x)$ 的精度就越好呢? 应该指出, 适当提高插值多项式次数, 可以提高逼近的精度, 但次数太高反而会产生不良的效果, 例如高到七八次以上的插值多项式很少被采用. 主要问题有: 次数越高, 计算工作量大, 积累误差也大; 在整个区间上做高次多项式, 当局部插值节点处的值有微小偏差时, 可能引起整个区间上函数值的很大变化, 使计算很不稳定. 另外, 还得注意的是, 对等距的插值节点, 当 $n \rightarrow \infty$ 时, $P_n(x)$ 不一定收敛于 $f(x)$. 最早由 Runge 给出了一个等距节点插值多项式 $L_n(x)$ 不收敛于 $f(x)$ 的例子: 函数 $f(x) = 1/(1+x^2)$, 它在 $[-5, 5]$ 上各阶导数均存在, 但在 $[-5, 5]$ 上取 $n+1$ 个等距节点 $x_i = -5 + 10 \times i/n (i=0, 1, 2, \dots, n)$ 所作的 Lagrange 插值多项式

$$L_n(x) = \sum_{i=0}^n \frac{1}{1+x_i^2} l_i(x),$$

当 $n \rightarrow \infty$ 时, 只在 $|x| \leq 3.63$ 内收敛, 而在这区间外是发散的. 图 2.3 给出了 $y = L_5(x)$ 、 $y = L_{10}(x)$ 与 $y = 1/(1+x^2)$ 的图形.

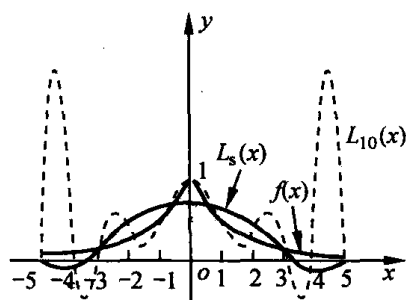


图 2.3

从图上看到在区间 $[-5, 5]$ 的两端点附近 $L_{10}(x)$ 与 $f(x)$ 偏离很远, 例如 $L_{10}(4.8) = 1.80438$, $f(4.8) = 0.04160$. 这种高次插值的不准确现象常称为 Runge 现象. 为了避免高次插值的上述缺点, 常常将插值区间分为若干个小区间, 在每个小区间上采用低次插值, 这就是分段插值的方法. 在区间 $[a, b]$ 上取 $n+1$ 个节点 $x_0 = a, x_1, \dots, x_n = b$, 对应的函数值 $f(x_0) = y_0, f(x_1) = y_1, \dots, f(x_n) = y_n$. 构造函数 $I_h(x)$ 满足

- (1) 插值条件: $I_h(x_k) = y_k \quad (k=0, 1, 2, \dots, n)$;
- (2) 在每个小区间 $[x_k, x_{k+1}] \quad (k=0, 1, 2, \dots, n-1)$ 上, $I_h(x)$ 是低次多项式;
- (3) 在整个区间 $[x_0, x_n]$ 上 $I_h(x)$ 连续.

$I_h(x)$ 称为分段低次插值函数, 下面讨论分段线性插值与分段 3 次插值函数.

2.6.2 分段线性插值

所谓分段线性插值, 就是通过插值点用折线段连接起来逼近 $f(x)$. 设已知节点 $a = x_0 < x_1 < \dots < x_n = b$ 上的函数值为 $f_0, f_1, \dots, f_n$, 记 $h_k = x_{k+1} - x_k$, $h = \max_k \{h_k\}$, 求一折线函数 $I_h(x)$ 满足

- (1) $I_h(x) \in C[a, b]$;
- (2) $I_h(x_k) = f_k \quad (k=0, 1, 2, \dots, n)$;
- (3) $I_h(x)$ 在每个小区间 $[x_k, x_{k+1}]$ 上是线性函数.

$I_h(x)$ 称为分段线性插值函数.

由定义可知 $I_h(x)$ 在每个小区间 $[x_k, x_{k+1}]$ 上可表示为

$$I_h(x) = \frac{x - x_{k+1}}{x_k - x_{k+1}} f_k + \frac{x - x_k}{x_{k+1} - x_k} f_{k+1} \quad (x_k \leq x \leq x_{k+1}).$$

若用插值基函数表示, 则在整个区间 $[a, b]$ 上为

$$I_h(x) = \sum_{j=0}^n f_j l_j(x), \quad (2.57)$$

其中, 基函数 $l_j(x)$ 满足条件 $l_j(x_k) = \delta_{jk} \quad (j, k=0, 1, 2, \dots, n)$, 其形式是

$$l_j(x) = \begin{cases} \frac{x - x_{j-1}}{x_j - x_{j-1}}, & x_{j-1} \leq x \leq x_j \quad (j=0 \text{ 略去}), \\ \frac{x - x_{j+1}}{x_j - x_{j+1}}, & x_j \leq x \leq x_{j+1} \quad (j=n \text{ 略去}), \\ 0, & x \in [a, b], \quad \text{但 } x \notin [x_{j-1}, x_{j+1}]. \end{cases} \quad (2.58)$$

分段线性插值基函数 $l_j(x)$ 只在 x_j 附近不为零, 在其他地方均为零, 这种性质称为局部非零性质. 如图 2.4 所示, 当 $x \in [x_k, x_{k+1}]$ 时

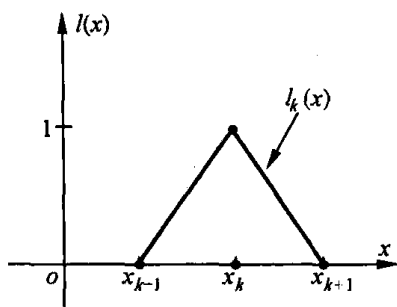


图 2.4

故

$$1 = \sum_{j=0}^n l_j(x) = l_k(x) + l_{k+1}(x),$$

$$f(x) = [l_k(x) + l_{k+1}(x)]f(x).$$

另一方面, 这时

$$I_h(x) = f_k l_k(x) + f_{k+1} l_{k+1}(x).$$

现在证明 $\lim_{h \rightarrow 0} I_h(x) = f(x)$. 考虑

$$\begin{aligned} |f(x) - I_h(x)| &\leq l_k(x) |f(x) - f_k| + l_{k+1}(x) |f(x) - f_{k+1}| \\ &\leq [l_k(x) + l_{k+1}(x)] \omega(h_k) = \omega(h_k) \leq \omega(h). \end{aligned}$$

这里 $\omega(h)$ 是函数 $f(x)$ 在区间 $[a, b]$ 上的连续模, 即对任意两点 $x', x'' \in [a, b]$, 只要 $|x' - x''| \leq h$, 就有

$$|f(x') - f(x'')| \leq \omega(h),$$

称 $\omega(h)$ 为 $f(x)$ 在 $[a, b]$ 上的连续模. 当 $f(x) \in C[a, b]$ 时, 就有

$$\lim_{h \rightarrow 0} \omega(h) = 0.$$

由前式可知, 当 $x \in [a, b]$ 时, 就有

$$\max_{a \leq x \leq b} |f(x) - I_h(x)| \leq \omega(h),$$

因此, 只要 $f(x) \in C[a, b]$, 就有

$$\lim_{h \rightarrow 0} I_h(x) = f(x)$$

在 $[a, b]$ 上一致成立, 故 $I_h(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$.

2.6.3 分段 3 次 Hermite 插值

分段线性插值函数 $I_h(x)$ 的导数是间断的, 若在节点 $x_k (k=0, 1, \dots, n)$ 上除已知函数值 f_k 外还给出导数值 $f'_k = m_k (k=0, 1, \dots, n)$, 这样就可构造一个导数连续的分段插值函数 $I_h(x)$, 它满足条件

- (1) $I_h(x) \in C^1[a, b]$ ($C^1[a, b]$ 代表区间 $[a, b]$ 上一阶导数连续的函数集合);
- (2) $I_h(x_k) = f_k, I'_h(x_k) = f'_k \quad (k=0, 1, \dots, n)$;
- (3) $I_h(x)$ 在每个小区间 $[x_k, x_{k+1}]$ 上是 3 次多项式.

$I_h(x)$ 称为分段 3 次 Hermite 插值.

根据 2 点 3 次插值多项式(2.48), 可知 $I_h(x)$ 在区间 $[x_k, x_{k+1}]$ 上的表达式为

$$\begin{aligned} I_h(x) &= \left(\frac{x - x_{k+1}}{x_k - x_{k+1}} \right)^2 \left(1 + 2 \frac{x - x_k}{x_{k+1} - x_k} \right) f_k + \left(\frac{x - x_k}{x_{k+1} - x_k} \right)^2 \left(1 + 2 \frac{x - x_{k+1}}{x_k - x_{k+1}} \right) f_{k+1} \\ &\quad + \left(\frac{x - x_{k+1}}{x_k - x_{k+1}} \right)^2 (x - x_k) f'_k + \left(\frac{x - x_k}{x_{k+1} - x_k} \right)^2 (x - x_{k+1}) f'_{k+1}. \end{aligned}$$

若在整个区间 $[a, b]$ 上定义一组分段3次插值基函数 $\alpha_j(x)$ 及 $\beta_j(x)$ ($j=0, 1, 2, \dots, n$), 则 $I_h(x)$ 可表示为

$$I_h(x) = \sum_{j=0}^n [f_j \alpha_j(x) + f'_j \beta_j(x)], \quad (2.59)$$

其中 $\alpha_j(x)$ 、 $\beta_j(x)$ 分别表示为

$$\alpha_j(x) = \begin{cases} \left(\frac{x-x_{j-1}}{x_j-x_{j-1}} \right)^2 \left(1 + 2 \frac{x-x_j}{x_{j-1}-x_j} \right), & x_{j-1} \leq x \leq x_j \quad (j=0 \text{ 略去}), \\ \left(\frac{x-x_{j+1}}{x_j-x_{j+1}} \right)^2 \left(1 + 2 \frac{x-x_j}{x_{j+1}-x_j} \right), & x_j \leq x \leq x_{j+1} \quad (j=n \text{ 略去}), \\ 0, & \text{其他;} \end{cases} \quad (2.60)$$

$$\beta_j(x) = \begin{cases} \left(\frac{x-x_{j-1}}{x_j-x_{j-1}} \right)^2 (x-x_j), & x_{j-1} \leq x \leq x_j \quad (j=0 \text{ 略去}), \\ \left(\frac{x-x_{j+1}}{x_j-x_{j+1}} \right)^2 (x-x_j), & x_j \leq x \leq x_{j+1} \quad (j=n \text{ 略去}), \\ 0, & \text{其他.} \end{cases} \quad (2.61)$$

由于 $\alpha_j(x)$ 、 $\beta_j(x)$ 的局部非零性质, 当 $x \in [x_k, x_{k+1}]$ 时, 只有 $\alpha_k(x)$ 、 $\alpha_{k+1}(x)$ 、 $\beta_k(x)$ 、 $\beta_{k+1}(x)$ 不为零, 于是式(2.59)中 $I_h(x)$ 可表示为

$$I_h(x) = f_k \alpha_k(x) + f_{k+1} \alpha_{k+1}(x) + f'_k \beta_k(x) + f'_{k+1} \beta_{k+1}(x) \quad (x_k \leq x \leq x_{k+1}). \quad (2.62)$$

为了研究 $I_h(x)$ 的收敛性, 由式(2.60)及式(2.61)直接得估计式

$$0 \leq \alpha_j(x) \leq 1, \quad (2.63)$$

$$|\beta_k(x)| \leq \frac{4}{27} h_k, \quad |\beta_{k+1}(x)| \leq \frac{4}{27} h_k. \quad (2.64)$$

此外, 当 $f(x)$ 是分段3次多项式时, $f(x)$ 的插值多项式 $I_h(x)$ 就是它本身. 例如, 当 $f(x)=1$ 时就有

$$\sum_{j=0}^n \alpha_j(x) = 1;$$

当 $x \in [x_k, x_{k+1}]$ 时, 就得

$$\alpha_k(x) + \alpha_{k+1}(x) = 1. \quad (2.65)$$

由式(2.62)~式(2.65), 当 $x \in [x_k, x_{k+1}]$ 时还可得

$$\begin{aligned} |f(x) - I_h(x)| &\leq \alpha_k(x) |f(x) - f_k| + \alpha_{k+1}(x) |f(x) - f_{k+1}| \\ &\quad + \frac{4}{27} h_k [|f'_k| + |f'_{k+1}|] \\ &\leq [\alpha_k(x) + \alpha_{k+1}(x)] \omega(h) + \frac{8}{27} h \max\{|f'_k|, |f'_{k+1}|\} \\ &\leq \omega(h) + \frac{8}{27} h \max_{0 \leq k \leq n} |f'_k| \end{aligned}$$

对 $x \in [a, b]$ 成立, 其中 $\omega(h)$ 是 $f(x)$ 在 $[a, b]$ 上的连续模. 因此, 当 $f(x) \in C^1[a, b]$ 时,

$$\lim_{h \rightarrow 0} I_h(x) = f(x).$$

这就证明了 $I_h(x)$ 在 $[a, b]$ 上一致收敛到 $f(x)$.

2.7 3 次样条(spline)插值

分段低次插值虽具有很多优点, 但是, 这种插值方法只能保证各段曲线在连接点上的连续, 而不能保证整条曲线在这些点上的充分光滑性, 这就不能满足某些工程技术上的要求. 例如, 在船体、飞机等外形曲线的设计中, 不仅要求曲线连续, 而且要求曲线的曲率也连续, 这就要求分段插值函数具有连续的二阶导数, 在这一节中介绍的 3 次样条插值函数就能满足上述要求.

样条本来是绘图员们用来画光滑曲线的一种细木条(或细金属条), 在画曲线时要求曲线过一些已知点, 且使木条强制性很小, 3 次样条就是从这个模型中抽象出来的.

2.7.1 3 次样条插值函数的概念

给定函数表, 如表 2.12 所示, 其中, $a = x_0 < x_1 < \cdots < x_n = b$, 要求构造一个函数 $s(x)$, 使其满足下面三个条件:

- (1) $s(x), s'(x), s''(x)$ 在 $[a, b]$ 上连续;
- (2) $s(x)$ 在每个子区间 $[x_{i-1}, x_i] (i=1, 2, \cdots, n)$ 上是不高于 3 次的多项式;
- (3) $s(x_i) = y_i (i=0, 1, 2, \cdots, n)$.

表 2.12

x	x_0	x_1	$\cdots$	x_n
$y=f(x)$	y_0	y_1	$\cdots$	y_n

满足条件(1)和(2)的函数称为节点 $x_0, x_1, \cdots, x_n$ 上的 3 次样条函数; 一个满足插值条件(3)的 3 次样条函数, 称为 3 次样条插值函数, 即

定义 2.3 若函数 $s(x) \in C^2[a, b]$, 且在每个小区间 $[x_{i-1}, x_i]$ 上是 3 次多项式, 其中 $a = x_0 < x_1 < \cdots < x_n = b$ 是给定节点, 则称 $s(x)$ 是节点 $x_0, x_1, \cdots, x_n$ 上的 3 次样条函数. 若在节点 x_i 上给定函数值 $y_i = f(x_i) (i=0, 1, 2, \cdots, n)$, 并成立 $s(x_i) = y_i (i=0, 1, 2, \cdots, n)$, 则称 $s(x)$ 为 3 次样条插值函数.

在每个子区间 $[x_{i-1}, x_i] (i=1, 2, \cdots, n)$ 上, $s(x)$ 是一个 3 次多项式, 可记为

$$s(x) = a + bx + cx^2 + dx^3.$$

这里有 4 个待定系数 a, b, c 和 d , 因此, 要确定整个 3 次样条插值函数 $s(x)$, 必须确定 $4n$ 个系数.

条件(1)表明 $s(x), s'(x), s''(x)$ 在节点 $x_1, x_2, \cdots, x_{n-1}$ 上连续, 于是它们在这些点

上的左、右极限相等,这样的等式共有 $3(n-1)$ 个,从而提供了包含未知数 a, b, c 和 d 的 $(3n-3)$ 个条件,条件(3)则提供了 $(n+1)$ 个条件,这样,可以得到 $(4n-2)$ 个条件.

显然,要唯一确定 $s(x)$ 的 $4n$ 个未知数还缺少两个条件,这两个条件通常在区间 $[a, b]$ 的两个端点处给出,我们把它叫做边界(或端点)条件.

边界条件应根据实际问题的要求来确定,其类型很多,常见的边界条件类型有如下一些.

(1) 第一种边界条件:给定端点处的一阶导数,即

$$s'(x_0) = y'_0, \quad s'(x_n) = y'_n.$$

(2) 第二种边界条件:给定端点处的二阶导数,即

$$s''(x_0) = y''_0, \quad s''(x_n) = y''_n.$$

作为特例, $s''(x_0) = s''(x_n) = 0$ 称为自然边界条件,满足自然边界条件的样条函数叫自然样条函数.

(3) 第三种边界条件:当 $y = f(x)$ 是以 $(b-a)$ 为周期的周期函数时,则要求 $s(x)$ 也是周期函数,这时的边界条件可以写成

$$\begin{cases} s(x_0 + 0) = s(x_n - 0), \\ s'(x_0 + 0) = s'(x_n - 0), \\ s''(x_0 + 0) = s''(x_n - 0). \end{cases}$$

注 由于 $y = f(x)$ 以 $(b-a)$ 为周期,故 $f(a) = f(b)$, 即 $y_0 = y_n$, 从而必有 $s(x_0 + 0) = s(x_n - 0)$, 因此,在第三种边界条件中,真正起作用的是后面的两个等式.

2.7.2 3次样条函数的构造

从原则上讲,可以利用 $s(x), s'(x), s''(x)$ 在点 $x_i (i=1, 2, \dots, n-1)$ 上的连续性、插值条件(3)以及边界条件,列出包含 $4n$ 个未知数 a_i, b_i, c_i 和 $d_i (i=1, 2, \dots, n)$ 的方程组,从而解出 $s(x)$ 在各子区间上的表达式 $s_i(x)$ 的各项系数,但是,这种做法的工作量相当大,以致在实际计算中很少用到,下面介绍两种计算工作量小得多且行之有效的方法.

1. 利用节点处的二阶导数来表示3次样条插值函数

实际上,3次样条插值函数的表达方法是多样的.还可以先利用一阶导数 $s'(x)$ 在节点 $x_i (i=1, 2, \dots, n-1)$ 上的连续性以及边界条件,列出确定二阶导数 $M_i = s''(x_i) (i=0, 1, 2, \dots, n)$ 的线性方程组(在力学上称为三弯矩方程组),并由此解出 M_i ; 然后用 M_i 来表达 $s(x)$. 在有些情况下,这种表达方法使用起来较方便,下面简要介绍这种方法的基本思路和公式.

首先指出,只要 $M_i = s''(x_i)$ 已知,就容易写出 $s(x)$ 的表达式了,事实上,在区间 $[x_{i-1}, x_i]$ 上, $s''(x)$ 是一次多项式. 设

$$s''(x_{i-1}) = M_{i-1}, \quad s''(x_i) = M_i,$$

则由线性插值

$$s''(x) = M_{i-1} \frac{x - x_i}{x_{i-1} - x_i} + M_i \frac{x - x_{i-1}}{x_i - x_{i-1}}, \quad x \in [x_{i-1}, x_i],$$

对上式连续积分两次,并利用 $s(x_{i-1}) = y_{i-1}$ 与 $s(x_i) = y_i$ 确定积分过程中出现的两个积分常数,即可得用 $M_i, i=1, 2, \dots, n$ 来表达的 $s(x)$ 公式

$$\begin{cases} s(x) = M_{i-1} \frac{(x_i - x)^3}{6h_i} + M_i \frac{(x - x_{i-1})^3}{6h_i} + \left(y_{i-1} - \frac{M_{i-1}}{6}h_i^2\right) \frac{x_i - x}{h_i} \\ \quad + \left(y_i - \frac{M_i}{6}h_i^2\right) \frac{x - x_{i-1}}{h_i}, \\ x \in [x_{i-1}, x_i], h_i = x_i - x_{i-1}, \quad i=1, 2, \dots, n. \end{cases} \quad (2.66)$$

利用 $s'(x_i)$ 在 $x_i (i=1, 2, \dots, n-1)$ 上的连续性,可得线性方程组

$$\mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = g_i, \quad i = 1, 2, \dots, n-1, \quad (2.67)$$

其中,

$$\begin{aligned} \mu_i &= \frac{h_i}{h_i + h_{i+1}}, \quad \lambda_i = 1 - \mu_i = \frac{h_{i+1}}{h_i + h_{i+1}}, \\ g_i &= \frac{6}{h_i + h_{i+1}} \left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \right) \\ &= \frac{6}{h_i + h_{i+1}} (f[x_i, x_{i+1}] - f[x_{i-1}, x_i]) \\ &= 6f[x_{i-1}, x_i, x_{i+1}]. \end{aligned}$$

方程组(2.67)中含有 $(n+1)$ 个未知数 $M_0, M_1, \dots, M_n$, 而方程只有 $(n-1)$ 个, 要完全确定 $M_i (i=0, 1, 2, \dots, n)$ 的值, 还必须用到两个边界条件.

(1) 从第一种边界条件 $s'(x_0) = y'_0$ 与 $s'(x_n) = y'_n$ 出发, 可以导出以下两个方程:

$$2M_0 + M_1 = \frac{6}{h_1} \left(\frac{y_1 - y_0}{h_1} - y'_0 \right), \quad (2.68)$$

$$M_{n-1} + 2M_n = \frac{6}{h_n} \left(y'_n - \frac{y_n - y_{n-1}}{h_n} \right). \quad (2.69)$$

将方程(2.67), 方程(2.68)与方程(2.69)合在一起, 即得确定 $M_0, M_1, \dots, M_n$ 的线性方程组为

$$\begin{bmatrix} 2 & 1 & & \\ \mu_1 & 2 & \lambda_1 & \\ \ddots & \ddots & \ddots & \\ & \mu_{n-1} & 2 & \lambda_{n-1} \\ & & 1 & 2 \end{bmatrix} \begin{bmatrix} M_0 \\ M_1 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} g_0 \\ g_1 \\ \vdots \\ g_{n-1} \\ g_n \end{bmatrix}, \quad (2.70)$$

其中,

$$g_0 = \frac{6}{h_1} \left(\frac{y_1 - y_0}{h_1} - y'_0 \right) = \frac{6}{h_1} (f[x_0, x_1] - y'_0),$$

$$g_n = \frac{6}{h_{n-1}} \left(y'_n - \frac{y_n - y_{n-1}}{h_n} \right) = \frac{6}{h_{n-1}} (y'_n - f[x_{n-1}, x_n]).$$

(2) 在第二种边界条件下, 由于在区间端点处二阶导数 $M_0 = y''_0$ 与 $M_n = y''_n$ 已知, 所以方程组(2.66)中实际上只包含有 $(n-1)$ 个未知数 $M_1, M_2, \dots, M_{n-1}$, 并且可表示成

$$\begin{bmatrix} 2 & \lambda_1 & & \\ \mu_2 & 2 & \lambda_2 & \\ & \ddots & \ddots & \ddots \\ & & \mu_{n-2} & 2 & \lambda_{n-2} \\ & & & \mu_{n-1} & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} g_1 - \mu_1 y''_0 \\ g_2 \\ \vdots \\ g_{n-2} \\ g_{n-1} - \lambda_{n-1} y''_n \end{bmatrix}. \quad (2.71)$$

(3) 从第三种边界条件 $s''(x_0+0) = s''(x_n-0)$ 与 $s'(x_0+0) = s'(x_n-0)$ 出发, 可以得到方程

$$M_0 = M_n, \quad (2.72)$$

$$\lambda_n M_1 + \mu_n M_{n-1} + 2M_n = g_n, \quad (2.73)$$

其中, $\lambda_n = \frac{h_1}{h_1 + h_n}, \quad \mu_n = 1 - \lambda_n = \frac{h_n}{h_1 + h_n},$

$$g_n = \frac{6}{h_1 + h_n} \left(\frac{y_1 - y_0}{h_1} - \frac{y_n - y_{n-1}}{h_n} \right) = \frac{6}{h_1 + h_n} (f[x_0, x_1] - f[x_{n-1}, x_n]).$$

将方程(2.67), 方程(2.72)和方程(2.73)合在一起, 即得确定 $M_1, M_2, \dots, M_n$ 的线性方程组

$$\begin{bmatrix} 2 & \lambda_1 & \cdots & \mu_1 \\ \mu_2 & 2 & \lambda_2 & \\ & \ddots & \ddots & \ddots \\ & & \mu_{n-1} & 2 & \lambda_{n-1} \\ \lambda_n & \cdots & \mu_n & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_{n-1} \\ g_n \end{bmatrix}. \quad (2.74)$$

因此, 只要根据给定数据和边界条件算出 λ_i, μ_i 和 g_i , 就能写出方程(2.70)、方程(2.71)或方程(2.74), 并由此解出 $s''(x)$ 在 x_i 上的值 M_i ; 将 M_i 代入方程(2.66), 就得到了 $s(x)$ 在各子区间上的表达式.

例 2.7 已知数据表如表 2.13 所示. 求满足自然边界条件 $s''(1) = s''(5) = 0$ 的 3 次样条插值函数 $s(x)$, 计算 $f(3)$ 的近似值.

表 2.13

x_i	1	2	4	5
f_i	1	3	4	2

解 作差商表如表 2.14 所示. 由自然边界条件得 $M_0 = M_3 = 0$, 故式(2.71)给出关于 M_1, M_2 的线性方程组

$$\begin{bmatrix} 2 & \lambda_1 \\ \mu_2 & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \end{bmatrix} = \begin{bmatrix} -3 \\ -4.999998 \end{bmatrix},$$

其中,

$$\lambda_1 = \frac{h_2}{h_1 + h_2} = \frac{2}{1+2} = 0.666667,$$

$$\mu_2 = \frac{h_2}{h_2 + h_3} = \frac{2}{2+1} = 0.666667.$$

表 2.14

i	x_i	f_i	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$
0	1	1		
1	2	3	2	
2	4	4	0.5	-0.5
3	5	2	-2	-0.833333

解此方程组,得

$$M_1 = -0.750000, \quad M_2 = -2.249999.$$

由于 $3 \in [2, 4]$, 所以根据式(2.66), 有

$$\begin{aligned} s(x) &= M_1 \frac{(x_2 - x)^3}{6h_2} + M_2 \frac{(x - x_1)^3}{6h_2} + \left(f_1 - \frac{M_1 h_2^2}{6}\right) \frac{x_2 - x}{h_2} \\ &\quad + \left(f_2 - \frac{M_2 h_2^2}{6}\right) \frac{x - x_1}{h_2} \\ &= -0.750000 \times \frac{(4-x)^3}{12} - 2.249999 \times \frac{(x-2)^3}{12} + 3.500000 \times \frac{4-x}{2} \\ &\quad + 5.499999 \times \frac{x-2}{2}. \end{aligned}$$

在上式中, 令 $x=3$, 得

$$f(3) \approx s(3) \approx 4.25000.$$

2. 利用节点处的一阶导数来表示 3 次样条插值函数

这种方法的基本思路和特点: 先利用二阶导数 $s''(x)$ 在节点 $x_i (i=1, 2, \dots, n-1)$ 上的连续性以及边界条件列出确定一阶导数 $m_i = s'(x_i) (i=0, 1, 2, \dots, n)$ 的线性方程组, 并由此解出 m_i , 然后通过 m_i 来表达 $s(x)$.

首先指出, 只要能求出 $s'(x)$ 在各个节点上的值 $m_i = s'(x_i)$, 那么就容易求 $s(x)$ 了. 实际上, 当已知 m_i 后, $s(x)$ 在子区间 $[x_{i-1}, x_i] (i=1, 2, \dots, n)$ 上就是一个满足条件

$$\begin{cases} s(x_{i-1}) = y_{i-1}, & s(x_i) = y_i, \\ s'(x_{i-1}) = m_{i-1}, & s'(x_i) = m_i \end{cases}$$

的 3 次 Hermite 插值多项式, 利用 Hermite 插值公式(2.48), 可得

$$s(x) = y_{i-1} \left(1 + 2 \frac{x - x_{i-1}}{x_i - x_{i-1}} \right) \left(\frac{x - x_i}{x_{i-1} - x_i} \right)^2 + y_i \left(1 + 2 \frac{x - x_i}{x_{i-1} - x_i} \right) \left(\frac{x - x_{i-1}}{x_i - x_{i-1}} \right)^2 \\ + m_{i-1} (x - x_{i-1}) \left(\frac{x - x_i}{x_{i-1} - x_i} \right)^2 + m_i (x - x_i) \left(\frac{x - x_{i-1}}{x_i - x_{i-1}} \right)^2.$$

或者写成

$$s(x) = \frac{(x - x_i)^2 [h_i + 2(x - x_{i-1})]}{h_i^3} y_{i-1} \\ + \frac{(x - x_{i-1})^2 [h_i + 2(x_i - x)]}{h_i^3} y_i \\ + \frac{(x - x_i)^2 (x - x_{i-1})}{h_i^2} m_{i-1} + \frac{(x - x_{i-1})^2 (x - x_i)}{h_i^2} m_i, \quad (2.75)$$

其中, $x \in [x_{i-1}, x_i]$, $h_i = x_i - x_{i-1}$ ($i = 1, 2, \dots, n$).

这样, 求 $4n$ 个未知数 a, b, c 和 d 的问题, 就转化为求 $(n+1)$ 个未知数 m_i ($i = 0, 1, 2, \dots, n$) 的问题了.

为了求得 m_i , 要用到 $s(x)$ 的二阶导数在 x_i 上连续的条件.

由式(2.75)可得 $s(x)$ 在 $[x_{i-1}, x_i]$ 上的二阶导数

$$s''(x) = \frac{6x - 2x_{i-1} - 4x_i}{h_i^2} m_{i-1} + \frac{6x - 4x_{i-1} - 2x_i}{h_i^2} m_i \\ + \frac{6(x_{i-1} + x_i - 2x)}{h_i^3} (y_i - y_{i-1}). \quad (2.76)$$

于是

$$s''(x_i - 0) = \frac{2}{h_i} m_{i-1} + \frac{4}{h_i} m_i - \frac{6}{h_i^2} (y_i - y_{i-1}).$$

同理可得, $s(x)$ 在 $[x_i, x_{i+1}]$ 上的二阶导数

$$s''(x) = \frac{6x - 2x_i - 4x_{i+1}}{h_{i+1}^2} m_i + \frac{6x - 4x_i - 2x_{i+1}}{h_{i+1}^2} m_{i+1} \\ + \frac{6(x_i + x_{i+1} - 2x)}{h_{i+1}^3} (y_{i+1} - y_i).$$

于是

$$s''(x_i + 0) = -\frac{4}{h_{i+1}} m_i - \frac{2}{h_{i+1}} m_{i+1} + \frac{6}{h_{i+1}^2} (y_{i+1} - y_i).$$

由 $s''(x)$ 在 x_i 上连续的条件

$$s''(x_i + 0) = s''(x_i - 0),$$

可得 $\frac{1}{h_i} m_{i-1} + 2 \left(\frac{1}{h_i} + \frac{1}{h_{i+1}} \right) m_i + \frac{1}{h_{i+1}} m_{i+1} = 3 \left(\frac{y_{i+1} - y_i}{h_{i+1}^2} + \frac{y_i - y_{i-1}}{h_i^2} \right).$

用 $\left(\frac{1}{h_i} + \frac{1}{h_{i+1}} \right)$ 除等式两边, 即得方程

$$\frac{h_{i+1}}{h_i + h_{i+1}} m_{i-1} + 2m_i + \frac{h_i}{h_i + h_{i+1}} m_{i+1}$$

$$= 3\left(\frac{h_i}{h_i + h_{i+1}} \frac{y_{i+1} - y_i}{h_{i+1}} + \frac{h_{i+1}}{h_i + h_{i+1}} \frac{y_i - y_{i-1}}{h_i}\right).$$

若记

$$\lambda_i = \frac{h_{i+1}}{h_i + h_{i+1}}, \quad \mu_i = 1 - \lambda_i = \frac{h_i}{h_i + h_{i+1}},$$

$$\begin{aligned} g_i &= 3\left(\mu_i \frac{y_{i+1} - y_i}{h_{i+1}} + \lambda_i \frac{y_i - y_{i-1}}{h_i}\right) \\ &= 3(\mu_i f[x_i, x_{i+1}] + \lambda_i f[x_{i-1}, x_i]), \quad i = 1, 2, \dots, n-1, \end{aligned}$$

则所得方程组可以写成

$$\lambda_i m_{i-1} + 2m_i + \mu_i m_{i+1} = g_i, \quad i = 1, 2, \dots, n-1. \quad (2.77)$$

这是一个含有 $(n+1)$ 个未知数($s(x)$ 在节点上的导数 $m_0, m_1, \dots, m_n$)的 $(n-1)$ 阶线性方程组(在力学上称为三转角方程组),要完全确定 $m_0, m_1, \dots, m_n$ 的值,还需用到两个边界条件.

(1) 在第一种边界条件下,由于 $s(x)$ 在区间端点处的一阶导数 $m_0 = y'_0$ 与 $m_n = y'_n$ 已知,所以在方程组(2.77)中实际上只包含 $(n-1)$ 个未知数 $m_1, m_2, \dots, m_{n-1}$,并且可改写成

$$\begin{bmatrix} 2 & \mu_1 & & \\ \lambda_2 & 2 & \mu_2 & \\ & \ddots & \ddots & \ddots \\ & & \lambda_{n-2} & 2 & \mu_{n-2} \\ & & & \lambda_{n-1} & 2 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ \vdots \\ m_{n-2} \\ m_{n-1} \end{bmatrix} = \begin{bmatrix} g_1 - \lambda_1 y'_0 \\ g_2 \\ \vdots \\ g_{n-2} \\ g_{n-1} - \mu_{n-1} y'_n \end{bmatrix}. \quad (2.78)$$

(2) 在第二种边界条件下,由于 $s''(x_0) = y''_0$ 与 $s''(x_n) = y''_n$ 已知,可以得到两个包含 $m_i (i=0, 1, 2, \dots, n)$ 的线性方程.事实上,由式(2.76)可知, $s''(x)$ 上的表达式为

$$\begin{aligned} s''(x) &= \frac{6x - 2x_0 - 4x_1}{h_1^2} m_0 + \frac{6x - 4x_0 - 2x_1}{h_1^2} m_1 \\ &\quad + \frac{6(x_0 + x_1 - 2x)}{h_1^3} (y_1 - y_0), \end{aligned}$$

于是,由条件 $s''(x_0) = y''_0$,立即可得

$$y''_0 = -\frac{4}{h_1} m_0 - \frac{2}{h_1} m_1 + \frac{6}{h_1^2} (y_1 - y_0),$$

即

$$2m_0 + m_1 = 3 \frac{y_1 - y_0}{h_1} - \frac{h_1}{2} y''_0. \quad (2.79)$$

同理,由条件 $s''(x_n) = y''_n$,可得

$$m_{n-1} + 2m_n = 3 \frac{y_n - y_{n-1}}{h_n} + \frac{h_n}{2} y''_n. \quad (2.80)$$

将式(2.77),式(2.79),式(2.80)合在一起,即得确定 $m_0, m_1, \dots, m_n$ 的线性方程组为

$$\begin{bmatrix} 2 & 1 & & \\ \lambda_1 & 2 & \mu_1 & \\ \ddots & \ddots & \ddots & \\ & \lambda_{n-1} & 2 & \mu_{n-1} \\ & & 1 & 2 \end{bmatrix} \begin{bmatrix} m_0 \\ m_1 \\ \vdots \\ m_{n-1} \\ m_n \end{bmatrix} = \begin{bmatrix} g_0 \\ g_1 \\ \vdots \\ g_{n-1} \\ g_n \end{bmatrix}. \quad (2.81)$$

其中,

$$g_0 = 3 \frac{y_1 - y_0}{h_1} - \frac{h_1}{2} y''_0 = 3f[x_0, x_1] - \frac{h_1}{2} y''_0,$$

$$g_n = 3 \frac{y_n - y_{n-1}}{h_n} + \frac{h_n}{2} y''_n = 3f[x_{n-1}, x_n] + \frac{h_n}{2} y''_n.$$

(3) 在第三种边界条件下, 由 $s'(x_0+0) = s'(x_n-0)$ 直接可得

$$m_0 = m_n. \quad (2.82)$$

由条件

$$s''(x_0+0) = s''(x_n-0),$$

可得

$$-\frac{4}{h_1}m_0 - \frac{2}{h_1}m_1 + \frac{6}{h_1^2}(y_1 - y_0) = \frac{2}{h_n}m_{n-1} + \frac{4}{h_n}m_n - \frac{6}{h_n^2}(y_n - y_{n-1}).$$

注意到 $m_0 = m_n$, 将上式整理后, 得

$$\frac{h_n}{h_1 + h_n}m_1 + \frac{h_1}{h_1 + h_n}m_{n-1} + 2m_n = 3\left(\frac{h_n}{h_1 + h_n} \frac{y_1 - y_0}{h_1} + \frac{h_1}{h_1 + h_n} \frac{y_n - y_{n-1}}{h_n}\right).$$

若记

$$\mu_n = \frac{h_n}{h_1 + h_n}, \quad \lambda_n = 1 - \mu_n = \frac{h_1}{h_1 + h_n},$$

$$g_n = 3\left(\mu_n \frac{y_1 - y_0}{h_1} + \lambda_n \frac{y_n - y_{n-1}}{h_n}\right) = 3(\mu_n f[x_0, x_1] + \lambda_n f[x_{n-1}, x_n]).$$

则上式可以写成

$$\mu_n m_1 + \lambda_n m_{n-1} + 2m_n = g_n. \quad (2.83)$$

将式(2.77), 式(2.82), 式(2.83)合在一起, 得

$$\begin{bmatrix} 2 & \mu_1 & \cdots & \lambda_1 \\ \lambda_2 & 2 & \mu_2 & \\ \ddots & \ddots & \ddots & \\ & \lambda_{n-1} & 2 & \mu_{n-1} \\ \mu_n & \cdots & \lambda_n & 2 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ \vdots \\ m_{n-1} \\ m_n \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_{n-1} \\ g_n \end{bmatrix}. \quad (2.84)$$

方程(2.78)、方程(2.81)、方程(2.84)的系数矩阵都是严格对角占优矩阵(即主对角线上的元素的绝对值都严格大于同一行中其他元素的绝对值之和), 因此可以证明它们都是非奇异的, 从而这些方程组都有唯一确定的解。

解上述方程组, 求出 $m_0, m_1, \dots, m_n$ 的值, 将它们代入公式(2.75), 立即可以得到 3 次样条插值函数 $s(x)$ 在各子区间上的表达式。

求解过程可以归纳为以下三步。

(1) 根据给定数据和边界条件算出 λ_i, μ_i 和 g_i , 从而写出确定 m_i 的方程(2.78)、方

程(2.81)或方程(2.84);

(2) 解方程(2.78)、方程(2.81)或方程(2.84)(用追赶法)得到 $s'(x)$ 在 x_i 上的值 $m_i = s'(x_i) (i=0, 1, 2, \dots, n)$;

(3) 将所得的 m_i 代入公式(2.75), 即得 $s(x)$ 在各子区间上的表达式 $s_i(x) (i=1, 2, \dots, n)$.

例 2.8 已知 $f(x)$ 的函数表如表 2.15 所示, 求在 $[0, 3]$ 上的 3 次样条插值函数.

表 2.15

x	0	1	2	3
$f(x)$	0	2	3	16
$f'(x)$	1			0

解 此时 $n=3$, 将 $h_i=1$ 代入公式计算, 得

$$\lambda_i = \frac{h_{i+1}}{h_i + h_{i+1}} = \frac{1}{1+1} = \frac{1}{2}, \quad \mu_i = 1 - \lambda_i = \frac{1}{2} \quad (i=1, 2).$$

$$g_1 = 3(\mu_1 f[x_1, x_2] + \lambda_1 f[x_0, x_1]) = \frac{9}{2},$$

$$g_2 = 3(\mu_2 f[x_2, x_3] + \lambda_2 f[x_1, x_2]) = 21,$$

将其代入式(2.77)得

$$\begin{cases} \frac{1}{2}m_0 + 2m_1 + \frac{1}{2}m_2 = \frac{9}{2}, \\ \frac{1}{2}m_1 + 2m_2 + \frac{1}{2}m_3 = 21. \end{cases}$$

因 $m_0=1, m_3=0$, 经整理得

$$\begin{cases} 2m_1 + \frac{1}{2}m_2 = 4, \\ \frac{1}{2}m_1 + 2m_2 = 21. \end{cases}$$

解得

$$m_1 = -\frac{2}{3}, \quad m_2 = \frac{32}{3}.$$

对式(2.75), 令 $i=1$, 且将 $y_0=0, y_1=2, x_0=0, x_1=1$ 及 $m_0=1, m_1=-\frac{2}{3}$ 代入得

$$\begin{aligned} s_0(x) &= 0 + 2 \times \frac{(x-0)^2[2(1-x)+1]}{1^3} + 1 \times \frac{(1-x)^2(x-0)}{1^2} \\ &\quad - \left(-\frac{2}{3}\right) \frac{(x-0)^2(1-x)}{1^2} = \frac{x}{3}(3+14x-11x^2). \end{aligned}$$

同理, 令 $i=2, 3$, 由式(2.75)求得 $s_1(x), s_2(x)$, 把它们写成如下三段函数形式:

$$s(x) = \begin{cases} \frac{x}{3}(-11x^2 + 14x + 3), & x \in [0, 1], \\ \frac{1}{3}(24x^3 - 91x^2 + 108x - 35), & x \in [1, 2], \\ \frac{1}{3}(-46x^3 + 329x^2 - 732x + 525), & x \in [2, 3], \end{cases}$$

$s(x)$ 即为满足已知函数表的三次样条插值函数.

2.7.3 误差界与收敛性

3次样条函数的收敛性与误差估计比较复杂,这里不加证明地给出一个主要结论.

定理 2.3 设 $f(x) \in C^4[a, b]$, $s(x)$ 为满足第一种或第二种边界条件的 3 次样条函数, 令 $h = \max_{1 \leq i \leq n} \{h_i\}$, $h_i = x_i - x_{i-1}$ ($i = 1, 2, \dots, n$), 则有估计式

$$\max_{a \leq x \leq b} |f^{(k)}(x) - s^{(k)}(x)| \leq C_k \max_{a \leq x \leq b} |f^{(4)}(x)| h^{4-k} \quad (k = 0, 1, 2),$$

其中, $C_0 = \frac{5}{384}$, $C_1 = \frac{1}{24}$, $C_2 = \frac{3}{8}$.

这个定理不但给出了 3 次样条插值函数 $s(x)$ 的误差估计, 且当 $h \rightarrow 0$ 时, $s(x)$ 及其一阶导数 $s'(x)$ 和二阶导数 $s''(x)$ 均分别一致收敛于 $f(x)$, $f'(x)$ 及 $f''(x)$.

习 题 2

1. 当 $x=1, -1, 2$ 时, $f(x)=0, -3, 4$. 求 $f(x)$ 的 2 次插值多项式.
2. 已知 $\sin(0.32)=0.314567, \sin(0.34)=0.333487$ 有 6 位有效数字.
 - (1) 用线性插值求 $\sin(0.33)$ 的近似值.
 - (2) 证明: 在区间 $[0.32, 0.34]$ 上用线性插值计算 $\sin x$ 时至少有 4 位有效数字.
3. 设 x_j 为互异节点 ($j=0, 1, 2, \dots, n$), 证明:

$$(1) \sum_{j=0}^n x_j^k l_j(x) \equiv x^k, k = 0, 1, 2, \dots, n.$$

$$(2) \sum_{j=0}^n (x_j - x)^k l_j(x) \equiv 0, k = 1, 2, \dots, n.$$

4. 给出 $\cos x, 0^\circ \leq x \leq 90^\circ$ 的函数表, 步长 $h=1'=(1/60)^\circ$, 若函数表具有 5 位有效数字, 研究用线性插值求 $\cos x$ 近似值时的总误差界.

5. 在 $-4 \leq x \leq 4$ 上给出 $f(x)=e^x$ 的等距节点函数表, 若用 2 次插值求 e^x 的近似值, 要使截断误差不超过 10^{-6} , 问: 函数表的步长 h 应取多少?

6. 如果 $f(x)$ 是 m 次多项式, 记 $\Delta f(x) = f(x+h) - f(x)$, 证明 $f(x)$ 的 k 阶差分 $\Delta^k f(x)$ ($0 \leq k \leq m$) 是 $(m-k)$ 次多项式, 并且 $\Delta^{m+l} f(x) = 0$ (l 为正整数).

7. 证明: $\Delta(f_k g_k) = f_k \Delta g_k + g_{k+1} \Delta f_k$.

8. 若 $f(x) = a_0 + a_1 x + \cdots + a_{n-1} x^{n-1} + a_n x^n$ 有 n 个不同实根 $x_1, x_2, \cdots, x_n$, 证明:

$$\sum_{j=1}^n \frac{x_j^k}{f'(x_j)} = \begin{cases} 0, & 0 \leq k \leq n-2, \\ a_n^{-1}, & k = n-1. \end{cases}$$

9. $f(x) = x^7 + x^4 + 3x + 1$, 求

$$f[2^0, 2^1, \cdots, 2^7] \text{ 及 } f[2^0, 2^1, \cdots, 2^8].$$

10. 证明: 2 点 3 次 Hermite 插值余项是

$$R_3(x) = f^{(4)}(\xi)(x-x_k)^2(x-x_{k+1})^2/4!, \quad \xi \in (x_k, x_{k+1}).$$

并由此求出分段 3 次 Hermite 插值的误差限.

11. 求一个次数不高于 4 次的多项式 $P(x)$, 使它满足 $P(0) = P'(0) = 0, P(1) = P'(1) = 1, P(2) = 1$, 并写出其余项表达式.

12. 已知数据如表 2.16 所示. 利用反插值, 求方程 $f(x) = 0.4500$ 的根的近似值.

表 2.16

x	0	0.2	0.4	0.6	0.8
$f(x)$	0.1995	0.3965	0.5881	0.7721	0.9461

13. 设 $f(x) = 1/(1+x^2)$, 在 $-5 \leq x \leq 5$ 上取 $n=10$, 按等距节点求分段线性插值函数 $I_h(x)$, 计算各节点间中点处的 $I_h(x)$ 与 $f(x)$ 的值, 并估计误差.

14. 求 $f(x) = x^4$ 在 $[a, b]$ 上的分段 Hermite 插值, 并估计误差.

15. 给定数据如表 2.17 所示.

表 2.17

x_j	0	1	2	3
y_j	0	0	0	0

(1) 求满足边界条件 $s''(0) = 1, s''(3) = 0$ 的 3 次样条插值函数 $s(x)$.

(2) 求满足边界条件 $s'(0) = 1, s'(3) = 0$ 的 3 次样条插值函数 $s(x)$.

16. 完成下列填空题.

(1) $f(x) = 3x^2 + 1$, 则 $f[1, 2, 3] = \underline{\hspace{2cm}}, f[1, 2, 3, 4] = \underline{\hspace{2cm}}.$

(2) 设 $x_i (i = 0, 1, 2, 3, 4, 5)$ 为互异节点, $l_i(x)$ 为对应的 5 次 Lagrange 插值基函数, 则 $\sum_{i=0}^5 x_i^5 l_i(0) = \underline{\hspace{2cm}}, \sum_{i=0}^5 (x_i^5 + 2x_i^4 + x_i^3 + 1) l_i(x) = \underline{\hspace{2cm}}.$

(3) $f(x) = x^5 + 1, x_i = \frac{1}{2}i (i=0, 1, 2, \dots)$, 则 $\Delta^6 f_0 = \underline{\hspace{2cm}}, \Delta^2 f_2 = \underline{\hspace{2cm}}$.

(4) $s(x) = \begin{cases} x^3 + x^2, & 0 \leq x \leq 1, \\ 2x^3 + bx^2 + cx - 1, & 1 < x \leq 2 \end{cases}$ 是以 0, 1, 2 为节点的 3 次样条函数, 则 $b = \underline{\hspace{2cm}}, c = \underline{\hspace{2cm}}$.

数值实验题 2

实验 2.1 多项式插值的振荡现象

问题提出: 考虑在一个固定的区间上用插值逼近一个函数. 显然 Lagrange 插值中使用的节点越多, 插值多项式的次数就越高. 我们自然关心插值多项式的次数增加时, $L_n(x)$ 是否也更加靠近被逼近的函数. Runge 给出的一个例子是极著名并富有启发性的. 设区间 $[-1, 1]$ 上函数

$$f(x) = \frac{1}{1 + 25x^2}.$$

实验内容: 考虑区间 $[-1, 1]$ 的一个等距划分, 分点为

$$x_i = -1 + \frac{2i}{n}, \quad i = 0, 1, 2, \dots, n,$$

则拉格朗日插值多项式为

$$L_n(x) = \sum_{i=0}^n \frac{1}{1 + 25x_i^2} l_i(x).$$

其中, $l_i(x), i=0, 1, 2, \dots, n$ 是 n 次 Lagrange 插值基函数.

实验要求:

(1) 选择不断增大的分点数目 $n=2, 3, \dots$, 画出原函数 $f(x)$ 及插值多项式函数 $L_n(x)$ 在 $[-1, 1]$ 上的图像, 比较并分析实验结果.

(2) 选择其他的函数, 例如定义在区间 $[-5, 5]$ 上的函数

$$h(x) = \frac{x}{1+x^4}, \quad g(x) = \arctan x,$$

重复上述的实验看其结果如何.

实验 2.2 样条插值的收敛

问题提出: 多项式插值是不收敛的, 即插值的节点多, 效果不一定就好. 对样条函数插值又如何呢? 理论上证明样条插值的收敛性是比较困难的, 也超出了本课程的内容.

第3章 函数逼近与曲线拟合

在实际问题中,我们常常会遇到用各种方式定义的函数.例如,用积分或无穷级数作为定义函数的表达式,这样的函数表达式,通常对于确定和分析函数的性质特别有效,但用来计算函数值一般就很不方便.因此,有时希望给出这种函数的一个形式简单的近似表达式.用一个简单的函数 $y(x)$ 近似代替给定的函数 $f(x)$ 的问题,称为函数逼近问题, $y(x)$ 称为逼近函数, $f(x)$ 称为被逼近函数.更确切一些可以叙述为:对于某个函数类 A 中给定的函数 $f(x)$,要求在另一较简单的函数类 $B \subset A$ 中,求函数 $y(x) \in B$,使 $y(x)$ 与 $f(x)$ 之差在某种度量意义下为最小.通常 A 是区间 $[a, b]$ 上的连续函数类 $C[a, b]$, B 可取为代数多项式,或有理分式函数,或三角多项式等.逼近误差的度量标准常用的有两种,一种是

$$\|f(x) - y(x)\|_{\infty} = \max_{a \leq x \leq b} |f(x) - y(x)|,$$

在这种度量意义下的函数逼近称为一致逼近;另一种度量标准是

$$\|f(x) - y(x)\|_2 = \sqrt{\int_a^b [f(x) - y(x)]^2 dx},$$

在这种度量意义下的函数逼近称为均方逼近或平方逼近.

3.1 内积空间

定义 3.1 若区间 (a, b) 上的非负函数 $\rho(x)$, 满足条件

(1) $\int_a^b |x|^n \rho(x) dx$ 存在 ($n = 0, 1, 2, \dots$);

(2) 对非负的连续函数 $g(x)$, 若

$$\int_a^b g(x) \rho(x) dx = 0, \text{ 则在 } (a, b) \text{ 上 } g(x) \equiv 0, \quad (3.1)$$

就称 $\rho(x)$ 为区间 (a, b) 上的权函数.

定义 3.2 设 $f(x), g(x) \in C[a, b]$, $\rho(x)$ 是 $[a, b]$ 上的权函数, 积分

$$(f, g) = \int_a^b \rho(x) f(x) g(x) dx \quad (3.2)$$

称为函数 $f(x)$ 与 $g(x)$ 在 $[a, b]$ 上的内积.

容易验证这样定义的内积满足下列四条公理:

(1) $(f, g) = (g, f)$;

(2) $(cf, g) = c(f, g)$, c 为常数;

$$(3) (f_1 + f_2, g) = (f_1, g) + (f_2, g);$$

$$(4) (f, f) \geq 0, \text{ 并且仅当 } f=0 \text{ 时 } (f, f)=0.$$

定义 3.3 内积的函数空间称为内积空间. 因此, 连续函数空间 $C[a, b]$ 上若定义了内积, 就形成一个内积空间, 这个内积的定义是 n 维欧氏空间 $\mathbf{R}^n$ 中两个向量内积定义的推广. 设

$$f = (f_1, f_2, \dots, f_n)^T, \quad g = (g_1, g_2, \dots, g_n)^T,$$

$\mathbf{R}^n$ 中内积定义是

$$(f, g) = \sum_{k=1}^n f_k g_k.$$

向量 $f \in \mathbf{R}^n$ 的模(范数)定义为

$$\|f\|_2 = \left(\sum_{k=1}^n f_k^2 \right)^{1/2}.$$

将它推广到任何内积空间中, 得出下面的定义.

定义 3.4 $f(x) \in C[a, b]$, 量

$$\|f\|_2 = \sqrt{\int_a^b \rho(x) f^2(x) dx} = \sqrt{(f, f)} \quad (3.3)$$

称为 $f(x)$ 的欧氏范数.

它同样满足范数三条性质, 头两条是显然满足的, 即(1) $\|f\|_2 \geq 0$, 当且仅当 $f=0$ 时 $\|f\|_2=0$; (2) $\|\alpha f\|_2 = |\alpha| \|f\|_2$ 对任意 $f \in C[a, b]$ 成立, α 为实数. 以下定理包含了第三条性质和其他重要结论.

定理 3.1 对任何 $f, g \in C[a, b]$, 下列结论成立:

$$(1) |(f, g)| \leq \|f\|_2 \|g\|_2 \text{ (此式称为 Cauchy-Schwarz (柯西-许瓦兹) 不等式);} \quad (3.4)$$

$$(2) \|f+g\|_2 \leq \|f\|_2 + \|g\|_2 \text{ (三角不等式);} \quad (3.5)$$

$$(3) \|f+g\|_2^2 + \|f-g\|_2^2 = 2(\|f\|_2^2 + \|g\|_2^2) \text{ (平行四边形定律).}$$

证 若 $g=0$, 则式(3.4)显然成立. 现考虑 $g \neq 0$, 对任何实数 λ , 有

$$0 \leq (f + \lambda g, f + \lambda g) = (f, f) + 2\lambda(f, g) + \lambda^2(g, g).$$

现取

$$\lambda = -\frac{(f, g)}{\|g\|_2^2},$$

代入上式得

$$\|f\|_2^2 - 2 \frac{|(f, g)|^2}{\|g\|_2^2} + \frac{|(f, g)|^2}{\|g\|_2^2} \geq 0,$$

即

$$|(f, g)|^2 \leq \|f\|_2^2 \|g\|_2^2.$$

两边开平方即得式(3.4).

利用式(3.4), 考虑

$$\|f+g\|_2^2 = (f+g, f+g) = (f, f) + 2(f, g) + (g, g)$$

$$\begin{aligned}
&\leq \|f\|_2^2 + 2|(f, g)| + \|g\|_2^2 \\
&\leq \|f\|_2^2 + 2\|f\|_2\|g\|_2 + \|g\|_2^2 \\
&= (\|f\|_2 + \|g\|_2)^2.
\end{aligned}$$

两边开方则得式(3.5). 平行四边形定律可直接计算得

$$\begin{aligned}
\|f+g\|_2^2 + \|f-g\|_2^2 &= (f+g, f+g) + (f-g, f-g) \\
&= (f, f) + 2(f, g) + (g, g) + (f, f) \\
&\quad - 2(f, g) + (g, g) \\
&= 2[(f, f) + (g, g)] = 2(\|f\|_2^2 + \|g\|_2^2).
\end{aligned}$$

证毕.

在 n 维空间中两个向量正交的定义也可推广到内积空间.

定义 3.5 若 $f(x), g(x) \in C[a, b]$, 满足

$$(f, g) = \int_a^b \rho(x) f(x) g(x) dx = 0,$$

则称 f 与 g 在 $[a, b]$ 上带权 $\rho(x)$ 正交. 若函数族 $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x), \dots$ 满足关系

$$(\varphi_j, \varphi_k) = \int_a^b \rho(x) \varphi_j(x) \varphi_k(x) dx = \begin{cases} 0, & j \neq k, \\ A_k > 0, & j = k. \end{cases}$$

就称 $\{\varphi_k\}$ 是 $[a, b]$ 上带权 $\rho(x)$ 的正交函数族; 若 $A_k \equiv 1$, 就称之为标准正交函数族.

例如, 三角函数族

$$1, \cos x, \sin x, \cos(2x), \sin(2x), \dots$$

就是在区间 $[-\pi, \pi]$ 上的正交函数族. 因为

$$(1, 1) = \int_{-\pi}^{\pi} 1^2 dx = 2\pi,$$

$$(\sin(kx), \sin(kx)) = \int_{-\pi}^{\pi} \sin^2(kx) dx = \pi,$$

$$(\cos(kx), \cos(kx)) = \int_{-\pi}^{\pi} \cos^2(kx) dx = \pi, \quad k = 1, 2, \dots,$$

而当 $j \neq k$ 时

$$\int_{-\pi}^{\pi} \cos(kx) \cos(jx) dx = 0, \quad \int_{-\pi}^{\pi} \sin(kx) \sin(jx) dx = 0, \quad \int_{-\pi}^{\pi} \cos(kx) \sin(jx) dx = 0.$$

在 $\mathbf{R}^n$ 空间中任一向量都可用它的一组线性无关的基表示, 对内积空间的任一元素 $f(x) \in C[a, b]$ 也同样可用线性无关的基表示, 此时相应地有以下结论.

定义 3.6 若 $\varphi_0(x), \dots, \varphi_{n-1}(x)$ 在 $[a, b]$ 上连续, 如果

$$a_0 \varphi_0(x) + a_1 \varphi_1(x) + \dots + a_{n-1} \varphi_{n-1}(x) = 0$$

当且仅当 $a_0 = a_1 = \dots = a_{n-1} = 0$ 时成立, 则称 $\varphi_0, \dots, \varphi_{n-1}$ 在 $[a, b]$ 上是线性无关的; 若函数族 $\{\varphi_k\} (k=0, 1, 2, \dots)$ 中的任何有限个 φ_k 线性无关, 则称 $\{\varphi_k\}$ 为线性无关函数族.

例如, $1, x, \dots, x^n, \dots$ 就是 $[a, b]$ 上的线性无关函数族. 若 $\varphi_0(x), \dots, \varphi_{n-1}(x)$ 是

$C[a, b]$ 中的线性无关函数,且 $a_0, a_1, \dots, a_{n-1}$ 是任意实数,则

$$s(x) = a_0 \varphi_0(x) + a_1 \varphi_1(x) + \dots + a_{n-1} \varphi_{n-1}(x)$$

的全体是 $C[a, b]$ 中的一个子集,记作

$$\Phi = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_{n-1}\}.$$

下面给出判断函数族 $\{\varphi_k\} (k=0, 1, 2, \dots, n-1)$ 线性无关的充要条件.

定理 3.2 $\varphi_0(x), \varphi_1(x), \dots, \varphi_{n-1}(x)$ 在 $[a, b]$ 上线性无关的充要条件是它的 Cramer 行列式 $G_{n-1} \neq 0$, 其中

$$G_{n-1} = G(\varphi_0, \varphi_1, \dots, \varphi_{n-1}) = \begin{vmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \cdots & (\varphi_0, \varphi_{n-1}) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \cdots & (\varphi_1, \varphi_{n-1}) \\ \vdots & \vdots & & \vdots \\ (\varphi_{n-1}, \varphi_0) & (\varphi_{n-1}, \varphi_1) & \cdots & (\varphi_{n-1}, \varphi_{n-1}) \end{vmatrix}.$$

3.2 函数的最佳平方逼近

设函数 $f(x) \in C[a, b]$, 用 n 次多项式 $s(x) = \sum_{k=0}^n a_k x^k$ 作最佳平方逼近, 就是要求

得以 $a_0^*, a_1^*, \dots, a_n^*$ 为系数的多项式 $s^*(x) = \sum_{k=0}^n a_k^* x^k$, 使

$$\|f(x) - s^*(x)\|_2^2 = \int_a^b [f(x) - s^*(x)]^2 dx = \min_{s(x) \in H_n} \|f(x) - s(x)\|_2^2.$$

推广到一般的情况, 就是对于给定的权函数 $\rho(x)$, 要求得 $a_k^* (k=0, 1, 2, \dots, n)$, 使

$$\begin{aligned} \|f(x) - s^*(x)\|_2^2 &= \int_a^b \rho(x) [f(x) - s^*(x)]^2 dx \\ &= \min_{s(x) \in H_n} \|f(x) - s(x)\|_2^2. \end{aligned}$$

n 次多项式 $s(x) = \sum_{k=0}^n a_k x^k$, 是以 $1, x, \dots, x^n$ 为基函数所作的线性组合构成的一类函数. 进一步推广可将 x^k 改为一般的线性无关的连续函数 $\varphi_k(x)$, 要求 $\varphi_0, \varphi_1, \dots, \varphi_n$ 的线性组合 $s(x) = \sum_{k=0}^n a_k \varphi_k(x)$ 的全体构成的 $C[a, b]$ 子空间, 即 $\Phi = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\}$, 最佳平方逼近问题的提法可叙述为: 求 $a_k^* (k=0, 1, 2, \dots, n)$, 使

$$\begin{aligned} \|f(x) - s^*(x)\|_2^2 &= \|f(x) - \sum_{k=0}^n a_k^* \varphi_k(x)\|_2^2 \\ &= \min_{s(x) \in \Phi} \|f(x) - s(x)\|_2^2. \end{aligned} \quad (3.6)$$

称 $s^*(x) = \sum_{k=0}^n a_k^* \varphi_k(x)$ 为 $f(x) \in C[a, b]$ 在子集 $\Phi \subset C[a, b]$ 中的最佳平方逼近函数.

为了求得 $s^*(x)$, 考察式(3.6), 这个问题等价于关于 $a_0, a_1, \dots, a_n$ 的多元函数

$$I(a_0, a_1, \dots, a_n) = \int_a^b \rho(x) [f(x) - \sum_{k=0}^n a_k \varphi_k(x)]^2 dx \quad (3.7)$$

的最小值问题.

为了确定参数 a_k^* ($k=0, 1, 2, \dots, n$), 由多元函数极值存在的必要条件, 有

$$\frac{\partial I}{\partial a_k} = 0, \quad k = 0, 1, 2, \dots, n,$$

$$\frac{\partial I}{\partial a_k} = 2 \int_a^b \rho(x) \left[\sum_{j=0}^n a_j \varphi_j(x) - f(x) \right] \varphi_k(x) dx = 0, \quad k = 0, 1, 2, \dots, n,$$

即有
$$\sum_{j=0}^n (\varphi_j, \varphi_k) a_j = (f, \varphi_k), \quad k = 0, 1, 2, \dots, n. \quad (3.8)$$

这是关于未知数 $a_0, a_1, \dots, a_n$ 的线性代数方程组, 称为法方程组. 由于 $\varphi_0, \varphi_1, \dots, \varphi_n$ 线性无关, 故系数行列式 $G(\varphi_0, \varphi_1, \dots, \varphi_n) \neq 0$, 于是方程组(3.8)有唯一解 $a_k = a_k^*$ ($k=0, 1, 2, \dots, n$), 从而得到

$$s^*(x) = a_0^* \varphi_0(x) + \dots + a_n^* \varphi_n(x).$$

下面证明 $s^*(x)$ 满足式(3.6), 即对任何 $s(x) \in \Phi$, 有

$$\int_a^b \rho(x) [f(x) - s^*(x)]^2 dx \leq \int_a^b \rho(x) [f(x) - s(x)]^2 dx. \quad (3.9)$$

为此只要考虑

$$\begin{aligned} D &= \int_a^b \rho(x) [f(x) - s(x)]^2 dx - \int_a^b \rho(x) [f(x) - s^*(x)]^2 dx \\ &= \int_a^b \rho(x) [s(x) - s^*(x)]^2 dx + 2 \int_a^b \rho(x) [s^*(x) - s(x)] [f(x) - s^*(x)] dx. \end{aligned}$$

由于 $s^*(x)$ 的系数 a_k^* 是方程(3.8)的解, 故

$$\int_a^b \rho(x) [f(x) - s^*(x)] \varphi_k(x) dx = 0, \quad k = 0, 1, 2, \dots, n,$$

从而上式第二个积分为 0, 于是有

$$D = \int_a^b \rho(x) [s(x) - s^*(x)]^2 dx \geq 0,$$

故式(3.9)成立, 这就证明了 $s^*(x)$ 是 $f(x)$ 在 Φ 中的最佳平方逼近函数.

如果令 $\delta = f(x) - s^*(x)$, 由式(3.8)易知 $(f - s^*, s^*) = 0$, 则平方误差为

$$\begin{aligned} \|\delta\|_2^2 &= (f - s^*, f - s^*) = (f, f) - (s^*, f) \\ &= \|f\|_2^2 - \sum_{k=0}^n a_k^* (\varphi_k, f). \end{aligned} \quad (3.10)$$

若取 $\varphi_k(x) = x^k, \rho(x) \equiv 1, f(x) \in C[0, 1]$, 要在 H_n 中求 n 次最佳平方逼近多项式

$$s^*(x) = a_0^* + a_1^* x + \dots + a_n^* x^n.$$

这时

$$(\varphi_j, \varphi_k) = \int_0^1 x^{k+j} dx = \frac{1}{k+j+1}, \quad (f, \varphi_k) = \int_0^1 x^k f(x) dx = d_k.$$

于是方程组(3.8)的系数矩阵为

$$H = \begin{bmatrix} 1 & \frac{1}{2} & \cdots & \frac{1}{n+1} \\ \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n+2} \\ \vdots & \vdots & & \vdots \\ \frac{1}{n+1} & \frac{1}{n+2} & \cdots & \frac{1}{2n+1} \end{bmatrix}. \quad (3.11)$$

这个矩阵称为 Hilbert(希尔伯特)矩阵,记 $d=(d_0, d_1, \dots, d_n)^T$, $a=(a_0, a_1, \dots, a_n)^T$, 则

$$Ha = d \quad (3.12)$$

的解 $a_k = a_k^*$ ($k=0, 1, 2, \dots, n$) 即为所求.

例 3.1 定义内积 $(f, g) = \int_0^1 f(x)g(x)dx$. 试在 $H_1 = \text{span}\{1, x\}$ 中寻求对于 $f(x) = \sqrt{x}$ 的最佳平方逼近元素 $P(x)$.

解 这里实际上要求的是 $(0, 1)$ 上的一次最佳平方逼近多项式 $P_1(x)$.

利用式(3.12)求出 $d=(d_0, d_1)^T$.

$$d_0 = (f, 1) = \int_0^1 \sqrt{x} dx = \frac{2}{3}, \quad d_1 = (f, x) = \int_0^1 x\sqrt{x} dx = \frac{2}{5},$$

得法方程组为

$$\begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{3} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \frac{2}{3} \\ \frac{2}{5} \end{bmatrix}.$$

解得 $a_0^* = \frac{4}{15}$, $a_1^* = \frac{12}{15}$, 所求的最佳平方逼近元素为

$$P(x) = \frac{4}{15} + \frac{12}{15}x, \quad 0 \leq x \leq 1.$$

平方误差

$$\begin{aligned} \|\delta\|_2^2 &= (f, f) - \sum_{k=0}^1 a_k^* (f, \varphi_k) = \int_0^1 x dx - \sum_{k=0}^1 a_k^* d_k \\ &= \frac{1}{2} - \frac{4}{15} \times \frac{2}{3} - \frac{12}{15} \times \frac{2}{5} = 0.002222. \end{aligned}$$

对于一般的基底 $\varphi_0, \varphi_1, \dots, \varphi_n$, 当 n 稍大时, 计算法方程组中的 (φ_k, φ_j) 以及求解法方程组的计算量都是很大的, 若采用 $1, x, \dots, x^n$ 作基底, 当 $\rho(x) \equiv 1$ 时, 虽然 $(\varphi_k, \varphi_j) = (x^k, x^j)$ 容易计算, 但由此形成的法方程组系数矩阵(3.11)当 $n \geq 4$ 时是病态矩阵, 用单字长在计算机上求解法方程组, 其结果往往不太可靠, 为避免上述的弊端, 可采用正交基底. 为此, 先介绍正交多项式.

3.3 正交多项式

定义 3.7 若首项系数 $a_n \neq 0$ 的 n 次多项式 $g_n(x)$, 满足

$$\int_a^b \rho(x) g_j(x) g_k(x) dx = \begin{cases} 0, & j \neq k, \\ A_k > 0, & j = k, \end{cases} \quad j, k = 0, 1, 2, \dots,$$

就称多项式序列 $g_0(x), g_1(x), \dots$ 在 $[a, b]$ 上带权 $\rho(x)$ 正交, 并称 $g_n(x)$ 是 $[a, b]$ 上带权 $\rho(x)$ 的 n 次正交多项式.

定理 3.3 设 $g_k(x) (k=0, 1, 2, \dots)$ 是 k 次多项式, 则多项式系 $\{g_k(x)\}$ 是 $[a, b]$ 上带权 $\rho(x)$ 的正交多项式系的充分必要条件是, 对任何次数不超过 $(k-1)$ 的多项式 $P(x)$, 都有 $\int_a^b \rho(x) P(x) g_k(x) dx = 0 (k=1, 2, \dots)$, 即 $g_k(x)$ 与任何次数不超过 $(k-1)$ 的多项式 $P(x)$ 在 $[a, b]$ 上带权 $\rho(x)$ 正交.

3.3.1 正交多项式的性质

设 $\{g_k(x)\}$ 为 $[a, b]$ 上的正交多项式序列, 其中 $g_k(x)$ 为 k 次正交多项式, 则具有下列基本性质.

性质 1 $\{g_k(x)\}$ 是线性无关的.

性质 2 $g_k(x)$ 的零点都是实的、相异的 (即单重的), 且全部在区间 (a, b) 内部.

性质 3 首项系数为 1 的正交多项式序列 $\{g_k(x)\}$ 中任何相邻 3 个多项式 $g_{k-1}(x), g_k(x), g_{k+1}(x)$, 存在如下递推关系:

$$g_{k+1}(x) = (x - a_{k+1})g_k(x) - b_k g_{k-1}(x), \quad k = 1, 2, \dots, \quad (3.13)$$

其中 a_k, b_k 都是与 x 无关的常数, 且

$$\begin{cases} a_{k+1} = \frac{(xg_k, g_k)}{(g_k, g_k)}, & k = 0, 1, 2, \dots, \\ b_0 = 0, \quad b_k = \frac{(g_k, g_k)}{(g_{k-1}, g_{k-1})}, & k = 1, 2, \dots. \end{cases} \quad (3.14)$$

一般说来, 当权 $\rho(x)$ 及区间 $[a, b]$ 给定后, 就可构造出正交多项式, 下面我们介绍几类.

3.3.2 Legendre(勒让德)多项式

当区间为 $[-1, 1]$, 权函数 $\rho(x) \equiv 1$ 时, 由 $\{1, x, \dots, x^n, \dots\}$ 正交化得到的多项式就称为 Legendre(勒让德)多项式, 并用 $P_0(x), P_1(x), \dots, P_n(x), \dots$ 表示. 这是 Legendre 于 1785 年引进的. 1814 年 Rodrigul(罗德利克)给出了简单的表达式

$$P_0(x) = 1, \quad P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} \{(x^2 - 1)^n\}, \quad n = 1, 2, \dots, \quad (3.15)$$

由于 $(x^2-1)^n$ 是 $2n$ 次多项式, 求 n 阶导数后得

$$P_n(x) = \frac{1}{2^n n!} (2n)(2n-1)\cdots(n+1)x^n + a_{n-1}x^{n-1} + \cdots + a_0,$$

于是得首项 x^n 的系数 $a_n = \frac{(2n)!}{2^n (n!)^2}$. 显然最高项系数为 1 的 Legendre 多项式为

$$\tilde{P}_n(x) = \frac{n!}{(2n)!} \frac{d^n}{dx^n} [(x^2-1)^n]. \quad (3.16)$$

Legendre 多项式有下述几个重要性质.

性质 1 正交性

$$\int_{-1}^1 P_n(x) P_m(x) dx = \begin{cases} 0, & m \neq n; \\ \frac{2}{2n+1}, & m = n. \end{cases} \quad (3.17)$$

证明 令 $\varphi(x) = (x^2-1)^n$, 则

$$\varphi^{(k)}(\pm 1) = 0, \quad k = 0, 1, 2, \dots, n-1.$$

设 $Q(x)$ 是在区间 $[-1, 1]$ 上有 n 阶连续可微的函数, 由分部积分知

$$\begin{aligned} \int_{-1}^1 P_n(x) Q(x) dx &= \frac{1}{2^n n!} \int_{-1}^1 Q(x) \varphi^{(n)}(x) dx \\ &= -\frac{1}{2^n n!} \int_{-1}^1 Q'(x) \varphi^{(n-1)}(x) dx \\ &= \cdots \\ &= \frac{(-1)^n}{2^n n!} \int_{-1}^1 Q^{(n)}(x) \varphi(x) dx. \end{aligned}$$

下面分两种情况讨论.

(1) 若 $Q(x)$ 是次数小于 n 的多项式, 则 $Q^{(n)}(x) \equiv 0$, 故得

$$\int_{-1}^1 P_m(x) P_n(x) dx = 0, \quad \text{当 } n \neq m.$$

(2) 若 $Q(x) = P_n(x) = \frac{1}{2^n n!} \varphi^{(n)}(x) = \frac{(2n)!}{2^n (n!)^2} x^n + \cdots$,

$$Q^{(n)}(x) = P_n^{(n)}(x) = \frac{(2n)!}{2^n n!},$$

于是

$$\int_{-1}^1 P_n^2(x) dx = \frac{(-1)^n (2n)!}{2^{2n} (n!)^2} \int_{-1}^1 (x^2-1)^n dx = \frac{(2n)!}{2^{2n} (n!)^2} \int_{-1}^1 (1-x^2)^n dx.$$

由于

$$\int_0^1 (1-x^2)^n dx = \int_0^{\frac{\pi}{2}} \cos^{2n+1} t dt = \frac{2 \cdot 4 \cdot \cdots \cdot (2n)}{1 \cdot 3 \cdot \cdots \cdot (2n+1)},$$

故

$$\int_{-1}^1 P_n^2(x) dx = \frac{2}{2n+1},$$

于是式(3.17)得证.

性质2 奇偶性

$$P_n(-x) = (-1)^n P_n(x). \quad (3.18)$$

由于 $\varphi(x) = (x^2 - 1)^n$ 是偶次多项式, 经过偶次求导仍为偶次多项式, 经过奇次求导则为奇次多项式, 故 n 为偶数时 $P_n(x)$ 为偶函数, n 为奇数时 $P_n(x)$ 为奇函数, 于是式(3.18)成立.

性质3 递推关系

考虑 $n+1$ 次多项式 $xP_n(x)$, 它可表示为

$$xP_n(x) = a_0 P_0(x) + a_1 P_1(x) + \cdots + a_{n+1} P_{n+1}(x).$$

两边乘 $P_k(x)$, 并从 -1 到 1 积分, 得

$$\int_{-1}^1 xP_n(x)P_k(x) dx = a_k \int_{-1}^1 P_k^2(x) dx.$$

当 $k \leq n-2$ 时, $xP_k(x)$ 次数小于等于 $n-1$, 上式左端积分为 0, 故得 $a_k = 0$. 当 $k=n$ 时, $xP_n^2(x)$ 为奇函数, 左端积分仍为 0, 故 $a_n = 0$. 于是

$$xP_n(x) = a_{n-1}P_{n-1}(x) + a_{n+1}P_{n+1}(x),$$

其中

$$\begin{aligned} a_{n-1} &= \frac{2n-1}{2} \int_{-1}^1 xP_n(x)P_{n-1}(x) dx = \frac{2n-1}{2} \cdot \frac{2n}{4n^2-1} = \frac{n}{2n+1}, \\ a_{n+1} &= \frac{2n+3}{2} \int_{-1}^1 xP_n(x)P_{n+1}(x) dx \\ &= \frac{2n+3}{2} \cdot \frac{2(n+1)}{(2n+1)(2n+3)} = \frac{n+1}{2n+1}, \end{aligned}$$

从而得到以下的递推公式

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \quad n = 1, 2, \cdots, \quad (3.19)$$

由 $P_0(x)=1, P_1(x)=x$, 利用式(3.19)就可推出

$$\begin{aligned} P_2(x) &= (3x^2 - 1)/2, \\ P_3(x) &= (5x^3 - 3x)/2, \\ P_4(x) &= (35x^4 - 30x^2 + 3)/8, \\ P_5(x) &= (63x^5 - 70x^3 + 15x)/8, \\ P_6(x) &= (231x^6 - 315x^4 + 105x^2 - 5)/16, \\ &\dots \end{aligned}$$

图 3.1 给出了 $P_0(x), P_1(x), P_2(x), P_3(x)$ 的图形.

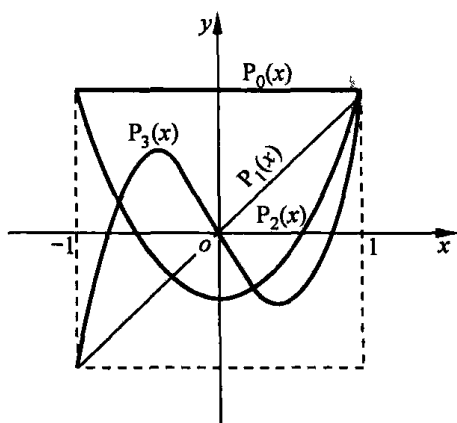


图 3.1

性质 4 $P_n(x)$ 在区间 $[-1, 1]$ 内有 n 个不同的实零点.

性质 5 在所有最高项系数为 1 的 n 次多项式中, Legendre 多项式 $\tilde{P}_n(x)$ 在 $[-1, 1]$ 上与零的平方误差最小.

3.3.3 切比雪夫(Chebyshev)多项式

当权函数 $\rho(x) = \frac{1}{\sqrt{1-x^2}}$, 区间为 $[-1, 1]$ 时,

由序列 $\{1, x, \dots, x^n, \dots\}$ 正交化得到的正交多项式就是 Chebyshev(切比雪夫)多项式, 它可表示为

$$T_n(x) = \cos(n \arccos x), \quad |x| \leq 1. \quad (3.20)$$

若令 $x = \cos \theta$, 则 $T_n(x) = \cos n\theta, 0 \leq \theta \leq \pi$.

Chebyshev 多项式有很多重要性质.

性质 1 递推关系

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \quad n = 1, 2, \dots$$

$$T_0(x) = 1, \quad T_1(x) = x. \quad (3.21)$$

这只要由三角恒等式

$$\cos(n+1)\theta = 2\cos\theta\cos n\theta - \cos(n-1)\theta \quad (n \geq 1),$$

令 $x = \cos \theta$ 即得. 由式(3.21)就可推出

$$T_2(x) = 2x^2 - 1,$$

$$T_3(x) = 4x^3 - 3x,$$

$$T_4(x) = 8x^4 - 8x^2 + 1,$$

$$T_5(x) = 16x^5 - 20x^3 + 5x,$$

$$T_6(x) = 32x^6 - 48x^4 + 18x^2 - 1,$$

...

$T_0(x), T_1(x), T_2(x), T_3(x)$ 的函数图形见图

3.2.

由递推关系(3.21)还可得到 $T_n(x)$ 的最高次项系数是 $2^{n-1} (n \geq 1)$.

性质 2 Chebyshev 多项式 $\{T_k(x)\}$ 在区间 $[-1, 1]$ 上带权 $\rho(x) = 1/\sqrt{1-x^2}$ 正交, 且

$$\int_{-1}^1 \frac{T_n(x)T_m(x)dx}{\sqrt{1-x^2}} = \begin{cases} 0, & n \neq m; \\ \frac{\pi}{2}, & n = m \neq 0; \\ \pi, & n = m = 0. \end{cases} \quad (3.22)$$

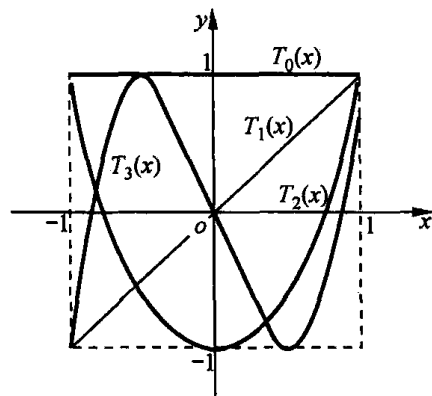


图 3.2

事实上,令 $x = \cos\theta$, 则 $dx = -\sin\theta d\theta$, 于是

$$\int_{-1}^1 \frac{T_n(x)T_m(x)}{\sqrt{1-x^2}} dx = \int_0^\pi \cos n\theta \cos m\theta d\theta = \begin{cases} 0, & n \neq m; \\ \frac{\pi}{2}, & n = m \neq 0; \\ \pi, & n = m = 0. \end{cases}$$

性质3 $T_{2k}(x)$ 只含 x 的偶次幂, $T_{2k+1}(x)$ 只含 x 的奇次幂.

这性质由递推关系直接得到.

性质4 $T_n(x)$ 在区间 $[-1, 1]$ 上有 n 个零点

$$x_k = \cos \frac{2k-1}{2n}\pi, \quad k = 1, 2, \dots, n.$$

此外, 实际计算中时常要求 x^n 用 $T_0(x), T_1(x), \dots, T_n(x)$ 的线性组合表示, 其公式为

$$x^n = 2^{1-n} \sum_{k=0}^{\left[\frac{n}{2}\right]} \binom{n}{k} T_{n-2k}(x). \quad (3.23)$$

这里规定 $T_0(x) = 1, n = 1, 2, \dots, 6$ 时的结果如下.

$$1 = T_0(x),$$

$$x = T_1(x),$$

$$x^2 = \frac{1}{2}(T_0(x) + T_2(x)),$$

$$x^3 = \frac{1}{4}(3T_1(x) + T_3(x)),$$

$$x^4 = \frac{1}{8}(3T_0(x) + 4T_2(x) + T_4(x)),$$

$$x^5 = \frac{1}{16}(10T_1(x) + 5T_3(x) + T_5(x)),$$

$$x^6 = \frac{1}{32}(10T_0(x) + 15T_2(x) + 6T_4(x) + T_6(x)).$$

3.3.4 其他常用的正交多项式

一般来说, 如果区间 $[a, b]$ 及权函数 $\rho(x)$ 不同, 则得到的正交多项式也不同. 除上述两种最重要的正交多项式外, 下面再给出三种较常用的正交多项式.

1. 第二类 Chebyshev 多项式

在区间 $[-1, 1]$ 上带权 $\rho(x) = \sqrt{1-x^2}$ 的正交多项式称为第二类 Chebyshev 多项式, 其表达式为

$$U_n(x) = \frac{\sin[(n+1)\arccos x]}{\sqrt{1-x^2}}. \quad (3.24)$$

令 $x = \cos \theta$, 可得

$$\int_{-1}^1 U_n(x) U_m(x) \sqrt{1-x^2} dx = \int_0^\pi \sin(n+1)\theta \sin(m+1)\theta d\theta = \begin{cases} 0, & m \neq n; \\ \frac{\pi}{2}, & m = n, \end{cases}$$

即 $\{U_n(x)\}$ 是 $[-1, 1]$ 上带权 $\sqrt{1-x^2}$ 的正交多项式族. 还可得到递推关系式

$$U_0(x) = 1, \quad U_1(x) = 2x,$$

$$U_{n+1}(x) = 2xU_n(x) - U_{n-1}(x), \quad n = 1, 2, \dots.$$

2. Laguerre(拉盖尔)多项式

在区间 $[0, +\infty)$ 上带权 e^{-x} 的正交多项式称为 Laguerre 多项式, 其表达式为

$$L_n(x) = e^x \frac{d^n}{dx^n} (x^n e^{-x}). \quad (3.25)$$

它也具有正交性质

$$\int_0^\infty e^{-x} L_n(x) L_m(x) dx = \begin{cases} 0, & m \neq n; \\ (n!)^2, & m = n, \end{cases}$$

和递推关系

$$L_0(x) = 1, \quad L_1(x) = 1 - x,$$

$$L_{n+1}(x) = (1 + 2n - x)L_n(x) - n^2 L_{n-1}(x), \quad n = 1, 2, \dots.$$

3. Hermite(埃尔米特)多项式

在区间 $(-\infty, +\infty)$ 上带权 e^{-x^2} 的正交多项式称为 Hermite 多项式, 其表达式为

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} (e^{-x^2}), \quad (3.26)$$

它满足正交关系

$$\int_{-\infty}^{+\infty} e^{-x^2} H_m(x) H_n(x) dx = \begin{cases} 0, & m \neq n; \\ 2^n n! \sqrt{\pi}, & m = n, \end{cases}$$

并有递推关系

$$H_0(x) = 1, \quad H_1(x) = 2x,$$

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x), \quad n = 1, 2, \dots.$$

3.4 用正交函数系作最佳平方逼近

若 $\{g_k(x)\} (k=0, 1, 2, \dots)$ 是带权 $\rho(x)$ 正交的函数系, 即

$$(g_k, g_j) = \int_a^b \rho(x) g_k(x) g_j(x) dx = 0 \quad (j \neq k),$$

那么,由法方程组 $\sum_{k=0}^n (g_k, g_j) a_k = (f, g_j) (j=0, 1, 2, \dots, n)$ 的各个方程可以独立地解得

$$a_k^* = \frac{(g_k, f)}{(g_k, g_k)}, \quad k = 0, 1, 2, \dots, n, \quad (3.27)$$

从而得出最佳平方逼近函数

$$s^*(x) = \sum_{k=0}^n \frac{(g_k, f)}{(g_k, g_k)} g_k(x).$$

这里的每个 a_k^* 与 n 是无关的,因此对于函数 $f(x) \in C[a, b]$ 与正交基函数系 $g_0, g_1, \dots$ 只要按公式(3.27)逐个计算出 $a_k^* (k=0, 1, 2, \dots)$ 即可以得出一个级数

$$a_0^* g_0(x) + a_1^* g_1(x) + \dots \quad (3.28)$$

这个级数称为 $f(x)$ 对应于基函数系 $\{g_k(x)\}_0^\infty$ 的广义 Fourier 级数;系数 a_k^* 称为广义 Fourier 系数;对任意固定的 n , 其部分和 $s_n(x) = \sum_{k=0}^n a_k^* g_k(x)$ 称为广义多项式, 就是所求的最佳平方逼近多项式. 当级数(3.28)是函数 $f(x)$ 对应的广义 Fourier 级数时, 记为

$$f(x) \sim a_0^* g_0(x) + a_1^* g_1(x) + \dots,$$

在一定条件下, 级数收敛于 $f(x)$, 这时上式中的号“ $\sim$ ”可以改为等号.

例如, 当 $\{g_k(x)\}_0^\infty$ 是三角函数系

$$1, \sin x, \cos x, \sin 2x, \cos 2x, \dots$$

时, 级数(3.28)就是通常的 Fourier 级数.

当 $f(x) \in C[-1, 1]$ 时, 可以用 Legendre 多项式作基函数 $\{g_k\} = \{P_k(x)\}$, 由式(3.28)可得到相应的级数

$$f(x) \sim a_0^* P_0(x) + a_1^* P_1(x) + \dots + a_n^* P_n(x) + \dots, \quad (3.29)$$

由式(3.27)知, 其中

$$a_k^* = \frac{(P_k, f)}{(P_k, P_k)} = \frac{2k+1}{2} \int_{-1}^1 f(x) P_k(x) dx.$$

当式(3.29)右端的级数收敛于 $f(x)$ 时, 则称它为 $f(x)$ 按 Legendre 多项式的展开式, 对于任意固定的 n , 级数(3.29)的部分和

$$s_n^*(x) = a_0^* P_0(x) + a_1^* P_1(x) + \dots + a_n^* P_n(x)$$

是 $[-1, 1]$ 上的一个 n 次多项式, 这是以 $\{P_k(x)\} (k=0, 1, 2, \dots, n)$ 为基函数, 使

$\|f(x) - \sum_{k=0}^n a_k P_k(x)\|_2^2$ 最小的最佳平方逼近多项式.

这时的平方误差为

$$\|\delta_n\|_2^2 = \int_{-1}^1 [f(x)]^2 dx - \sum_{k=0}^n \frac{2}{2k+1} a_k^{*2}.$$

例 3.2 求 $f(x) = e^x$ 在 $[-1, 1]$ 上的 3 次最佳平方逼近多项式, 要求用 $\{P_0, P_1,$

P_2, P_3 作基函数.

解 先计算 (P_k, f) ($k=0, 1, 2, 3$).

$$(P_0, f) = \int_{-1}^1 e^x dx = e - \frac{1}{e} = 2.3504,$$

$$(P_1, f) = \int_{-1}^1 x e^x dx = 2e^{-1} = 0.7358,$$

$$(P_2, f) = \int_{-1}^1 \left(\frac{3}{2}x^2 - \frac{1}{2} \right) e^x dx = e - \frac{7}{e} = 0.1431,$$

$$(P_3, f) = \int_{-1}^1 \left(\frac{5}{2}x^3 - \frac{3}{2}x \right) e^x dx = 37 \frac{1}{e} - 5e = 0.02013.$$

$$a_0^* = \frac{(P_0, f)}{2} = 1.1752, \quad a_1^* = \frac{3}{2}(P_1, f) = 1.1036,$$

$$a_2^* = \frac{5}{2}(P_2, f) = 0.3578, \quad a_3^* = \frac{7}{2}(P_3, f) = 0.07046.$$

于是得

$$s_3^*(x) = 0.9963 + 0.9979x + 0.5367x^2 + 0.1762x^3,$$

平方误差为

$$\|\delta_3\|_2^2 = \|e^x - s_3^*(x)\|_2^2 = \int_{-1}^1 e^{2x} dx - \sum_{k=0}^3 \frac{2}{2k+1} a_k^{*2} = 0.88 \times 10^{-4}.$$

注 如果所给的区间不是 $[-1, 1]$, 而是一般的有限区间 $[a, b]$, 那么, 可以通过变量置换

$$x = \frac{a+b}{2} + \frac{b-a}{2}t$$

将它转化为区间 $-1 \leq t \leq 1$ 上的情形来处理.

例 3.3 求 $f(x) = \sqrt{x}$ 在区间 $[0, 1]$ 上的一次最佳平方逼近多项式.

解 令 $x = \frac{1}{2}(1+t)$, 则

$$f(x) = \frac{1}{\sqrt{2}}\sqrt{1+t} = \varphi(t), \quad -1 \leq t \leq 1.$$

先求 $\varphi(t)$ 在区间 $[-1, 1]$ 上的一次最佳平方逼近多项式 $q_1(t)$, 由

$$a_0^* = \frac{1}{2}(\varphi, P_0) = \frac{1}{2} \int_{-1}^1 \frac{1}{\sqrt{2}} \sqrt{1+t} dt = \frac{2}{3},$$

$$a_1^* = \frac{3}{2}(\varphi, P_1) = \frac{3}{2} \int_{-1}^1 \frac{t}{\sqrt{2}} \sqrt{1+t} dt = \frac{2}{5}.$$

可知
$$q_1(t) = \frac{2}{3}P_0(t) + \frac{2}{5}P_1(t) = \frac{2}{3} + \frac{2}{5}t, \quad -1 \leq t \leq 1.$$

将 $t=2x-1$ 代入 $q_1(t)$, 就得 $\sqrt{x}$ 在区间 $[0, 1]$ 上的一次最佳平方逼近多项式

$$s_1^*(x) = \frac{2}{3} + \frac{2}{5}(2x-1) = \frac{4}{15} + \frac{4}{5}x.$$

3.5 曲线拟合的最小二乘法

3.5.1 最小二乘曲线拟合的概念

在科学实验的统计分析研究工作中,常常需要从一组观测数据 $(x_i, y_i) (i=1, 2, \dots, N)$ 中,求得变量 x 与 y 之间的某种近似关系 $y = \varphi(x)$,从几何图形上看,就是根据 N 个给定的点 $(x_i, y_i) (i=1, 2, \dots, N)$ 求一条近似曲线,这类问题称为曲线拟合问题.由于一般实验所得的数据很多,而且观测数据本身带有误差,因此所求的近似曲线并不要求通过所有的给定点 (x_i, y_i) ,即不要求满足 $\varphi(x_i) = y_i (i=1, 2, \dots, N)$,而只要求函数 $\varphi(x)$ 能反映数据的基本变化趋势.由于不同类型的函数能表达不同的特性,因此常常规定在某种确定的函数类 Φ 中寻求一个最好的函数 $\varphi(x)$ 拟合已给的数据 (x_i, y_i) .所谓“最好”的标准通常是要求 $\varphi(x_i)$ 与 y_i 的偏差 $[y_i - \varphi(x_i)]$ 的平方和 $\sum_{i=1}^N [y_i - \varphi(x_i)]^2$ 为最小,按这样的标准确定拟合函数,称为最小二乘曲线拟合,这实际上是在离散情形下的最佳平方逼近.

问题更一般的提法是:对于给定的数据 $(x_i, y_i) (i=1, 2, \dots, N)$,选取线性无关的函数族 $\{\varphi_0, \varphi_1, \dots, \varphi_m\}$ 及权函数 $\omega(x_i)$ ($\omega(x_i)$ 表示数据 (x_i, y_i) 的比重),要求在函数类 $\Phi = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_m\}$ 中寻找一个函数

$$\varphi^*(x) = a_0^* \varphi_0 + a_1^* \varphi_1 + \dots + a_m^* \varphi_m \quad (m < N),$$

使

$$\sum_{i=1}^N \omega(x_i) [y_i - \varphi(x_i)]^2$$

达到极小.显然上式是 $(m+1)$ 个变量 $a_0, a_1, \dots, a_m$ 的二次函数

$$I(a_0, a_1, \dots, a_m) = \sum_{i=1}^N \omega(x_i) \left[y_i - \sum_{k=0}^m a_k \varphi_k(x_i) \right]^2. \quad (3.30)$$

由多元函数极值的必要条件,有

$$\frac{1}{2} \frac{\partial I}{\partial a_j} = \sum_{i=1}^N \omega(x_i) \varphi_j(x_i) \left[y_i - \sum_{k=0}^m a_k \varphi_k(x_i) \right] = 0, \quad j = 0, 1, 2, \dots, m. \quad (3.31)$$

引入内积

$$(f, g) = \sum_{i=1}^N \omega(x_i) f(x_i) g(x_i), \quad (3.32)$$

方程组(3.31)可以表示为

$$a_0(\varphi_j, \varphi_0) + a_1(\varphi_j, \varphi_1) + \dots + a_m(\varphi_j, \varphi_m) = (\varphi_j, y_i), \quad j = 0, 1, 2, \dots, m,$$

即有

$$\begin{bmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \cdots & (\varphi_0, \varphi_m) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \cdots & (\varphi_1, \varphi_m) \\ \vdots & \vdots & & \vdots \\ (\varphi_m, \varphi_0) & (\varphi_m, \varphi_1) & \cdots & (\varphi_m, \varphi_m) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} (\varphi_0, Y) \\ (\varphi_1, Y) \\ \vdots \\ (\varphi_m, Y) \end{bmatrix}. \quad (3.33)$$

这里 $Y = (y_1, y_2, \dots, y_N)^T$, 这个方程组称为法方程组或正规方程组. 若用 $\varphi_0, \varphi_1, \dots, \varphi_m$ 构成 $N \times (m+1)$ 阶矩阵 A , 即

$$A = \begin{bmatrix} \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_m(x_1) \\ \varphi_0(x_2) & \varphi_1(x_2) & \cdots & \varphi_m(x_2) \\ \vdots & \vdots & & \vdots \\ \varphi_0(x_N) & \varphi_1(x_N) & \cdots & \varphi_m(x_N) \end{bmatrix}.$$

又引入向量 $a = (a_0, a_1, \dots, a_m)^T$, $W = \text{diag}(w_1, w_2, \dots, w_N)$ ($w_i = w(x_i)$), 则法方程组 (3.33) 可写成以下矩阵形式:

$$A^T W A a = A^T W Y \quad (3.34)$$

由于 $\varphi_0, \varphi_1, \dots, \varphi_m$ 线性无关, 可知法方程组 (3.33) 存在唯一解

$$a_0 = a_0^*, \quad a_1 = a_1^*, \quad \dots, \quad a_m = a_m^*,$$

从而得到函数

$$\varphi^*(x) = a_0^* \varphi_0 + a_1^* \varphi_1 + \cdots + a_m^* \varphi_m.$$

下面证明 $\varphi^*(x)$ 使 I 取最小值. 由式 (3.31) 有

$$\begin{aligned} (y - \varphi^*, \varphi^* - \varphi) &= (y - \sum_{k=0}^m a_k^* \varphi_k, \sum_{j=0}^m (a_j^* - a_j) \varphi_j) \\ &= \sum_{j=0}^m (a_j^* - a_j) \left[(y, \varphi_j) - \sum_{k=0}^m a_k^* (\varphi_k, \varphi_j) \right] = 0. \end{aligned}$$

因此

$$\begin{aligned} \|y - \varphi\|_2^2 &= \|y - \varphi^* + \varphi^* - \varphi\|_2^2 \\ &= (y - \varphi^* + \varphi^* - \varphi, y - \varphi^* + \varphi^* - \varphi) \\ &= (y - \varphi^*, y - \varphi^*) + (\varphi^* - \varphi, \varphi^* - \varphi) + 2(y - \varphi^*, \varphi^* - \varphi) \\ &= \|y - \varphi^*\|_2^2 + \|\varphi^* - \varphi\|_2^2. \end{aligned}$$

于是可知

$$\|y - \varphi\|_2^2 \geq \|y - \varphi^*\|_2^2,$$

且等号当且仅当 $\varphi = \varphi^*$ 时成立, 故 $\varphi^*(x)$ 使 I 取最小值.

由式 (3.31), 有 $(\varphi^*, y - \varphi^*) = 0$, 所以最小平方误差为

$$\begin{aligned} \delta^2 &= \|y - \varphi^*\|_2^2 = (y - \varphi^*, y - \varphi^*) \\ &= (y - \varphi^*, y) - (y - \varphi^*, \varphi^*) = \|y\|_2^2 - \sum_{j=0}^m a_j^* (\varphi_j, y). \end{aligned}$$

或

$$\delta^2 = Y^T(Y - Aa)$$

从上述推导过程可知,当 $\varphi_0, \varphi_1, \dots, \varphi_m$ 不但线性无关,若还是标准正交系,即

$$(\varphi_j, \varphi_k) = \delta_{j,k}, \quad j, k = 0, 1, 2, \dots, m$$

时,正规方程组(3.33)的系数矩阵为单位矩阵,这时

$$a_j^* = (y, \varphi_j), \quad j = 0, 1, 2, \dots, m,$$

最小平方误差为

$$\delta^2 = \|y\|_2^2 - \sum_{j=0}^m a_j^{*2}.$$

3.5.2 用多项式进行最小二乘曲线拟合

若取 $\varphi_0 = 1, \varphi_1 = x, \dots, \varphi_m = x^m$, 即取 $\{1, x, \dots, x^m\}$ 为基函数的代数多项式拟合时,相应的正规方程组(3.33)就是

$$\begin{bmatrix} \sum_{i=1}^N w_i & \sum_{i=1}^N w_i x_i & \cdots & \sum_{i=1}^N w_i x_i^m \\ \sum_{i=1}^N w_i x_i & \sum_{i=1}^N w_i x_i^2 & \cdots & \sum_{i=1}^N w_i x_i^{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^N w_i x_i^m & \sum_{i=1}^N w_i x_i^{m+1} & \cdots & \sum_{i=1}^N w_i x_i^{2m} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^N w_i y_i \\ \sum_{i=1}^N w_i x_i y_i \\ \vdots \\ \sum_{i=1}^N w_i x_i^m y_i \end{bmatrix}.$$

由此看出,只要逐个计算出方程组的系数矩阵与右端项的各个元素,就可以形成正规方程组,然后求出正规方程组的解 $a_0, a_1, \dots, a_m$, 就可得到拟合多项式

$$\varphi(x) = a_0 + a_1 x + \cdots + a_m x^m.$$

特别地,取 $w_i = 1$ 时,系数矩阵及右端项各元素为 $N, \sum_{i=1}^N x_i, \sum_{i=1}^N x_i^2, \dots, \sum_{i=1}^N x_i^m, \sum_{i=1}^N x_i^{m+1}, \dots, \sum_{i=1}^N x_i^{2m}, \sum_{i=1}^N y_i, \sum_{i=1}^N x_i y_i, \sum_{i=1}^N x_i^2 y_i, \dots, \sum_{i=1}^N x_i^m y_i$.

例 3.4 设有一组数据为表 3.1 中第(2),(3)两列所示,求用代数多项式拟合这组数据.

表 3.1

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)
i	x_i	y_i	$x_i y_i$	x_i^2	$x_i^2 y_i$	x_i^3	x_i^4
1	1	10	10	1	10	1	1
2	3	5	15	9	45	27	81
3	4	4	16	16	64	64	256
4	5	2	10	25	50	125	625

续表

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)
5	6	1	6	36	36	216	1296
6	7	1	7	49	49	343	2401
7	8	2	16	64	128	512	4096
8	9	3	27	81	243	729	6561
9	10	4	40	100	400	1000	10000
$\sum_{i=1}^9$	53	32	147	381	1025	3017	25317

解 通常可按下列步骤求解.

(1) 描草图. 由已知数据描出粗略的图形, 如图 3.3 所示, 从图看出近似为一条抛物线.

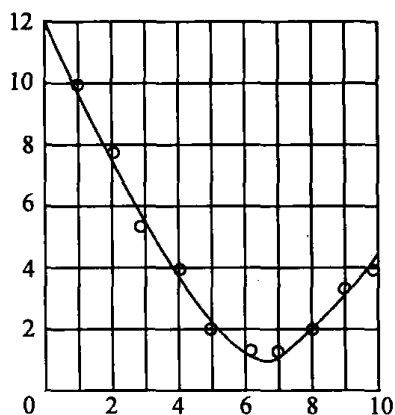


图 3.3

(2) 造型. 从草图可设拟合曲线为

$$y = \varphi(x) = a_0 + a_1x + a_2x^2.$$

(3) 建立包含未知数 a_0, a_1, a_2 的正规方程组. 为此列表算出以下各数值:

$$\sum_{i=1}^9 x_i, \sum_{i=1}^9 x_i^2, \sum_{i=1}^9 x_i^3, \sum_{i=1}^9 x_i^4, \\ \sum_{i=1}^9 y_i, \sum_{i=1}^9 x_i y_i, \sum_{i=1}^9 x_i^2 y_i.$$

这里权函数 $w(x) \equiv 1$, 由表的最后一行的数值可得正规方程组为

$$\begin{cases} 9a_0 + 53a_1 + 381a_2 = 32, \\ 53a_0 + 381a_1 + 3017a_2 = 147, \\ 381a_0 + 3017a_1 + 25317a_2 = 1025. \end{cases}$$

(4) 求解正规方程组得

$$a_0 = 13.45966, \quad a_1 = -3.60531, \quad a_2 = 0.26757,$$

故所求的二次拟合多项式为

$$y = \varphi(x) = 13.45966 - 3.60531x + 0.26757x^2.$$

例 3.5 给定数据如表 3.2 所示. 试分别用 2 次和 3 次多项式以最小二乘法拟合表中的数据, 并比较优劣.

表 3.2

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-0.1	0.1	0.4	0.9	1.6

解 (1) 设拟合函数 $y(x) = a_1 + a_2x + a_3x^2$, 此例中权函数 $w(x) = 1$, 即 $W = \text{diag}(1, 1, 1, 1, 1)$. 利用法方程组(3.34)

$$A^T A a = A^T Y.$$

这里,

$$\varphi_0(x) = 1, \quad \varphi_1(x) = x, \quad \varphi_2(x) = x^2,$$

$$a = (a_1, a_2, a_3), \quad Y = (-0.1, 0.1, 0.4, 0.9, 1.6)^T,$$

$$A = \begin{bmatrix} 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix}, \quad A^T A = \begin{bmatrix} 5 & 0 & 10 \\ 0 & 10 & 0 \\ 10 & 0 & 34 \end{bmatrix}, \quad A^T Y = \begin{bmatrix} 2.9 \\ 4.2 \\ 7 \end{bmatrix},$$

即法方程组为

$$\begin{bmatrix} 5 & 0 & 10 \\ 0 & 10 & 0 \\ 10 & 0 & 34 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 2.9 \\ 4.2 \\ 7 \end{bmatrix}.$$

解之得

$$a_1 = 0.4086, \quad a_2 = 0.42, \quad a_3 = 0.0857,$$

故所求的 2 次多项式为

$$y(x) = 0.4086 + 0.42x + 0.0857x^2.$$

误差平方和为

$$\delta_2^2 = Y^T(Y - Aa) = 0.00116.$$

(2) 设拟合函数 $y(x) = a_1 + a_2x + a_3x^2 + a_4x^3$, 利用法方程的矩阵形式(3.34), 则其中 $\varphi_0(x) = 1, \varphi_1(x) = x, \varphi_2(x) = x^2, \varphi_3(x) = x^3, a = (a_1, a_2, a_3, a_4)$. 于是有

$$A = \begin{bmatrix} 1 & -2 & 4 & -8 \\ 1 & -1 & 1 & -1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{bmatrix}, \quad A^T A = \begin{bmatrix} 5 & 0 & 10 & 0 \\ 0 & 10 & 0 & 34 \\ 10 & 0 & 34 & 0 \\ 0 & 34 & 0 & 130 \end{bmatrix},$$

$$A^T Y = (2.9, 4.2, 7, 14.4)^T,$$

即法方程组为

$$\begin{bmatrix} 5 & 0 & 10 & 0 \\ 0 & 10 & 0 & 34 \\ 10 & 0 & 34 & 0 \\ 0 & 34 & 0 & 130 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} = \begin{bmatrix} 2.9 \\ 4.2 \\ 7 \\ 14.4 \end{bmatrix}.$$

解之得

$$a_1 = 0.4086, \quad a_2 = 0.39167, \quad a_3 = 0.0857, \quad a_4 = 0.00833.$$

得所求的3次多项式为

$$y(x) = 0.4086 + 0.39167x + 0.0857x^2 + 0.0083x^3.$$

误差平方和为

$$\delta_3^2 = Y^T(Y - Aa) = 0.000626.$$

由于 $\delta_3^2 < \delta_2^2$, 所以, 用3次多项式拟合所给数据优于用2次多项式去拟合.

例 3.6 求矛盾方程的最小二乘解, 已知

$$\begin{cases} x_1 - x_2 = 1, \\ -x_1 + x_2 = 2, \\ 2x_1 - 2x_2 = 3, \\ -3x_1 + x_2 = 4. \end{cases}$$

解 依据最小二乘原理, 应使偏差的平方和

$$I(x_1, x_2) = (x_1 - x_2 - 1)^2 + (-x_1 + x_2 - 2)^2 + (2x_1 - 2x_2 - 3)^2 + (-3x_1 + x_2 - 4)^2$$

达到最小. 令

$$\frac{\partial I}{\partial x_1} = 0, \quad \frac{\partial I}{\partial x_2} = 0$$

即得法方程组

$$\begin{bmatrix} 15 & -9 \\ -9 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} -7 \\ -1 \end{bmatrix}.$$

解之得

$$x_1 = -29/12, \quad x_2 = -39/12.$$

例 3.7 求形如 $y = ae^{bx}$ (a, b 为常数) 的经验公式, 使它能和表 3.3 给出的数据相拟合.

表 3.3

x	1	2	3	4	5	6	7	8
y	15.3	20.5	27.4	36.6	49.1	65.5	87.8	117.6

解 对经验公式两边取常用对数得

$$\lg y = \lg a + bx \lg e,$$

作代换

$$u = \lg y, \quad A = \lg a, \quad B = b \lg e,$$

则有

$$u = A + Bx.$$

这样将原来的指数型拟合问题转化为了一次多项式拟合问题, 为了得出正规方程组, 需算出以下数值

$$\sum_{i=1}^8 x_i, \quad \sum_{i=1}^8 u_i = \sum_{i=1}^8 \lg y_i, \quad \sum_{i=1}^8 x_i u_i, \quad \sum_{i=1}^8 x_i^2.$$

算出这些数值如表 3.4 所示,得出正规方程组为

$$\begin{cases} 8A + 36B = 13.0189, \\ 36A + 204B = 63.89595. \end{cases}$$

求解得 $A = 1.0583375, B = 0.12645.$

由此得 $a = 11.43767, b = 0.291162.$

因此,所求的经验公式为

$$y = 11.43767e^{0.291162x}.$$

表 3.4

x_i	y_i	$\lg y_i$	$x_i \lg y_i$	x_i^2
1	15.3	1.184691	1.184691	1
2	20.5	1.311754	2.623508	4
3	27.4	1.437751	4.313252	9
4	36.6	1.563481	6.253924	16
5	49.1	1.691081	8.455407	25
6	65.5	1.816241	10.89745	36
7	87.8	1.943495	13.60446	49
8	117.6	2.070407	16.56326	64
$\sum_{i=1}^8 = 36$	419.8	13.01890	63.89595	204

3.5.3 正交多项式在曲线拟合中的应用

为了避免多项式拟合时求解病态法方程组,与 3.4 节的讨论相同,一个可行的办法是在子空间

$$\Phi = \text{span}\{1, x, x^2, \dots, x^m\}$$

中寻找另一组基函数 $\{\varphi_k(x)\}_{k=0}^m$,使得它们关于点列 $\{x_i\}_{i=1}^N$ 正交,即向量组

$$\varphi_k = (\varphi_k(x_1), \varphi_k(x_2), \dots, \varphi_k(x_N))^T, \quad k = 0, 1, 2, \dots, m \quad (3.35)$$

具有正交性

$$(\varphi_k, \varphi_l) = \begin{cases} 0, & k \neq l, \\ A_k \neq 0, & k = l, \end{cases} \quad k, l = 0, 1, 2, \dots, m. \quad (3.36)$$

这时,法方程(3.33),法方程(3.34)的系数矩阵为对角阵

$$G = \text{diag}\{(\varphi_0, \varphi_0), (\varphi_1, \varphi_1), \dots, (\varphi_m, \varphi_m)\},$$

令 $y = (y_1, y_2, \dots, y_N)^T$,从而法方程简化为

$$(\varphi_k, \varphi_k)a_k = (y, \varphi_k), \quad k = 0, 1, 2, \dots, m.$$

它的解分量仅用简单的除法即可获得,即

$$a_k^* = \frac{(y, \varphi_k)}{(\varphi_k, \varphi_k)}, \quad k = 0, 1, 2, \dots, m. \quad (3.37)$$

于是,离散数据 $\{x_i, y_i\}_{i=1}^N$ 的多项式拟合为

$$\varphi^*(x) = \sum_{k=0}^m a_k^* \varphi_k(x) = \sum_{k=0}^m \frac{(y, \varphi_k)}{(\varphi_k, \varphi_k)} \varphi_k(x). \quad (3.38)$$

若记 $\varphi^*(x)$ 在点列 $\{x_i\}_{i=0}^m$ 处的值向量为

$$\boldsymbol{\varphi}^* = (\varphi^*(x_1), \varphi^*(x_2), \dots, \varphi^*(x_N))^T,$$

则 $\boldsymbol{\varphi}^* = \sum_{k=0}^m a_k^* \boldsymbol{\varphi}_k$. 根据正交性(3.36)和关系式(3.37), 有

$$\|\boldsymbol{\varphi}^*\|_2^2 = (\boldsymbol{\varphi}^*, \boldsymbol{\varphi}^*) = \sum_{k=0}^m (a_k^*)^2 (\boldsymbol{\varphi}_k, \boldsymbol{\varphi}_k) = \sum_{k=0}^m a_k^* (y, \boldsymbol{\varphi}_k).$$

于是, 得到平方误差

$$\delta^2 = \|y - \boldsymbol{\varphi}^*\|_2^2 = \|y\|_2^2 - \sum_{k=0}^m a_k^* (y, \boldsymbol{\varphi}_k) = \|y\|_2^2 - \|\boldsymbol{\varphi}^*\|_2^2. \quad (3.39)$$

这样的基函数 $\{\varphi_k(x)\}_{k=0}^m$ 并不难得到, 如果 $m < N+1$, 并且点列 $\{x_i\}_{i=1}^N$ 中至少有 $(m+1)$ 个两两不相等, 那么根据正交多项式性质 3 可用 3 项递推公式求出多项式序列 $\{\varphi_k(x)\}_{k=0}^m$, 它们是子空间 $\Phi = \text{span}\{1, x, x^2, \dots, x^m\}$ 的一组基, 并且满足关于点列 $\{x_i\}_{i=1}^N$ 的正交性(3.36), 得到多项式序列 $\{\varphi_k(x)\}_{k=0}^m$ 后, 就可用式(3.35), 式(3.37)~式(3.39), 求出离散数据 $\{x_i, y_i\}_{i=1}^N$ 的拟合多项式 $\varphi^*(x)$, 以及平方误差 δ^2 . 这种方法称为正交多项式拟合.

例 3.8 给定数据表 3.5, 用正交化方法求数据的二次多项式拟合.

表 3.5

i	1	2	3	4	5
x_i	0	0.25	0.5	0.75	1
y_i	0.10	0.35	0.81	1.09	1.96
w_i	1	1	1	1	1

解 这里 $N=5, m=2, m < N+1$. 令 $\varphi_0(x)=1$, 由此得

$$\boldsymbol{\varphi}_0 = \{\varphi_0(x_i)\}_{i=1}^N = (1, 1, 1, 1, 1)^T,$$

$$(\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_0) = \sum_{i=1}^N w_i [\varphi_0(x_i)]^2 = 5, \quad (x\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_0) = \sum_{i=1}^N w_i x_i [\varphi_0(x_i)]^2 = 2.5,$$

$$a_1 = \frac{(x\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_0)}{(\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_0)} = 0.5.$$

于是

$$\varphi_1(x) = x - a_1 = x - 0.5.$$

又

$$\boldsymbol{\varphi}_1 = \{\varphi_1(x_i)\}_{i=1}^N = (-0.5, -0.25, 0, 0.25, 0.5)^T,$$

$$(\boldsymbol{\varphi}_1, \boldsymbol{\varphi}_1) = \sum_{i=1}^N w_i [\varphi_1(x_i)]^2 = 0.625, \quad (x\boldsymbol{\varphi}_1, \boldsymbol{\varphi}_1) = \sum_{i=1}^N w_i x_i [\varphi_1(x_i)]^2 = 0.3125,$$

$$a_2 = \frac{(x\boldsymbol{\varphi}_1, \boldsymbol{\varphi}_1)}{(\boldsymbol{\varphi}_1, \boldsymbol{\varphi}_1)} = 0.5, \quad b_1 = \frac{(\boldsymbol{\varphi}_1, \boldsymbol{\varphi}_0)}{(\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_0)} = 0.125.$$

从而 $\varphi_2(x) = (x - a_2)\varphi_1(x) - b_1\varphi_0(x) = (x - 0.5)^2 - 0.125$,

进而有 $\varphi_2 = (0.125, -0.0625, -0.125, -0.0625, 0.125)^T$,

$$(\varphi_2, \varphi_2) = 0.0546875,$$

$$(y, \varphi_0) = \sum_{i=1}^N w_i y_i \varphi_0(x_i) = 4.31, \quad a_0^* = \frac{(y, \varphi_0)}{(\varphi_0, \varphi_0)} = 0.862,$$

$$(y, \varphi_1) = \sum_{i=1}^N w_i y_i \varphi_1(x_i) = 1.115, \quad a_1^* = \frac{(y, \varphi_1)}{(\varphi_1, \varphi_1)} = 1.784,$$

$$(y, \varphi_2) = \sum_{i=1}^N w_i y_i \varphi_2(x_i) = 0.06625, \quad a_2^* = \frac{(y, \varphi_2)}{(\varphi_2, \varphi_2)} = 1.211428571.$$

最后得到拟合多项式

$$\begin{aligned} \varphi^*(x) &= a_0^* \varphi_0(x) + a_1^* \varphi_1(x) + a_2^* \varphi_2(x) \\ &= 0.862 + 1.784(x - 0.5) + 1.2114[(x - 0.5)^2 - 0.125] \\ &= 0.1214 + 0.5726x + 1.2114x^2. \end{aligned}$$

由 $\|y\|_2^2 = (y, y) = 5.8183$,

可求出平方误差

$$\delta^2 = 0.0337.$$

3.6 最佳一致逼近多项式及其求法

3.6.1 最佳一致逼近多项式

定义 3.8 设 $f \in C[a, b]$, $p_n \in H_n = \text{span}\{1, x, \dots, x^n\}$, 称

$$\Delta(f, p_n) = \|f - p_n\|_\infty = \max_{a \leq x \leq b} |f(x) - p_n(x)| \quad (3.40)$$

为 p_n 与 f 的偏差;

$$E_n = \inf_{p_n \in H_n} \Delta(f, p_n) = \min_{p_n \in H_n} \Delta(f, p_n) \quad (3.41)$$

称为 p_n 与 f 的最小偏差.

定义 3.9 设 $f \in C[a, b]$, 若存在 $p_n^* \in H_n$, 使

$$\Delta(f, p_n^*) = E_n = \min_{p_n \in H_n} \Delta(f, p_n), \quad (3.42)$$

则称 p_n^* 为 f 在 $[a, b]$ 上的最佳一致逼近多项式, 简称最佳逼近多项式.

可以证明最佳逼近多项式是存在的, 即有下述定理.

定理 3.4 若 $f \in C[a, b]$, 则存在 $p_n^* \in H_n$ 使式(3.42)成立.

根据定义, 求最佳逼近多项式是很困难的, 因此需要进一步研究最佳逼近多项式特征.

定义 3.10 设 $f \in C[a, b]$, $p \in H_n$, 若 $\exists x_0 \in [a, b]$ 使

$$|f(x_0) - p(x_0)| = \Delta(f, p) = \max_{a \leq x \leq b} |f(x) - p(x)| = \mu,$$

则称 x_0 是 p 关于 f 的偏差点.

若 $p(x_0) - f(x_0) = \mu$, 则称 x_0 为正偏差点;

若 $p(x_0) - f(x_0) = -\mu$, 则称 x_0 为负偏差点.

下面讨论最佳逼近多项式的偏差点性质.

定理 3.5 若 $p^*(x) \in H_n$ 是 $f(x) \in C[a, b]$ 的最佳逼近多项式, 则 $p^*(x)$ 同时存在正、负偏差点.

证明 因 $p^*(x)$ 是 $f(x)$ 的最佳逼近多项式, 故 $\mu = E_n$. 由于 $p^*(x)$ 在 $[a, b]$ 上总有偏差点存在, 故可用反证法求证定理. 不妨假定只有正偏差点, 没有负偏差点, 于是, 对于一切 $x \in [a, b]$ 都有

$$p^*(x) - f(x) > -E_n.$$

因 $p^*(x) - f(x)$ 在 $[a, b]$ 上连续, 故有最小值大于 $-E_n$, 用 $-E_n + 2h$ 表示, 其中 $h > 0$. 于是, 对于一切 $x \in [a, b]$ 都有

$$-E_n + 2h \leq p^*(x) - f(x) \leq E_n,$$

$$-E_n + h \geq [p^*(x) - h] - f(x) \leq E_n - h,$$

即

$$|[p^*(x) - h] - f(x)| \leq E_n - h.$$

它表示多项式 $p^*(x) - h$ 与 $f(x)$ 的偏差小于 E_n , 与 E_n 是最小偏差的假定矛盾.

同样, 可证明只有负偏差点没有正偏差点也是不成立的. 证毕.

定理 3.5 的证明从几何上看是十分明显的. 如图 3.4 所示, 考察两曲线

$$y = f(x) + E_n \quad \text{与} \quad y = f(x) - E_n$$

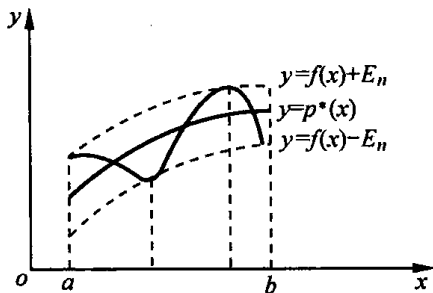


图 3.4

在 $[a, b]$ 间形成带状区域, 曲线 $y = p^*(x)$ 在 $[a, b]$ 上就位于这带状区域之间. 定理 3.5 表明, $p^*(x)$ 的图形应当与这两条曲线至少各接触一次, 这是十分明显的. 若不与 $y = f(x) - E_n$ 接触, 则可将曲线 $y = p^*(x)$ 稍向下移动就得到位于曲线 $y = f(x)$ 较窄带状区域内的曲线 $y = p^*(x) - h$.

下面给出反映最佳逼近多项式特征的 Chebyshev 定理.

定理 3.6 (Chebyshev 定理) $p_n^* \in H_n$ 是 $f \in C[a, b]$ 的最佳逼近多项式的充分必要条件是, 在 $[a, b]$ 上至少有 $n+2$ 个轮流为正负的偏差点, 即至少有 $n+2$ 个点 $a \leq x_1 < x_2 < \cdots < x_{n+2} \leq b$, 使得

$$p_n^*(x_k) - f(x_k) = (-1)^k \sigma \|f - p_n^*\|_\infty, \quad \sigma = \pm 1, \quad k = 1, 2, \cdots, n+2. \quad (3.43)$$

上述点 $\{x_k\}_{k=1}^{n+2}$ 称为 Chebyshev 交错点组.

证明 只证充分性.

假设 $p_n^*(x) \in H_n$ 在 $[a, b]$ 上有 $n+2$ 个点 $a \leq x_1 < x_2 < \cdots < x_{n+2} \leq b$, 使式(3.43)成立. 用反证法, 假定 p_n^* 不是 f 的最佳逼近多项式, 则由定理 3.4 知, 存在 $Q \in H_n$, 使得

$$E_n = \|Q - f\|_\infty < \|p_n^* - f\|_\infty.$$

由于 $Q - p_n^* = (f - p_n^*) - (f - Q) \in H_n$, 由式(3.43)可知, 在点 $x_1, x_2, \dots, x_{n+2}$ 上 $Q - p_n^*$ 符号与 $f - p_n^*$ 相同, 于是 $Q - p_n^*$ 在点 $x_1, \dots, x_{n+2}$ 上轮流取正负号, 故它在 (a, b) 内有 $n+1$ 个零点. 另一方面, H_n 中的任何多项式最多只有 n 个零点, 这一矛盾表明只能 $Q = p_n^*$. 证毕.

定理 3.6 说明用 p_n^* 逼近 f 的误差曲线 $y = p_n^* - f$ 是均匀分布的, 由这一定理还可得到以下重要推论.

推论 1 (唯一性定理) 设 $f \in C[a, b]$, 则在 H_n 中的最佳逼近多项式是唯一的.

证明 设 $p_n^*, q_n^* \in H_n$ 均为 f 的最佳逼近多项式, 由于

$$-E_n \leq p_n^*(x) - f(x) \leq E_n, \quad -E_n \leq q_n^*(x) - f(x) \leq E_n,$$

于是

$$E_n \leq \left\| f - \frac{p_n^* + q_n^*}{2} \right\|_\infty \leq \frac{1}{2} \left\| f - p_n^* \right\|_\infty + \frac{1}{2} \left\| f - q_n^* \right\|_\infty \leq E_n.$$

它表明 $r(x) = \frac{p_n^*(x) + q_n^*(x)}{2}$ 也是 f 的最佳逼近多项式, 于是由定理 3.6 知, $r(x)$ 关于 f 的 $n+2$ 个偏差点 $x_1, \dots, x_{n+2}$ 有

$$r(x_k) - f(x_k) = (-1)^k \sigma E_n, \quad \sigma = \pm 1; \quad k = 1, 2, \dots, n+2,$$

从而得到

$$E_n = |r(x_k) - f(x_k)| \leq \frac{1}{2} |p_n^*(x_k) - f(x_k)| + \frac{1}{2} |q_n^*(x_k) - f(x_k)| = E_n.$$

于是有

$$|p_n^*(x_k) - f(x_k)| = E_n, \quad |q_n^*(x_k) - f(x_k)| = E_n,$$

它表明 $x_1, \dots, x_{n+2}$ 也是 p_n^* 和 q_n^* 关于 f 的偏差点.

$$\text{由于} \quad \left| \frac{1}{2} [p_n^*(x_k) - f(x_k)] + \frac{1}{2} [q_n^*(x_k) - f(x_k)] \right| = E_n,$$

故 $p_n^*(x_k) - f(x_k)$ 与 $q_n^*(x_k) - f(x_k)$ 同号, 于是

$$p_n^*(x_k) - f(x_k) = q_n^*(x_k) - f(x_k) = (-1)^k \sigma E_n, \quad k = 1, 2, \dots, n+2,$$

因此有 $p_n^*(x_k) = q_n^*(x_k)$, $k = 1, 2, \dots, n+2$, 故 $p_n^*(x) \equiv q_n^*(x)$. 证毕.

推论 2 若 $f \in C[a, b]$, 则其最佳逼近多项式 $p_n^*(x) \in H_n$ 是 f 的一个 Lagrange 插值多项式.

证明 由定理 3.6 知, 在 $[a, b]$ 上有 $n+2$ 个点 $\{x_k\}_{k=1}^{n+2}$, 使

$$p_n^*(x_k) - f(x_k) = (-1)^k \sigma E_n, \quad k = 1, 2, \dots, n+2.$$

于是, 存在 $y_k \in (x_k, x_{k+1}), k=1, 2, \dots, n+1$, 使 $p_n^*(y_k) = f(y_k)$, 从而以 $\{y_k\}_{k=1}^{n+1}$ 为插值点得到的 Lagrange 插值多项式就是 $p_n^*(x)$.

定理 3.6 从理论上给出了求最佳一致逼近多项式的方法, 但当 $n \geq 2$ 时计算是很困难的. Remes(里姆斯)给出了一种逐次逼近的算法.

设 $f(x) \in C[a, b]$ 的最佳逼近多项式为

$$p_n(x) = \sum_{k=0}^n a_k x^k,$$

又设它的 $n+2$ 个点 $a \leq x_1 < x_2 < \dots < x_{n+2} \leq b$ 为 Chebyshev 交错点组, 最小偏差为 E_n , 由定理 3.6 可得方程组

$$\begin{cases} \sum_{k=0}^n a_k x_j^k - f(x_j) = (-1)^j \sigma E_n, & \sigma = \pm 1, \\ (x_1 - a)(x_{n+2} - b)[f'(x_j) - p'_n(x_j)] = 0, \end{cases} \quad j = 1, 2, \dots, n+2. \quad (3.44)$$

这是关于 $x_1, \dots, x_{n+2}, a_0, \dots, a_n, E_n$ 的 $2n+4$ 个未知量的 $2n+4$ 个方程的方程组, 方程组(3.44)第二式表示当 $f'(x) - p'_n(x) = 0$ 只有 n 个根时, $x_1 = a, x_{n+2} = b$. 方程组(3.44)是一个很复杂的非线性方程组, 且在 $n \geq 2$ 时很难求解. Remes 算法是构造偏差点组 $\{x_j^{(i)}\}_{j=1}^{n+2} (i=0, 1, 2, \dots)$ 逐次逼近 Chebyshev 交错点组 $\{x_j\}_{j=1}^{n+2}$, 当 $i \rightarrow \infty$ 时, 相应 $p_n^{(i)}(x)$ 的系数 $a_k^{(i)} \rightarrow a_k, k=0, 1, 2, \dots, n$. 但是此法计算非常复杂, 故很少用. 只当 $n=0$ 及 $n=1$ 时才能由式(3.44)求出最佳逼近多项式, $n \geq 2$ 时通常可用 Chebyshev 多项式性质求近似的最佳逼近多项式.

下面讨论 $n=1$ 时求 $p_1(x) = a_0 + a_1 x$ 的方法. 设 $f \in C^2[a, b]$ 且 f'' 在 $[a, b]$ 上不变号, 由定理 3.6 可知, 存在点 $a \leq x_1 < x_2 < x_3 \leq b$, 使

$$p_1(x_k) - f(x_k) = (-1)^k \sigma E_n, \quad k = 1, 2, 3; \quad \sigma = \pm 1.$$

由于 $f'' \neq 0$, 故 f' 在 $[a, b]$ 上单调, 于是 $f'(x) - p'_1(x) = f'(x) - a_1 = 0$ 在 $[a, b]$ 上只有一个根 x_2 . p_1 对 f 另两个偏差点只能在 $[a, b]$ 的端点, 故有

$$x_1 = a, \quad x_3 = b,$$

由此可得

$$p_1(a) - f(a) = -[p_1(x_2) - f(x_2)] = p_1(b) - f(b),$$

$$\text{或} \quad a_0 + a_1 a - f(a) = a_0 + a_1 b - f(b), \quad (3.45)$$

$$a_0 + a_1 a - f(a) = -[a_0 + a_1 x_2 - f(x_2)]. \quad (3.46)$$

由式(3.45)得

$$a_1 = \frac{f(b) - f(a)}{b - a}, \quad (3.47)$$

代入式(3.46)得

$$a_0 = \frac{1}{2}[f(a) + f(x_2)] - \frac{a+x_2}{2} \frac{f(b) - f(a)}{b-a}, \quad (3.48)$$

其中 x_2 由 $f'(x_2) = a_1$ 得到, 由此得 f 在 $[a, b]$ 上的最佳一次逼近多项式 $p_1(x) = a_0 + a_1x$. 其几何意义如图 3.5 所示.

例 3.9 设 $f(x) = \sqrt{1+x^2}$, 在 $[0, 1]$ 上求最佳一次逼近多项式 $p_1(x) = a_0 + a_1x$.

解 由式(3.47)求得

$$a_1 = \frac{\sqrt{2}-1}{1} \approx 0.414,$$

由 $f'(x) = \frac{x}{\sqrt{1+x^2}}$, 得到 $f'(x_2) = \frac{x_2}{\sqrt{1+x_2^2}} = 0.414$, 于是

$$x_2 = \sqrt{\frac{\sqrt{2}-1}{2}} = 0.4551, \quad f(x_2) = \sqrt{1+x_2^2} = 1.0987.$$

由式(3.48)可求得 $a_0 = \frac{1+\sqrt{1+x_2^2}}{2} - a_1 \frac{x_2}{2} = 0.955$, 于是得 f 的最佳逼近一次多项式为

$$p_1(x) = 0.955 + 0.414x,$$

即

$$\sqrt{1+x^2} \approx 0.955 + 0.414x, \quad 0 \leq x \leq 1;$$

误差限为

$$\max_{0 \leq x \leq 1} |\sqrt{1+x^2} - p_1(x)| \leq 0.0452.$$

这里可顺便得到一个求根式的公式

$$\sqrt{a^2 + b^2} \approx 0.955a + 0.414b.$$

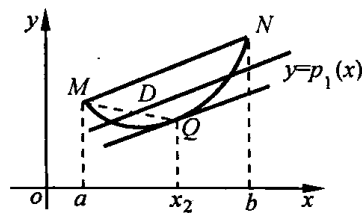


图 3.5

3.6.2 最佳一致逼近多项式的近似解法

通常, 在讨论连续函数的多项式逼近中, 要为某一连续函数求取某种最佳一致逼近多项式常常是十分困难的. 因此, 在实际使用中, 往往采用一个近似的多项式来代替.

下面, 介绍两种最佳一致逼近多项式的近似求法, 它们都要用到 Chebyshev 多项式.

1. Chebyshev 插值法

由定理 3.6 可以看出, 若 $p_n^*(x)$ 是 $f(x)$ 的最佳一致逼近多项式, 那么连续函数 $f(x) - p_n^*(x)$ 在区间 $[a, b]$ 上至少变号 $n+1$ 次, 从而至少有 $n+1$ 个点 $x_i \in [a, b]$ ($i = 0, 1, 2, \dots, n$), 使

$$p_n^*(x_i) = f(x_i), \quad i = 0, 1, 2, \dots, n,$$

也就是说, $p_n^*(x)$ 实际上是以 x_i ($i = 0, 1, 2, \dots, n$) 为节点的 $f(x)$ 的 n 次插值多项式(见

定理 3.6 的推论 2). 因此, 如果能适当选择节点, 求出使最大误差最小的 n 次插值多项式, 那么就得到了 n 次最佳一致逼近多项式 $p_n^*(x)$.

这里, 求使最大误差最小的插值多项式的问题虽然还没有圆满地解决, 但是已给我们指出了一条求近似的 n 次最佳一致逼近多项式的道路. 因为根据插值余项公式有

$$\max_{x \in [a, b]} |R_n(x)| = \max_{x \in [a, b]} |f(x) - p_n(x)| = \max_{x \in [a, b]} \left| \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) \right|.$$

若记

$$M_{n+1} = \max_{x \in [a, b]} |f^{(n+1)}(x)|,$$

则

$$\max_{x \in [a, b]} |R_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \max_{x \in [a, b]} |\omega_{n+1}(x)|.$$

此式表明, 只要能选取使

$$\max_{x \in [a, b]} |\omega_{n+1}(x)| = \max_{x \in [a, b]} |(x-x_0)(x-x_1)\cdots(x-x_n)|$$

取到最小值的点 $x_0, x_1, \dots, x_n$, 那么以这些点为节点的插值多项式 $p_n(x)$ 就可以作为 n 次最佳一致逼近多项式 $p_n^*(x)$ 的近似了.

为了解决节点 $x_i (i=0, 1, 2, \dots, n)$ 的求法, 先证明 Chebyshev 多项式的一个重要性质.

定理 3.7 在区间 $[-1, 1]$ 上所有最高次项系数为 1 的 n 次多项式中, $\frac{1}{2^{n-1}} T_n(x)$ 的最大绝对值最小, 即

$$\max_{x \in [-1, 1]} \left| \frac{1}{2^{n-1}} T_n(x) \right| = \min_{p_n(x) \in \tilde{H}_n} \max_{x \in [-1, 1]} |p_n(x)|, \quad (3.49)$$

其中, $\tilde{H}_n$ 表示由所有最高次项系数为 1 的 n 次多项式所组成的一个集合.

证明 由于 $\frac{1}{2^{n-1}} T_n(x)$ 与 $p_n(x)$ 都是首项系数为 1 的 n 次多项式, 故可分别表示为

$$\frac{1}{2^{n-1}} T_n(x) = x^n - p_{n-1}^*(x),$$

$$p_n(x) = x^n - p_{n-1}(x),$$

其中, $p_{n-1}^*(x) = x^n - \frac{1}{2^{n-1}} T_n(x)$ 是一个特定的 $n-1$ 次多项式, $p_{n-1}(x) = x^n - p_n(x)$ 是任意一个 $n-1$ 次多项式. 于是, 式(3.49)可以改写为

$$\max_{x \in [-1, 1]} |x^n - p_{n-1}^*(x)| = \min_{p_{n-1}(x) \in \tilde{H}_{n-1}} \max_{x \in [-1, 1]} |x^n - p_{n-1}(x)|.$$

因此, 由定义 3.9 知, 证明式(3.49)等价于证明 $p_{n-1}^*(x)$ 在 $[-1, 1]$ 上是函数 $f(x) = x^n$ 的 $n-1$ 次最佳一致逼近多项式, 这是容易办到的. 事实上, 有

$$\max_{x \in [-1, 1]} |x^n - p_{n-1}^*(x)| = \max_{x \in [-1, 1]} \left| \frac{1}{2^{n-1}} T_n(x) \right| = \frac{1}{2^{n-1}},$$

而且在 $n+1$ 个点

$$x_k = \cos \frac{k\pi}{n}, \quad k = 0, 1, 2, \dots, n$$

上有

$$x_k^n - p_{n-1}^*(x_k) = \frac{1}{2^{n-1}} T_n(x_k) = \frac{1}{2^{n-1}} \cos k\pi = (-1)^k \frac{1}{2^{n-1}}, \quad k = 0, 1, 2, \dots, n.$$

于是由 Chebyshev 定理, 立即得到所要证明的结论.

定理 3.7 也给出了某些特殊函数的最佳一致逼近多项式的构造方法.

例 3.10 求 $f(x) = 2x^3 + x^2 + 2x - 1$ 在 $[-1, 1]$ 上的最佳 2 次逼近多项式.

解 由题意, 所求最佳逼近多项式 $p_2^*(x)$ 应满足

$$\max_{-1 \leq x \leq 1} |f(x) - p_2^*(x)| = \min.$$

由定理 3.7 可知, 当

$$f(x) - p_2^*(x) = \frac{1}{2} T_3(x) = 2x^3 - \frac{3}{2}x$$

时, 多项式 $f(x) - p_2^*(x)$ 与零偏差最小, 故

$$p_2^*(x) = f(x) - \frac{1}{2} T_3(x) = x^2 + \frac{7}{2}x - 1.$$

这就是 $f(x)$ 在 $[-1, 1]$ 上的最佳 2 次逼近多项式.

定理 3.7 说明, 在所讨论的区间是 $[-1, 1]$ 的情况下, 当

$$\omega_{n+1}(x) = \frac{1}{2^n} T_{n+1}(x)$$

时, $\max_{x \in [-1, 1]} |\omega_{n+1}(x)|$ 取到最小值. 也就是说, 选取的节点应是 $n+1$ 次 Chebyshev 多项式的零点

$$x_k = \cos \frac{2k+1}{2(n+1)}\pi, \quad k = 0, 1, 2, \dots, n. \quad (3.50)$$

此时所得插值多项式的最大误差满足不等式

$$\max_{x \in [-1, 1]} |R_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \cdot \frac{1}{2^n}. \quad (3.51)$$

如果所讨论的是任一有限区间 $[a, b]$, 只要通过变量替换

$$x = \frac{a+b}{2} + \frac{b-a}{2}t,$$

使

$$\begin{aligned} \omega_{n+1}(x) &= (x-x_0)(x-x_1)\cdots(x-x_n) \\ &= \left(\frac{b-a}{2}\right)^{n+1} (t-t_0)(t-t_1)\cdots(t-t_n), \end{aligned}$$

其中, t_i 与 x_i 相对应. 容易看出, $(t-t_0)(t-t_1)\cdots(t-t_n)$ 是 $[-1, 1]$ 上最高次项系数为 1 的 $n+1$ 次多项式, 为了使其最大绝对值取到最小值, 应取

$$t_k = \cos \frac{2k+1}{2(n+1)}\pi, \quad k = 0, 1, 2, \dots, n$$

为节点.也就是说,当节点

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos \frac{2k+1}{2(n+1)}\pi, \quad k = 0, 1, 2, \dots, n \quad (3.52)$$

时, $\max_{x \in [a, b]} |\omega_{n+1}(x)|$ 取到最小值, 并且有

$$\max_{x \in [a, b]} |\omega_{n+1}(x)| \leq \left(\frac{b-a}{2}\right)^{n+1} \frac{1}{2^n}.$$

此时所得插值多项式的最大误差满足不等式

$$\max_{x \in [a, b]} |R_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \left(\frac{b-a}{2}\right)^{n+1} \frac{1}{2^n} = \frac{(b-a)^{n+1}}{2^{2n+1}} \cdot \frac{M_{n+1}}{(n+1)!}. \quad (3.53)$$

例 3.11 对于函数 $f(x) = xe^x (x \in [0, 1.5])$, 用 3 种不同方法求出 3 次逼近多项式.

(1) 先将 $f(x)$ 展开成 x 的幂级数

$$xe^x = x + x^2 + \frac{1}{2!}x^3 + \dots + \frac{1}{n!}x^{n+1} + \dots,$$

然后取前面 3 项之和 $S_3(x)$ 作为 xe^x 的近似, 得

$$xe^x \approx S_3(x) = x + x^2 + \frac{1}{2}x^3.$$

(2) 以 $x_0=0, x_1=0.5, x_2=1.0, x_3=1.5$ 为节点, 做 $f(x)$ 的 3 次插值多项式,

$$p_3(x) = 1.3877x^3 + 0.05758x^2 + 1.2730x,$$

于是有

$$xe^x \approx p_3(x) = 1.3877x^3 + 0.05758x^2 + 1.2730x.$$

(3) 做 3 次 Chebyshev 插值. 由公式(3.52)可以算出该插值多项式相应的节点:

$$x_0 = \frac{1.5}{2} + \frac{1.5}{2} \cos \frac{\pi}{8} = 1.44291,$$

$$x_1 = \frac{1.5}{2} + \frac{1.5}{2} \cos \frac{3\pi}{8} = 1.03701,$$

$$x_2 = \frac{1.5}{2} + \frac{1.5}{2} \cos \frac{5\pi}{8} = 0.46299,$$

$$x_3 = \frac{1.5}{2} + \frac{1.5}{2} \cos \frac{7\pi}{8} = 0.05709.$$

以 x_0, x_1, x_2, x_3 为节点作 $f(x)$ 的 3 次插值多项式, 即得

$$\tilde{p}_3(x) = 1.3811x^3 + 0.04465x^2 + 1.3031x - 0.01435.$$

于是有

$$xe^x \approx \tilde{p}_3(x) = 1.3811x^3 + 0.04465x^2 + 1.3031x - 0.01435.$$

把所得到的 3 种近似多项式在部分点上的误差列在表 3.6 中. 由表中看出, 用 Chebyshev 插值法得到的插值多项式在所讨论的区间上能比较均匀地逼近 $f(x)$, 而且准确程度也比较高, 因此用它来近似地代替 $f(x)$ 的 3 次最佳一致逼近多项式是比较合理的.

表 3.6

x	$ xe^x - S_3(x) $	$ xe^x - p_3(x) $	$ xe^x - \tilde{p}_3(x) $
0.15	0.0001	0.0227	0.0125
0.25	0.0007	0.0225	0.0148
0.35	0.0027	0.0154	0.0097
0.65	0.0353	0.0122	0.0143
0.75	0.0643	0.0152	0.0170
0.85	0.1091	0.0128	0.0150
1.15	0.3990	0.0187	0.0118
1.25	0.5739	0.0286	0.0188
1.35	0.8048	0.0302	0.0167

2. 缩减幂级数法

该法特点是:在取 $f(x)$ 的 Taylor 级数的前 $m+1$ 项 ($m > n$) 之和 $S_m(x)$ 作为 $f(x)$ 的近似的基礎上,利用 Chebyshev 多项式的值落在 $[-1, 1]$ 中的事实,以及在对函数值影响不大的情况下,将所得 m 次多项式缩减为 n 次多项式,从而得到 $f(x)$ 的一个逼近效果较好(以最大误差为标准)的 n 次多项式。

为了适应这里的需要,把 Chebyshev 多项式用 x 的乘幂来表示,见表 3.7.

利用缩减幂级数法求近似最佳一致逼近多项式的具体做法,参见下面例子.

表 3.7

$1 = T_0$
$x = T_1$
$x^2 = \frac{1}{2}(T_0 + T_2)$
$x^3 = \frac{1}{4}(3T_1 + T_3)$
$x^4 = \frac{1}{8}(3T_0 + 4T_2 + T_4)$
$x^5 = \frac{1}{16}(10T_1 + 5T_3 + T_5)$
$x^6 = \frac{1}{32}(10T_0 + 15T_2 + 6T_4 + T_6)$
$x^7 = \frac{1}{64}(35T_1 + 21T_3 + 7T_5 + T_7)$
$x^8 = \frac{1}{128}(35T_0 + 56T_2 + 28T_4 + 8T_6 + T_8)$
$x^9 = \frac{1}{256}(126T_1 + 84T_3 + 36T_5 + 9T_7 + T_9)$
$x^{10} = \frac{1}{512}(126T_0 + 210T_2 + 120T_4 + 45T_6 + 10T_8 + T_{10})$

例 3.12 求 $f(x)=e^x$ 在区间 $[-1,1]$ 上的 4 次逼近多项式.

解 先将 e^x 展开成 x 的幂级数

$$e^x = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^n}{n!} + \cdots,$$

若取前 7 项之和

$$S_6(x) = 1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3 + \frac{1}{24}x^4 + \frac{1}{120}x^5 + \frac{1}{720}x^6,$$

作为 e^x 的近似,那么

$$e^x = S_6(x) + R_6(x),$$

其中 $R_6(x)$ 为截断误差.

由高等数学知识,有

$$|R_6(x)| = \left| \frac{f^{(7)}(\xi)}{7!} x^7 \right| = \left| \frac{e^\xi}{7!} x^7 \right| \leq \frac{e}{7!} < 0.00054,$$

其中, $x, \xi \in [-1, 1]$. 利用表 3.7 之结果,可得

$$\begin{aligned} e^x &= S_6(x) + R_6(x) \\ &= \frac{2917}{2304}T_0 + \frac{217}{192}T_1 + \frac{417}{1536}T_2 + \frac{17}{384}T_3 \\ &\quad + \frac{21}{3840}T_4 + \frac{1}{1920}T_5 + \frac{1}{23040}T_6 + R_6(x). \end{aligned}$$

若把等式右端前 5 项之和记为 $S_{6 \rightarrow 4}(x)$ (下标“6→4”表示由 6 次多项式缩减得到的 4 次多项式),即

$$S_{6 \rightarrow 4}(x) = \frac{2917}{2304}T_0 + \frac{217}{192}T_1 + \frac{417}{1536}T_2 + \frac{17}{384}T_3 + \frac{21}{3840}T_4, \quad (3.54)$$

那么由 $|T_k(x)| \leq 1 (k=0, 1, 2, \cdots)$ 知,用 $S_{6 \rightarrow 4}(x)$ 去逼近 e^x 时其截断误差为

$$\begin{aligned} |e^x - S_{6 \rightarrow 4}(x)| &= \left| \frac{1}{1920}T_5 + \frac{1}{23040}T_6 + R_6(x) \right| \\ &\leq \frac{1}{1920} + \frac{1}{23040} + 0.00054 < 0.00111. \end{aligned}$$

如果用 x 的乘幂重写式(3.54)中的 Chebyshev 多项式,那么 $S_{6 \rightarrow 4}(x)$ 可以写成

$$S_{6 \rightarrow 4}(x) = 1.0000434 + 0.9973958x + 0.4992188x^2 + 0.1770833x^3 + 0.043750x^4.$$

例 3.12 中所得 4 次多项式类似于 e^x 展开式中的前 5 项之和

$$S_4(x) = 1.0000000 + 1.0000000x + 0.5000000x^2 + 0.1666667x^3 + 0.0416667x^4.$$

但是用 $S_4(x)$ 去逼近 $f(x)$ 的最大误差为 $\frac{e}{5!} \approx 0.0227$, 显然,在以最大误差作为衡量误差大小的意义下,用缩减幂级数法得到的 4 次多项式 $S_{6 \rightarrow 4}(x)$,比单纯用级数展开法得到的 4 次多项式 $S_4(x)$ 好得多. 如果在 e^x 的展开式中取更多的项之和作为 e^x 的近似,然后按上面方法缩减为 4 次多项式,那么所得 4 次逼近多项式的系数将稍有变动,而逼

近的效果可以进一步提高. 例如, 按上法得到的 4 次多项式 $S_{8 \rightarrow 4}$ 的最大误差 (估计方法同上) 为

$$|e^x - S_{8 \rightarrow 4}(x)| \leq 0.0006,$$

其准确程度几乎与 $S_8(x)$ 相同.

对于任一有限区间 $[a, b]$ 上的逼近问题, 可以通过变量替换

$$x = \frac{a+b}{2} + \frac{b-a}{2}t, \quad \text{即} \quad t = \frac{2x-a-b}{b-a},$$

转化为 $[-1, 1]$ 上的逼近问题.

上述方法也可用于求任意次逼近多项式.

习 题 3

1. $f(x), g(x) \in C^1[a, b]$, 定义

$$(1) (f, g) = \int_a^b f'(x)g'(x)dx, \quad (2) (f, g) = \int_a^b f'(x)g'(x)dx + f(a)g(a).$$

问: 它们是否构成内积?

2. 设 $\Phi_1 = \text{span}(1, x)$, $\Phi_2 = \text{span}(x^{100}, x^{101})$, 分别在 Φ_1, Φ_2 上求一元素, 使其为 $x^2 \in C[0, 1]$ 的最佳平方逼近, 并比较其结果.

3. $f(x) = |x|$ 在 $[-1, 1]$ 上定义, 求在 $\Phi = \text{span}(1, x^2, x^4)$ 上的最佳平方逼近.

4. 已知 $T_n^*(x) = T_n(2x-1)$, 证明 $\{T_n^*(x)\}$ 是在 $[0, 1]$ 上带权 $\rho = \frac{1}{\sqrt{x-x^2}}$ 的正交多项式.

5. $u_n(x) = \frac{\sin(n+1)\arccos x}{\sqrt{1-x^2}}$ 是第二类 Chebyshev 多项式. 证明: 它有递推关系

$$u_{n+1}(x) = 2xu_n(x) - u_{n-1}(x).$$

6. 将 $f(x) = \sin \frac{\pi}{2}x$ 在 $[-1, 1]$ 上按 Legendre 多项式, 求 3 次最佳平方逼近多项式, 再计算均方误差.

7. 用最小二乘法求一个形如 $y = a + bx^2$ 的经验公式, 使它与表 3.8 中的数据相拟合, 并计算均方误差.

表 3.8

x_i	19	25	31	38	44
y_i	19.0	32.3	49.0	73.3	97.8

8. 观测物体的直线运动, 得出表 3.9 中的数据:

表 3.9

时间 t/s	0	0.9	1.9	3.0	3.9	5.0
距离 s/s	0	10	30	50	80	110

求运动方程.

9. 在某化学反应里, 根据实验所得分解物的浓度与时间关系如表 3.10 所示.

表 3.10

时间 t/min	0	5	10	15	20	25
浓度 $y(\times 10^{-4})$	0	1.27	2.16	2.86	3.44	3.87
时间 t/min	30	35	40	45	50	55
浓度 $y(\times 10^{-4})$	4.15	4.37	4.51	4.58	4.62	4.64

用最小二乘拟合求 $y=F(t)$.

10. 求 $f(x)=e^x$ 在 $[0,1]$ 上的最佳一致逼近多项式.

11. 如何选取 r , 使 $p(x)=x^2+r$ 在 $[-1,1]$ 上与零的偏差最小? 是否唯一?

12. 设 $f(x)=x^4+3x^2-1$ 在 $[0,1]$ 上求 3 次最佳一致逼近多项式.

13. 在 $[-1,1]$ 上利用插值极小化, 求 $f(x)=\arctan x$ 的 3 次近似最佳一致逼近多项式.

14. 在 $[-1,1]$ 上利用幂级数项数节约求 $f(x)=\sin x$ 的 3 次逼近多项式, 使误差不超过 0.005.

15. 完成下列填空题.

(1) n 次 Chebyshev 多项式 $T_n(x)$ 在 $[-1,1]$ 中有 _____ 个不同的实零点, 其零点 $x_k =$ _____.

(2) $\{\varphi_k(x)\}_{k=0}^{\infty}$ 是区间 $[0,1]$ 上权函数 $\rho(x)=x$ 的最高项系数为 1 的正交多项式族, 其中 $\varphi_0(x)=1$, 则 $\int_0^1 x\varphi_3(x)dx =$ _____, $\varphi_1(x) =$ _____.

(3) 设 $P_n(x)$ 为 Legendre 多项式, 则 $\int_{-1}^1 P_n(x)P_m(x)dx =$ _____.

(4) 在所有首项系数为 1 的 n 次多项式中, 首项系数为 1 的 n 次 _____ 多项式在 $[-1,1]$ 上与零的平方逼近误差最小.

数值实验题 3

实验 3.1

编制以函数 $\{x^k\}_{k=0}^n$ 为基的多项式最小二乘拟合程序, 并用于对表 3.11 中的数据

作3次多项式最小二乘拟合.

表 3.11

x_i	-1.0	-0.5	0.0	0.5	1.0	1.5	2.0
y_i	-4.447	-0.452	0.551	0.048	-0.447	0.549	4.552

取权数 $w_i \equiv 1$, 求拟合曲线 $\varphi^* = \sum_{k=0}^n \alpha_k^* x^k$ 中的参数 $\{\alpha_k\}$ 、平方误差 δ^2 , 并作离散数据 $\{x_i, y_i\}$ 的拟合函数 $y = \varphi^*(x)$ 的图形.

实验 3.2

编制正交化多项式最小二乘拟合程序, 并用于求解上题中的3次多项式最小二乘拟合问题, 作拟合曲线的图形, 计算平方误差, 并与上题结果进行比较.

第4章 数值积分

实际问题中常常需要计算积分. 有些数值方法, 如微分方程和积分方程的求解, 也都和积分计算相联系.

依据人们所熟知的微积分基本定理, 对于积分

$$I = \int_a^b f(x) dx,$$

只要求出被积函数 $f(x)$ 的原函数 $F(x)$, 便有下列 Newton-Leibniz (牛顿-莱布尼兹) 公式

$$\int_a^b f(x) dx = F(b) - F(a).$$

但实际运用这种方法往往有困难, 因为大量的被积函数, 找不到用初等函数表示的原函数, 例如 $\frac{\sin x}{x} \sin x^2$ 等; 另外, 当 $f(x)$ 是由实验测量或数值计算给出的一张数据表时, Newton-Leibniz 公式也不能直接运用, 因此有必要研究积分的数值计算问题.

4.1 数值求积公式的基本概念

4.1.1 数值求积的基本思想

依据积分中值定理, 对于连续函数 $f(x)$, 在 $[a, b]$ 内存在一点 ξ , 成立

$$\int_a^b f(x) dx = (b-a)f(\xi),$$

也就是说, 底为 $(b-a)$, 而高为 $f(\xi)$ 的矩形面积恰等于所求曲边梯形的面积 I (见图 4.1). 问题在于点 ξ 的具体位置一般是不知道的, 因而难以准确地算出 $f(\xi)$ 的值. 我们称 $f(\xi)$ 为区间 $[a, b]$ 上的平均高度. 这样, 只要对平均高度 $f(\xi)$ 提供一种算法, 相应地便获得一种数值求积方法.

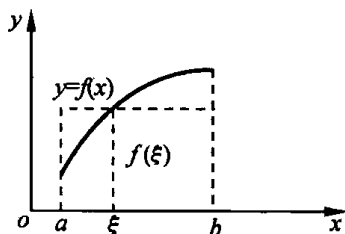


图 4.1

按照这种理解, 人们所熟知的梯形公式

$$\int_a^b f(x) dx \approx \frac{(b-a)}{2} [f(a) + f(b)] \quad (4.1)$$

中矩形公式

$$\int_a^b f(x) dx \approx (b-a) f\left(\frac{a+b}{2}\right) \quad (4.2)$$

和 Simpson (辛普森) 公式

$$\int_a^b f(x) dx \approx \frac{(b-a)}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right], \quad (4.3)$$

分别可以看作是用 $a, b, c = \frac{a+b}{2}$ 三点高度的加权平均值 $\frac{1}{2}[f(a) + f(b)]$, $f\left(\frac{a+b}{2}\right)$ 或 $\frac{1}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$ 作为平均高度 $f(\xi)$ 的近似值. 更一般地, 取 $[a, b]$ 内若干个节点 x_k 处的高度 $f(x_k)$, 通过加权平均的方法近似地得出平均高度 $f(\xi)$, 这类求积公式的一般形式是

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k), \quad (4.4)$$

式中, x_k 称为求积节点, A_k 称为求积系数, 亦称伴随节点 x_k 的权.

由于求积系数 A_k 仅仅与节点 x_k 的选取有关, 而不依赖于被积函数 $f(x)$ 的具体形式, 因此求积式(4.4)具有通用性.

这类求积方法通常称作机械求积法, 其特点是直接利用某些节点上的函数值计算积分值, 而将积分求值问题归结为函数值的计算, 这就避开了 Newton-Leibniz 公式需要寻求原函数的困难.

4.1.2 代数精度的概念

数值求积方法是近似方法, 为要保证精度, 自然希望求积公式能对“尽可能多”的函数准确地成立, 这就提出了所谓代数精度的概念.

定义 4.1 如果某个求积公式对于次数不超过 m 的多项式均能准确地成立, 但对于 $(m+1)$ 次多项式就不一定准确成立, 则称该求积公式具有 m 次代数精度.

一般地, 欲使求积式(4.4)具有 m 次代数精度, 只要令它对于 $f(x) = 1, x, \dots, x^m$ 都能准确成立, 这就要求

$$\begin{cases} \sum_{k=0}^n A_k = b-a, \\ \sum_{k=0}^n A_k x_k = \frac{1}{2}(b^2 - a^2), \\ \dots \\ \sum_{k=0}^n A_k x_k^m = \frac{1}{m+1}(b^{m+1} - a^{m+1}). \end{cases} \quad (4.5)$$

直接验证易知, 梯形式(4.1)与中矩式(4.2)均具有一次代数精度, 而 Simpson 式(4.3)则具有 3 次代数精度.

可以用代数精度作为标准来构造求积公式.

例 4.1 构造两点公式

$$\int_a^b f(x) dx \approx A_0 f(a) + A_1 f(b). \quad (4.6)$$

解 因式(4.6)中含有两个待定参数 A_0, A_1 , 为了确定 A_0, A_1 , 令式(4.6)对于 $f(x)=1, x$ 准确成立, 有

$$\begin{cases} A_0 + A_1 = b - a, \\ A_0 a + A_1 b = \frac{1}{2}(b^2 - a^2). \end{cases}$$

解之得 $A_0 = A_1 = \frac{b-a}{2}$. 这说明, 形如式(4.6)且具有一次代数精度的求积公式必为梯形公式(4.1), 这一论断从几何角度来看是十分明显的(见图 4.2).

由此还可推出如下定理.

定理 4.1 在区间 $[a, b]$ 上, 对于给定的 $(n+1)$ 个互异节点

$$a \leq x_0 < x_1 < \cdots < x_n \leq b,$$

总存在求积系数 $A_0, A_1, \cdots, A_n$, 使求积式(4.4)至少具有 n 次代数精确度.

证 在节点 $x_k (k=0, 1, 2, \cdots, n)$ 确定以后, 式(4.5)就是关于待定系数 $A_k (k=0, 1, 2, \cdots, n)$ 的线性方程组, 由于节点互异, 式(4.5)的系数行列式

$$\begin{vmatrix} 1 & 1 & \cdots & 1 \\ x_0 & x_1 & \cdots & x_n \\ \vdots & \vdots & & \vdots \\ x_0^n & x_1^n & \cdots & x_n^n \end{vmatrix} \neq 0,$$

故存在唯一解 $A_k (k=0, 1, 2, \cdots, n)$, 使以它为系数的求积公式至少具有 n 次代数精确度.

4.1.3 插值型的求积公式

设已给出 $f(x)$ 在节点 $x_k (k=0, 1, 2, \cdots, n)$ 的函数值, 作插值多项式

$$L_n(x) = \sum_{k=0}^n f(x_k) l_k(x),$$

式中

$$l_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}.$$

由于多项式 $L_n(x)$ 的求积是容易的, 我们取 $\int_a^b L_n(x) dx$ 作为 $\int_a^b f(x) dx$ 的近似值, 即令

$$\int_a^b f(x) dx \approx \int_a^b L_n(x) dx = \sum_{k=0}^n A_k f(x_k), \quad (4.7)$$

因而这类求积公式具有式(4.4)的形式, 而其求积系数

$$A_k = \int_a^b l_k(x) dx. \quad (4.8)$$

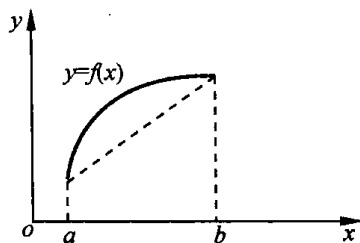


图 4.2

定义 4.2 若求积公式 $\int_a^b f(x)dx \approx \sum_{k=0}^n A_k f(x_k)$ 的求积系数 A_k 由式(4.8)确定, 则称求积公式是插值型的.

由插值余项定理知, 对于插值型求积式(4.7), 其余项为

$$\begin{aligned} R(f) &= \int_a^b f(x)dx - \sum_{k=0}^n \int_a^b l_k(x) f(x_k) dx \\ &= \int_a^b [f(x) - \sum_{k=0}^n l_k(x) f(x_k)] dx \\ &= \int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j) dx \quad (\xi \text{ 与 } x \text{ 有关}). \end{aligned} \quad (4.9)$$

从 $R(f)$ 的表达式极易推得如下定理的充分性.

定理 4.2 形如 $\int_a^b f(x)dx \approx \sum_{k=0}^n A_k f(x_k)$ 的求积公式至少有 n 次代数精度的充分必要条件为它是插值型的.

证 充分性由其余项表达式极易证得, 下证必要性.

如果求积式(4.4)至少有 n 次代数精度, 则它对于插值基函数 $l_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}$

应准确成立, 即有

$$\int_a^b l_k(x) dx = \sum_{j=0}^n A_j l_k(x_j).$$

注意到 $l_k(x_j) = \delta_{kj} = \begin{cases} 1 & k=j, \\ 0 & k \neq j, \end{cases}$ 上式右端即等于 A_k , 因而式(4.8)成立, 证毕.

例 4.2 对 $\int_0^3 f(x)dx$ 构造一个至少具有 3 次代数精度的求积公式.

解 具有 4 个求积节点的插值型求积公式, 至少有 3 次代数精度. 如果在 $[0, 3]$ 上取节点为 0, 1, 2, 3, 则插值型求积公式为

$$\int_0^3 f(x)dx \approx A_0 f(0) + A_1 f(1) + A_2 f(2) + A_3 f(3).$$

下面求 $A_i (i=0, 1, 2, 3)$.

$$\begin{aligned} A_0 &= \int_0^3 l_0(x) dx = \int_0^3 \frac{(x-1)(x-2)(x-3)}{(0-1)(0-2)(0-3)} dx \\ &= -\frac{1}{6} \int_0^3 (x^3 - 6x^2 + 11x - 6) dx = \frac{3}{8}. \end{aligned}$$

同理可求得

$$A_1 = \frac{9}{8}, \quad A_2 = \frac{9}{8}, \quad A_3 = \frac{3}{8},$$

即有

$$\int_0^3 f(x) dx \approx \frac{3}{8}f(0) + \frac{9}{8}f(1) + \frac{9}{8}f(2) + \frac{3}{8}f(3).$$

4.1.4 求积公式的收敛性与稳定性

定义 4.3 在求积公式(4.4)中,若

$$\lim_{\substack{n \rightarrow \infty \\ h \rightarrow 0}} \sum_{k=0}^n A_k f(x_k) = \int_a^b f(x) dx,$$

其中, $h = \max_{1 \leq i \leq n} (x_i - x_{i-1})$, 则称求积式(4.4)是收敛的.

在求积式(4.4)中, 由于计算 $f(x_k)$ 可能产生误差 δ_k , 实际得到 $\tilde{f}_k$, 即 $f(x_k) = \tilde{f}_k + \delta_k$. 记

$$I_n(f) = \sum_{k=0}^n A_k f(x_k), \quad I_n(\tilde{f}_k) = \sum_{k=0}^n A_k \tilde{f}_k.$$

如果对任给小正数 $\epsilon > 0$, 只要误差 $|\delta_k|$ 充分小, 就有

$$|I_n(f) - I_n(\tilde{f}_k)| = \left| \sum_{k=0}^n A_k [f(x_k) - \tilde{f}_k] \right| \leq \epsilon, \quad (4.10)$$

则表明求积式(4.4)计算是稳定的.

定义 4.4 对任意 $\epsilon > 0$, 若 $\exists \delta > 0$, 只要 $|f(x_k) - \tilde{f}_k| \leq \delta (k=0, 1, 2, \dots, n)$ 就有式(4.10)成立, 则称求积式(4.4)是稳定的.

定理 4.3 若求积式(4.4)中系数 $A_k > 0 (k=0, 1, 2, \dots, n)$, 则此求积公式是稳定的.

证 对任意 $\epsilon > 0$, 取 $\delta = \frac{\epsilon}{b-a}$, 对 $k=0, 1, 2, \dots, n$ 都有 $|f(x_k) - \tilde{f}_k| \leq \delta$, 则有

$$\begin{aligned} |I_n(f) - I_n(\tilde{f})| &= \left| \sum_{k=0}^n A_k (f(x_k) - \tilde{f}_k) \right| \\ &\leq \sum_{k=0}^n |A_k| |f(x_k) - \tilde{f}_k| \\ &\leq \delta \sum_{k=0}^n A_k = \delta(b-a) = \epsilon. \end{aligned}$$

由定义 4.4 可知求积式(4.4)是稳定的, 证毕.

定理 4.3 表明, 只要求积系数 $A_k > 0$, 就能保证计算的稳定性.

4.2 Newton-Cotes(牛顿-柯特斯)公式

作为插值型求积公式的一种应用, 我们介绍利用等距节点构造出的一类求积公式, 这类求积公式称 Newton-Cotes 公式.

1. Cotes 系数

在区间 $[a, b]$ 上取 $(n+1)$ 个等距节点 $x_k = a + kh$ ($k=0, 1, 2, \dots, n$), $h = \frac{b-a}{n}$, 由它构造出的插值型求积公式

$$I_n = (b-a) \sum_{k=0}^n C_k^{(n)} f(x_k) \quad (4.11)$$

称为 Newton-Cotes 公式, 式中 $C_k^{(n)}$ 称为 Cotes 系数.

若令 $x = a + th, x_j = a + jh$, 则可得 $x - x_j = h(t - j)$,

$$C_k^{(n)} = \frac{1}{b-a} A_k = \frac{1}{b-a} \int_a^b \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j} dx = \frac{(-1)^{n-k}}{n \cdot k! (n-k)!} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt. \quad (4.12)$$

按式(4.12)计算 Cotes 系数不会遇到实质性困难, 表 4.1 列出了 Cotes 系数表开头的一部分.

表 4.1

$n \backslash k$	$C_k^{(n)}$								
	0	1	2	3	4	5	6	7	8
1	$\frac{1}{2}$	$\frac{1}{2}$							
2	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$						
3	$\frac{1}{8}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{1}{8}$					
4	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$				
5	$\frac{19}{288}$	$\frac{25}{96}$	$\frac{25}{144}$	$\frac{25}{144}$	$\frac{25}{96}$	$\frac{19}{288}$			
6	$\frac{41}{840}$	$\frac{9}{35}$	$\frac{9}{280}$	$\frac{34}{105}$	$\frac{9}{280}$	$\frac{9}{35}$	$\frac{41}{840}$		
7	$\frac{751}{17280}$	$\frac{3577}{17280}$	$\frac{1323}{17280}$	$\frac{2989}{17280}$	$\frac{2989}{17280}$	$\frac{1323}{17280}$	$\frac{3577}{17280}$	$\frac{751}{17280}$	
8	$\frac{989}{28350}$	$\frac{5888}{28350}$	$\frac{-928}{28350}$	$\frac{10496}{28350}$	$\frac{-4540}{28350}$	$\frac{10496}{28350}$	$\frac{-928}{28350}$	$\frac{5888}{28350}$	$\frac{989}{28350}$

从表 4.1 可以看到, 一阶和二阶 Newton-Cotes 公式分别是梯形公式

$$T = \frac{b-a}{2} [f(a) + f(b)]$$

和 Simpson 公式

$$S = \frac{b-a}{6} [f(a) + 4f(c) + f(b)], \quad c = \frac{a+b}{2},$$

又 4 阶 Newton-Cotes 公式为

$$C = \frac{b-a}{90} [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)], \quad (4.13)$$

式中, $x_k = a + kh (k=0, 1, 2, 3, 4)$, $h = \frac{b-a}{4}$, 式(4.13)特别称作 Cotes 公式.

由表 4.1 可看到, 当 $n=8$ 时, Cotes 系数有正有负, 这时稳定性得不到保证. 因此, 实际计算不用高阶的 Newton-Cotes 公式.

2. 偶阶求积公式的代数精度

先看一个数值例子.

例 4.3 用 Newton-Cotes 式(4.11) 计算积分 $I = \int_0^1 \frac{\sin x}{x} dx$.

解 计算结果见表 4.2, 表中最末一列指明有效数字的位数 m (I 的准确值为 0.9460831).

表 4.2

n	I_n	m
1	0.9207355	1
2	0.9461459	3
3	0.9461109	3
4	0.9460830	6
5	0.9460830	6

我们知道, n 阶的 Newton-Cotes 公式至少有 n 次代数精度, 但从上面的计算中可以看到, 2 阶公式与 3 阶公式的精度相当, 四阶和五阶也是如此, 这种现象不是偶然的. 事实上, 2 阶 Newton-Cotes 公式, 即 Simpson 公式

$$S = \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

至少具有二次代数精度. 进一步用 $f(x) = x^3$ 进行检验, 按上式计算得

$$S = \frac{b-a}{6} \left[a^3 + 4\left(\frac{a+b}{2}\right)^3 + b^3 \right].$$

另一方面, 直接求积得

$$I = \int_a^b x^3 dx = \frac{b^4 - a^4}{4},$$

这时有 $S=I$, 即 Simpson 公式对次数不超过 3 次的多项式均能准确成立. 又容易验证它对 $f(x) = x^4$ 通常是不准确的, 因此, Simpson 公式实际上具有 3 次代数精度.

一般地, 可以证明下述论断.

定理 4.4 当阶 n 为偶数时, Newton-Cotes 式(4.11)至少有 $(n+1)$ 次代数精度.

证 只要验证当 n 为偶数时, Newton-Cotes 公式对 $f(x)=x^{n+1}$ 的余项为零.

按余项式(4.9), 由于这里 $f^{(n+1)}(x)=(n+1)!$, 从而有

$$R(f) = I - I_n = \int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega(x) dx = \int_a^b \prod_{j=0}^n (x - x_j) dx.$$

引进变换 $x=a+th$, 并注意到 $x_j=a+jh$, 有

$$R(f) = h^{n+2} \int_0^1 \prod_{j=0}^n (t-j) dt.$$

若 n 为偶数, 则 $\frac{n}{2}$ 为整数. 再令 $t=u+\frac{n}{2}$, 进一步有

$$R(f) = h^{n+2} \int_{-\frac{n}{2}}^{\frac{n}{2}} \prod_{j=0}^n \left(u + \frac{n}{2} - j\right) du.$$

据此可断定 $R(f)=0$, 因为被积函数

$$H(u) = \prod_{j=0}^n \left(u + \frac{n}{2} - j\right) = \prod_{j=-n/2}^{n/2} (u-j)$$

是个奇函数.

3. 几种低阶求积公式的余项

求积公式的余项式(4.9)又称为截断误差. 下面讨论几个常用公式的截断误差.

定理 4.5 若 $f(x)$ 在 $[a, b]$ 上具有二阶连续导数, 则梯形式(4.1)的截断误差为

$$\begin{aligned} R[f] &= \int_a^b f(x) dx - (b-a) \left[\frac{1}{2} f(a) + \frac{1}{2} f(b) \right] \\ &= -\frac{(b-a)^3}{12} f''(\eta) \quad (a \leq \eta \leq b). \end{aligned} \quad (4.14)$$

证 由式(4.9)有

$$R[f] = \int_a^b \frac{f''(\xi)}{2!} (x-a)(x-b) dx \quad (\xi \text{ 依赖于 } x).$$

由于 $f''(\xi)$ 是依赖于 x 的函数, 在 $[a, b]$ 上连续, 且 $(x-a)(x-b) \leq 0$ 不变号, 故可应用积分中值定理, 在 $[a, b]$ 上存在一点 η , 使

$$\begin{aligned} \int_a^b \frac{f''(\xi)}{2!} (x-a)(x-b) dx &= \frac{f''(\eta)}{2!} \int_a^b (x-a)(x-b) dx \\ &= -\frac{(b-a)^3}{12} f''(\eta). \end{aligned}$$

证毕.

定理 4.6 若 $f(x)$ 在 $[a, b]$ 上有 4 阶连续导数, 则 Simpson 式(4.3)的截断误差为

$$R[f] = -\frac{(b-a)^5}{2880} f^{(4)}(\eta) \quad (a \leq \eta \leq b). \quad (4.15)$$

证 已知 Simpson 公式具有 3 次代数精确度, 考虑在 $[a, b]$ 上构造 3 次插值多项式 $H_3(x)$, 使它满足条件

$$\begin{aligned} H_3(a) &= f(a), \quad H_3(b) = f(b), \\ H_3\left(\frac{a+b}{2}\right) &= f\left(\frac{a+b}{2}\right), \quad H'_3\left(\frac{a+b}{2}\right) = f'\left(\frac{a+b}{2}\right). \end{aligned}$$

这样的插值误差为

$$f(x) - H_3(x) = \frac{f^{(4)}(\xi)}{4!} (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b) \quad (a \leq \xi \leq b).$$

由于 Simpson 公式对 $H_3(x)$ 准确成立, 由上式两边积分, 考虑到 $x \in [a, b]$ 时 $(x-a)$

$\cdot \left(x - \frac{a+b}{2}\right)^2 (x-b) \leq 0$ 不变号, 故有

$$\begin{aligned} R[f] &= \int_a^b f(x) dx - \frac{(b-a)}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \\ &= \int_a^b \frac{f^{(4)}(\xi)}{4!} (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b) dx \\ &= \frac{f^{(4)}(\eta)}{4!} \int_a^b (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b) dx \\ &= -\frac{(b-a)^5}{2880} f^{(4)}(\eta) \quad (a \leq \eta \leq b). \end{aligned}$$

可以证明 Cotes 式(4.13)的截断误差为

$$R(f) = -\frac{2(b-a)}{945} \left(\frac{b-a}{4}\right)^6 f^{(6)}(\eta). \quad (4.16)$$

例 4.4 试分别使用梯形公式和 Simpson 公式计算积分 $\int_1^2 e^{\frac{1}{x}} dx$ 的近似值, 并估计截断误差.

解 用梯形公式计算.

$$\int_1^2 e^{1/x} dx \approx \frac{2-1}{2} (e + e^{1/2}) = 2.1835.$$

$$f(x) = e^{1/x}, \quad f'(x) = -\frac{1}{x^2} e^{1/x}, \quad f''(x) = \left(\frac{2}{x^3} + \frac{1}{x^4}\right) e^{1/x}.$$

$$\max_{1 \leq x \leq 2} |f''(x)| = f''(1) = 8.1548.$$

截断误差估计为

$$|R_1| \leq \frac{(2-1)^3}{12} \max_{1 \leq x \leq 2} |f''(x)| = 0.6796.$$

用 Simpson 公式计算.

$$\int_1^2 e^{1/x} dx \approx \frac{2-1}{6} (e + 4e^{\frac{1}{1.5}} + e^{\frac{1}{2}}) \approx 2.0263,$$

$$f^{(4)} = \left(\frac{1}{x^8} + \frac{12}{x^7} + \frac{36}{x^6} + \frac{24}{x^5} \right) e^{\frac{1}{x}},$$

$$\max_{1 \leq x \leq 2} |f^{(4)}(x)| = f^{(4)}(1) = 198.43.$$

截断误差估计为

$$|R_2| \leq \frac{(2-1)^5}{2880} \max_{1 \leq x \leq 2} |f^{(4)}(x)| = 0.06890.$$

4.3 复化求积公式及其收敛性

4.2 节讨论的 Newton-Cotes 公式是基于插值公式推导的,一般说来, n 增大可提高公式的代数精确度,但提高插值公式次数来逼近 f 的效果并不佳,所以可以预料到当 $n \rightarrow \infty$ 时, $I_n(f)$ 不一定收敛到 $I(f)$. 下面给出一个例子,令

$$I = \int_{-4}^4 \frac{dx}{1+x^2} = 2\arctan 4 \approx 2.6516.$$

用 n 阶 Newton-Cotes 公式计算 I_n 的值为

$$I_2 = 5.4902, \quad I_4 = 2.2776, \quad I_6 = 3.3288,$$

$$I_8 = 1.9411, \quad I_{10} = 3.5956.$$

可以看出这样计算是不收敛的.

即使对收敛的情形, n 大时, Newton-Cotes 系数也不容易求出,又由于稳定性的讨论,一般不用 $n \geq 8$ 时的公式. 所以,为了提高近似积分的准确度,可以将 $[a, b]$ 剖分为若干个子区间,在每个子区间上使用低阶的 Newton-Cotes 公式,然后求和,这种方法称为复化求积法,用这种方法导出的新的求积公式称为复化求积公式.

4.3.1 复化梯形公式

将区间 $[a, b]$ 划分为 n 等分,分点 $x_k = a + kh$ ($h = \frac{b-a}{n}, k=0, 1, 2, \dots, n$), 在每个子区间 $[x_k, x_{k+1}]$ ($k=0, 1, 2, \dots, n-1$) 上采用梯形式(4.1), 则得

$$\begin{aligned} I &= \int_a^b f(x) dx = \sum_{k=0}^{n-1} \int_{x_k}^{x_{k+1}} f(x) dx \\ &= \frac{h}{2} \sum_{k=0}^{n-1} [f(x_k) + f(x_{k+1})] + R_n(f). \end{aligned} \quad (4.17)$$

记

$$T_n = \frac{h}{2} \sum_{k=0}^{n-1} [f(x_k) + f(x_{k+1})] = \frac{h}{2} \left[f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right], \quad (4.18)$$

称为复化梯形公式,其余项可由式(4.14),得

$$R_n(f) = I - T_n = \sum_{k=0}^{n-1} \left[-\frac{h^3}{12} f''(\eta_k) \right], \quad \eta_k \in (x_k, x_{k+1}).$$

由于 $f(x) \in C^2[a, b]$, 且

$$\min_{0 \leq k \leq n-1} f''(\eta_k) \leq \frac{1}{n} \sum_{k=0}^{n-1} f''(\eta_k) \leq \max_{0 \leq k \leq n-1} f''(\eta_k).$$

所以 $\exists \eta \in (a, b)$, 使

$$f''(\eta) = \frac{1}{n} \sum_{k=0}^{n-1} f''(\eta_k),$$

于是复化梯形公式余项为

$$R_n(f) = -\frac{b-a}{12} h^2 f''(\eta). \quad (4.19)$$

可以看出误差是 h^2 阶, 且由式(4.19)立即得到, 当 $f(x) \in C^2[a, b]$ 时, 则

$$\lim_{n \rightarrow \infty} T_n = \int_a^b f(x) dx,$$

即复化梯形公式是收敛的. 事实上, 只要设 $f(x) \in C[a, b]$, 则可得其收敛性, 因为只要把 T_n 改写为

$$T_n = \frac{1}{2} \left[\frac{b-a}{n} \sum_{k=0}^{n-1} f(x_k) + \frac{b-a}{n} \sum_{k=1}^n f(x_k) \right],$$

当 $n \rightarrow \infty$ 时, 上式右端括号内的两个和式均收敛到积分 $\int_a^b f(x) dx$, 所以复化梯形公式(4.18)收敛. 此外, T_n 的求积系数为正, 由定理 4.3 知复化梯形公式是稳定的.

4.3.2 复化 Simpson 求积公式

将区间 $[a, b]$ 分为 n 等分, 在每个子区间 $[x_k, x_{k+1}]$ 上采用 Simpson 式(4.3), 若记 $x_{k+1/2} = x_k + \frac{1}{2}h$, 则得

$$\begin{aligned} I &= \int_a^b f(x) dx = \sum_{k=0}^{n-1} \int_{x_k}^{x_{k+1}} f(x) dx \\ &= \frac{h}{6} \sum_{k=0}^{n-1} [f(x_k) + 4f(x_{k+1/2}) + f(x_{k+1})] + R_n(f). \end{aligned} \quad (4.20)$$

记

$$\begin{aligned} S_n &= \frac{h}{6} \sum_{k=0}^{n-1} [f(x_k) + 4f(x_{k+1/2}) + f(x_{k+1})] \\ &= \frac{h}{6} \left[f(a) + 4 \sum_{k=0}^{n-1} f(x_{k+1/2}) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right], \end{aligned} \quad (4.21)$$

称为复化 Simpson 求积公式, 其余项由式(4.15), 得

$$R_n(f) = I - S_n = -\frac{h}{180} \left(\frac{h}{2}\right)^4 \sum_{k=0}^{n-1} f^{(4)}(\eta_k), \quad \eta_k \in (x_k, x_{k+1}).$$

于是当 $f(x) \in C^4[a, b]$ 时, 与复化梯形公式相似有

$$R_n(f) = I - S_n = -\frac{b-a}{180} \left(\frac{h}{2}\right)^4 f^{(4)}(\eta), \quad \eta \in (a, b). \quad (4.22)$$

由式(4.22)看出, 误差阶为 h^4 , 收敛性是显然的. 实际上, 只要 $f(x) \in C[a, b]$, 则可得收敛性, 即

$$\lim_{n \rightarrow \infty} S_n = \int_a^b f(x) dx.$$

此外, 由于 S_n 中求积系数均为正数, 故知复化 Simpson 公式计算稳定.

如果将每个小区间 $[x_k, x_{k+1}]$ 划分为 4 等分, 内分点依次记作 $x_{k+\frac{1}{4}}, x_{k+\frac{2}{4}}, x_{k+\frac{3}{4}}$, 则复化 Cotes 公式具有形式

$$C_n = \frac{h}{90} \left[7f(a) + 32 \sum_{k=0}^{n-1} f(x_{k+\frac{1}{4}}) + 12 \sum_{k=0}^{n-1} f(x_{k+\frac{2}{4}}) + 32 \sum_{k=0}^{n-1} f(x_{k+\frac{3}{4}}) + 14 \sum_{k=1}^{n-1} f(x_k) + 7f(b) \right]$$

依据式(4.16), 容易得到复化 Cotes 公式的余项是

$$R_n(f) = I - C_n = -\frac{2(b-a)}{945} \left(\frac{h}{4}\right)^6 f^{(6)}(\eta), \quad \eta \in (a, b)$$

例 4.5 对于函数 $f(x) = \frac{\sin x}{x}$, 给出 $n=8$ 的函数表(见表 4.3), 试用复化梯形式(4.18)及复化 Simpson 式(4.21)计算积分

$$I = \int_0^1 \frac{\sin x}{x} dx,$$

并估计误差(I 的准确值为 0.9460831).

解 将积分区间 $[0, 1]$ 划分为 8 等分, 应用复化梯形法求得

$$T_8 = 0.9456909,$$

而如果将 $[0, 1]$ 分为 4 等分, 应用复化 Simpson 法有

$$S_4 = 0.9460833$$

两种算法计算量基本相当, 但精度却差别很大. 同准确值 I 比较, T_8 只有 3 位有效数字, 而复化 Simpson 法的结果 S_4 却有 6 位有效数字.

表 4.3

x	$f(x)$
0	1
1/8	0.9973979
1/4	0.9896158
3/8	0.9767267
1/2	0.9588511
5/8	0.9361556
3/4	0.9088517
7/8	0.8771926
1	0.8414710

为了利用余项公式估计误差, 要求 $f(x) = \frac{\sin x}{x}$ 的高阶导数, 由于

$$f(x) = \frac{\sin x}{x} = \int_0^1 \cos(xt) dt,$$

所以有 $f^{(k)}(x) = \int_0^1 \frac{d^k}{dx^k} \cos(xt) dt = \int_0^1 t^k \cos\left(xt + \frac{k\pi}{2}\right) dt$, 于是 $\max_{0 \leq x \leq 1} |f^{(k)}(x)| \leq \int_0^1 \left| \cos\left(xt + \frac{k\pi}{2}\right) \right| t^k dt \leq \int_0^1 t^k dt = \frac{1}{k+1}$.

由式(4.19)得复化梯形公式误差

$$\begin{aligned} |R_8(f)| &= |I - T_8| \leq \frac{h^2}{12} \max_{0 \leq x \leq 1} |f''(x)| \\ &\leq \frac{1}{12} \left(\frac{1}{8}\right)^2 \frac{1}{3} = 0.000434. \end{aligned}$$

对复化 Simpson 公式误差, 由式(4.22), 得

$$|R_4(f)| = |I - S_4| \leq \frac{1}{2880} \left(\frac{1}{4}\right)^4 \frac{1}{5} = 0.271 \times 10^{-6}.$$

例 4.6 分别用复化梯形公式与复化 Simpson 公式计算积分

$$I = \int_0^1 e^x dx$$

的近似值, 要求其截断误差 $|R_N[f]| \leq \frac{1}{2} \times 10^{-4}$. 问: 各需取多少个节点?

解 $f(x) = e^x$, $f''(x) = f^{(4)}(x) = e^x$. 在区间 $[0, 1]$ 上, $\max |f''(x)| = \max |f^{(4)}(x)| = e$. 用复化梯形公式求积时, 由式(4.19), 有

$$|R_N[f]| \leq \frac{e}{12} h^2 \leq \frac{1}{2} \times 10^{-4},$$

由此得 $h \leq 0.0149$, 取 $h = 0.0148$, 则 $N > 67.6$, 需取 $N+1 = 69$ 个节点.

用复化 Simpson 公式, 由式(4.22), 有

$$|R_N[f]| \leq \frac{e}{2880} h^4 \leq \frac{1}{2} \times 10^{-4},$$

则

$$h \leq 0.4798,$$

由此可知 $N \geq \frac{1}{h} = 2.085$, 取 $N = 3$, 则只需取 $2N+1 = 7$ 个节点.

定义 4.5 对于复化求积公式 $\int_a^b f(x) dx \approx I_n$, 若当 $h \rightarrow 0$ 时, 有

$$\frac{\int_a^b f(x) dx - I_n}{h^p} \rightarrow C \quad (C \text{ 为非零常数}),$$

则称 I_n 是 p 阶收敛的.

注 p 阶收敛与 p 次代数精度的意义是不同的, 不能混为一谈, 式(4.18), 式(4.21)分别具有二、四阶收敛性.

4.3.3 区间逐次分半求积法

从理论上说, 可以用误差式(4.19), 式(4.22)来估计近似值的误差, 或者根据精度

要求来确定区间的等分数 n (即确定步长 h), 而实际上, 由于公式中包含有被积函数的高阶导数, 要对它做出估计往往很困难, 因此在实际计算时, 最有效的方法是“事后估计误差法”, 即将区间逐次分半进行计算 (每分一次就进行一次计算), 并利用前后两次计算结果来判断误差的大小, 其原理如下.

先看梯形公式, 由误差公式 (4.19), 知

$$I - T_n = -\frac{b-a}{12} \left(\frac{b-a}{n} \right)^2 f''(\xi_1), \quad \xi_1 \in (a, b),$$

其中, I 表示积分 $\int_a^b f(x) dx$ 的真值, $T_n = \frac{h}{2} \left[f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b) \right]$.

同理, 还可以得到

$$I - T_{2n} = -\frac{b-a}{12} \left(\frac{b-a}{2n} \right)^2 f''(\xi_2), \quad \xi_2 \in [a, b].$$

若 $f''(x)$ 在 $[a, b]$ 上变化不大, 则 $f''(\xi_1) \approx f''(\xi_2)$, 且由上面两式可得

$$\frac{I - T_n}{I - T_{2n}} \approx 4.$$

于是有
$$I \approx T_{2n} + \frac{1}{3}(T_{2n} - T_n) = T_{2n} + \frac{1}{4-1}(T_{2n} - T_n). \quad (4.23)$$

此式表明, 若用 T_{2n} 作为 I 的近似值, 则其截断误差约为 $\frac{T_{2n} - T_n}{3}$. 因此, 在逐次二分进行计算时, 可以用 T_n 与 T_{2n} 来估计误差, 这种直接用计算结果来估计误差的方法通常称作误差的事后估计法, 若 $|T_{2n} - T_n| < \epsilon' = 3\epsilon$ (ϵ 为计算结果的允许误差), 则停止计算, 并取 T_{2n} 作为积分的近似值; 否则将区间再次二分后算出 T_{4n} , 并检验不等式 $|T_{4n} - T_{2n}| < \epsilon'$ 是否满足.

通过类似的推导, 还可得到下列结论.

对于 Simpson 公式, 若 $f^{(4)}(x)$ 在 $[a, b]$ 上连续且变化不大, 则有

$$I \approx S_{2n} + \frac{1}{15}(S_{2n} - S_n) = S_{2n} + \frac{1}{4^2 - 1}(S_{2n} - S_n). \quad (4.24)$$

对于 Cotes 公式, 若 $f^{(6)}(x)$ 在 $[a, b]$ 上连续且变化不大, 则有

$$I \approx C_{2n} + \frac{1}{63}(C_{2n} - C_n) = C_{2n} + \frac{1}{4^3 - 1}(C_{2n} - C_n). \quad (4.25)$$

由此可以如同使用梯形公式那样来估计误差与确定步长 h .

上述事后估计误差法, 给计算带来了很大方便.

例 4.7 用复化 Simpson 方法计算积分

$$\int_0^1 \frac{\sin x}{x} dx = 0.9460831$$

的近似值, 使误差不超过 $\frac{1}{2} \times 10^{-6}$.

解 可先算出 $S_1 = 0.9461459$, 然后将区间二分(即二等分), 并计算 $S_2 = 0.9460869$; 显然 S_2 不合要求, 故再次将区间二分(即四等分), 并计算 $S_4 = 0.9460833$, 因为

$$|S_4 - S_2| < \frac{15}{2} \times 10^{-6},$$

故 $S_4 = 0.9460833$ 是满足精度要求的近似解. 此过程也容易在电子计算机上实现.

4.4 Romberg(龙贝格)算法

4.4.1 梯形法的递推化

下面以梯形法为例, 介绍上述变步长(即 h 次逐次减半)算法.

将积分区间 $[a, b]$ n 等分, 则一共有 $(n+1)$ 个分点, 按梯形公式计算积分的近似值 T_n , 需要计算 $(n+1)$ 个函数值, 当 T_n 不满足精度要求时, 就将各个小区间二分, 此时分点增至 $(2n+1)$ 个, 如果再直接用梯形公式计算积分新的近似值 T_{2n} , 就需要计算 $(2n+1)$ 个函数值, 实际上, 这 $(2n+1)$ 个点中有 $(n+1)$ 个点是“老分点”, 对应的函数值在计算 T_n 时已经算出, 为了避免这种重复计算, 将计算 T_{2n} 的式子作如下改造.

由梯形公式, 有

$$T_{2n} = \frac{b-a}{4n} \left[f(a) + 2 \sum_{k=1}^{2n-1} f\left(a + k \frac{b-a}{2n}\right) + f(b) \right]. \quad (4.26)$$

注意到在分点 $x_k = a + k \frac{b-a}{2n}$ ($k=1, 2, \dots, 2n-1$) 中, 当 k 取偶数时是“老分点”, 当 k 取奇数时是新增加的点, 将“老分点”的函数值从右端求和记号中分离出来, 即得

$$\begin{aligned} T_{2n} &= \frac{b-a}{4n} \left[f(a) + 2 \sum_{k=1}^{n-1} f\left(a + 2k \frac{b-a}{2n}\right) \right. \\ &\quad \left. + f(b) + 2 \sum_{k=1}^n f\left(a + (2k-1) \frac{b-a}{2n}\right) \right] \\ &= \frac{b-a}{4n} \left[f(a) + 2 \sum_{k=1}^{n-1} f\left(a + 2k \frac{b-a}{2n}\right) + f(b) \right] \\ &\quad + \frac{b-a}{2n} \sum_{k=1}^n f\left[a + (2k-1) \frac{b-a}{2n}\right]. \end{aligned}$$

再由梯形公式, 就得到递推算式

$$T_{2n} = \frac{1}{2} T_n + \frac{b-a}{2n} \sum_{k=1}^n f\left[a + (2k-1) \frac{b-a}{2n}\right]. \quad (4.27)$$

式(4.27)表明, 只要算出新增加的 n 个点上的函数值, 就可以求出 T_{2n} , 与直接用梯形式(4.26)求 T_{2n} 相比较, 计算工作量几乎节省了一半.

为了便于编制程序, 通常将积分区间 $[a, b]$ 的等分数依次取 $1=2^0, 2=2^1, 4=2^2$,

$8=2^3, \dots$, 并将递推式(4.27)改写成

$$\begin{cases} T_1 = \frac{b-a}{2}[f(a) + f(b)], \\ T_{2^k} = \frac{1}{2}T_{2^{k-1}} + \frac{b-a}{2^k} \sum_{i=1}^{2^{k-1}} f\left[a + (2i-1) \frac{b-a}{2^k}\right]. \end{cases} \quad (4.28)$$

利用式(4.28)在电子计算机上求积分的计算步骤如下:

(1) 计算初值 T_1 ;

(2) $k \leftarrow 1$;

(3) 计算新的梯形值 T_{2^k} ;

(4) 精度控制: 若 $|T_{2^k} - T_{2^{k-1}}| < \epsilon$, 则停止计算, 并输出 T_{2^k} 作为积分的近似值; 否则 $k \leftarrow k+1$, 并转第(3)步继续计算(其中 ϵ 根据问题的精度要求确定).

例 4.8 利用复化梯形公式计算积分 $\int_0^1 \frac{\sin x}{x} dx$, 使误差不超过 $\frac{1}{2} \times 10^{-6}$.

解 利用递推公式计算.

$$T_{2^0} = T_1 = \frac{1}{2}[f(0) + f(1)] = \frac{1}{2}[1 + 0.8414710] = 0.9207355,$$

$$T_{2^1} = T_2 = \frac{1}{2}T_1 + \frac{1}{2}f\left(\frac{1}{2}\right) = 0.9397933 \quad (x = \frac{1}{2} \text{ 是新分点}),$$

$$T_{2^2} = T_4 = \frac{1}{2}T_2 + \frac{1}{4}\left[f\left(\frac{1}{4}\right) + f\left(\frac{3}{4}\right)\right] = 0.9445135 \quad (x = \frac{1}{4}, \frac{3}{4} \text{ 是新分点}),$$

$$\begin{aligned} T_{2^3} = T_8 &= \frac{1}{2}T_4 + \frac{1}{8}\left[f\left(\frac{1}{8}\right) + f\left(\frac{3}{8}\right) + f\left(\frac{5}{8}\right) + f\left(\frac{7}{8}\right)\right] \\ &= 0.9456909 \quad (x = \frac{1}{8}, \frac{3}{8}, \frac{5}{8}, \frac{7}{8} \text{ 是新分点}), \end{aligned}$$

...

仿上可得

$$T_{2^7} = T_{128} = 0.9460815 \quad (\text{出现的新分点有 } 64 \text{ 个}),$$

$$T_{2^8} = T_{256} = 0.9460827 \quad (\text{出现的新分点有 } 128 \text{ 个}).$$

因为 $|T_{2^8} - T_{2^7}| < 3 \times \frac{1}{2} \times 10^{-6},$

故取 $\int_0^1 \frac{\sin x}{x} dx \approx T_{2^8} = 0.9460827.$

4.4.2 Romberg 算法

梯形法计算简单但收敛慢, 如何提高收敛速度是本节讨论的中心问题. 按式(4.23), 积分近似值 T_{2^n} 的误差大致等于 $\frac{1}{3}(T_{2^n} - T_n)$, 因此如果用这个误差值作为 T_{2^n}

的一种补偿,可以期望所得到的 $\tilde{T} = T_{2n} + \frac{1}{3}(T_{2n} - T_n)$ 可能是更好的结果,这是因为

$$S_n = \frac{4}{3}T_{2n} - \frac{1}{3}T_n, \quad (4.29)$$

即用梯形法二分前后的两个积分值 T_n 与 T_{2n} , 按式(4.23)做线性组合, 结果得到较梯形公式精度高的 Simpson 的积分值 S_n .

这说明从梯形公式出发, 将区间 $[a, b]$ 逐次二分可提高求积公式的精度, 上述加速过程还可继续下去, 其理论依据是梯形公式的余项展开. 设

$$I - T_n = -\frac{b-a}{12}h^2 f''(\eta) \quad \left(\eta \in [a, b], h = \frac{b-a}{n} \right).$$

若记 $T_n = T(h)$, 当区间 $[a, b]$ 分为 $2n$ 等分时, 则有 $T_{2n} = T\left(\frac{h}{2}\right)$, 并且有

$$T(h) = I + \frac{b-a}{12}h^2 f''(\eta), \quad \lim_{h \rightarrow 0} T(h) = T(0) = I,$$

可证明梯形公式余项可展开为级数形式.

定理 4.7 设 $f(x) \in C^\infty[a, b]$, 则有

$$T(h) = I + \alpha_1 h^2 + \alpha_2 h^4 + \cdots + \alpha_l h^{2l} + \cdots, \quad (4.30)$$

其中, 系数 $\alpha_l (l=1, 2, \cdots)$ 与 h 无关.

此定理可利用 $f(x)$ 的 Taylor 展开推导得到, 此处从略.

定理 4.7 表明 $T(h) \approx I$ 是 $O(h^2)$ 阶, 在式(4.30)中, 若用 $h/2$ 代替 h , 则有

$$T\left(\frac{h}{2}\right) = I + \alpha_1 \frac{h^2}{4} + \alpha_2 \frac{h^4}{16} + \cdots + \alpha_l \left(\frac{h}{2}\right)^{2l} + \cdots. \quad (4.31)$$

若用 4 乘式(4.31), 减去式(4.30)再除 3, 记之为 $T_1(h)$, 则得

$$T_1(h) = \frac{4T\left(\frac{h}{2}\right) - T(h)}{3} = I + \beta_1 h^4 + \beta_2 h^6 + \cdots. \quad (4.32)$$

这里 $\beta_1, \beta_2, \cdots$ 以及后面将要出现的 r_k, δ_k 均为与 h 无关的系数, 这样构造的 $T_1(h)$ 与积分值 I 近似的阶为 $O(h^4)$. 比较式(4.32)与式(4.29)可知, 这样构造的序列 $T_1(h)$,

$T_1\left(\frac{h}{2}\right), \cdots$, 就是 Simpson 公式序列 $S_n, S_{2n}, \cdots$.

又根据式(4.32), 有

$$T_1\left(\frac{h}{2}\right) = I + \beta_1 \frac{h^4}{16} + \beta_2 \frac{h^6}{64} + \beta_3 \frac{h^8}{256} + \cdots,$$

若令

$$T_2(h) = \frac{16}{15}T_1\left(\frac{h}{2}\right) - \frac{1}{15}T_1(h),$$

则又可进一步从余项展开式中消去 h^4 项, 而有

$$T_2(h) = I + \gamma_1 h^6 + \gamma_2 h^8 + \cdots.$$

这样构造出的 $\{T_2(h)\}$, 其实就是 Cotes 公式序列, 它与积分值 I 的逼近阶为 $O(h^6)$. 如此继续下去, 每加速一次, 误差的量级便提高 2 阶, 一般地, 若记 $T_0(h) = T(h)$, 则有

$$T_m(h) = \frac{4^m}{4^m - 1} T_{m-1}\left(\frac{h}{2}\right) - \frac{1}{4^m - 1} T_{m-1}(h). \quad (4.33)$$

经过 $m(m=1, 2, \dots)$ 次加速后, 余项便取下列形式:

$$T_m(h) = I + \delta_1 h^{2(m+1)} + \delta_2 h^{2(m+2)} + \dots \quad (4.34)$$

上述处理方法通常称为理查森(Richardson)外推加速方法.

设以 $T_0^{(k)}$ 表示二分 k 次后求得的梯形值, 且以 $T_m^{(k)}$ 表示序列 $\{T_0^{(k)}\}$ 的 m 次加速值, 则依递推式(4.33), 可得

$$T_m^{(k)} = \frac{4^m}{4^m - 1} T_{m-1}^{(k+1)} - \left(\frac{1}{4^m - 1}\right) T_{m-1}^{(k)}, \quad k = 1, 2, \dots \quad (4.35)$$

式(4.35)也称为 Romberg 求积算法, 计算过程如下.

(1) 取 $k=0, h=b-a$, 求 $T_0(h) = \frac{h}{2}[f(a) + f(b)]$,

令 $1 \rightarrow k$ (k 记区间 $[a, b]$ 的二分次数).

(2) 求梯形值 $T_0\left(\frac{b-a}{2^k}\right)$, 即按递推式(4.28)计算 $T_0^{(k)}$.

(3) 求加速值, 按式(4.35)逐个求出 T 表(见表 4.4)的第 k 行其余各元素 $T_j^{(k-j)}$ ($j=1, 2, \dots, k$).

(4) 若 $|T_k^{(0)} - T_{k-1}^{(0)}| < \varepsilon$ (预先给定的精度), 则终止计算, 并取 $T^{(0)}(k) \approx I$; 否则令 $k+1 \rightarrow k$ 转(2)继续计算.

表 4.4

k	h	$T_0^{(k)}$	$T_1^{(k)}$	$T_2^{(k)}$	$T_3^{(k)}$	$T_4^{(k)}$	...
0	$b-a$	$T_0^{(0)}$					
1	$\frac{b-a}{2}$	$T_0^{(1)} \downarrow \xrightarrow{\textcircled{1}} T_1^{(0)}$					
2	$\frac{b-a}{4}$	$T_0^{(2)} \downarrow \xrightarrow{\textcircled{2}} T_1^{(1)} \downarrow \xrightarrow{\textcircled{3}} T_2^{(0)}$					
3	$\frac{b-a}{8}$	$T_0^{(3)} \downarrow \xrightarrow{\textcircled{4}} T_1^{(2)} \downarrow \xrightarrow{\textcircled{5}} T_2^{(1)} \downarrow \xrightarrow{\textcircled{6}} T_3^{(0)}$					
4	$\frac{b-a}{16}$	$T_0^{(4)} \downarrow \xrightarrow{\textcircled{7}} T_1^{(3)} \downarrow \xrightarrow{\textcircled{8}} T_2^{(2)} \downarrow \xrightarrow{\textcircled{9}} T_3^{(1)} \downarrow \xrightarrow{\textcircled{10}} T_4^{(0)}$					
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

表 4.4 指出了计算过程, 第 2 列 $h = \frac{b-a}{2^k}$ 给出了子区间长度, $\textcircled{i}$ 表示第 i 步外推.

可以证明, 如果 $f(x)$ 充分光滑, 那么 T 表每一列的元素及对角线元素均收敛到所求的积分值 I , 即

$$\lim_{k \rightarrow \infty} T_m^{(k)} = I \quad (m \text{ 固定}),$$

$$\lim_{m \rightarrow \infty} T_m^{(0)} = I.$$

对于不充分光滑的函数 $f(x)$ 也可用 Romberg 算法计算,只是收敛慢一些,这时也可以直接使用复化 Simpson 公式计算.

例 4.9 用 Romberg 算法计算积分 $I = \int_0^1 x^{3/2} dx$.

解 $f(x) = x^{3/2}$ 在 $[0, 1]$ 上仅是一次连续可微,用 Romberg 算法计算结果见表 4.5. 从表中看到 Romberg 算到 $k=5$ 的精度与 Simpson 求积精度相当. 这里 I 的精确值为 0.4.

表 4.5

k	$T_0^{(k)}$	$T_1^{(k)}$	$T_2^{(k)}$	$T_3^{(k)}$	$T_4^{(k)}$	$T_5^{(k)}$
0	0.500000					
1	0.426777	0.402369				
2	0.407018	0.400432	0.400303			
3	0.401812	0.400077	0.400054	0.400050		
4	0.400463	0.400014	0.400009	0.400009	0.400009	
5	0.400118	0.400002	0.400002	0.400002	0.400002	0.400002

4.5 Gauss(高斯)型求积公式

在前面构造的插值型求积公式中,如梯形公式和 Simpson 公式,为了简化过程,任意给定(不是选定)了求积节点,这样做就限定了求积公式的代数精确度. 下面按照以求积公式的代数精确度达到最大的原则去选取求积节点和求出相应的求积系数. 首先以 $n=1$ 为例论证这种想法是可行的,然后给出构造原理.

例 4.10 构造形如 $\int_{-1}^1 f(x) dx \approx A_0 f(x_0) + A_1 f(x_1)$ 的求积公式.

解 上述公式中,有 4 个待定参数,至少要建立 4 个方程. 按代数精确度,分别令 $f(x)=1, x, x^2, x^3$, 代入上式,有

$$f(x) = 1, \quad A_0 + A_1 = 2;$$

$$f(x) = x, \quad A_0 x_0 + A_1 x_1 = 0;$$

$$f(x) = x^2, \quad A_0 x_0^2 + A_1 x_1^2 = \frac{2}{3};$$

$$f(x) = x^3, \quad A_0 x_0^3 + A_1 x_1^3 = 0.$$

改写第二式

$$(A_0 + A_1)x_0 + A_1(x_1 - x_0) = 0,$$

利用第一式,则上式简化为

$$2x_0 + A_1(x_1 - x_0) = 0.$$

同样,利用第二式简化第三式,利用第三式简化第四式,分别得

$$(x_1 - x_0)x_1A_1 = \frac{2}{3}, \quad \frac{2}{3}x_0 + (x_1 - x_0)x_1^2A_1 = 0.$$

从前面3个式子中,消去 $A_1(x_1 - x_0)$, 易求出

$$x_0 = -\frac{1}{\sqrt{3}}, \quad x_1 = \frac{1}{\sqrt{3}}, \quad A_0 = A_1 = 1.$$

于是得到求积公式

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

它至少有3次代数精度,而以两个端点为节点的梯形公式却只有1次代数精度.

为了一般性,考虑带权定积分

$$I(f) = \int_a^b \rho(x) f(x) dx, \quad (4.36)$$

其中, $\rho(x) \geq 0$ 为权函数. 当 $\rho(x) = 1$ 时,即是普通积分.

若选定 $(n+1)$ 个节点,还需求出 $(n+1)$ 个求积系数,共有 $(2n+2)$ 个未知量. 利用代数精确度的概念可建立 $(2n+2)$ 个方程,即令 $f(x) = 1, x, x^2, \dots, x^{2n+1}$, 使求积公式

$$\int_a^b \rho(x) f(x) dx = \sum_{k=0}^n A_k f(x_k) \quad (4.37)$$

成立. 从而可使求积公式至少达到 $(2n+1)$ 次代数精确度,与给定 $(n+1)$ 个节点的一般插值型求积公式至少达到 n 次代数精确度相比,其代数精度的次数大大提高了.

定义 4.6 选互异节点 $x_0, x_1, \dots, x_n$, 使插值型求积式(4.37)的代数精确度为 $(2n+1)$ 次,则称该求积公式为 Gauss 型的. 称这些节点为 Gauss 点.

如果像例 4.10 那样,直接利用代数精确度的概念去求 $(n+1)$ 个求积节点和 $(n+1)$ 个求积系数,则要联立求解 $(2n+2)$ 个方程的非线性方程组. 方程组是可解的,但当 n 稍大时,解析地求解就很难,数值求解非线性方程组也不容易.

一般利用正交多项式来确定 Gauss 点 $x_0, x_1, \dots, x_n$, 然后,利用插值原理确定 Gauss 求解系数

$$A_k = \int_a^b \rho(x) l_k(x) dx. \quad (4.38)$$

其中, $l_k(x)$ ($k=0, 1, 2, \dots, n$) 是关于 Gauss 点的 Lagrange 插值基函数,从而得到插值型求积公式

$$\int_a^b \rho(x) f(x) dx \approx \sum_{k=0}^n A_k f(x_k), \quad (4.39)$$

它是 Gauss 型的.

4.5.1 Gauss 点与正交多项式零点的关系

定理 4.8 对于插值型求积式(4.39),其节点 $x_0, x_1, \dots, x_n$ 是 Gauss 点的充分必要条件是, $\omega_{n+1}(x) = (x-x_0)(x-x_1)\cdots(x-x_n)$ 与任意不超过 n 次的多项式 $P(x)$ 带权正交,即

$$\int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = 0.$$

证 先证必要性,即 $x_0, x_1, \dots, x_n$ 是 Gauss 点 $\Rightarrow \int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = 0$.

因为 $x_0, x_1, \dots, x_n$ 是 Gauss 点,由 Gauss 点的定义,它们使求积式(4.39)具有 $(2n+1)$ 次代数精确度.对于不超过 $(2n+1)$ 次的多项式 $P(x)\omega_{n+1}(x)$,求积公式(4.39)准确成立,即

$$\int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = \sum_{k=0}^n A_k P(x_k) \omega_{n+1}(x_k),$$

其中, $\omega_{n+1}(x_k) = 0 (k = 0, 1, 2, \dots, n)$, 所以, $\int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = 0$.

再证充分性,即 $\int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = 0 \Rightarrow x_0, x_1, \dots, x_n$ 是 Gauss 点.

因为使求积式(4.39)具有 $(2n+1)$ 次代数精确度的点,才称为 Gauss 点,而要说明求积公式有 $(2n+1)$ 次代数精确度,只要证明在条件 $\int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = 0$ 成立下,当 $f(x)$ 是任意不超过 $(2n+1)$ 次多项式时,求积公式准确成立.

设 $f(x)$ 为不超过 $(2n+1)$ 次多项式,用 $\omega_{n+1}(x)$ 除 $f(x)$,其商为 $P(x)$,余项为 $Q(x)$,即

$$f(x) = P(x)\omega_{n+1}(x) + Q(x), \quad (4.40)$$

其中, $P(x)$ 和 $Q(x)$ 均是不超过 n 次的多项式.对上式两边积分,且利用

$$\int_a^b \rho(x) P(x) \omega_{n+1}(x) dx = 0,$$

得
$$\int_a^b \rho(x) f(x) dx = \int_a^b \rho(x) Q(x) dx. \quad (4.41)$$

对于一般插值型求积公式,至少有 n 次代数精确度,对次数不超过 n 次的多项式,求积公式准确成立,则有

$$\int_a^b \rho(x) Q(x) dx = \sum_{k=0}^n A_k Q(x_k).$$

因为 $\omega_{n+1}(x_k) = 0 (k = 0, 1, 2, \dots, n)$, 由式(4.40), $f(x_k) = Q(x_k)$. 又由式(4.41), 则

$$\int_a^b \rho(x) f(x) dx = \sum_{k=0}^n A_k f(x_k).$$

说明求积式(4.39)对于一切次数不超过 $(2n+1)$ 的多项式均成立,因此, $x_0, x_1, \dots, x_n$ 是 Gauss 点. 证毕.

推论 区间 $[a, b]$ 上带权 $\rho(x)$ 的正交多项式 $\varphi_{n+1}(x)$ 的零点就是 Gauss 点.

因为正交多项式的两个重要的性质是:其一,正交多项式 $\varphi_{n+1}(x)$ 与比它次数低的任意多项式 $P(x)$ 均正交,即 $\int_a^b \rho(x)P(x)\varphi_{n+1}(x)dx = 0$;其二, $(n+1)$ 次正交多项式 $\varphi_{n+1}(x)$ 恰好有 $(n+1)$ 个互异的实单根,且都在 $[a, b]$ 内. 所以推论成立.

选定了 Gauss 点 $x_0, x_1, \dots, x_n$ 后,由式(4.38)求得求积系数 $A_k (k=0, 1, 2, \dots, n)$,这样就构成了 Gauss 型求积公式.

例 4.11 确定 x_0, x_1, A_0 和 A_1 使以下公式为 Gauss 求积公式

$$\int_0^1 \sqrt{1-x} f(x) dx \approx A_0 f(x_0) + A_1 f(x_1). \quad (4.42)$$

解 设 $\varphi_2(x) = x^2 + bx + c$,为区间 $[0, 1]$ 上带权 $\rho(x) = \sqrt{1-x}$ 的正交多项式,利用正交性,有

$$(\varphi_2, \varphi_0) = \int_0^1 \sqrt{1-x}(x^2 + bx + c)dx = 0 \Rightarrow \frac{8}{35} + \frac{2}{5}b + c = 0,$$

$$(\varphi_2, \varphi_1) = \int_0^1 \sqrt{1-x}(x^2 + bx + c)x dx = 0 \Rightarrow b = -\frac{8}{9}.$$

从而得 $c = \frac{8}{63}$, $\varphi_2(x) = x^2 - \frac{8}{9}x + \frac{8}{63}$, 易求出其零点 $x_0 = 0.1788, x_1 = 0.7101$. 令 $f(x) = 1, x$, 分别代入式(4.42), 即

$$f(x) = 1, \quad A_0 + A_1 = \frac{2}{3};$$

$$f(x) = x, \quad 0.1788A_0 + 0.7101A_1 = \frac{4}{15}.$$

求得 $A_0 = 0.3891, A_1 = 0.2776$, 从而得到 Gauss 求积公式

$$\int_0^1 \sqrt{1-x} f(x) dx \approx 0.3891 f(0.1788) + 0.2776 f(0.7101).$$

4.5.2 常用的 Gauss 型求积公式

1. Gauss-Legendre 求积公式

区间 $[-1, 1]$ 上权函数为 $\rho(x) = 1$ 的正交多项式是 Legendre 正交多项式

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} [(x^2 - 1)^n],$$

以 Legendre 多项式的零点为 Gauss 点的求积公式

$$\int_{-1}^1 f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$$

称为 Gauss-Legendre 求积公式.

当 $n=1$ 时,即

$$\int_{-1}^1 f(x) dx \approx A_0 f(x_0) + A_1 f(x_1),$$

选 2 次 Legendre 多项式 $P_2(x) = \frac{1}{2}(3x^2 - 1)$ 的零点 $x_0 = -\frac{1}{\sqrt{3}}, x_1 = \frac{1}{\sqrt{3}}$ 作为 Gauss 点, 代入上式, 则有

$$\int_{-1}^1 f(x) dx \approx A_0 f\left(-\frac{1}{\sqrt{3}}\right) + A_1 f\left(\frac{1}{\sqrt{3}}\right). \quad (4.43)$$

由求积系数公式

$$A_k = \int_{-1}^1 l_k(x) dx \quad (k=0,1),$$

其中, $l_k(x)$ 是关于节点 x_0, x_1 的线性 Lagrange 插值基函数, 求得 $A_0 = A_1 = 1$. 或利用代数精确度的概念, 分别令 $f(x) = 1, x$, 代入式(4.43), 使它准确成立, 同样得到 $A_0 = A_1 = 1$. 从而构造了具有 $2n+1=3$ 次代数精确度的 2 点 Gauss-Legendre 求积公式

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right). \quad (4.44)$$

当 $n=2$ 时, 选取 3 次 Legendre 正交多项式 $P_3(x) = (5x^3 - 3x)/2$ 的零点 $x_0 = -\frac{\sqrt{15}}{5}, x_1 = 0, x_2 = \frac{\sqrt{15}}{5}$ 作为 Gauss 点, 仿 2 点 Gauss-Legendre 求积公式, 求出相应的求积系数, 可构造出具有 $2n+1=5$ 次代数精确度的 3 点 Gauss-Legendre 求积公式

$$\int_{-1}^1 f(x) dx \approx \frac{5}{9} f\left(-\frac{\sqrt{15}}{5}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\frac{\sqrt{15}}{5}\right). \quad (4.45)$$

当 $n=3, 4, 5, \dots$, 可求得相应的 x_k 和 A_k 如表 4.6 所示.

表 4.6

n	节点数	x_k	A_k
1	2	$\pm 1/\sqrt{3}$	1.0
2	3	$\pm \sqrt{3/5}$	5/9
		0	8/9
3	4	± 0.861136	0.347855
		± 0.339981	0.652145
4	5	± 0.906180	0.236927
		± 0.538469	0.478629
		0	0.568889

对于构造任意区间 $[a, b]$ 上的 Gauss 公式, 只要作变量置换

$$x = (a+b)/2 + (b-a)t/2,$$

使 $x \in [a, b]$ 时 $t \in [-1, 1]$, 则有

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{a+b}{2} + \frac{b-a}{2}t\right) dt \approx \frac{b-a}{2} \sum_{k=0}^n A_k f\left(\frac{a+b}{2} + \frac{b-a}{2}t_k\right). \quad (4.46)$$

其中, t_k 和 A_k 用表 4.6 中相应的值.

例 4.12 构造 $\int_1^5 f(x) dx$ 的 Gauss-Legendre 求积公式, 使其具有 5 次代数精确度.

解 由代数精确度 $2n+1=5$, 得 $n=2$, 即 3 个节点. 作变量置换 $x=3+2t$, $dx=2dt$, 则有

$$\int_1^5 f(x) dx = 2 \int_{-1}^1 f(3+2t) dt \approx 2 \sum_{k=0}^2 A_k f(3+2t_k).$$

查表得, $t_0 = -\sqrt{3/5}$, $t_1 = 0$, $t_2 = \sqrt{3/5}$, $A_0 = A_2 = 5/9$, $A_1 = 8/9$. 代入上式, 得到 Gauss-Legendre 求积公式

$$\int_1^5 f(x) dx \approx 2 \left\{ \frac{5}{9} [f(3-2\sqrt{3/5}) + f(3+2\sqrt{3/5})] + \frac{8}{9} f(3) \right\}.$$

下面构造复化 Gauss-Legendre 求积公式. 将 $[a, b]$ 分成 n 等分, 分点为 $x_k = a + kh$ ($k=0, 1, 2, \dots, n; h=(b-a)/n$), 在每个子区间 $[x_k, x_{k+1}]$ 上用 2 点 Gauss-Legendre 求积公式

$$\begin{aligned} \int_{x_k}^{x_{k+1}} f(x) dx &\approx \frac{x_{k+1}-x_k}{2} \left[f\left(\frac{x_k+x_{k+1}}{2} - \frac{x_{k+1}-x_k}{2} \frac{1}{\sqrt{3}}\right) \right. \\ &\quad \left. + f\left(\frac{x_k+x_{k+1}}{2} + \frac{x_{k+1}-x_k}{2} \frac{1}{\sqrt{3}}\right) \right] \end{aligned}$$

在 $[a, b]$ 区间上的复化积分公式为

$$\int_a^b f(x) dx \approx \frac{h}{2} \sum_{k=0}^{n-1} \left[f\left(x_{k+\frac{1}{2}} - \frac{h}{2\sqrt{3}}\right) + f\left(x_{k+\frac{1}{2}} + \frac{h}{2\sqrt{3}}\right) \right], \quad (4.47)$$

称为复化 Gauss-Legendre I 型求积公式.

将区间 $[a, b]$ 分成 $2n$ 等份, $x_k = a + kh$ ($k=0, 1, 2, \dots, 2n; h=(b-a)/(2n)$), 在每个区间 $[x_k, x_{k+2}]$ 上用 3 点 Gauss-Legendre 求积公式

$$\begin{aligned} \int_{x_k}^{x_{k+2}} f(x) dx &\approx \frac{x_{k+2}-x_k}{2} \left\{ \frac{5}{9} \left[f\left(\frac{x_k+x_{k+2}}{2} - \frac{x_{k+2}-x_k}{2} \sqrt{\frac{3}{5}}\right) \right. \right. \\ &\quad \left. \left. + f\left(\frac{x_k+x_{k+2}}{2} + \frac{x_{k+2}-x_k}{2} \sqrt{\frac{3}{5}}\right) \right] + \frac{8}{9} f(x_{k+1}) \right\} \end{aligned}$$

在 $[a, b]$ 区间上的复化积分公式为

$$\int_a^b f(x) dx \approx \frac{h}{9} \sum_{k=0}^{n-1} \left\{ 5 \left[f\left(x_{2k+1} - h\sqrt{\frac{3}{5}}\right) + f\left(x_{2k+1} + h\sqrt{\frac{3}{5}}\right) \right] + 8f(x_{2k+1}) \right\}, \quad (4.48)$$

称为复化 Gauss-Legendre II 型求积公式.

例 4.13 给定积分 $\int_0^{\frac{\pi}{2}} x^2 \cos x dx$, 分别用 3 段复化梯形公式, 2 段复化 Simpson 公式, 4 点 Gauss-Legendre 公式和 2 段 Gauss-Legendre I 型求积公式计算, 并比较计算结果.

解 该定积分的准确值是 0.4674011.

(1) 3 段 4 点复化梯形公式: $h = \frac{\pi}{6}$.

$$T(f) = \frac{h}{2} \left[f(0) + 2(f(h) + f(2h)) + f\left(\frac{\pi}{2}\right) \right] = 0.411411.$$

(2) 2 段 5 点复化 Simpson 公式: $h = \frac{\pi}{8}$.

$$S(f) = \frac{h}{3} \left[f(0) + 4(f(h) + f(3h)) + 2f(2h) + f\left(\frac{\pi}{2}\right) \right] = 0.466890.$$

(3) 4 点 Gauss-Legendre 公式.

$$G(f) = \frac{\pi}{4} \int_{-1}^1 g(t) dt \approx \frac{\pi}{4} \sum_{k=0}^3 A_k g(t_k) = 0.467402.$$

其中, $g(t) = \left[\frac{\pi}{4}(1+t) \right]^2 \cos\left(\frac{\pi}{4}(1+t)\right)$, $t_3 = -t_0 = 0.861136$, $t_2 = -t_1 = 0.339981$, $A_3 = A_0 = 0.347855$, $A_2 = A_1 = 0.652145$.

(4) 2 段复化 Gauss-Legendre I 型求积公式: $h = \frac{\pi}{4}$, $x_0 = 0$, $x_1 = \frac{\pi}{4}$, $x_2 = \frac{\pi}{2}$.

$$G_2(f) \approx \frac{\pi}{8} \left[f\left(\frac{\pi}{8} - \frac{\pi}{8\sqrt{3}}\right) + f\left(\frac{\pi}{8} + \frac{\pi}{8\sqrt{3}}\right) + f\left(\frac{3\pi}{8} - \frac{\pi}{8\sqrt{3}}\right) + f\left(\frac{3\pi}{8} + \frac{\pi}{8\sqrt{3}}\right) \right] \\ = 0.467744.$$

显然, Gauss 求积公式的精确度最高, 特别是 4 点 Gauss 公式.

2. Gauss-Chebyshev 求积公式

区间 $[-1, 1]$ 上权函数为 $\rho(x) = \frac{1}{\sqrt{1-x^2}}$ 的正交多项式是 Chebyshev 正交多项式

$$T_n(x) = \cos(n \arccos x).$$

选取其 $(n+1)$ 次多项式的零点

$$x_k = \cos \frac{2k+1}{2(n+1)} \pi, \quad k = 0, 1, 2, \dots, n$$

为 Gauss 点. 经证明(证明略), 相应的求积系数为

$$A_k = \int_{-1}^1 \frac{1}{\sqrt{1-x^2}} l_k(x) dx = \frac{\pi}{n+1}, \quad k = 0, 1, 2, \dots, n, \quad (4.49)$$

其中, $l_k(x)$ 是关于所选节点的 Lagrange 插值基函数. 从而有 Gauss-Chebyshev 求积公式

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \frac{\pi}{n+1} \sum_{k=0}^n f(x_k). \quad (4.50)$$

当 $n=1$ 时, 取 2 次 Chebyshev 多项式 $T_2(x)=2x^2-1$ 的零点为 Gauss 点, 即

$$x_0 = -\frac{1}{\sqrt{2}}, \quad x_1 = \frac{1}{\sqrt{2}}.$$

由式(4.49), 求得 $A_0=A_1=\frac{\pi}{2}$, 于是得到 2 点 Gauss-Chebyshev 求积公式

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \frac{\pi}{2} \left[f\left(-\frac{1}{\sqrt{2}}\right) + f\left(\frac{1}{\sqrt{2}}\right) \right]. \quad (4.51)$$

同理, 当 $n=2$ 时, 可构造出 3 点 Gauss-Chebyshev 求积公式

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \frac{\pi}{3} \left[f\left(-\frac{\sqrt{3}}{2}\right) + f(0) + f\left(\frac{\sqrt{3}}{2}\right) \right]. \quad (4.52)$$

带权 Gauss 求积方法, 能把复杂积分化简, 还可以直接计算奇异积分.

例 4.14 选择适当的数值积分法计算下列奇异积分 $I = \int_{-1}^1 \frac{e^x}{\sqrt{1-x^2}} dx$.

解 这里, 可将 $\frac{1}{\sqrt{1-x^2}}$ 当作权, 设 $f(x)=e^x$, 则可选用 Gauss-Chebyshev 求积公式计算.

取 $n=4$, 由式(4.50)可得

$$I \approx \frac{\pi}{5} \sum_{k=0}^4 e^{\cos \frac{2k+1}{10}\pi} \approx 3.977463.$$

当 $n=3, 4, 5, \dots$ 时, 用 Matlab 库函数 roots, 求出相应 Chebyshev 多项式的零点 x_k 如表 4.7 所示.

表 4.7

n	x_k	n	x_k
1	± 0.7071068	5	$\pm 0.2588190, \pm 0.7071068, \pm 0.9659258$
2	$0, \pm 0.8660254$	6	$0, \pm 0.4338837, \pm 0.7818315, \pm 0.9749279$
3	$\pm 0.3826834 \quad \pm 0.9238795$	7	$\pm 0.1950903, \pm 0.5555702, \pm 0.8314696$
4	$0, \pm 0.9510565 \quad \pm 0.5877853$		0.9807853

4.5.3 Gauss 公式的余项

定理 4.9 设 $f \in C^{2n+2}[a, b]$, 则 Gauss 求积公式

$$\int_a^b \rho(x) f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$$

的余项是

$$\begin{aligned} R_G(f) &= \int_a^b \rho(x) f(x) dx - \sum_{k=0}^n A_k f(x_k) \\ &= \frac{f^{(2n+2)}(\eta)}{(2n+2)!} \int_a^b \rho(x) \omega_{n+1}^2(x) dx, \quad \eta \in (a, b). \end{aligned} \quad (4.53)$$

证 因为 Gauss 公式也是插值型的, 仍然用插值原理来证明. Gauss 点 $x_k (k=0, 1, 2, \dots, n)$ 是使求积公式达到 $(2n+1)$ 次代数精确度的点. 现构造在 Gauss 点上满足如下插值条件的不超过 $(2n+1)$ 次的 Hermite 插值多项式 $H(x)$

$$H(x_k) = f(x_k), \quad H'(x_k) = f'(x_k), \quad k = 0, 1, 2, \dots, n.$$

其插值余项是

$$f(x) - H(x) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \omega_{n+1}^2(x), \quad \xi = \xi(x) \in (a, b).$$

代入定积分, 则有

$$\int_a^b \rho(x) f(x) dx - \int_a^b \rho(x) H(x) dx = \int_a^b \rho(x) \frac{f^{(2n+2)}(\xi(x))}{(2n+2)!} \omega_{n+1}^2(x) dx. \quad (4.54)$$

根据 Gauss 公式具有 $(2n+1)$ 次代数精确度, 它对不超过 $(2n+1)$ 次的多项式 $H(x)$ 应准确成立, 又由插值公式, 得到

$$\int_a^b \rho(x) H(x) dx = \sum_{k=0}^n A_k H(x_k) = \sum_{k=0}^n A_k f(x_k). \quad (4.55)$$

因为 $\omega_{n+1}^2(x)$ 在 $[a, b]$ 内保号, 因此, 对式(4.54)中的右端项积分可用积分中值定理

$$\int_a^b \frac{f^{(2n+2)}(\xi(x))}{(2n+2)!} \rho(x) \omega_{n+1}^2(x) dx = \frac{f^{(2n+2)}(\eta)}{(2n+2)!} \int_a^b \rho(x) \omega_{n+1}^2(x) dx. \quad (4.56)$$

将式(4.55)和式(4.56)代入式(4.54), 最后有

$$R_G(f) = \int_a^b \rho(x) f(x) dx - \sum_{k=0}^n A_k f(x_k) = \frac{f^{(2n+2)}(\eta)}{(2n+2)!} \int_a^b \rho(x) \omega_{n+1}^2(x) dx.$$

证毕.

例如, 对于两点 Gauss-Legendre 求积公式, 其 Gauss 点为 $x_0 = -1/\sqrt{3}, x_1 = 1/\sqrt{3}$, 得到余项

$$\begin{aligned} R_{GL}(f) &= \frac{f^{(4)}(\eta)}{4!} \int_{-1}^1 \left(x + \frac{1}{\sqrt{3}}\right)^2 \left(x - \frac{1}{\sqrt{3}}\right)^2 dx \\ &= \frac{f^{(4)}(\eta)}{135}, \quad \eta \in (-1, 1). \end{aligned}$$

对于 2 点 Gauss-Chebyshev 求积公式, 其 Gauss 点为 $x_0 = -1/\sqrt{2}, x_1 = 1/\sqrt{2}$, 得到余项

$$\begin{aligned} R_{GC}(f) &= \frac{f^{(4)}(\eta)}{4!} \int_{-1}^1 \frac{1}{\sqrt{1-x^2}} \left(x + \frac{1}{\sqrt{2}}\right)^2 \left(x - \frac{1}{\sqrt{2}}\right)^2 dx \\ &= \frac{\pi f^{(4)}(\eta)}{192}, \quad \eta \in (-1, 1). \end{aligned}$$

下面讨论 Gauss 求积公式的稳定性与收敛性.

定理 4.10 Gauss 求积式(4.39)的求积系数 $A_k (k=0, 1, 2, \dots, n)$ 全是正的.

证 考察

$$l_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j},$$

它是 n 次多项式, 因而 $l_k^2(x)$ 是 $2n$ 次多项式, 故 Gauss 求积式(4.39)对于它能准确成立, 即有

$$0 < \int_a^b l_k^2(x) \rho(x) dx = \sum_{i=0}^n A_i l_k^2(x_i).$$

注意到 $l_k(x_i) = \delta_{ki}$, 上式右端实际上即等于 A_k , 从而有

$$A_k = \int_a^b l_k^2(x) \rho(x) dx > 0.$$

证毕.

由本定理及定理 4.3, 则得以下推论.

推论 Gauss 求积式(4.39)是稳定的.

定理 4.11 设 $f(x) \in C[a, b]$, 则 Gauss 求积式(4.39)是收敛的, 即

$$\lim_{n \rightarrow \infty} \sum_{k=0}^n A_k f(x_k) = \int_a^b f(x) \rho(x) dx.$$

证明 记 $I_n(f) = \sum_{k=0}^n A_k f(x_k)$, 由于 $f \in C[a, b]$, 利用 Weierstrass 定理有, 任给 $\epsilon > 0$, 存在多项式 $P(x)$ (设其为 m 次)满足

$$\|f(x) - P(x)\|_{\infty} < \frac{\epsilon}{2 \int_a^b \rho(x) dx},$$

$$\begin{aligned} \text{则} \quad & \left| \int_a^b f(x) \rho(x) dx - I_n(f) \right| \\ &= \left| \int_a^b f(x) \rho(x) dx - \int_a^b P(x) \rho(x) dx \right. \\ &\quad \left. + \int_a^b P(x) \rho(x) dx - I_n(P) + I_n(P) - I_n(f) \right| \\ &\leq \left| \int_a^b f(x) \rho(x) dx - \int_a^b P(x) \rho(x) dx \right| \\ &\quad + \left| \int_a^b P(x) \rho(x) dx - I_n(P) \right| + |I_n(P) - I_n(f)|. \end{aligned}$$

对上式右端三项分别估计如下:

$$\begin{aligned} \left| \int_a^b f(x) \rho(x) dx - \int_a^b P(x) \rho(x) dx \right| &\leq \int_a^b |f(x) - P(x)| \rho(x) dx \\ &\leq \|f - P\|_{\infty} \int_a^b \rho(x) dx < \frac{\epsilon}{2}. \end{aligned}$$

因多项式 $P(x)$ 为 m 次, 当 $m \leq 2n+1$, 即 $n \geq \frac{m-1}{2}$ 时, Gauss 求积公式准确成立, 有

$$\begin{aligned} \left| \int_a^b P(x) \rho(x) dx - I_n(P) \right| &= 0, \\ |I_n(P) - I_n(f)| &= \left| \sum_{k=0}^n A_k [P(x_k) - f(x_k)] \right| \\ &\leq \sum_{k=1}^n |A_k| |P(x_k) - f(x_k)| \\ &\leq \frac{\varepsilon}{2 \int_a^b \rho(x) dx} \cdot \int_a^b \rho(x) dx = \frac{\varepsilon}{2}. \end{aligned}$$

由上可知, 当 $n > N \left(> \frac{m-1}{2} \right)$ 时, 定理结论成立.

4.6 数值微分

4.6.1 中点方法与误差分析

数值微分就是用函数值的线性组合近似函数在某点的导数值. 按导数定义可以简单地用差商近似导数, 这样立即得到几种数值微分公式

$$\begin{aligned} f'(a) &\approx \frac{f(a+h) - f(a)}{h}, \\ f'(a) &\approx \frac{f(a) - f(a-h)}{h}, \\ f'(a) &\approx \frac{f(a+h) - f(a-h)}{2h}, \end{aligned} \quad (4.57)$$

其中, h 为一增量, 称为步长. 后一种数值微分方法称为中点方法, 它其实是前两种方法的算术平均. 但它的误差阶却由 $O(h)$ 提高到 $O(h^2)$. 上面给出的三个公式是很实用的, 尤其是中点公式更为常用.

为要利用中点公式

$$G(h) = \frac{f(a+h) - f(a-h)}{2h}$$

计算导数 $f'(a)$ 的近似值, 首先必须选取合适的步长, 为此需要进行误差分析. 分别将 $f(a \pm h)$ 在 $x=a$ 处做 Taylor 展开, 有

$$\begin{aligned} f(a \pm h) &= f(a) \pm hf'(a) + \frac{h^2}{2!} f''(a) \pm \frac{h^3}{3!} f'''(a) \\ &\quad + \frac{h^4}{4!} f^{(4)}(a) \pm \frac{h^5}{5!} f^{(5)}(a) + \dots, \end{aligned}$$

代入上式得

$$G(h) = f'(a) + \frac{h^2}{3!}f'''(a) + \frac{h^4}{5!}f^{(5)}(a) + \dots$$

由此得知,从截断误差的角度看,步长越小,计算结果越准确,且

$$|f'(a) - G(h)| \leq \frac{h^2}{6}M. \quad (4.58)$$

其中, $M \geq \max_{|x-a| \leq h} |f'''(x)|$.

再考察舍入误差,按中点公式计算,当 h 很小时,因 $f(a+h)$ 与 $f(a-h)$ 很接近,直接相减会造成有效数字的严重损失. 因此,从舍入误差的角度来看,步长是不宜太小的.

例如,用中点公式求 $f(x) = \sqrt{x}$ 在 $x=2$ 处的一阶导数

$$G(h) = \frac{\sqrt{2+h} - \sqrt{2-h}}{2h},$$

设取 4 位数字计算,结果见表 4.8(导数的准确值 $f'(2) = 0.353553$).

表 4.8

h	$G(h)$	h	$G(h)$	h	$G(h)$
1	0.3660	0.05	0.3540	0.001	0.3500
0.5	0.3564	0.01	0.3500	0.0005	0.4000
0.1	0.3535	0.005	0.3600	0.0001	0.0000

从表 4.8 中看到 $h=0.1$ 的逼近效果最好,如果进一步缩小步长,则逼近效果反而变差. 这里因为当 $f(a+h)$ 及 $f(a-h)$ 分别有舍入误差 ϵ_1 及 ϵ_2 . 若令 $\epsilon = \max\{|\epsilon_1|, |\epsilon_2|\}$, 则计算 $f'(a)$ 的舍入误差上界为

$$\delta(f'(a)) = |f'(a) - G(a)| \leq \frac{|\epsilon_1| + |\epsilon_2|}{2h} = \frac{\epsilon}{h}.$$

它表明 h 越小,舍入误差 $\delta(f'(a))$ 越大,故它是病态的. 用中点式(4.57)计算 $f'(a)$ 的误差上界为

$$E(h) = \frac{h^2}{6}M + \frac{\epsilon}{h},$$

要使误差 $E(h)$ 最小,步长 h 不宜太大,也不宜太小,其最优步长应为 $h_{\text{opt}} = \sqrt[3]{3\epsilon/M}$.

4.6.2 插值型的求导公式

对于列表函数 $y=f(x)$ (见表 4.9)运用插值原理,可以建立插值多项式 $y=P_n(x)$ 作为它的近似. 由于多项式的求导比较容易,取 $P'_n(x)$ 的值作为 $f'(x)$ 的近似值,这样建立的数值公式

表 4.9

x	x_0	x_1	x_2	$\dots$	x_n
y	y_0	y_1	y_2	$\dots$	y_n

$$f'(x) \approx P'_n(x) \quad (4.59)$$

统称插值型的求导公式.

必须指出,即使 $f(x)$ 与 $P_n(x)$ 的值相差不多,导数的近似值 $P'_n(x)$ 与导数的真值 $f'(x)$ 仍然可能差别很大,因而在使用求导式(4.59)时应特别注意误差的分析.

依据插值余项定理,求导式(4.59)的余项为

$$f'(x) - P'_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega'_{n+1}(x) + \frac{\omega_{n+1}(x)}{(n+1)!} \frac{d}{dx} f^{(n+1)}(\xi),$$

其中,

$$\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i).$$

在这一余项公式中,由于 ξ 是 x 的未知函数,无法对它的第二项 $\frac{\omega_{n+1}(x)}{(n+1)!} \frac{d}{dx} f^{(n+1)}(\xi)$ 做出进一步的说明,因此,对于随意给出的点 x ,误差 $f'(x) - P'_n(x)$ 是无法预估的.但是,如果限定求某个节点 x_k 上的导数值,那么上面的第二项因式 $\omega_{n+1}(x_k)$ 变为零,这时有余项公式

$$f'(x_k) - P'_n(x_k) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega'_{n+1}(x_k). \quad (4.60)$$

下面仅仅考察节点处的导数值.为简化讨论,假定所给的节点是等距的.

1. 两点公式

设已给出两个节点 x_0, x_1 上的函数值 $f(x_0), f(x_1)$, 做线性插值,得公式

$$P_1(x) = \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1).$$

对上式两端求导,记 $x_1 - x_0 = h$, 有

$$P'_1(x) = \frac{1}{h} [-f(x_0) + f(x_1)].$$

于是有下列求导公式

$$P'_1(x_0) = \frac{1}{h} [f(x_1) - f(x_0)],$$

$$P'_1(x_1) = \frac{1}{h} [f(x_1) - f(x_0)].$$

而利用余项式(4.60)知,带余项的两点公式是

$$f'(x_0) = \frac{1}{h} [f(x_1) - f(x_0)] - \frac{h}{2} f''(\xi),$$

$$f'(x_1) = \frac{1}{h} [f(x_1) - f(x_0)] + \frac{h}{2} f''(\xi).$$

2. 3点公式

设已给出3个节点 $x_0, x_1 = x_0 + h, x_2 = x_0 + 2h$ 上的函数值,做二次插值多项式.

$$P_2(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}f(x_0) + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}f(x_1) \\ + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}f(x_2).$$

令 $x = x_0 + th$, 上式可表示为

$$P_2(x_0 + th) = \frac{1}{2}(t-1)(t-2)f(x_0) - t(t-2)f(x_1) + \frac{1}{2}t(t-1)f(x_2),$$

两端对 t 求导, 有

$$P'_2(x_0 + th) = \frac{1}{2h}[(2t-3)f(x_0) - (4t-4)f(x_1) \\ + (2t-1)f(x_2)]. \quad (4.61)$$

这里撇号(')表示对变量 t 求导数. 上式分别取 $t=0, 1, 2$, 得到三种三点公式:

$$P'_2(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_1) - f(x_2)],$$

$$P'_2(x_1) = \frac{1}{2h}[-f(x_0) + f(x_2)],$$

$$P'_2(x_2) = \frac{1}{2h}[f(x_0) - 4f(x_1) + 3f(x_2)].$$

而带余项的 3 点求导公式如下:

$$f'(x_0) = \frac{1}{2h}[-3f(x_0) + 4f(x_1) - f(x_2)] + \frac{h^2}{3}f'''(\xi), \\ f'(x_1) = \frac{1}{2h}[-f(x_0) + f(x_2)] - \frac{h^2}{6}f'''(\xi), \quad (4.62) \\ f'(x_2) = \frac{1}{2h}[f(x_0) - 4f(x_1) + 3f(x_2)] + \frac{h^2}{3}f'''(\xi).$$

其中的式(4.62)是中点公式. 在这 3 点公式中, 它由于少用了一个函数值 $f(x_1)$ 而引起注目.

用插值多项式 $P_n(x)$ 作为 $f(x)$ 的近似函数, 还可以建立高阶数值微分公式:

$$f^{(k)}(x) \approx P_n^{(k)}(x), \quad k = 1, 2, \dots.$$

例如, 将式(4.61)再对 t 求导一次, 有

$$P''_2(x_0 + th) = \frac{1}{h^2}[f(x_0) - 2f(x_1) + f(x_2)],$$

于是有

$$P''_2(x_1) = \frac{1}{h^2}[f(x_1 - h) - 2f(x_1) + f(x_1 + h)].$$

而带余项的 2 阶 3 点公式如下:

$$f''(x_1) = \frac{1}{h^2}[f(x_1 - h) - 2f(x_1) + f(x_1 + h)] - \frac{h^2}{12}f^{(4)}(\xi). \quad (4.63)$$

4.6.3 数值微分的外推算法

利用中点公式计算导数值时,

$$f'(x) \approx G(h) = \frac{1}{2h}[f(x+h) - f(x-h)],$$

对 $f(x)$ 在点 x 做 Taylor 级数展开有

$$f'(x) = G(h) + \alpha_1 h^2 + \alpha_2 h^4 + \cdots,$$

其中, $\alpha_i (i=1, 2, \cdots)$ 与 h 无关. 利用 Richardson 外推(见本章第 4 节)对 h 逐次二分, 若记 $G_0(h) = G(h)$, 则有

$$G_m(h) = \frac{4^m G_{m-1}\left(\frac{h}{2}\right) - G_{m-1}(h)}{4^m - 1}, \quad m = 1, 2, \cdots. \quad (4.64)$$

式(4.64)的计算过程见表 4.10, 表中 ① 为外推步数.

表 4.10

$G(h)$				
$G\left(\frac{h}{2}\right) \downarrow \xrightarrow{\textcircled{1}}$	$G_1(h)$			
$G\left(\frac{h}{2^2}\right) \downarrow \xrightarrow{\textcircled{2}}$	$G_1\left(\frac{h}{2}\right) \downarrow \xrightarrow{\textcircled{3}}$	$G_2(h)$		
$G\left(\frac{h}{2^3}\right) \downarrow \xrightarrow{\textcircled{4}}$	$G_1\left(\frac{h}{2^2}\right) \downarrow \xrightarrow{\textcircled{5}}$	$G_2\left(\frac{h}{2}\right) \downarrow \xrightarrow{\textcircled{6}}$	$G_3(h)$	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

根据 Richardson 外推方法, 式(4.64)的误差为

$$f'(x) - G_m(h) = O(h^{2(m+1)}).$$

由此看出当 m 较大时, 计算是很精确的. 考虑到舍入误差, 一般 m 不能取太大.

例 4.15 用外推法计算 $f(x) = x^2 e^{-x}$ 在 $x = 0.5$ 的导数.

解 令 $G(h) = \frac{1}{2h} \left[\left(\frac{1}{2} + h \right)^2 e^{-(\frac{1}{2}+h)} - \left(\frac{1}{2} - h \right)^2 e^{-(\frac{1}{2}-h)} \right]$, 当 $h = 0.1, 0.05, 0.025$ 时, 由外推法表 4.10, 可算得

$$G(0.1) = 0.4516049081,$$

$$G(0.05) = 0.4540761694 \downarrow \xrightarrow{\textcircled{1}} G_1(h) = 0.4548999232,$$

$$G(0.025) = 0.4546926288 \downarrow \xrightarrow{\textcircled{2}} G_1\left(\frac{h}{2}\right)$$

$$= 0.4548981153 \downarrow \xrightarrow{\textcircled{3}} G_2 = 0.454897995.$$

$f'(0.5)$ 的精确值为 0.454897995, 可见当 $h = 0.025$ 时用中点微分公式只有 3 位有效数字, 外推一次达到 5 位有效数字, 外推两次达到 9 位有效数字.

习 题 4

1. 确定下列求积公式中的待定参数,使其代数精度尽量高,并指明所构造出的求积公式所具有的代数精度.

$$(1) \int_{-h}^h f(x) dx \approx A_{-1} f(-h) + A_0 f(0) + A_1 f(h);$$

$$(2) \int_{-1}^1 f(x) dx \approx [f(-1) + 2f(x_1) + 3f(x_2)]/3.$$

2. 分别运用梯形公式、Simpson公式、Cotes公式计算积分 $\int_0^1 e^x dx$, 并估计各种方法的误差(要求小数点后至少保留5位).

3. 推导下列三种矩形求积公式.

$$\int_a^b f(x) dx = (b-a)f(a) + \frac{f'(\eta)}{2}(b-a)^2;$$

$$\int_a^b f(x) dx = (b-a)f(b) - \frac{f'(\eta)}{2}(b-a)^2;$$

$$\int_a^b f(x) dx = (b-a)f\left(\frac{a+b}{2}\right) + \frac{f''(\eta)}{24}(b-a)^3.$$

4. 已知 $x_0 = \frac{1}{4}, x_1 = \frac{1}{2}, x_2 = \frac{3}{4}$.

(1) 推导以这3个点作为求积节点在 $[0, 1]$ 上的插值型求积公式;

(2) 指明求积公式所具有的代数精度;

(3) 用所求公式计算 $\int_0^1 x^2 dx$.

5. 给定积分 $I = \int_0^1 \frac{\sin x}{x} dx$.

(1) 利用复化梯形公式计算上述积分值,使其截断误差不超过 $\frac{1}{2} \times 10^{-3}$;

(2) 取同样的求积节点,改用复化 Simpson 公式计算时,截断误差是多少?

(3) 要求截断误差不超过 10^{-6} ,如果用复化 Simpson 公式,应取多少个函数值?

6. 从地面发射一枚火箭,在最初 80 s 内记录其加速度如表 4.11 所示,试求火箭在第 80 s 时的速度.

表 4.11

t/s	0	10	20	30	40	50	60	70	80
$a/(m \cdot s^{-2})$	30.00	31.63	33.44	35.47	37.75	40.33	43.29	46.69	50.67

7. 用复化梯形公式和复化 Simpson 公式的事后误差估计计算定积分 $I = \int_0^1 \frac{4}{1+x^2} dx$, 要求精确到 10^{-3} .

8. 用 Romberg 算法计算积分 $\frac{2}{\sqrt{\pi}} \int_0^1 e^{-x} dx$, 要求误差不超过 10^{-5} .

9. 用下列方法计算积分 $\int_1^3 \frac{1}{y} dy$.

(1) Romberg 算法;

(2) 3 点及 5 点 Gauss-Legendre 求积公式;

(3) 将积分区间分为四等分, 用复化两点 Gauss 公式.

10. 建立 Gauss 型求积公式 $\int_0^1 \frac{1}{\sqrt{x}} f(x) dx \approx A_0 f(x_0) + A_1 f(x_1)$.

11. 证明: 求积公式 $\int_{-1}^1 f(x) dx \approx \frac{1}{9} [5f(\sqrt{0.6}) + 8f(0) + 5f(-\sqrt{0.6})]$ 对于次数不高于 5 的多项式准确成立, 并计算积分 $\int_0^1 \frac{\sin x}{1+x} dx$.

12. 设 $\{\varphi_n(x)\}$ 是 $[a, b]$ 上带权 $\rho(x)$ 的正交多项式序列, $x_i (i = 0, 1, 2, \dots, n)$ 为 $\varphi_{n+1}(x)$ 的零点, $l_i(x) (i = 0, 1, 2, \dots, n)$ 是以 $\{x_i\}$ 为节点的 Lagrange 插值基函数,

$\int_a^b \rho(x) f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$ 为 Gauss 型求积公式, 证明:

(1) 当 $0 \leq k, l \leq n, k \neq l$ 时 $\sum_{i=0}^n A_i \varphi_k(x_i) \varphi_l(x_i) = 0$;

(2) $\int_a^b \rho(x) l_k(x) l_j(x) dx = 0, k \neq j$;

(3) $\sum_{k=0}^n \int_a^b \rho(x) l_k^2(x) dx = \int_a^b \rho(x) dx$.

13. 应用两点 Gauss-Chebyshev 求积公式计算 $I = \int_0^{\frac{1}{3}} \frac{6x}{\sqrt{x(1-3x)}} dx$.

14. 设 $P_2(x)$ 是对 $f(x)$ 的二次插值多项式, 插值节点为 $0, h, 2h$, 用 $P_2(x)$ 导出积分 $I = \int_0^{3h} f(x) dx$ 的数值积分公式 I_h , 并用 Taylor 展开证明:

$$I - I_h = \frac{3}{8} h^4 f'''(0) + O(h^5).$$

15. 证明: 等式

$$n \sin \frac{\pi}{n} = \pi - \frac{\pi^3}{3!n^2} + \frac{\pi^5}{5!n^4} - \dots$$

成立. 试依据 $n \sin(\pi/n) (n=3, 6, 12)$ 的值, 用外推算法求 π 的近似值.

16. 用3点公式求 $f(x) = \frac{1}{(1+x)^2}$ 在 x 分别在 1.0, 1.1, 1.2 处的导数值, 并估计误差. $f(x)$ 的函数值由表 4.12 给出.

表 4.12

x_i	1.0	1.1	1.2
$f(x_i)$	0.250000	0.226757	0.206612

17. 用中点差商公式

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}$$

计算 $f(x) = \sqrt{x}$ 在 $x = 2$ 处的一阶导数的近似值, 取 $h = 0.5, 0.05, 0.005, 0.0005, 0.00005$, 观测计算结果, 解释所发生的现象. 设计高精度的算法, 并重新计算.

18. 完成下列填空题.

(1) 若用复化梯形公式计算积分 $\int_0^{1.5} e^{-x} dx$, 区间 $[0, 1.5]$ 应等分为 $n = \underline{\hspace{2cm}}$ 才能使截断误差不超过 $\frac{1}{2} \times 10^{-4}$.

(2) 当 n 为奇数时, N-C 求积公式 $I_n = (b-a) \sum_{i=1}^n C_i f(x_i)$ 的代数精度至少为 $\underline{\hspace{2cm}}$.

(3) 插值型求积公式 $\sum_{k=0}^n A_k f(x_k) \approx \int_a^b f(x) dx$ 的求积系数之和 $\sum_{k=0}^n A_k = \underline{\hspace{2cm}}$.

(4) 设 $f(x) = x^5 + 2x^4 + 3x^2 + 4$, 求积公式 $\sum_{k=0}^2 A_k f(x_k) \approx \int_a^b f(x) dx$ 是 Gauss 型的, 则 $\int_a^b f(x) dx - \sum_{k=0}^2 A_k f(x_k) = \underline{\hspace{2cm}}$.

数值实验题 4

实验 4.1

实验目的: 复化求积公式计算定积分.

实验题目: 数值计算下列各式右端定积分的近似值.

$$(1) \ln 2 - \ln 3 = -2 \int_2^3 \frac{1}{x^2 - 1} dx; \quad (2) \pi = 4 \int_0^1 \frac{1}{1+x^2} dx;$$

$$(3) \frac{2}{\ln 3} = \int_0^1 3^x dx;$$

$$(4) e^2 = \int_1^2 x e^x dx.$$

实验要求:

(1) 若用复化梯形公式、复化 Simpson 公式和复化 Gauss-Legendre I 型公式做计算,要求绝对误差限为 $\epsilon = \frac{1}{2} \times 10^{-7}$,分别利用它们的余项对每种算法做出步长的事前估计.

(2) 分别用复化梯形公式、复化 Simpson 公式和复化 Gauss-Legendre I 型公式作计算.

(3) 将计算结果与精确解做比较,并比较各种算法的计算量.

实验 4.2

实验目的:比较复化 Simpson 方法与变步长(区间逐次分半求积法)Simpson 方法.

实验题目:计算下列定积分.

$$(1) \int_0^2 \left(\frac{x^6}{10} - x^2 + x \right) dx \quad (2) \int_0^1 x \sqrt{x} dx \quad (3) \int_5^{200} \frac{1}{\sqrt{x}} dx$$

实验要求:

(1) 分别用复化 Simpson 公式与变步长 Simpson 公式计算,要求绝对误差限为 $\epsilon = \frac{1}{2} \times 10^{-7}$,输出每种方法所需的节点数和积分近似值.

(2) 分析比较计算结果.

第5章 常微分方程的数值方法

在许多物理、力学、空间科学和其他科学技术问题中,建立的数学模型很多均以常微分方程的形式表示,在常微分方程课程中介绍过方程的各种性质和解法,然而要给出一般方程解析解的表达式是十分困难的,而且往往从解析解得到解的数值也不容易,所以在实际工作中要使用计算机计算解的数值.

比如,求解一阶常微分方程初值问题只有少数十分简单的微分方程才能求得其精确解,即使求出解,也往往由于复杂或在解的表达式中含有 $e^x, \ln x$ 等初等函数值的计算,得到的仍不是准确值,多数情形只能用近似方法求其近似解. 在微分方程一章中已经介绍过的级数解法、逐次逼近法等就是近似解法,这些方法可以求出解的近似表达式,称为近似解析方法;还有一类近似方法,它可以给出解在一些离散点上的近似值,通常称为数值方法.

本章主要研究求解一阶常微分方程初值问题的几个常用的数值方法. 初值问题

$$\begin{cases} y'(x) = f(x, y(x)), & a \leq x \leq b, \\ y(a) = y_0, \end{cases} \quad (5.1)$$

且 $f(x, y)$ 满足李普希茨(Lipschitz)条件,即存在常数 L , 使

$$|f(x, y) - f(x, \bar{y})| \leq L|y - \bar{y}|,$$

则由常微分方程的理论知道,初值问题(5.1)的解在区间 $[a, b]$ 上存在,而且唯一.

5.1 建立常微分方程数值方法的基本思想与途径

所谓微分方程数值解法,其基本思想是把求解区间和方程离散化,求出方程的解 $y(x)$ 在一系列离散点上的近似值. 因此,不同的离散方式就产生不同的数值解法,下面介绍离散化方法.

1. 将求解区间 $[a, b]$ 离散化

将求解区间 $[a, b]$ 离散化,是在 $[a, b]$ 上插入一系列分点 $\{x_k\}$, 使

$$a = x_0 < x_1 < \cdots < x_n < x_{n+1} < \cdots < x_N = b.$$

记 $h_n = x_{n+1} - x_n$ ($n=0, 1, 2, \cdots, N-1$), 称为步长,一般取 $h_n = h$ (常数), 节点 $x_n = x_0 + nh$ ($n=0, 1, 2, \cdots, N; h = \frac{b-a}{N}$) 称为等步长节点.

2. 将微分方程离散化

将微分方程离散化,通常有以下三种方法.

(1) 差商逼近法,即是用适当的差商逼近导数值.

(2) 数值积分法,其基本思想是先将初值问题(5.1)转化为积分方程

$$y(x_m) - y(x_n) = \int_{x_n}^{x_m} f(x, y(x)) dx \quad (y(x_0) = y_0, m > n).$$

然后将上式右端采用第4章介绍的数值积分离散化,从而获得原初值问题的一个离散差分格式.

(3) Taylor 展开法,其基本思想是首先构造一个关于真解及其有关信息的含参量算子,将算子中诸项在某点处按 Taylor 展式展开,合并该展式中的同类项并截去余项,然后令诸同类项系数为零,由此即可确定出原算子中的全部(或部分)参数,从而获得一个(或一类)关于数值解的差分方程.

5.2 Euler(欧拉)方法及其截断误差和阶

5.2.1 Euler 公式

Euler 方法是一种最简单的显式单步法.

用 x_n 处的向前、向后差商分别代替式(5.1)左边的微商,实现微分算子离散化,即

$$\frac{y(x_n + h) - y(x_n)}{h} \approx f(x_n, y(x_n)), \quad (5.2)$$

$$\frac{y(x_n + h) - y(x_n)}{h} \approx f(x_{n+1}, y(x_{n+1})). \quad (5.3)$$

令 y_n 为 $y(x_n)$ 的近似值,将上面两个近似式写成等式,整理后得

$$y_{n+1} = y_n + hf(x_n, y_n), \quad (5.4)$$

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}). \quad (5.5)$$

从 x_0 处的初值 y_0 开始,按式(5.4),逐点计算以后各点的值,即 $y_0 \rightarrow y_1 \rightarrow y_2 \rightarrow \cdots$ 称式(5.4)为显式 Euler 公式.由于式(5.5)的右端隐含有待求函数值 y_{n+1} ,不能逐步显式计算,称式(5.5)为隐式 Euler 公式或后退 Euler 公式.如果将两式做算术平均,就得到梯形公式

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]. \quad (5.6)$$

梯形公式也是隐式公式.统称以上的离散公式为差分公式.以上差分公式都是由 y_n 去计算 y_{n+1} ,故称它们为单步法.

5.2.2 梯形公式的计算

对于隐式方法,如果 $f(x, y)$ 是 y 的线性函数,则隐式公式可显式计算. 如 $\frac{dy}{dx} = xy + 5$, 其隐式 Euler 公式为 $y_{n+1} = y_n + h(x_{n+1}y_{n+1} + 5)$, 它有显形式 $y_{n+1} = (y_n + 5h)/(1 - hx_{n+1})$. 但当 $f(x, y)$ 是 y 的非线性函数时, 如 $\frac{dy}{dx} = \sqrt[3]{y} + 5x$, 其隐式 Euler 公式为 $y_{n+1} = y_n + h(\sqrt[3]{y_{n+1}} + 5x_{n+1})$, 显然, 它是关于 y_{n+1} 的非线性方程. 可以选择第 7 章的迭代法求解.

对于隐式公式, 每前进一步, 都要进行若干次迭代, 而且需要提供好的迭代初始值, 使迭代收敛尽量快. 下面以梯形公式为例, 介绍隐式公式的迭代算法.

梯形公式通常用迭代法求解. 例如, 先用 Euler 公式求出一个值 $y_{n+1}^{(0)}$ 作为初始近似值, 然后再用梯形公式进行迭代, 直至满足精度要求为止, 这时迭代公式为

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n), \\ y_{n+1}^{(m)} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(m-1)})], \quad m = 1, 2, \dots \end{cases} \quad (5.7)$$

当 h 充分小时, 迭代过程 (5.7) 是收敛的, 因为 $f(x, y)$ 关于 y 满足 Lipschitz 条件, 故

$$\begin{aligned} |y_{n+1}^{(m+1)} - y_{n+1}^{(m)}| &= \frac{h}{2} |f(x_{n+1}, y_{n+1}^{(m)}) - f(x_{n+1}, y_{n+1}^{(m-1)})| \\ &\leq \frac{hL}{2} |y_{n+1}^{(m)} - y_{n+1}^{(m-1)}|. \end{aligned}$$

由此可见, 只要 $\frac{hL}{2} < 1$, 序列 $y_{n+1}^{(0)}, y_{n+1}^{(1)}, y_{n+1}^{(2)}, \dots$ 便收敛. (5.8)

因此当 h 取得较小时, 可有 $\frac{hL}{2} < 1$, 这也是迭代过程 (5.7) 收敛的充分条件.

5.2.3 改进的 Euler 法

梯形法虽提高了精度, 但它是一种隐式算法, 需要借助于迭代过程求解, 计算量大, 而 Euler 法虽精度低, 但它是一种显式算法, 其计算量小.

综合使用这两种方法, 先用 Euler 方法求得一个初步的近似值, 记为 $\bar{y}_{n+1}$, 称之为预报值, 预报值 $\bar{y}_{n+1}$ 的精度不高, 用它替代梯形公式右端的 y_{n+1} 再直接计算, 得到校正值 y_{n+1} , 这样建立预报-校正系统.

$$\text{预报} \quad \bar{y}_{n+1} = y_n + hf(x_n, y_n),$$

$$\text{校正} \quad y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})]$$

称作改进的 Euler 公式. 这是一种一步显式公式, 它可表为嵌套形式

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_n + hf(x_n, y_n))], \quad (5.9)$$

或表为平均化形式

$$\begin{cases} y_p = y_n + hf(x_n, y_n), \\ y_c = y_n + hf(x_{n+1}, y_p), \\ y_{n+1} = \frac{1}{2}(y_p + y_c). \end{cases}$$

改进的 Euler 法的优点是实现了隐式显算的目的,且减少了计算量,式(5.9)还可写成

$$\begin{cases} y_{n+1} = y_n + h\left(\frac{1}{2}k_1 + \frac{1}{2}k_2\right), \\ k_1 = f(x_n, y_n), \\ k_2 = f(x_{n+1}, y_n + hk_1). \end{cases} \quad (5.9)'$$

公式中用到的斜率是两个点的斜率的加权平均,它为构造新的算法提供了新的途径,下节介绍的 R-K 方法就是这种思想的体现和发展.

例 5.1 初值问题

$$\begin{cases} y' = -2y - 4x, \\ y(0) = 2, \end{cases}$$

其精确解为 $y(x) = e^{-2x} - 2x + 1$. 取 $h=0.1$, 分别用显式 Euler 法、隐式 Euler 法和梯形公式计算,并与精确解比较.

解 此处 $f(x, y) = -2y - 4x$ 是 y 的线性函数,因此,隐式的 Euler 公式和梯形公式都可以改写成显式,分别为

$$\begin{aligned} y_{n+1} &= \frac{1}{1+2h}(y_n - 4hx_{n+1}), \\ y_{n+1} &= \frac{1}{1+h}(y_n - h(y_n + 2(x_n + x_{n+1}))). \end{aligned}$$

显式 Euler 公式为

$$y_{n+1} = (1-2h)y_n - 4hx_n,$$

取 $h=0.1$, 用 Matlab 计算结果列于表 5.1 中.

表 5.1

x_n	0.0	0.1	0.2	0.3	0.4	0.5
显式 Euler 法	2	1.6000	1.2400	0.9120	0.6096	0.3277
隐式 Euler 法	2	1.6333	1.2944	0.9787	0.6823	0.4019
梯形公式	2	1.6182	1.2694	0.9477	0.6481	0.3666
精确解	2	1.6187	1.2703	0.9488	0.6493	0.3679

在 $x_5=0.5$ 处,比较三种方法的误差 $|y_5 - y(x_5)|$. 显式 Euler 法和隐式 Euler 法的误差分别是 4.02×10^{-2} 和 3.4×10^{-2} , 而梯形公式的误差是 1.3×10^{-3} . 显然,不同的

方法具有不同的精确度,需要讨论这样的方法误差.

5.2.4 局部截断误差

初值问题(5.1)的单步法可用一般形式表示为

$$y_{n+1} = y_n + h\varphi(x_n, y_n, y_{n+1}, h), \quad (5.10)$$

其中,多元函数 φ 与 $f(x, y)$ 有关,当 φ 含有 y_{n+1} 时,方法是隐式的,若不含 y_{n+1} 则为显式方法.所以显式单步法可表示为

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h), \quad (5.11)$$

其中, $\varphi(x, y, h)$ 称为增量函数.例如对 Euler 式(5.4),有

$$\varphi(x, y, h) = f(x, y).$$

从 x_0 开始计算,如果考虑每一步产生的误差,直到 x_n ,则误差 $e_n = y(x_n) - y_n$ 称为该方法在 x_n 点的整体截断误差.分析和求得整体截断误差 e_n 是复杂的.为此,仅考虑从 x_n 到 x_{n+1} 的局部情况,并假设 x_n 之前的计算没有误差,即 $y_n = y(x_n)$,给出单步法的局部截断误差概念.

对一般显式单步法的局部截断误差可如下定义.

定义 5.1 设 $y(x)$ 是初值问题(5.1)的准确解,称

$$T_{n+1} = y(x_{n+1}) - y(x_n) - h\varphi(x_n, y(x_n), h) \quad (5.12)$$

为显式单步法(5.11)的局部截断误差.

例 5.2 试给出显式 Euler 法

$$y_{n+1} = y_n + hf(x_n, y_n)$$

的局部截断误差表达式.

解 设 $y_n = y(x_n)$,则

$$\begin{aligned} T_{n+1} &= y(x_{n+1}) - y_{n+1} \\ &= y(x_{n+1}) - (y_n + hf(x_n, y_n)) \\ &= y(x_{n+1}) - y(x_n) - hy'(x_n) \\ &= y(x_n) + y'(x_n)h + \frac{h^2}{2}y''(x_n) - y(x_n) - y'(x_n)h + O(h^3) \\ &= \frac{y''(x_n)}{2}h^2 + O(h^3). \end{aligned}$$

如图 5.1 所示的是显式 Euler 公式的整体截断误差和局部截断误差的示意图.

这里 $\frac{h^2}{2}y''(x_n)$ 称为局部截断误差主项,显然 $T_{n+1} = O(h^2)$,一般情形的定义如下.

定义 5.2 设 $y(x)$ 是初值问题(5.1)的准确解,若存在最大整数 p 使显式单步法(5.11)的局部截断误差满足

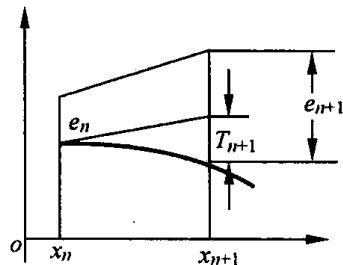


图 5.1

$$T_{n+1} = y(x+h) - y(x) - h\varphi(x, y, h) = O(h^{p+1}), \quad (5.13)$$

则称方法(5.10)是 p 阶的, 或称具有 p 阶精度.

若将式(5.13)写成

$$T_{n+1} = \psi(x_n, y(x_n))h^{p+1} + O(h^{p+2}),$$

则 $\psi(x_n, y(x_n))h^{p+1}$ 称为局部截断误差主项.

以上定义对隐式单步法(5.10)也是适用的. 例如, 对后退 Euler 法(5.5)其局部截断误差为

$$\begin{aligned} T_{n+1} &= y(x_{n+1}) - y(x_n) - hf(x_{n+1}, y(x_{n+1})) \\ &= hy'(x_n) + \frac{h^2}{2}y''(x_n) + O(h^3) - h[y'(x_n) + hy''(x_n) + O(h^2)] \\ &= -\frac{h^2}{2}y''(x_n) + O(h^3), \end{aligned}$$

这里 $p=1$, 是 1 阶方法, 局部截断误差主项为 $-\frac{h^2}{2}y''(x_n)$.

例 5.3 证明: 梯形法(5.6)是二阶方法.

$$\begin{aligned} \text{证} \quad T_{n+1} &= y(x_{n+1}) - y(x_n) - \frac{h}{2}[y'(x_n) + y'(x_{n+1})] \\ &= hy'(x_n) + \frac{h^2}{2}y''(x_n) + \frac{h^3}{3!}y'''(x_n) \\ &\quad - \frac{h}{2}[y'(x_n) + y'(x_n) + hy''(x_n) + \frac{h^2}{2}y'''(x_n)] + O(h^4) \\ &= -\frac{h^3}{12}y'''(x_n) + O(h^4). \end{aligned}$$

所以, 梯形方法(5.6)是二阶的, 其局部误差主项为 $-\frac{h^3}{12}y'''(x_n)$.

5.3 Runge-Kutta(龙格-库塔)方法

1. Runge-Kutta 方法

方法的阶越高, 方法就越精确. Euler 法和隐式 Euler 法各用了区间 $[x_n, x_{n+1}]$ 两端的一个导数, 它们都仅是一阶方法. 而梯形公式和改进 Euler 法方法都用了两端的两个导数, 它们的精度的阶提高了, 都是二阶方法. 这启发人们用区间上若干个点的导数 f , 将它们作线性组合得到平均斜率, 以提高方法的阶, 这就是 Runge-Kutta 方法的基本思想. 其一般形式为

$$y_{n+1} = y_n + h \sum_{i=1}^L \lambda_i k_i, \quad (5.14)$$

$$\begin{aligned}
k_1 &= f(x_n, y_n), \\
k_2 &= f(x_n + c_2 h, y_n + c_2 h k_1), \\
k_3 &= f(x_n + c_3 h, y_n + c_3 h(a_{31} k_1 + a_{32} k_2)), \\
&\dots\dots \\
k_i &= f\left(x_n + c_i h, y_n + c_i h \sum_{j=1}^{i-1} a_{ij} k_j\right), \quad i = 2, 3, \dots, L,
\end{aligned}$$

其中, $\sum_{i=1}^L \lambda_i = 1, 0 < c_i \leq 1, \sum_{j=1}^{i-1} a_{ij} = 1$. 它的局部截断误差是

$$T_{n+1} = y(x_{n+1}) - y(x_n) - h \sum_{i=1}^L \lambda_i \bar{k}_i, \quad (5.15)$$

其中, $\bar{k}_i$ 与 k_i 的区别在于: 用微分方程精确解 $y(x_n)$ 代替 k_i 中所含的 y_n 就得到 $\bar{k}_i$. 参数 λ_i, c_i 和 a_{ij} 待定, 确定它们的原则和方法是: 将式(5.15)中的 $y(x_{n+1})$ 在 x_n 做 Taylor 展开, $\bar{k}_i$ 在 $(x_n, y(x_n))$ 点做二元 Taylor 展开, 将展开式按 h 的幂次整理后, 令 T_{n+1} 中 $h^0, h^1, h^2, \dots$ 项的系数为零, 使 T_{n+1} 首项中 h 的幂次尽量高. 比如, 使 $T_{n+1} = O(h^{p+1})$, 则称式(5.14)为 L 级 p 阶 Runge-Kutta 方法(简称 R-K 法).

令 $L=2$, 则

$$\begin{aligned}
y_{n+1} &= y_n + h(\lambda_1 k_1 + \lambda_2 k_2), \\
k_1 &= f(x_n, y_n), \\
k_2 &= f(x_n + c_2 h, y_n + c_2 h k_1).
\end{aligned}$$

其局部截断误差为

$$T_{n+1} = y(x_{n+1}) - y(x_n) - h(\lambda_1 \bar{k}_1 + \lambda_2 \bar{k}_2). \quad (5.16)$$

将 T_{n+1} 中各项做 Taylor 展开, 特别是 $\bar{k}_2$ 在 $(x_n, y(x_n))$ 点做二元 Taylor 展开, 并利用

$$y'(x_n) = f(x_n, y(x_n)), \quad \frac{df}{dx} = f_x + f \cdot f_y = y''(x_n),$$

则有 $y(x_{n+1}) = y(x_n) + h y'(x_n) + \frac{h^2}{2} y''(x_n) + \frac{h^3}{3!} y'''(x_n) + O(h^4)$,

$$\begin{aligned}
\bar{k}_1 &= f(x_n, y(x_n)) = y'(x_n), \\
\bar{k}_2 &= f(x_n + c_2 h, y(x_n) + c_2 h y'(x_n)) \\
&= y'(x_n) + c_2 h y''(x_n) \\
&\quad + \frac{c_2^2 h^2}{2} [(f_{xx} + 2f_{xy} y'(x_n) + f_{yy} (y'(x_n))^2)] + O(h^3).
\end{aligned}$$

将它们代入式(5.16), 整理后得

$$\begin{aligned}
T_{n+1} &= (1 - \lambda_1 - \lambda_2) h y'(x_n) + \left(\frac{1}{2} - \lambda_2 c_2\right) h^2 y''(x_n) \\
&\quad + h^3 \left[\frac{1}{6} y'''(x) - \frac{\lambda_2 c_2^2}{2} (f_{xx} + 2f_{xy} y'_n + f_{yy} y'^2) \right] + O(h^4).
\end{aligned}$$

选 λ_1, λ_2 和 c_2 使方法的阶尽可能高, 就是使 h 和 h^2 的系数为零. 因为 h^3 的系数一般不为零, 于是得到方程组

$$\begin{cases} \lambda_1 + \lambda_2 = 1, \\ \lambda_2 c_2 = \frac{1}{2}. \end{cases}$$

这是有 3 个未知量的方程组, 其中有一个自由参数, 方程组有无穷多组解, 从而得到一族 2 级 2 阶 R-K 方法. 利用

$$y'''(x_n) = f_{xx} + 2f_{xy}y' + f_{yy}y'^2 + f_y y''(x_n),$$

得到它们的局部截断误差是

$$T_{n+1} = h^3 \left[\frac{1}{6} y''(x_n) f_y + \left(\frac{1}{6} - \frac{\lambda_2 c_2^2}{2} \right) (f_{xx} + 2f_{xy}y' + f_{yy}(y')^2) \right] + O(h^4).$$

若以 c_2 为自由参数, 分别取 $c_2 = \frac{1}{2}, \frac{2}{3}, 1$, 依次得到如下的三个著名方法:

(1) 中点公式

$$y_{n+1} = y_n + hf\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}f(x_n, y_n)\right); \quad (5.17)$$

(2) Heun(休恩)公式

$$y_{n+1} = y_n + \frac{h}{4} \left[f(x_n, y_n) + 3f\left(x_n + \frac{2h}{3}, y_n + \frac{2h}{3}f(x_n, y_n)\right) \right]; \quad (5.18)$$

(3) 改进 Euler 公式

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_n + h, y_n + hf(x_n, y_n))], \quad (5.19)$$

其中, $c_2 = \frac{2}{3}$ 即 Heun 公式, 它的截断误差主项为 $\frac{h^3}{6} y''(x_n) f_y$.

以上给出了构造 2 级 2 阶 R-K 方法的思路和方法, 可推广到 $L=3, 4, 5, \dots$, 构造出一族 3 级 3 阶, 一族 4 级 4 阶和一族 5 级 4 阶等 R-K 方法. 最常用的 4 级 4 阶公式是如下的经典 R-K 方法

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad (5.20)$$

$$k_1 = f(x_n, y_n),$$

$$k_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right),$$

$$k_4 = f(x_n + h, y_n + hk_3).$$

前面已看到, L 级 R-K 方法在 $L=1, 2, 3, 4$ 时, 可分别得到最高阶数 1, 2, 3, 4 阶. 但是, 通常 L 级 R-K 方法的最高阶不一定是 L 阶. 设 $P(L)$ 是 L 级 R-K 方法可达到的

最高阶数,可以证明如下.

$$P(5) = 4, \quad P(6) = 5, \quad P(7) = 6, \quad P(8) = 6, \quad P(9) = 7.$$

例 5.4 分别用显式 Euler 法(5.4),改进的 Euler 法(5.9)'及经典 R-K 方法(5.20)求解初值问题

$$\begin{cases} y' = 1 - \frac{2xy}{1+x^2}, & 0 \leq x \leq 2, \\ y(0) = 0. \end{cases}$$

取步长 $h=0.5$,并与精确解作比较.

解 用显式 Euler 法求解,计算公式为

$$\begin{cases} y_0 = 0, \\ y_{n+1} = y_n + h \left(1 - \frac{2x_n y_n}{1+x_n^2} \right) = y_n + h \left(1 - \frac{2nh y_n}{1+n^2 h^2} \right), \end{cases}$$

其中, $h=0.5, x_n=0.5n, y_0=0$. 计算结果见表 5.2.

用改进的 Euler 法求解,计算公式为

$$\begin{cases} k_1 = 1 - \frac{2x_n y_n}{1+x_n^2}, \\ k_2 = 1 - \frac{2(x_n+h)(y_n+hk_1)}{1+(x_n+h)^2}, \\ y_{n+1} = y_n + \frac{h}{2}(k_1+k_2), \end{cases}$$

其中, $h=0.5, x_n=0.5n, y_0=0$, 计算结果见表 5.2.

用经典 R-K 法(5.20)求解,计算公式为

$$\begin{cases} k_1 = 1 - \frac{2x_n y_n}{1+x_n^2}, \\ k_2 = 1 - \frac{2\left(x_n + \frac{1}{2}h\right)\left(y_n + \frac{1}{2}hk_1\right)}{1+\left(x_n + \frac{1}{2}h\right)^2}, \\ k_3 = 1 - \frac{2\left(x_n + \frac{1}{2}h\right)\left(y_n + \frac{1}{2}hk_2\right)}{1+\left(x_n + \frac{1}{2}h\right)^2}, \\ k_4 = 1 - \frac{2(x_n+h)(y_n+hk_3)}{1+(x_n+h)^2}, \\ y_{n+1} = y_n + \frac{h}{6}(k_1+2k_2+2k_3+k_4), \end{cases}$$

其中, $h=0.5, x_n=0.5n, y_0=0$. 计算结果见表 5.2.

表 5.2

x_n	0.0	0.5	1.0	1.5	2.0
显式 Euler 法	0	0.500000	0.800000	0.900000	0.984615
改进 Euler 法	0	0.400000	0.635000	0.787596	0.921025
经典 R-K 法	0	0.433218	0.666312	0.807423	0.933156
精确解	0	0.433333	0.666667	0.807692	0.933333

从表 5.2 可看出,用 Euler 法、改进的 Euler 法和经典 R-K 方法,使用同一步长 $h=0.5$ 求解同一个初值问题,经典 R-K 方法精确性最好,改进的 Euler 法次之,Euler 法的精确性最低.但经典 R-K 方法计算量较大.

类似于显式 R-K 公式,稍加改变,就得到隐式 R-K 公式

$$y_{n+1} = y_n + h \sum_{i=1}^L \lambda_i k_i,$$

$$k_i = f\left(x_n + c_i h, y_n + c_i h \sum_{j=1}^L a_{ij} k_j\right), \quad i = 1, 2, \dots, L.$$

它与显式 R-K 公式的区别在于:显式公式中,对系数 a_{ij} 求和的上限是 $(i-1)$,从而 a_{ij} 构成的矩阵是一个严格下三角阵;而在隐式公式中,对系数 a_{ij} 求和的上限是 L ,从而 a_{ij} 构成的矩阵是方阵,需要用迭代法求出 $k_i (i=1, 2, \dots, L)$. 推导隐式公式的思路和方法与显式 R-K 法类似.通常,同级的隐式公式能获得比显式公式更高的阶.常用的隐式 R-K 法有

1 级 2 阶中点公式

$$y_{n+1} = y_n + hf\left(x_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2}\right); \quad (5.21)$$

2 级 2 阶梯形公式

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]; \quad (5.22)$$

2 级 4 阶 R-K 公式

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2), \\ k_1 = f\left(x_n + \frac{3+\sqrt{3}}{6}h, y_n + \frac{1}{4}k_1h + \frac{3+2\sqrt{3}}{12}k_2h\right), \\ k_2 = f\left(x_n + \frac{3-\sqrt{3}}{6}h, y_n + \frac{3-2\sqrt{3}}{12}k_1h + \frac{1}{4}k_2h\right). \end{cases} \quad (5.23)$$

为了分析经典 R-K 公式的计算量和计算精度,将 4 阶经典 R-K 式(5.20)与 1 阶显式 Euler 式(5.4)及 2 阶改进 Euler 公式相比较.一般说来,公式的级数越大,计算右端项 f 的次数越多,计算量越大.4 阶 R-K 法的计算量差不多是改进 Euler 公式的 2 倍,

是显式 Euler 公式的 4 倍. 下面举例说明这 3 种方法计算量相当的情况下, 它们所能达到的计算精度.

例 5.5 考虑常微分方程初值问题

$$\begin{cases} y' = \frac{1}{2}(-y + x^2 + 4x - 1), \\ y(0) = 0, \end{cases}$$

其解析解为 $y(x) = e^{-\frac{x}{2}} + x^2 - 1$. 试分别用 $h=0.001$ 的一阶显式 Euler 公式, $h=0.002$ 的改进 Euler 公式和 $h=0.004$ 的经典 R-K 公式计算到 $x=0.020$. 三种方法在 x 方向每前进 0.004 都要计算 4 个右端函数值, 计算量相当. 用 Matlab 计算, 计算结果见表 5.3 中的计算结果均乘以 10^{-2} . 从计算结果看, 它们的最大绝对误差: Euler 法是 2×10^{-5} , 改进 Euler 法是 2×10^{-8} , 经典 R-K 法是 10^{-14} . 因此, 经典 R-K 法步长最大, 计算精度最高, 对多数好条件问题, 都能获得好的效果, 然而, 它不是万能的, 用它解刚性方程组时, 效果则不好.

表 5.3

x_n	Euler 法($\times 10^{-2}$)	改进 Euler 法($\times 10^{-2}$)	经典 R-K 法($\times 10^{-2}$)	精确值($\times 10^{-2}$)
0	0	0	0	0
0.004	-0.1986	-0.1981997	-0.198200133266	-0.198200133267
0.008	-0.3937	-0.3928002	-0.392801065600	-0.392801065601
0.012	-0.5851	-0.5838023	-0.583803594606	-0.583803594606
0.016	-0.7730	-0.7712068	-0.771208516293	-0.771208516294
0.020	-0.9573	-0.9550145	-0.955016625082	-0.955016625083

2. 变步长的 R-K 方法

单从每一步看, 步长越小, 截断误差就越小. 但随着步长的缩小, 在一定求解范围内所要完成的步数就增加了. 步数的增加不但引起计算量的增大, 而且可能导致舍入误差的严重积累. 在实际计算时, 可以利用 Richardson(李查逊)外推技巧来估计误差, 并自动选取步长. 设从 x_n 出发, 以 h 为步长, 经过一步计算, 得到 $y(x_{n+1})$ 的近似值, 记为 $y_{n+1}^{(h)}$. 若计算方法是 p 阶的, 则有

$$y(x_{n+1}) - y_{n+1}^{(h)} = ch^{p+1} + O(h^{p+2}). \quad (5.24)$$

然后, 将步长减半, 即取 $\frac{h}{2}$ 为步长, 从 x_n 出发经两步计算, 求得 $y(x_{n+1})$ 的近似值, 记为

$y_{n+1}^{(\frac{h}{2})}$, 有

$$y(x_{n+1}) - y_{n+1}^{(\frac{h}{2})} = 2c\left(\frac{h}{2}\right)^{p+1} + O(h^{p+2}). \quad (5.25)$$

当 h 很小时, 以上两式中的 c 可看作同一常数. 用 2^p 乘式(5.25)的两端后与式(5.24)相减, 得

$$y(x_{n+1}) = \frac{2^p y_{n+1}^{(\frac{h}{2})} - y_{n+1}^{(h)}}{2^p - 1} + O(h^{p+2}).$$

因此, 取 $y_{n+1} = \frac{2^p y_{n+1}^{(\frac{h}{2})} - y_{n+1}^{(h)}}{2^p - 1}$ 显然比 $y_{n+1}^{(h)}$ 或 $y_{n+1}^{(\frac{h}{2})}$ 的精度高. 当 $p=4$ 时, 即可取

$$y_{n+1} = \frac{1}{15} [16 y_{n+1}^{(\frac{h}{2})} - y_{n+1}^{(h)}].$$

由式(5.24)与式(5.25)中舍去 $O(h^{p+2})$ 后相比, 可得近似式

$$|y_{n+1}^{(\frac{h}{2})} - y(x_{n+1})| \approx \frac{1}{2^p - 1} |y_{n+1}^{(\frac{h}{2})} - y_{n+1}^{(h)}| = \delta.$$

上式表明, 以 $y_{n+1}^{(\frac{h}{2})}$ 作为 $y(x_{n+1})$ 的近似值, 其误差可用先后两次计算结果之差来表示. 由此可知, 只需考察 $\delta < \epsilon$ (允许误差) 是否成立. 如果 $\delta < \epsilon$, 则反复将步长扩大一倍进行计算, 直到 $\delta > \epsilon$, 再以上次的步长计算得到的值作为 y_{n+1} ; 如果 $\delta > \epsilon$, 则反复将步长减半计算, 直到 $\delta < \epsilon$, 将最后一次计算所得到的值作为 y_{n+1} . 这就是在计算中自动选择步长的方法, 也常称为变步长方法.

5.4 单步法收敛性与稳定性

5.4.1 单步法的收敛性

不论是显式 Euler 法还是 R-K 法, 它们都是由 y_n 推进到 y_{n+1} , 单步逐点计算, 统称为单步法, 其显形式由式(5.11)给出. 将式(5.11)改写为

$$\varphi(x_n, y_n, h) = \frac{y_{n+1} - y_n}{h}. \quad (5.26)$$

这样的差分公式在理论上是否合理, 要看差分方程的解 y_n 是否收敛于原微分方程的精确解 $y(x_n)$, 这是差分格式的收敛性问题. 当 $h \rightarrow 0$ 时, 式(5.26)是否逼近于微分方程(5.1), 即 $\lim_{h \rightarrow 0} \frac{y_{n+1} - y_n}{h} = y'(x_n)$, $\lim_{h \rightarrow 0} \varphi(x, y, h) = f(x, y)$ 是否成立, 这是差分格式是否与微分方程相容的问题. 若计算中某一步 y_n 有舍入误差, 逐步推进, 舍入误差传播情况如何, 这是差分格式的稳定性问题. 本节讨论的单步法都是相容的, 下面简单介绍收敛性. 对于一种收敛的相容的差分格式, 重点讨论其数值稳定性, 一个不稳定的差分格式会使计算解失真或计算失败.

定义 5.3 对于任意固定的 $x_n = x_0 + nh$, 当 $h \rightarrow 0$ (必然同时 $n \rightarrow \infty$) 时, 若单步法(5.11)产生的近似解 y_n 均有 $\lim_{h \rightarrow 0, n \rightarrow \infty} y_n = y(x_n)$, 则称方法是收敛的.

定理 5.1 对于一个 p 阶的显式单步法(5.11),若满足如下条件

(1) 增量函数 φ 关于 y 满足 Lipschitz 条件,即存在常数 $L_\varphi > 0$,使

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L_\varphi |y - \bar{y}|, \quad \forall y, \bar{y} \in \mathbb{R}$$

成立;

(2) 微分方程的初值是精确的,则该方法收敛,其整体截断误差为

$$|e_n| = |y(x_n) - y_n| = O(h^p).$$

证 令 $\bar{y}_{n+1}$ 表示当 $y_n = y(x_n)$ 时用式(5.11)求得的准确差分解,而方法(5.11)具有 p 阶精度,即其局部截断误差满足

$$|y(x_{n+1}) - \bar{y}_{n+1}| \leq Ch^{p+1},$$

其中, $\bar{y}_{n+1} = y_n + h\varphi(x_n, y_n, h) = y(x_n) + h\varphi(x_n, y(x_n), h)$.

若 $y_n \neq y(x_n)$,则各步的积累误差为

$$\bar{y}_{n+1} - y_{n+1} = y(x_n) - y_n + h[\varphi(x_n, y(x_n), h) - \varphi(x_n, y_n, h)],$$

从而,得

$$\begin{aligned} |\bar{y}_{n+1} - y_{n+1}| &\leq |y(x_n) - y_n| + h|\varphi(x_n, y(x_n), h) - \varphi(x_n, y_n, h)| \\ &\leq |y(x_n) - y_n| + hL_\varphi |y(x_n) - y_n| = (1 + hL_\varphi) |y(x_n) - y_n|. \end{aligned}$$

整体截断误差为

$$y(x_{n+1}) - y_{n+1} = \bar{y}_{n+1} - y_{n+1} + y(x_{n+1}) - \bar{y}_{n+1},$$

故

$$\begin{aligned} |y(x_{n+1}) - y_{n+1}| &\leq |\bar{y}_{n+1} - y_{n+1}| + |y(x_{n+1}) - \bar{y}_{n+1}| \\ &\leq (1 + hL_\varphi) |y(x_n) - y_n| + Ch^{p+1}, \end{aligned}$$

即得

$$|e_n| \leq (1 + hL_\varphi) |e_{n-1}| + Ch^{p+1}.$$

反复利用上式,得

$$|e_n| \leq (1 + hL_\varphi)^n |e_0| + \frac{Ch^p}{L_\varphi} [(1 + hL_\varphi)^n - 1].$$

因为 $h = \frac{b-a}{N}$, $n \leq N$, 故

$$(1 + hL_\varphi)^n \leq \left(1 + \frac{L_\varphi(b-a)}{N}\right)^N \leq e^{L_\varphi(b-a)}.$$

又 $e_0 = 0$, 故有

$$|e_n| \leq \frac{Ch^p}{L_\varphi} (e^{L_\varphi(b-a)} - 1).$$

显然,当 $h \rightarrow 0$ 时(同时 $n \rightarrow \infty$),有 $e_n \rightarrow 0$,故方法收敛,且收敛阶为 $O(h^p)$.

由证明的过程知,单步法的整体误差,由初值误差 ϵ_0 及局部截断误差界 Ch^{p+1} 所决定.当初值 $\epsilon_0 = 0$ 时, $|e_n| = O(h^p)$,可见整体截断误差比局部截断误差的阶数低一阶,因此,要构造高精度的计算方法,只需设法提高局部截断误差阶数即可.

依据此定理,判断单步法(5.11)的收敛性,归结为验证增量函数 φ 能否满足 Lipschitz 条件.

例 5.6 验证 Euler 方法及改进的 Euler 方法的收敛性.

解 对于 Euler 方法, 由于其增量函数 φ 就是 f , 故当 f 满足 Lipschitz 条件时它是收敛的.

再考察改进的 Euler 方法, 其增量函数有

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq \frac{1}{2} [|f(x, y) - f(x, \bar{y})| + |f(x+h, y+hf(x, y)) - f(x+h, \bar{y}+hf(x, \bar{y}))|].$$

假设 f 关于 y 满足 Lipschitz 条件, 记 Lipschitz 常数为 L , 则由上式推得

$$|\varphi(x, y, h) - \varphi(x, \bar{y}, h)| \leq L \left(1 + \frac{h}{2} L \right) |y - \bar{y}|,$$

限定 $h \leq h_0$ (h_0 为定数). 上式表明 φ 关于 y 的 Lipschitz 常数为 $L_\varphi = L \left(1 + \frac{h}{2} L \right)$, 因此改进的 Euler 方法也是收敛的.

类似地, 可验证其他 R-K 方法的收敛性.

定理 5.1 表明 $p \geq 1$ 时单步法收敛, 并且当 $y(x)$ 是初值问题(5.1)的解及式(5.11)具有 p 阶精度时, 则有展开式

$$\begin{aligned} T_{n+1} &= y(x+h) - y(x) - h\varphi(x, y(x), h) \\ &= y'(x)h + \frac{y''(x)}{2}h^2 + \cdots - h[\varphi(x, y(x), 0) + \varphi'_x(x, y(x), 0)h + \cdots] \\ &= h[y'(x) - \varphi(x, y(x), 0)] + O(h^2). \end{aligned}$$

所以, $p \geq 1$ 的充要条件是 $y'(x) - \varphi(x, y(x), 0) = 0$, 而 $y'(x) = f(x, y(x))$, 于是可给出如下定义.

定义 5.4 若单步法(5.11)的增量函数 φ 满足

$$\varphi(x, y, 0) = f(x, y),$$

则称单步法(5.11)与初值问题(5.1)相容.

以上讨论表明 p 阶方法(5.11)当 $p \geq 1$ 时与初值问题(5.1)相容, 反之相容方法至少是 1 阶的.

于是由定理 5.1 可知方法(5.11)收敛的充分必要条件是此方法是相容的.

5.4.2 单步法的稳定性

一个收敛的方法, 在实际进行计算时, 由于初始值不一定完全精确, 一般都带有一定的误差, 同时在计算过程中还会产生舍入误差. 初值误差和计算中的舍入误差传播下去, 对以后的计算结果都有影响, 稳定性问题是指这种误差的积累能否得到控制的问题, 稳定性的定义很多, 这里只介绍所谓绝对稳定性的概念.

定义 5.5 若一种数值方法在节点值 y_n 上有大小为 δ 的扰动, 对以后各节点值 y_m ($m > n$) 上产生的偏差均不超过 δ , 则称该方法是稳定的.

对于方法的数值稳定性, 为了讨论方便起见, 将方程(5.1)中的 $f(x, y)$ 在解域内某

一点 (a, b) 做 Taylor 展开并局部线性化

$$\begin{aligned} y' = f(x, y) &= f(a, b) + (x - a)f_x(a, b) + (y - b)f_y(a, b) + \cdots \\ &= f_y(a, b)y + c_1x + c_2 + \cdots \end{aligned}$$

忽略高阶项, 令 $\lambda = f_y(a, b)$, $y = u - \frac{c_1}{\lambda}x - \frac{c_2}{\lambda^2} - \frac{c_3}{\lambda}$. 对上式作变量置换, 得 $u' = \lambda u$.

因此, 一般形式的一阶微分方程总能化成如下的模型方程

$$y' = \lambda y \quad (\operatorname{Re} \lambda < 0). \quad (5.27)$$

本章数值方法的稳定性都是通过模型方程讨论的, 前提是求解好条件问题. 坏条件问题的解本身有固有的不稳定性, 不在讨论之列; 也不去讨论 $h \rightarrow 0$ 时差分方法的渐近稳定性. 因为实际计算时, 只能取有限的步长.

现在讨论显式 Euler 方法的稳定性. 将显式 Euler 法用于模型方程 (5.27), 有 $y_{n+1} = (1 + \lambda h)y_n$. 当 y_n 有舍入误差时, 其近似解为 $\tilde{y}_n$, 从而有 $\tilde{y}_{n+1} = (1 + \lambda h)\tilde{y}_n$. 令 $\epsilon_n = y_n - \tilde{y}_n$, 得到误差传播方程

$$\epsilon_{n+1} = (1 + \lambda h)\epsilon_n.$$

令 $E(\lambda h) = 1 + \lambda h$, 只要 $|E(\lambda h)| \leq 1$, 则显式 Euler 方法的解和误差都不会恶性发展, 即当 λ 为实数 $-2 \leq \lambda h \leq 0$ 时, 显式 Euler 方法是绝对稳定的, 当然, 它是条件稳定的.

对于梯形公式, 应用于模型方程后, 有 $y_{n+1} = y_n + \frac{h}{2}(\lambda y_n + \lambda y_{n+1})$, 整理后, 得

$$y_{n+1} = \frac{1 + \lambda h/2}{1 - \lambda h/2} y_n.$$

同理, 有误差方程 $\epsilon_{n+1} = E(\lambda h)\epsilon_n$, 其中

$$E(\lambda h) = \frac{1 + \lambda h/2}{1 - \lambda h/2}.$$

因为 $\lambda < 0$, 所以, 对任何 λh 都有 $|E(\lambda h)| < 1$, 梯形公式是无条件稳定的.

任何一种单步法应用于模型方程 (5.27), 其中, $\lambda = \frac{\partial f(x, y)}{\partial y}$ 均有

$$y_{n+1} = E(\lambda h)y_n. \quad (5.28)$$

对于不同的差分方法, $E(\lambda h)$ 有不同的表达式.

定义 5.6 若式 (5.28) 中 $|E(\lambda h)| \leq 1$, 则说该单步法是绝对稳定的. 在复平面上, λh 满足 $|E(\lambda h)| \leq 1$ 的区域, 称为方法的绝对稳定区域, 它与实轴的交称为绝对稳定区间.

如图 5.2(a)、(b)、(c) 所示的阴影部分分别是显式 Euler 法、隐式 Euler 法及梯形法的绝对稳定性区域.

各种单步法的 $E(\lambda h)$ 表达式和它们的绝对稳定区间见表 5.4.

从表上看到, 显式单步法的 $E(\lambda h)$ 是 $e^{\lambda h}$ 的 Taylor 近似展开式, 对 p 阶方法, 取展开式中的 $(p+1)$ 项. 原因很简单, 模型方程 (5.27) 的解析解是 $y(x_n) = e^{\lambda x_n}$, 显然,

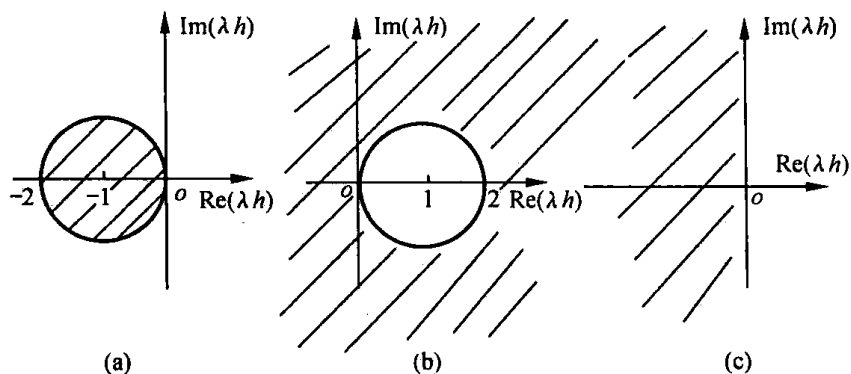


图 5.2

$y(x_{n+1}) = e^{\lambda h} y(x_n)$, 而一个 p 阶方法应用于模型方程后, 得到 $y_{n+1} = E(\lambda h) y_n$, 其局部截断误差为

$$T_{n+1} = y(x_{n+1}) - E(\lambda h) y(x_n) = e^{\lambda h} - E(\lambda h) y(x_n) = O(h^{p+1}).$$

因此, $E(\lambda h)$ 是 $e^{\lambda h}$ 的 p 阶近似. 但隐式单步法不符合这一规则, 如表 5.4 所示, $E(\lambda h)$ 是 $e^{\lambda h}$ 的一个有理分式逼近, 这里不再赘述.

表 5.4

方 法	$E(\lambda h)$	绝对稳定区间
Euler 法	$1 + \lambda h$	$-2 \leq \lambda h \leq 0$
改进 Euler 法, Heun 法	$1 + \lambda h + \frac{(\lambda h)^2}{2}$	$-2 \leq \lambda h \leq 0$
3 阶 R-K 法	$1 + \lambda h + \frac{(\lambda h)^2}{2} + \frac{(\lambda h)^3}{3!}$	$-2.51 \leq \lambda h \leq 0$
4 阶 R-K 法	$1 + \lambda h + \frac{(\lambda h)^2}{2} + \frac{(\lambda h)^3}{3!} + \frac{(\lambda h)^4}{4!}$	$-2.785 \leq \lambda h \leq 0$
隐式 Euler 法	$\frac{1}{1 - \lambda h}$	$-\infty \leq \lambda h \leq 0$
中点隐公式	$\frac{1 + \lambda h/2}{1 - \lambda h/2}$	$-\infty \leq \lambda h \leq 0$
梯形公式	$\frac{1 + \lambda h/2}{1 - \lambda h/2}$	$-\infty \leq \lambda h \leq 0$
2 级 4 阶 R-K 隐公式	$\frac{1 + \lambda h/2 + (\lambda h)^2/12}{1 - \lambda h/2 + (\lambda h)^2/12}$	$-\infty \leq \lambda h \leq 0$

绝对稳定区间(或区域)越大, 那么, 为使数值解法的计算过程具有数值稳定性, 对步长 h 的限制就越小.

例 5.7 用显式 Euler 法求解

$$\begin{cases} y' = -5y + x, \\ y(x_0) = y_0, \end{cases} \quad x_0 \leq x \leq Z,$$

从绝对稳定性考虑, 对步长 h 有何限制?

解 这里 $\lambda = f_y = -5$, 由

$$|1 + h\lambda| = |1 - 5h| < 1$$

得到对 h 的限制, $0 < h < 0.4$.

对于一般初值问题(5.1), 对其右端 $f(x, y)$ 进行线性化处理后可知 $\lambda = \frac{\partial f}{\partial y}$. 这时 λ 将是变化的, 但只要 $h\lambda = h \frac{\partial f}{\partial y}$ 属于所用方法的绝对稳定区域, 则该方法就是绝对稳定的.

例 5.8 用经典 R-K 方法(5.20)求解

$$\begin{cases} y' = 1 - \frac{10xy}{1+x^2}, & 0 \leq x \leq 10, \\ y(0) = 0, \end{cases}$$

从绝对稳定性考虑, 对步长 h 有何限制?

解 对于所给微分方程, 有

$$\lambda = \frac{\partial f}{\partial y} = -\frac{10x}{1+x^2} < 0, \quad 0 \leq x \leq 10,$$

$$\max_{0 \leq x \leq 10} |\lambda| = \max_{0 \leq x \leq 10} \frac{10x}{1+x^2} = 5.$$

根据经典 R-K 方法的绝对稳定区间 $-2.78 \leq h\lambda < 0$, 步长 h 应满足

$$-2.78 < -5h < 0, \quad 0 < h < 0.556.$$

在使用任何数值解法时, 步长 h 若不满足绝对稳定性的要求, 由于计算机的位数有限, 可能会产生很大的误差, 计算结果会完全失真. 试看下例.

例 5.9 常微分方程初值问题

$$\begin{cases} y' = -y + x - e^{-1}, \\ y(1) = 0, \end{cases}$$

其精确解为 $y(x) = e^{-x} + x - 1 - e^{-1}$. 分别取 $h=1, 2, 4$, 用经典 R-K 法计算, 结果见表 5.5.

表 5.5

x_n	$h=1$ 的解	$h=2$ 的解	$h=4$ 的解	解析解
1	0	0	0	0
3	1.6839	1.7547		1.6819
5	3.6394	3.6730	5.4715	3.6389
7	5.6331	5.6457		5.6330
9	7.6323	7.6367	16.8291	7.6322
11	9.6321	9.6336		9.6321
13	11.6321	11.6326	57.6171	11.6321

由于 R-K 法的绝对稳定区间是 $-2.785 \leq \lambda h \leq 0$, 本题中, $\lambda = f_y = -1$, 因此, 当 $h \leq 2.785$ 时, 该方法才稳定. $h=1$ 和 $h=2$ 在稳定范围内, 计算结果确实稳定. $h=4$ 超出了

稳定范围,结果发散.另一方面, $h=1$ 的计算精度比 $h=2$ 的计算精度高.因为 h 越小,方法的截断误差越小.但若 h 过分小的话,计算步数非常多,其累积误差会增加.所以,实际计算时,应选取合适的步长,常常采用自动变步长的R-K方法.

5.5 线性多步法

当数值求解常微分方程初值问题(5.1)时,若计算 y_{n+1} 仅用到它前面的一个函数值 y_n ,则称为单步法.如果利用前面已经算出的 r 个值 $y_{n-r}, \dots, y_{n-1}, y_n$,这就构成线性多步法的基本思想.

设想构造如下形式的线性多步公式,使它具有 p 阶精确度:

$$\begin{aligned} y_{n+1} &= \alpha_0 y_n + \alpha_1 y_{n-1} + \dots + \alpha_r y_{n-r} \\ &\quad + h(\beta_{-1} f_{n+1} + \beta_0 f_n + \beta_1 f_{n-1} + \dots + \beta_r f_{n-r}) \\ &= \sum_{k=0}^r \alpha_k y_{n-k} + h \sum_{k=-1}^r \beta_k f_{n-k}. \end{aligned} \quad (5.29)$$

当 $\beta_{-1}=0$ 时,称式(5.29)为显式多步法;当 $\beta_{-1} \neq 0$ 时,称式(5.29)是隐式多步法.它们关于 y_k 和 f_k 是线性的,所以,又称它们为线性多步法,推导线性多步公式的方法主要有Taylor展开法与数值积分法两种.本节首先用数值积分法导出亚当姆斯(Adams)公式等几个方法,最后介绍基于Taylor展开的构造方法.

1. 基于数值积分的构造法

将初值问题(5.1)写成等价的积分形式

$$y(x_{n+1}) = y(x_n) + \int_{x_n}^{x_{n+1}} f(x, y(x)) dx, \quad n = 0, 1, 2, \dots,$$

只要近似地算出右边的积分 $I_n \approx \int_{x_n}^{x_{n+1}} f(x, y(x)) dx$,则可通过 $y_{n+1} = y_n + I_n$ 近似 $y(x_{n+1})$,而选用不同近似式 I_n .可得到不同的计算公式.

(1) Adams 显式公式.

利用 $(r+1)$ 个节点上的被积函数 $f_n, f_{n-1}, \dots, f_{n-r}$,构造 r 阶Newton后插多项式 $N_r(x_n+th), t \in [0, 1]$,有

$$\int_{x_n}^{x_{n+1}} f(x, y(x)) dx = \int_0^1 N_r(x_n+th) h dt + \int_0^1 R_r(x_n+th) h dt,$$

其中, $R_r(x_n+th)$ 为Newton后插多项式的余项.于是有

$$y_{n+1} = y_n + h \int_0^1 N_r(x_n+th) dt = y_n + h \sum_{j=0}^r \alpha_j \nabla^j f_n, \quad (5.30)$$

其中, $\alpha_j = (-1)^j \int_0^1 \binom{-t}{j} dt, \binom{-t}{j} = \frac{-t(-t-1)\dots(-t-j+1)}{j!}.$

将式(5.30)中的差分用函数值表示,即 $\nabla^j f_n = \sum_{i=0}^j (-1)^i \binom{j}{i} f_{n-i}$. 这样 $\sum_{j=0}^r \alpha_j \sum_{i=0}^j (-1)^i$

$\binom{j}{i} f_{n-i} = \sum_{i=0}^r (-1)^i \sum_{j=i}^r \binom{j}{i} \alpha_j f_{n-i}$, 从而式(5.30)可写成

$$y_{n+1} = y_n + h \sum_{i=0}^r \beta_i f_{n-i}, \quad (5.31)$$

式中, $\beta_i = (-1)^i \sum_{j=i}^r \binom{j}{i} \alpha_j$, β_i 与 r, i 有关, 列表 5.6 如下.

表 5.6

i	0	1	2	3	4	5
β_{0i}	1					
$2\beta_{1i}$	3	-1				
$12\beta_{2i}$	23	-16	5			
$24\beta_{3i}$	55	-59	37	-9		
$720\beta_{4i}$	1901	-2774	2616	-1274	251	
$1440\beta_{5i}$	4277	-7923	9982	-7298	2877	-475

当 $r=0$ 时, 式(5.31)即为 Euler 公式.

当 $r=1$ 时, 式(5.31)为

$$y_{n+1} = y_n + \frac{h}{2} (3f_n - f_{n-1}). \quad (5.32)$$

当 $r=2$ 时, 式(5.31)为

$$y_{n+1} = y_n + \frac{h}{12} (23f_n - 16f_{n-1} + 5f_{n-2}). \quad (5.33)$$

当 $r=3$ 时, 式(5.31)为

$$y_{n+1} = y_n + \frac{h}{24} (55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}). \quad (5.34)$$

此即常用的 Adams 4 步显式方法是由前面的 4 个点上数值来计算下一点的值, 而式(5.31)统称为 Adams 显式方法, 其局部截断误差为

$$T_{n+1} = h \int_0^1 R_r(x_n + th) dt = \alpha_{r+1} h^{r+2} y^{(r+2)}(\xi_n),$$

其中 α_{r+1} 的数值见表 5.7.

表 5.7

r	0	1	2	3	4	...
α_r	1	1/2	5/12	3/8	251/720	...

由此可知,式(5.31)是 $r+1$ 阶方法. 例如,当 $r=3$ 时,式(5.34)的局部截断误差为

$$T_{n+1} = \alpha_4 h^5 y^{(5)}(\xi) = \frac{251}{720} h^5 y^{(5)}(\xi),$$

即式(5.34)是一个 4 阶方法.

(2) Adams 隐式公式.

利用 $(r+1)$ 个节点上的被积函数值 $f_{n+1}, f_n, \dots, f_{n-r+1}$ 构造 r 阶 Newton 插值多项式 $N_r(x_{n+1}+th), t \in [-1, 0]$. 类似于 Adams 显式公式的推导,可以得出如下 Adams 隐式公式

$$y_{n+1} = y_n + h \sum_{j=0}^r \alpha_j^* \nabla^j f_{n+1}, \quad (5.35)$$

其中, $\alpha_j^* = (-1)^j \int_{-1}^0 \binom{-t}{j} dt$. 将式(5.35)中的差分用函数值表示,即 $\nabla^j f_{n+1} =$

$$\sum_{i=0}^j (-1)^i \binom{j}{i} f_{n-i+1}, \text{ 得}$$

$$y_{n+1} = y_n + h \sum_{i=0}^r \beta_{r,i}^* f_{n-i+1}, \quad (5.36)$$

式中, $\beta_{r,i}^* = (-1)^i \sum_{j=i}^r \binom{j}{i} \alpha_j^*$, $\beta_{r,i}^*$ 与 r, i 有关,列表 5.8 如下.

表 5.8

i	0	1	2	3	4	5
$\beta_{0,i}^*$	1					
$2\beta_{1,i}^*$	1	1				
$12\beta_{2,i}^*$	5	8	-1			
$24\beta_{3,i}^*$	9	19	-5	1		
$720\beta_{4,i}^*$	251	646	-264	106	-19	
$1440\beta_{5,i}^*$	475	1427	-798	482	-173	27

当 $r=0$ 时,式(5.36)为

$$y_{n+1} = y_n + hf_{n+1}, \quad n = 0, 1, 2, \dots,$$

此即隐式 Euler 方法.

当 $r=1$ 时,式(5.36)为

$$y_{n+1} = y_n + \frac{h}{2}(f_n + f_{n+1}), \quad n = 0, 1, 2, \dots,$$

此即梯形方法.

当 $r=2$ 时, 式(5.36)为

$$y_{n+1} = y_n + \frac{h}{12}(5f_{n+1} + 8f_n - f_{n-1}), \quad n = 0, 1, 2, \dots \quad (5.37)$$

当 $r=3$ 时, 式(5.36)为

$$y_{n+1} = y_n + \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}), \quad n = 0, 1, 2, \dots, \quad (5.38)$$

此即常用的 Adams 三步隐式方法. 由前面 3 个点的数值用迭代方法来求出下一点的值. 式(5.36)统称为 Adams 隐式方法. 其局部截断误差为

$$T_{n+1} = \alpha_{r+1}^* h^{r+2} y^{(r+2)}(\eta),$$

其中, α_{r+1}^* 的数值见表 5.9.

表 5.9

r	0	1	2	3	4	...
α_r^*	1	$-\frac{1}{2}$	$-\frac{1}{12}$	$-\frac{1}{24}$	$-\frac{19}{720}$	...

由此可知式(5.36)是 $(r+1)$ 阶的方法. 例如当 $r=3$ 时, 式(5.38)的局部截断误差为

$$T_{n+1} = \alpha_4^* h^5 y^{(5)}(\eta) = -\frac{19}{720} h^5 y^{(5)}(\eta).$$

将 Adams 显式及隐式方法进行比较, 可得出如下结论.

- ① 从局部截断误差 T_{n+1} 来看, 若 h 的阶数一样, 显式比隐式方法多用一点预报值. 如 $T_{n+1} = O(h^5)$, 显式用预估值 $y_n, y_{n-1}, y_{n-2}, y_{n-3}$, 而隐式只用 y_n, y_{n-1}, y_{n-2} .
- ② 从局部截断误差主部的系数看, 显式比隐式大, 以 $O(h^5)$ 来比较. 显式系数为 $\frac{251}{720}$, 隐式为 $-\frac{19}{720}$.
- ③ 从计算上看, 显式比隐式计算量小.

2. 基于 Taylor 展开的方法

并非所有的线性多步式(5.29)都能由数值积分法得到, 但是都能用 Taylor 展开法构造出来, 因而 Taylor 展开法更具有一般性.

线性多步式(5.29)的局部截断误差为

$$T_{n+1} = y(x_{n+1}) - \left[\sum_{k=0}^r \alpha_k y(x_{n-k}) + h \sum_{k=-1}^r \beta_k f(x_{n-k}, y(x_{n-k})) \right]. \quad (5.39)$$

利用原微分方程后, 成为

$$T_{n+1} = y(x_{n+1}) - \left[\sum_{k=0}^r \alpha_k y(x_{n-k}) + h \sum_{k=-1}^r \beta_k y'(x_{n-k}) \right]. \quad (5.40)$$

现利用 Taylor 展开原理, 去确定线性多步式(5.31)中的待定参数 α_k, β_k , 使它达到 p 阶

精度, 即 $T_{n+1} = O(h^{p+1})$. 对式(5.40)的右端各项在 x_n 点作 Taylor 展开

$$\begin{aligned} y(x_{n-k}) &= y(x_n) + (-kh)y'(x_n) + \frac{(-kh)^2}{2}y''(x_n) \\ &\quad + \cdots + \frac{(-kh)^p}{p!}y^{(p)}(x_n) + \frac{(-kh)^{p+1}}{(p+1)!}y^{(p+1)}(x_n) + O(h^{p+2}) \\ &= \sum_{j=0}^p \frac{(-kh)^j}{j!}y^{(j)}(x_n) + \frac{(-kh)^{p+1}}{(p+1)!}y^{(p+1)}(x_n) + O(h^{p+2}), \\ y'(x_{n-k}) &= y'(x_n) + (-kh)y''(x_n) + \frac{(-kh)^2}{2!}y'''(x_n) + \cdots \\ &\quad + \frac{(-kh)^{p-1}}{(p-1)!}y^{(p)}(x_n) + \frac{(-kh)^p}{p!}y^{(p+1)}(x_n) + O(h^{p+1}) \\ &= \sum_{j=1}^p \frac{(-kh)^{j-1}}{(j-1)!}y^{(j)}(x_n) + \frac{(-kh)^p}{p!}y^{(p+1)}(x_n) + O(h^{p+1}). \end{aligned}$$

将它们代入式(5.40)整理后, 得

$$\begin{aligned} T_{n+1} &= \left(1 - \sum_{k=0}^r \alpha_k\right)y(x_n) + \sum_{j=1}^p \frac{h^j}{j!} \left[1 - \left(\sum_{k=1}^r (-k)^j \alpha_k \right. \right. \\ &\quad \left. \left. + j \sum_{k=-1}^r (-k)^{j-1} \beta_k \right) \right] y^{(j)}(x_n) + \frac{h^{p+1}}{(p+1)!} \left[1 - \left(\sum_{k=1}^r (-k)^{p+1} \alpha_k \right. \right. \\ &\quad \left. \left. + (p+1) \sum_{k=-1}^r (-k)^p \beta_k \right) \right] y^{(p+1)}(x_n) + O(h^{p+2}). \end{aligned}$$

使 $y(x_n), h^1, h^2, \dots, h^p$ 的系数为零, 就得到关于 α_k 和 β_k 的方程组

$$\begin{cases} \sum_{k=0}^r \alpha_k = 1, \\ \sum_{k=1}^r (-k)^j \alpha_k + j \sum_{k=-1}^r (-k)^{j-1} \beta_k = 1, \quad j = 1, 2, \dots, p. \end{cases} \quad (5.41)$$

而且得到线性多步法的局部截断误差

$$\begin{aligned} T_{n+1} &= \frac{h^{p+1}}{(p+1)!} \left\{ 1 - \left[\sum_{k=1}^r (-k)^{p+1} \alpha_k + (p+1) \sum_{k=-1}^r (-k)^p \beta_k \right] \right\} \\ &\quad \cdot y^{(p+1)}(x_n) + O(h^{p+2}). \end{aligned} \quad (5.42)$$

下面构造几个常用的著名的4阶线性多步公式, 即当 $r=3, p=4$ 时,

$$\begin{aligned} y_{n+1} &= \alpha_0 y_n + \alpha_1 y_{n-1} + \alpha_2 y_{n-2} + \alpha_3 y_{n-3} \\ &\quad + h(\beta_{-1} f_{n+1} + \beta_0 f_n + \beta_1 f_{n-1} + \beta_2 f_{n-2} + \beta_3 f_{n-3}), \end{aligned} \quad (5.43)$$

$$T_{n+1} = \frac{h^5}{5!} \left[1 - \sum_{k=1}^3 (-k)^5 \alpha_k + 5 \sum_{k=-1}^3 (-k)^4 \beta_k \right] y^{(5)}(x_n) + O(h^6). \quad (5.44)$$

式(5.43)共含9个未知量, 从式(5.41)得到5个方程, 其中有4个自由度.

此时式(5.41)即为

$$\begin{cases} \alpha_0 + \alpha_1 + \alpha_2 + \alpha_3 = 1, \\ -\alpha_1 - 2\alpha_2 - 3\alpha_3 + \beta_{-1} + \beta_0 + \beta_1 + \beta_2 + \beta_3 = 1, \\ \alpha_1 + 4\alpha_2 + 9\alpha_3 + 2\beta_{-1} - 2\beta_1 - 4\beta_2 - 6\beta_3 = 1, \\ -\alpha_1 - 8\alpha_2 - 27\alpha_3 + 3\beta_{-1} + 3\beta_1 + 12\beta_2 + 27\beta_3 = 1, \\ \alpha_1 + 16\alpha_2 + 81\alpha_3 + 4\beta_{-1} - 4\beta_1 - 32\beta_2 - 108\beta_3 = 1. \end{cases}$$

若取 $\alpha_0 = \alpha_1 = \alpha_2 = \beta_{-1} = 0$, 则解得 $\alpha_3 = 1, \beta_0 = \frac{8}{3}, \beta_1 = -\frac{4}{3}, \beta_2 = \frac{8}{3}, \beta_3 = 0$. 构成著名的 Milne(米尔尼)4步4阶显式公式

$$y_{n+1} = y_{n-3} + \frac{4h}{3}(2f_n - f_{n-1} + 2f_{n-2}), \quad (5.45)$$

其局部截断误差可由式(5.44)得到

$$T_{n+1} = \frac{14}{45}h^5 y^{(5)}(\xi_n), \quad \xi_n \in (x_{n-3}, x_{n+1}). \quad (5.46)$$

又若取 $\alpha_1 = \alpha_3 = \beta_2 = \beta_3 = 0$, 则得到著名的 Hamming(哈明)3步4阶隐式公式

$$y_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3h}{8}(f_{n+1} + 2f_n - f_{n-1}), \quad (5.47)$$

其局部截断误差为

$$T_{n+1} = -\frac{1}{40}h^5 y^{(5)}(\xi_n), \quad \xi_n \in (x_{n-2}, x_{n+1}). \quad (5.48)$$

取 $\alpha_0 = \alpha_2 = \alpha_3 = 0, \beta_3 = 0$, 则得到 Simpson 隐公式及其局部截断误差

$$y_{n+1} = y_{n-1} + \frac{h}{3}(f_{n+1} + 4f_n + f_{n-1}),$$

$$T_{n+1} = \frac{-1}{90}h^5 y^{(5)}(\xi_n), \quad \xi_n \in (x_{n-1}, x_{n+1}).$$

当取 $\beta_{-1} = \alpha_1 = \alpha_2 = \alpha_3 = 0$, 可得到4步4阶显式 Adams 公式及其局部截断误差.

当取 $\alpha_1 = \alpha_2 = \alpha_3 = \beta_3 = 0$, 可得到3步4阶隐式 Adams 公式及其局部截断误差.

上面介绍的线性多步公式, 除 Hamming 公式外, 均可由数值积分法得到.

线性多步公式与同阶的单步公式相比, 由于不需要反复地计算函数 $f(x, y)$ 的值, 所以使计算工作量大为减少, 但线性多步法需要已知前面若干个数值解才能使用, 所以开头几个数值解必须先用同阶的一步法求得.

例 5.10 设有

$$\begin{cases} y' = f(x, y), \\ y(x_0) = y_0, \end{cases}$$

试构造形如

$$y_{n+1} = \alpha(y_n + y_{n-1}) + h(\beta_0 f_n + \beta_1 f_{n-1})$$

的二阶方法, 并推导其局部截断误差的主项.

解 设 $y_n = y(x_n), y_{n-1} = y(x_{n-1})$, 则

$$y_{n+1} = \alpha[y(x_n) + y(x_{n-1})] + h[\beta_0 y'(x_n) + \beta_1 y'(x_{n-1})],$$

而
$$y(x_{n-1}) = y(x_n) - y'(x_n)h + \frac{y''(x_n)}{2}h^2 - \frac{y'''(x_n)}{3!}h^3 + O(h^4),$$

$$y'(x_{n-1}) = y'(x_n) - y''(x_n)h + \frac{y'''(x_n)}{2}h^2 + O(h^3),$$

将 $y(x_{n-1}), y'(x_{n-1})$ 在 x_n 处的 Taylor 展式代入 y_{n+1} 中整理得

$$\begin{aligned} y_{n+1} = & 2\alpha y(x_n) + (-\alpha + \beta_0 + \beta_1)y'(x_n)h + \left(\frac{\alpha}{2} - \beta_1\right)y''(x_n)h^2 \\ & + \left(-\frac{1}{6}\alpha + \frac{\beta_1}{2}\right)y'''(x_n)h^3 + O(h^4), \end{aligned}$$

又
$$y(x_{n+1}) = y(x_n) + y'(x_n)h + \frac{y''(x_n)}{2}h^2 + \frac{y'''(x_n)}{3!}h^3 + O(h^4).$$

比较 y_{n+1} 与 $y(x_{n+1})$ 在 x_n 处的 Taylor 展开式, 有

$$\begin{cases} 2\alpha = 1, \\ -\alpha + \beta_0 + \beta_1 = 1, \\ \frac{\alpha}{2} - \beta_1 = \frac{1}{2}, \end{cases}$$

解之得

$$\alpha = \frac{1}{2}, \quad \beta_0 = \frac{7}{4}, \quad \beta_1 = -\frac{1}{4},$$

故

$$y_{n+1} = \frac{1}{2}(y_n + y_{n-1}) + \frac{h}{4}(7f_n - f_{n-1}).$$

局部截断误差

$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{3}{8}y'''(x_n)h^3 + O(h^4),$$

则 T_{n+1} 的主项为 $\frac{3}{8}y'''(x_n)h^3$.

此题也可直接用式(5.39)、式(5.40)求解.

例 5.11 常微分方程初值问题

$$\begin{cases} y' = -y + x - e^{-1}, \\ y(1) = 0, \end{cases}$$

其精确解为 $y(x) = e^{-x} + x - 1 - e^{-1}$. 分别取 $h=0.2, 2$, 用 4 步 4 阶显式 Milne 公式和 3 步 4 阶隐式 Hamming 公式计算, 由 4 阶经典 R-K 公式为 Milne 公式提供初值 y_0, y_1, y_2, y_3 , 为 Hamming 公式提供 y_0, y_1, y_2 , 计算结果及精确值之间的误差分别见表 5.10 和表 5.11.

从表 5.10 看出, 两种多步法的计算精度都很高, Hamming 公式比 Milne 公式更精确, 这是因为 Hamming 公式的截断误差系数比 Milne 公式的小. 从表 5.11 看到, 在计算步长变大后, 显式多步法 Milne 公式的计算结果出现了振荡, 不稳定了, 而隐式多步

法 Hamming 公式的计算结果仍然是稳定的,这说明隐式公式的稳定性比同阶的显式公式好.

表 5.10

x_n	Milne 方法	误 差	Hamming	误 差
1.8	0.59744302	-2.4×10^{-5}	0.59741744	2.0×10^{-6}
2.0	0.76746411	-8.3×10^{-6}	0.76745251	3.3×10^{-6}
2.2	0.94294268	-1.9×10^{-5}	0.94291955	4.2×10^{-6}
2.4	1.12283349	-5.0×10^{-6}	1.12283386	4.7×10^{-6}
2.6	1.30643214	-3.8×10^{-5}	1.30638930	4.8×10^{-6}
2.8	1.49291625	1.4×10^{-5}	1.49292582	4.8×10^{-6}
3.0	1.68195450	-4.7×10^{-5}	1.68190299	4.6×10^{-6}

表 5.11

x_n	Milne 方法	误 差	Hamming	误 差
7	5.64574572	-1.3×10^{-2}	5.64963863	-1.7×10^{-2}
9	7.38232588	-2.5×10^{-1}	7.63712572	-4.9×10^{-3}
11	10.90531648	-1.27318	9.63563609	-3.5×10^{-3}
13	4.14383108	7.48829	11.63226101	-1.4×10^{-4}
15	58.31071703	-44.67860	13.63323961	-1.1×10^{-4}
17	-249.66267219	265.29479	15.63168985	4.3×10^{-5}
19	1592.89134564	-1575.25923	17.63268241	-5.6×10^{-5}
21	-9334.97733122	9354.60945	19.63173564	-3.8×10^{-6}

经典 R-K 法和上述 4 阶线性多步公式都是 4 阶精度,但每前进一步,前者要计算 4 次微分方程的右端函数,而后者只要计算一次新的右端函数值,计算量减少了.

5.6 预测-校正技术和外推技巧

不论单步法还是多步法,隐式公式都比显式公式的稳定性好.但在实际使用隐式公式时,都会遇到两个问题:一个是隐式公式如何能方便地进行计算;另一个是实际计算步长取多大.如隐式梯形公式,每往前推进一步,不必进行多次迭代,而是采用一阶显式 Euler 公式预测,二阶隐式梯形公式校正一次,构成显式的改进 Euler 公式,能达到与梯形公式同阶的精度,即二阶精度.对于线性多步公式,一般地,不难验证,如果预测公式是 p 阶或 $(p+1)$ 阶精度,校正公式为 $(p+1)$ 阶精度,则用预测公式提供初值,校正公式迭代一次的效果也能达到 $(p+1)$ 阶精度,再迭代下去的话,效果就不明显了.预测-校

正技术既保证了计算精度,又使隐式计算显式化,克服了隐式公式要反复迭代计算的困难.至于实际计算步长的选取,对预测-校正公式使用外推原理,得到误差估计式,用来调整计算步长,使达到控制误差要求.

对于线性多步法,常用的预测-校正公式有 Milne-Hamming 方法和 4 阶 Adams 显隐预测-校正公式.

5.6.1 修正 Hamming 公式

将 Milne 公式和 Hamming 公式结合,构成预测-校正公式

$$y_{n+1}^p = y_{n-3} + \frac{4h}{3}(2f_n - f_{n-1} + 2f_{n-2}),$$

$$y_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3h}{8}(f(x_{n+1}, y_{n+1}^p) + 2f_n - f_{n-1}).$$

由式(5.46)和式(5.48),Milne 公式和 Hamming 公式的局部截断误差分别为

$$T_{n+1}^M = y(x_{n+1}) - y_{n+1}^M = \frac{14}{45}h^5 y^{(5)}(x_n) + O(h^6),$$

$$T_{n+1}^H = y(x_{n+1}) - y_{n+1}^H = -\frac{1}{40}h^5 y^{(5)}(x_n) + O(h^6).$$

利用外推原理,即 $\frac{1}{40} \times T_{n+1}^M$ 加上 $\frac{14}{45} \times T_{n+1}^H$ 后,消去局部截断误差主项,使

$$y(x_{n+1}) - \frac{\frac{1}{40}y_{n+1}^M + \frac{14}{45}y_{n+1}^H}{\frac{1}{40} - \frac{14}{45}} = O(h^6), \quad (5.49)$$

说明经过外推后的算法其精度提高了一阶.忽略误差项,上式可改写为

$$y(x_{n+1}) - y_{n+1}^M \approx \frac{112}{121}(y_{n+1}^H - y_{n+1}^M), \quad (5.50)$$

$$y(x_{n+1}) - y_{n+1}^H \approx -\frac{9}{121}(y_{n+1}^H - y_{n+1}^M). \quad (5.51)$$

由于 y_{n+1}^M 和 y_{n+1}^H 是在计算过程中获得的数据,称式(5.50)和式(5.51)为 Milne 公式和 Hamming 公式的事后误差估计式.可以用它们来调节计算步长 h 的大小,即选择一个合适的步长,使 $\left| -\frac{9}{121}(y_{n+1}^H - y_{n+1}^M) \right| < \epsilon$, 其中 ϵ 是要求达到的计算精度.

也可以从式(5.50)和式(5.51)得到的 Milne 公式和 Hamming 公式的修正公式,它们分别是

$$y_{n+1}^{Mm} = y_{n+1}^M + \frac{112}{121}(y_{n+1}^H - y_{n+1}^M), \quad (5.52)$$

$$y_{n+1}^{Hm} = y_{n+1}^H - \frac{9}{121}(y_{n+1}^H - y_{n+1}^M). \quad (5.53)$$

从而构成如下的修正 Hamming 预测-校正公式, 简称为修正 Hamming 公式.

$$\begin{aligned}
 \text{预测} \quad y_{n+1}^p &= y_{n-3} + \frac{4h}{3}(2f_n - f_{n-1} + 2f_{n-2}), \\
 \text{修正} \quad y_{n+1}^{pm} &= y_{n+1}^p + \frac{112}{121}(y_n^c - y_n^p), \\
 \text{校正} \quad y_{n+1}^c &= \frac{1}{8}(9y_n - y_{n-2}) + \frac{3h}{8}[f(x_{n+1}, y_{n+1}^{pm}) + 2f_n - f_{n-1}], \\
 \text{修正} \quad y_{n+1} &= y_{n+1}^c - \frac{9}{121}(y_{n+1}^c - y_{n+1}^p).
 \end{aligned} \tag{5.54}$$

在应用这套公式时, 先由同阶单步法提供初值 y_0, y_1, y_2, y_3 . 当计算 y_4 时, 可取 $y_3^c = y_3^p$.

5.6.2 Adams 预测-校正公式

将 4 步 4 阶显式 Adams 公式作为预测公式, 并将 3 步 4 阶隐式 Adams 公式作为校正公式, 构成 Adams 预测-校正公式

$$\begin{aligned}
 y_{n+1}^p &= y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}), \\
 y_{n+1}^c &= y_n + \frac{h}{24}(9f(x_{n+1}, y_{n+1}^p) + 19f_n - 5f_{n-1} + f_{n-2}).
 \end{aligned}$$

它们的局部截断误差分别是

$$\begin{aligned}
 T_{n+1}^p &= y(x_{n+1}) - y_{n+1}^p = \frac{251}{720}h^5 y^{(5)}(x_n) + O(h^6), \\
 T_{n+1}^c &= y(x_{n+1}) - y_{n+1}^c = -\frac{19}{720}h^5 y^{(5)}(x_n) + O(h^6).
 \end{aligned}$$

利用外推原理, 将上两式作线性组合, 消去局部截断误差主项, 使计算精度至少提高一阶. 同时得到两个修正公式, 将它们和上两式组合构成了如下修正的 Adams 预测-校正公式.

$$\text{预测} \quad y_{n+1}^p = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}), \tag{5.55}$$

$$\text{修正} \quad y_{n+1}^{pm} = y_{n+1}^p + \frac{251}{270}(y_n^c - y_n^p), \tag{5.56}$$

$$\text{校正} \quad y_{n+1}^c = y_n + \frac{h}{24}[9f(x_{n+1}, y_{n+1}^{pm}) + 19f_n - 5f_{n-1} + f_{n-2}], \tag{5.57}$$

$$\text{修正} \quad y_{n+1} = y_{n+1}^c - \frac{19}{270}(y_{n+1}^c - y_{n+1}^p). \tag{5.58}$$

同理, 在计算时, 调节计算步长 h , 使 $\left| -\frac{19}{270}(y_{n+1}^c - y_{n+1}^p) \right| < \epsilon$, 由同阶单步法提供初值 y_0, y_1, y_2, y_3 . 当计算 y_4 时, 可取 $y_3^c = y_3^p$.

理论分析得出了它们的绝对稳定区间,修正 Hamming 法: $-0.69 \leq \lambda h \leq 0, 4$ 阶
 Adams 预测-校正法: $-0.75 \leq \lambda h \leq 0, \lambda = \frac{\partial f}{\partial y}(x, y(x))$.

例 5.12 初值问题

$$\begin{cases} y' = -y + x - e^{-1}, \\ y(1) = 0, \end{cases}$$

其精确解为 $y(x) = e^{-x} + x - 1 - e^{-1}$. 取 $h = 0.2$, 用经典 R-K 法提供初值, 分别用 Milne-Hamming 预测-校正公式和修正 Hamming 法计算, 并将计算结果与精确解比较. 结果见表 5.12.

表 5.12

x_n	Milne-Hamming	误差	修正 Hamming	误差
1.8	0.59741783	1.6×10^{-6}	0.59741971	-2.6×10^{-7}
2.0	0.76745120	4.7×10^{-6}	0.76745646	-6.2×10^{-7}
2.2	0.94291625	7.5×10^{-6}	0.94292449	-7.8×10^{-7}
2.4	1.12282872	9.8×10^{-6}	1.12283955	-1.0×10^{-6}
2.6	1.30638271	1.1×10^{-5}	1.30639537	-1.2×10^{-6}
2.8	1.49291816	1.2×10^{-5}	1.49293184	-1.2×10^{-6}
3.0	1.68189467	1.3×10^{-5}	1.68190879	-1.2×10^{-6}

将表 5.10 和表 5.12 所示的计算结果进行比较, 特别是在 $x_n = 1.8, 2.0$ 处, 它们的计算精度排列次序是: 隐式 Hamming 公式的精度最好, 其次是显式修正 Hamming 公式, 再次是 Milne-Hamming 预测-校正公式, 最后是 Milne 公式. 计算结果表明预测-校正技术和外推技巧的效果很好.

习 题 5

1. 取步长 $h = 0.2$, 用 Euler 法解初值问题

$$\begin{cases} y' = -y - xy^2, \\ y(0) = 1 \end{cases} \quad (0 \leq x \leq 0.6).$$

2. 用改进的 Euler 公式计算初值问题

$$\begin{cases} y' = \frac{1}{x}y - \frac{1}{x}y^2, \\ y(1) = 0.5 \end{cases} \quad (1 \leq x \leq 1.5),$$

取步长 $h = 0.1$, 小数点后至少保留 5 位.

3. 写出用梯形格式的迭代算法求解初值问题

$$\begin{cases} y' + y = 0, \\ y(0) = 1 \end{cases}$$

的计算公式,取步长 $h=0.1$,并求 $y(0.2)$ 的近似值,要求迭代误差不超过 10^{-5} .

4. 证明:公式

$$\begin{cases} y_{n+1} = y_n + \frac{h}{9}(2K_1 + 3K_2 + 4K_3), \\ K_1 = f(x_n, y_n), \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right), \\ K_3 = f\left(x_n + \frac{3}{4}h, y_n + \frac{3}{4}hK_2\right) \end{cases}$$

至少是 3 阶方法.

5. 设有常微分方程初值问题

$$\begin{cases} y' = f(x, y), \\ y(x_0) = y_0 \end{cases}$$

的单步法

$$y_{n+1} = y_n + \frac{h}{3}[f(x_n, y_n) + 2f(x_{n+1}, y_{n+1})],$$

证明:该方法是无条件稳定的.

6. 讨论求解初值问题

$$\begin{cases} y' = \lambda y, \quad \lambda < 0, \\ y(0) = a \end{cases}$$

的 2 阶中点公式

$$y_{n+1} = y_n + hf\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}f(x_n, y_n)\right)$$

的稳定性.

7. 证明:线性二步法

$$y_{n+1} + (b-1)y_n - by_{n-1} = \frac{1}{4}h[(b+3)f_{n+1} + (3b+1)f_{n-1}].$$

当 $b \neq -1$ 时方法为 2 阶的,当 $b = -1$ 时方法为 3 阶的.

8. 对于初值问题

$$\begin{cases} y' = -1000(y - g(x)) + g'(x), \\ y(0) = g(0), \end{cases}$$

其中 $g(x)$ 为已知函数,其解 $y(x) = g(x)$. 问:

(1) 若用 Euler 法求解,从稳定性考虑步长应在什么范围内选取?

(2) 若用隐式 Euler 法解题,从稳定性考虑,步长有没有限制?

(3) 若 $g(x)$ 为不超过一次的多项式,用 Euler 法求解此问题时,从精确阶考虑,步长的选取有没有限制?

9. 试构造形如

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + h(\beta_0 f_n + \beta_1 f_{n-1})$$

的差分格式,使其为 3 阶方法,并求出其局部截断误差的首项.

10. 已知 3 阶 Adams 显式公式及截断误差

$$y_{n+1} = y_n + \frac{h}{12}(23f_n - 16f_{n-1} + 5f_{n-2}),$$

$$T_1 = y(x_{n+1}) - y_{n+1} = \frac{3}{8}h^4 y^{(4)}(\xi_1), x_{n-2} < \xi_1 < x_{n+1},$$

和 3 阶 Adams 隐式公式及截断误差

$$y_{n+1} = y_n + \frac{h}{12}(5f_{n+1} + 8f_n - f_{n-1}),$$

$$T_2 = y(x_{n+1}) - y_{n+1} = \frac{-1}{24}h^4 y^{(4)}(\xi_2), x_{n-1} < \xi_2 < x_{n+1},$$

试求其预估—改进—校正系统.

数值实验题 5

实验 5.1 常微分方程性态和 R-K 法稳定性试验

实验目的:考察下面微分方程右端项中函数 y 前面的参数对方程性态的影响(它可使方程为好条件的或坏条件的)和研究计算步长对 R-K 法计算稳定性的影响.

实验题目:常微分方程初值问题

$$\begin{cases} y' = \alpha y - \alpha x + 1, & 0 < x < 1, \\ y(0) = 1, \end{cases}$$

其中, $-50 \leq \alpha \leq 50$. 其精确解为 $y(x) = e^{\alpha x} + x$.

实验要求:本实验题都用 4 阶经典 R-K 法计算.

(1) 对参数 α , 分别取 4 个不同的数值: 一个大的正值, 一个小的正值, 一个绝对值小的负值和一个绝对值大的负值. 取步长 $h=0.01$, 分别用经典 R-K 法计算, 将四组计算结果画在同一张图上, 进行比较并说明相应初值问题的性态.

(2) 取参数 α 为一个绝对值不大的负值和两个计算步长, 一个步长使参数 ah 在经典 R-K 法的稳定域内, 另一个步长在经典 R-K 法的稳定域外. 分别用经典 R-K 法计算

并比较计算结果. 取全域等距的 10 个点上的计算值, 列表说明.

实验 5.2 比较同阶精度的单步法和预测-校正多步法的计算精度和所花费的计算机 CPU 时间

实验目的: 比较 4 阶 Adams 预测-校正修正法式(5.55)~式(5.58)与经典 R-K 法的计算精度. 验证对于微分方程右端项较复杂的初值问题, 线性多步法所花的计算机 CPU 时间比经典 R-K 法省.

实验 5.3 常微分方程初值问题

$$\begin{cases} y' = -y + \cos 2x - 2\sin 2x + 2xe^{-x}, & 0 < x < 2, \\ y(0) = 1 \end{cases}$$

有精确解 $y(x) = x^2 e^{-x} + \cos 2x$.

实验要求: (1) 选择一个步长 h 使 4 阶 Adams 预测-校正修正法和经典 R-K 法均稳定, 分别用这两种方法计算初值问题, 以表格形式列出 10 个等距节点上的计算值和精确值并比较它们的计算精确度. 其中多步法需要的初值由经典 R-K 法提供. 运算时, 取足以表示计算精度的有效值.

(2) 取 $h=0.001$, 仍然用这两种方法计算(不必保留中间结果, 只需显示最后一步的计算值), 比较两种方法所花的计算机 CPU 时间.

注 在编制多步法计算程序时, 要避免重复计算微分方程的右端函数值.

第 6 章 线性代数方程组的解法

6.1 引言及预备知识

6.1.1 引言

线性代数方程组经常出现在工程与科学的许多领域中,而且很多数值求解问题最后也是归结于求解某些线性代数方程组. 众多的事实表明,解线性代数方程组的有效方法在计算数学和科学计算中具有特殊的重要性.

线性代数方程组的解法可分为直接法和迭代法两大类.

直接法的特征是:对于给定的方程组,在没有舍入误差的假设下,能在预定的运算次数内求得精确解. 然而原始数据的误差和运算的舍入误差是不可避免的,实际上只能获得方程组的近似解,因此存在误差分析问题. 对等规模(阶数 <100)的方程组及某些大型的带型或稀疏方程组,由于直接法所具有的准确性和可靠性,所以经常选用它来求解. 最基本的直接法是 Gauss 消去法,其他重要的直接法全都受到 Gauss 消去法的启示.

迭代法是基于一定的递推格式,产生逼近方程组精确解的近似序列,收敛性是迭代法的前提,然后还有收敛速度与误差估计问题. 对较高阶的方程组,特别是某些偏微分方程数值求解过程中出现的方程组,迭代法因简单好用而非常诱人,直接法则计算代价较高,因而迭代法更具有竞争力.

6.1.2 向量与矩阵的范数

如何衡量一个方程组的近似解的好坏,实质上涉及如何衡量一个向量的大小,最常用的是用向量的 Euclid(欧几里得)长度表示它的大小,但只有这一大小的标准是不够的,范数就是 Euclid 长度的推广.

我们用 $\mathbf{R}^n$ 表示 n 维实向量空间,用 $\mathbf{C}^n$ 表示 n 维复向量空间.

定义 6.1 (向量的范数)如果向量 $\mathbf{X} \in \mathbf{R}^n$ (或 $\mathbf{C}^n$) 的某个实值函数 $N(\mathbf{X}) = \|\mathbf{X}\|$, 满足条件

- (1) $\|\mathbf{X}\| \geq 0$ (当且仅当 $\mathbf{X} = \mathbf{0}$ 时, $\|\mathbf{X}\| = 0$) (正定性);
- (2) $\|\alpha\mathbf{X}\| = |\alpha| \|\mathbf{X}\|$, $\forall \alpha \in \mathbf{R}$ (或 $\mathbf{C}$), (齐次性);
- (3) $\|\mathbf{X} + \mathbf{Y}\| \leq \|\mathbf{X}\| + \|\mathbf{Y}\|$ (三角不等式),

则称 $N(X)$ 是 $\mathbf{R}^n$ (或 $\mathbf{C}^n$) 上的一个向量范数 (或模).

由条件(3)可推出不等式

$$(4) \quad | \|X\| - \|Y\| | \leq \|X - Y\|.$$

按以上方式定义的范数,其具体形式是多种多样的,常用的范数有

(1) ∞ -范数 (最大范数), 即

$$\|X\|_{\infty} = \max_{1 \leq i \leq n} |x_i|;$$

(2) 1-范数, 即

$$\|X\|_1 = \sum_{i=1}^n |x_i|;$$

(3) 2-范数 (长度), 即

$$\|X\|_2 = \left(\sum_{i=1}^n x_i^2 \right)^{1/2};$$

(4) p -范数, 即

$$\|X\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

∞ -范数、1-范数、2-范数都是 p -范数的特例. 事实上, 1-范数和 2-范数显然是 p -范数当 $p=1$ 和 $p=2$ 时的特殊情形. 而 ∞ -范数, 则是当 $p \rightarrow \infty$ 时的情形, 即 $\|X\|_{\infty} = \lim_{p \rightarrow \infty} \|X\|_p$, 因为

$$\left(\max_{1 \leq i \leq n} |x_i|^p \right)^{1/p} \leq \left(\sum_{i=1}^n |x_i|^p \right)^{1/p} \leq \left(n \max_{1 \leq i \leq n} |x_i|^p \right)^{1/p},$$

故有

$$\|X\|_{\infty} \leq \|X\|_p \leq n^{1/p} \|X\|_{\infty},$$

令 $p \rightarrow \infty$, 得 $n^{1/p} \rightarrow 1$, 即有

$$\lim_{p \rightarrow \infty} \|X\|_p = \|X\|_{\infty}.$$

例 6.1 计算向量 $X = (1, -2, 3)^T$ 的各种范数.

解 $\|X\|_1 = 6$, $\|X\|_{\infty} = 3$, $\|X\|_2 = \sqrt{14}$.

定义 6.2 设 $\{X^{(k)}\}$ 为 $\mathbf{R}^n$ 中一向量序列, $X^* \in \mathbf{R}^n$, 记 $X^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T$, $X^* = (x_1^*, \dots, x_n^*)^T$, 如果 $\lim_{k \rightarrow \infty} x_i^{(k)} = x_i^* (i=1, 2, \dots, n)$, 则称 $X^{(k)}$ 收敛于向量 X^* , 记为 $\lim_{k \rightarrow \infty} X^{(k)} = X^*$.

定理 6.1 (连续性定理) 设非负函数 $N(X) = \|X\|$ 为 $\mathbf{R}^n$ 上任一向量范数, 则 $N(X)$ 是分量 $x_1, x_2, \dots, x_n$ 的连续函数.

证 令 $e_j = (0, \dots, 0, 1, 0, \dots, 0)^T (j=1, 2, \dots, n)$, 即第 j 个元素为 1, 有

$$X = (x_1, x_2, \dots, x_n)^T = \sum_{j=1}^n x_j e_j;$$

$$Y = (y_1, y_2, \dots, y_n)^T = \sum_{j=1}^n y_j e_j.$$

由范数定义,有

$$\begin{aligned} |N(X) - N(Y)| &= |\|X\| - \|Y\|| \leq \|X - Y\| \leq \sum_{j=1}^n |x_j - y_j| \|e_j\| \\ &\leq \max_{1 \leq j \leq n} |x_j - y_j| \sum_{j=1}^n \|e_j\|, \end{aligned}$$

所以,当 $\max |x_j - y_j| \rightarrow 0 (j=1, 2, \dots, n)$ 时,有 $|N(X) - N(Y)| \rightarrow 0$.

有了范数函数的连续性后,便极易得到范数的等价性. 我们知道,按照不同方式规定的范数,其值一般不相同,但各种范数在考虑向量序列的收敛性时,却表示出明显的一致性,这就是向量范数的等价性.

定理 6.2 (等价性定理) 设 $\|X\|_i, \|X\|_l$ 为 $\mathbf{R}^n$ 上向量的任意两种范数,则存在常数 $C_1, C_2 > 0$,使得

$$C_1 \|X\|_i \leq \|X\|_l \leq C_2 \|X\|_i, \quad (6.1)$$

对一切 $X \in \mathbf{R}^n$ 均成立.

证 只要 $\|X\|_i = \|X\|_\infty$ 证明上式成立即可,即证明存在常数 $C_1, C_2 > 0$,使

$$C_1 \|X\|_\infty \leq \|X\|_i \leq C_2 \|X\|_\infty$$

成立即可.

事实上,当 $X=0$ 时,式(6.1)显然成立,现设 $X \neq 0$,则

$$\frac{X}{\|X\|_\infty} \in S = \{X \mid \|X\|_\infty = 1, X \in \mathbf{R}^n\}.$$

又假定 C_1, C_2 是连续函数 $\|X\|_i$ 在有界闭集 S 上的最小值和最大值,则对任何 $X \neq 0$,有

$$C_1 \leq \left\| \frac{X}{\|X\|_\infty} \right\|_i \leq C_2,$$

即 $C_1 \|X\|_\infty \leq \|X\|_i \leq C_2 \|X\|_\infty$ 对一切 $X \in \mathbf{R}^n$ 均成立.

注 此式说明任何一种 t -范数均与 ∞ -范数等价,因而任何两种 t -范数 ($t \leq \infty$) 彼此都是等价的. 特别地,前述三种常用范数 $\|\cdot\|_1, \|\cdot\|_2$ 和 $\|\cdot\|_\infty$ 彼此等价.

范数的等价性保证了运用具体范数研究收敛性在理论上的合法性,容易看出有下述定理成立.

定理 6.3 $\lim_{k \rightarrow \infty} X^{(k)} = X^* \Leftrightarrow \|X^{(k)} - X^*\| \rightarrow 0$ (当 $k \rightarrow \infty$ 时).

证 显然 $\lim_{k \rightarrow \infty} X^{(k)} = X^* \Leftrightarrow \|X^{(k)} - X^*\|_\infty \rightarrow 0$ (当 $k \rightarrow \infty$ 时),而对于 $\mathbf{R}^n$ 上任一种范数 $\|\cdot\|$, $\|\cdot\|$ 表示向量的任一种范数,由范数的等价性,存在常数 $C_1, C_2 > 0$,使

$$C_1 \|X^{(k)} - X^*\|_\infty \leq \|X^{(k)} - X^*\| \leq C_2 \|X^{(k)} - X^*\|_\infty$$

成立,于是又有

$$\|X^{(k)} - X^*\|_\infty \rightarrow 0 \Leftrightarrow \|X^{(k)} - X^*\| \rightarrow 0 \quad (\text{当 } k \rightarrow \infty).$$

类似于向量范数,在 $n \times n$ 矩阵集合 $\mathbf{R}^{n \times n}$ 中也可以定义矩阵范数,但由于矩阵之间

有乘法运算,为了使用方便通常规定如下.

定义 6.3 (矩阵的范数) 如果矩阵 $A \in \mathbf{R}^{n \times n}$ 的某个非负的实值函数 $N(A) = \|A\|$, 满足条件

- (1) $\|A\| \geq 0$ (当且仅当 $A=0$ 时, $\|A\|=0$) (正定条件);
- (2) $\|CA\| = |C| \|A\|$, C 为实数 (齐次条件);
- (3) $\|A+B\| \leq \|A\| + \|B\|$ (三角不等式);
- (4) $\|AB\| \leq \|A\| \|B\|$,

则称 $N(A)$ 是 $\mathbf{R}^{n \times n}$ 上的一个矩阵范数 (或模).

在矩阵计算中, 矩阵与向量的乘积经常出现, 因而应让所用的矩阵范数与向量范数有某种关系.

定义 6.4 对于给定的向量范数 $\|\cdot\|$ 和矩阵范数 $\|\cdot\|$, 如果对任一个 $X \in \mathbf{R}^n$ 和任一个 $A \in \mathbf{R}^{n \times n}$, 满足

$$\|AX\| \leq \|A\| \|X\|,$$

则称所给的矩阵范数与向量范数是相容的.

为此我们再引进一种矩阵范数.

定义 6.5 (矩阵的算子范数) 设 $X \in \mathbf{R}^n$, $A \in \mathbf{R}^{n \times n}$, 给出一种向量范数 $\|X\|_p$ (如 $p=1, 2$, 或 ∞), 相应地定义一个矩阵的非负函数

$$\|A\|_p = \max_{X \neq 0} \frac{\|AX\|_p}{\|X\|_p} = \max_{\|X\|_p=1} \|AX\|_p.$$

可验证 $\|A\|_p$ 满足矩阵范数定义, 所以 $\|A\|_p$ 是 $\mathbf{R}^{n \times n}$ 上矩阵的一个范数, 称为 A 的算子范数.

可以证明这样定义的范数是满足相容性条件:

$$\|AX\|_p \leq \|A\|_p \|X\|_p.$$

定理 6.4 设 $X \in \mathbf{R}^n$, $A \in \mathbf{R}^{n \times n}$, 则

$$(1) \|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|, \text{ 称为 } A \text{ 的行范数};$$

$$(2) \|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|, \text{ 称为 } A \text{ 的列范数};$$

$$(3) \|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}, \text{ 称为 } A \text{ 的 } 2\text{-范数}, \text{ 其中, } \lambda_{\max}(A^T A) \text{ 表示 } A^T A \text{ 的最大特征值}.$$

凡是从属于某种向量范数的矩阵范数, 对于单位矩阵 I , 都有

$$\|I\| = \max_{\|X\|=1} \|IX\| = 1.$$

还有一种常用的矩阵范数, 就是

$$\|A\|_F = \sqrt{\sum_{i,j=1}^n a_{ij}^2},$$

其中, $A=(a_{ij}) \in \mathbb{R}^{n \times n}$, $\|\cdot\|_F$ 称为 Frobenius(佛罗比尼乌斯)范数, 又称 Euclid 范数, 它也可记为 $\|\cdot\|_E$. 可以证明, $\|\cdot\|_F$ 与向量范数 $\|\cdot\|_2$ 相容, 即

$$\|AX\|_2 \leq \|A\|_F \|X\|_2, \quad A \in \mathbb{R}^{n \times n}, X \in \mathbb{R}^n.$$

也可以证明, $\|\cdot\|_F$ 满足矩阵范数定义的 4 个条件, 需要指出的是, 矩阵范数 $\|\cdot\|_F$ 不从属于任何向量范数.

例 6.2 设 $A = \begin{bmatrix} 1 & 5 & -2 \\ -2 & 1 & 0 \\ 3 & -8 & 2 \end{bmatrix}$, 试求 $\|A\|_p (p=1, 2, \infty)$ 以及 $\|A\|_F$.

解 $\|A\|_1 = \max\{1+2+3, 5+1+8, 2+0+2\} = 14$;

$\|A\|_\infty = \max\{1+5+2, 2+1+0, 3+8+2\} = 13$;

$\|A\|_F = \sqrt{1+25+4+4+1+0+9+64+4} = \sqrt{112}$;

$$A^T A = \begin{bmatrix} 14 & -21 & 4 \\ -21 & 90 & -26 \\ 4 & -26 & 8 \end{bmatrix};$$

$A^T A$ 的特征方程

$$|A^T A - \lambda I| = -\lambda^3 + 112\lambda^2 - 959\lambda + 16 = 0$$

的最大根为 $\lambda_1 \approx 102.66$, 所以

$$\|A\|_2 = \sqrt{\lambda_1} \approx 10.132.$$

对于复矩阵(即 $A \in \mathbb{C}^{n \times n}$)定理 6.4 中(1)、(2)显然也成立, 对于(3)应改为

$$\|A\|_2 = \max_{X \neq 0} \left(\frac{X^H A^H A X}{X^H X} \right)^{1/2} = \sqrt{\lambda_{\max}(A^H A)},$$

其中, $A^H = \overline{A}^T$ (A 共轭转置).

定义 6.6 设 $A \in \mathbb{R}^{n \times n}$ 的特征值为 $\lambda_i (i=1, 2, \dots, n)$, 称 $\rho(A) = \max_{1 \leq i \leq n} |\lambda_i|$ 为 A 的谱半径.

定理 6.5 (特征值上界) 设 $A \in \mathbb{R}^{n \times n}$, 则 $\rho(A) \leq \|A\|$, 即 A 的谱半径不超过 A 的任何一种算子范数(对 $\|A\|_F$ 亦成立).

证 设 λ 是 A 的任一特征值, X 为相应的特征向量, 则 $AX = \lambda X$. 由向量范数、矩阵范数的相容性, 得

$$|\lambda| \|X\| = \|\lambda X\| = \|AX\| \leq \|A\| \|X\|,$$

即

$$|\lambda| \leq \|A\|.$$

定理 6.6 如果 $A \in \mathbb{R}^{n \times n}$ 为对称矩阵, 则 $\|A\|_2 = \rho(A)$.

定理 6.7 如果 $\|B\| < 1$, 则 $I \pm B$ 为非奇异矩阵, 且

$$\|(I \pm B)^{-1}\| \leq \frac{1}{1 - \|B\|},$$

其中, $\|\cdot\|$ 是指矩阵的算子范数.

证 用反证法. 若 $\det(I-B)=0$, 则 $(I-B)X=0$ 有非零解, 即存在 $X_0 \neq 0$, 使 $BX_0 = X_0$, 而 $\frac{\|BX_0\|}{\|X_0\|} = 1$, 故 $\|B\| \geq 1$, 与题设矛盾. 又由 $(I-B)(I-B)^{-1} = I$, 有

$$(I-B)^{-1} = I + B(I-B)^{-1},$$

从而

$$\|(I-B)^{-1}\| \leq \|I\| + \|B\| \|(I-B)^{-1}\|,$$

$$\|(I-B)^{-1}\| \leq \frac{1}{1-\|B\|}.$$

类似可证, $(I+B)$ 可逆, 且

$$\|(I+B)^{-1}\| \leq \frac{1}{1-\|B\|}.$$

6.2 Gauss(高斯)消去法

求解线性代数方程组的消去法是一个众所周知的古老方法, 这种方法用在现代电子计算机上仍然十分有效.

消去法的基本思想是, 通过将一方程乘或除某个定数以及两个方程相加减这两种运算手续, 逐步减少方程中变元的数目, 最终使每个方程仅含一个变元, 从而得出所求的解.

我们知道, 线性代数方程组求解的难易程度与其系数矩阵的特征有关, 如果系数矩阵是三角阵, 则这类方程组求解比较容易, 如:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ a_{22}x_2 + \cdots + a_{2n}x_n = b_2, \\ \cdots \\ a_{nn}x_n = b_n, \end{cases}$$

通过回代即可求得全部的 $x_i (i=1, 2, \cdots, n)$.

Gauss 消去法的目的就是要将线性代数方程组化为三角方程组去求解.

考察一般形式的 n 阶方程组

$$AX = b,$$

或写成分量形式
$$\sum_{j=1}^n a_{ij}x_j = b_i, \quad i = 1, 2, \cdots, n. \quad (6.2)$$

记 $a_{ij} = a_{ij}^{(1)}$, $b_i = b_i^{(1)}$.

第一步, 若 $a_{11}^{(1)} \neq 0$, 先让第一个方程保持不变, 利用它从其余的方程中消去变元 x_1 , 使之变成关于变元 $x_2, x_3, \cdots, x_n$ 的 $n-1$ 阶方程组, 即

$$\sum_{j=2}^n a_{ij}^{(2)}x_j = b_i^{(2)}, \quad i = 2, 3, \cdots, n.$$

为此,只要从方程(6.2)的第 i 行减去第一行的 a_{i1}/a_{11} 倍,即施行运算

$$\begin{cases} l_{i1} = a_{i1}/a_{11}, \\ a_{ij}^{(2)} = a_{ij}^{(1)} - l_{i1}a_{1j}^{(1)}, \quad i, j = 2, 3, \dots, n, \\ b_i^{(2)} = b_i^{(1)} - l_{i1}b_1^{(1)}, \end{cases}$$

这样每做一步,都保留其中的一个方程,而归结得出低一阶的方程组.

经过 k 步消元后,得到 k 个方程

$$\sum_{j=i}^n a_{ij}^{(i)} x_j = b_i^{(i)}, \quad i = 1, 2, \dots, k$$

和下列 $(n-k)$ 阶方程组

$$\begin{cases} \sum_{j=k+1}^n a_{ij}^{(k+1)} x_j = b_i^{(k+1)}, \quad i = k+1, k+2, \dots, n. \\ l_{ik} = a_{ik}^{(k)}/a_{kk}^{(k)}, \quad a_{kk}^{(k)} \neq 0, \\ a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik}a_{kj}^{(k)}, \quad i, j = k+1, \dots, n, \\ b_i^{(k+1)} = b_i^{(k)} - l_{ik}b_k^{(k)}, \quad i = k+1, \dots, n. \end{cases} \quad (6.3)$$

而按上述手续做 $n-1$ 步后,所给方程组最终被加工成如下形式:

$$\sum_{j=i}^n a_{ij}^{(i)} x_j = b_i^{(i)}, \quad i = 1, 2, \dots, n, \quad (6.4)$$

这是一个三角方程组.

由方程组(6.2)约化为方程组(6.4)的过程称为消元过程.

求解三角方程组(6.4),有求解公式

$$\begin{cases} x_n = b_n^{(n)}/a_{nn}^{(n)} \quad (a_{nn}^{(n)} \neq 0), \\ x_k = (b_k^{(k)} - \sum_{j=k+1}^n a_{kj}^{(k)} x_j)/a_{kk}^{(k)}, \quad k = n-1, n-2, \dots, 2, 1. \end{cases} \quad (6.5)$$

方程组(6.4)的求解过程称为回代过程.

例 6.3 用 Gauss 消去法解线性方程组

$$\begin{cases} 2x_1 + x_2 + 3x_3 + x_4 = 6, \\ 4x_1 + 2x_2 + 7x_3 - x_4 = 10, \\ 6x_1 + 4x_2 + 2x_4 = 6, \\ -2x_1 + 4x_2 + 5x_3 + 2x_4 = -5. \end{cases} \quad (6.6)$$

解 Gauss 消去法的消元过程是对方程组(6.6)的增广矩阵进行下列一系列行初等变换;用 r_i 表示第 i 个方程,即增广矩阵的第 i 行,用 $r_i + xr_j \rightarrow r_i$ 表示第 j 个方程(行)乘数 x 加至第 i 个方程(行).

$$[A : b] = \left[\begin{array}{cccc|c} 2 & 1 & 3 & 1 & 6 \\ 4 & 2 & 7 & -1 & 10 \\ 6 & 4 & 0 & 2 & 6 \\ -2 & 4 & 5 & 2 & -5 \end{array} \right] \xrightarrow{\substack{r_2 - 2r_1 \\ r_3 - 3r_1 \\ r_4 - (-1)r_1}} \left[\begin{array}{cccc|c} 2 & 1 & 3 & 1 & 6 \\ 0 & 0 & 1 & -3 & -2 \\ 0 & 1 & -9 & -1 & -12 \\ 0 & 5 & 8 & 3 & 1 \end{array} \right]$$

$$\begin{array}{c}
 \xrightarrow{r_2 \leftrightarrow r_3} \left[\begin{array}{cccc|c} 2 & 1 & 3 & 1 & 6 \\ 0 & 1 & -9 & -1 & -12 \\ 0 & 0 & 1 & -3 & -2 \\ 0 & 5 & 8 & 3 & 1 \end{array} \right] \xrightarrow{r_4 - 5r_2} \left[\begin{array}{cccc|c} 2 & 1 & 3 & 1 & 6 \\ 0 & 1 & -9 & -1 & -12 \\ 0 & 0 & 1 & -3 & -2 \\ 0 & 0 & 53 & 8 & 61 \end{array} \right] \\
 \\
 \xrightarrow{r_4 - 53r_3} \left[\begin{array}{cccc|c} 2 & 1 & 3 & 1 & 6 \\ 0 & 1 & -9 & -1 & -12 \\ 0 & 0 & 1 & -3 & -2 \\ 0 & 0 & 0 & 167 & 167 \end{array} \right].
 \end{array}$$

由此得到与方程组(6.6)同解的上三角形方程组

$$\begin{cases} 2x_1 + x_2 + 3x_3 + x_4 = 6, \\ x_2 - 9x_3 - x_4 = -12, \\ x_3 - 3x_4 = -2, \\ 167x_4 = 167. \end{cases} \quad (6.7)$$

Gauss 消去法的回代过程是解上三角形方程组(6.7). 首先,由方程组(6.7)的第4个方程得 $x_4=1$. 其次,将 $x_4=1$ 代入第3个方程得

$$x_3 = -2 + 3 \times 1 = 1.$$

再将 $x_3=1, x_4=1$ 代入第2个方程得

$$x_2 = -12 + 9 \times 1 + 1 \times 1 = -2.$$

最后,将 $x_2=-2, x_3=1, x_4=1$ 代入第1个方程得

$$x_1 = [6 - 1 \times (-2) - 3 \times 1 - 1 \times 1] / 2 = 2.$$

于是,我们得到方程组(6.6)的解(向量)是 $[2, -2, 1, 1]^T$.

从上面对方法的描述看到,使用 Gauss 消去法实现方程组的求解条件是

$$a_{11}^{(1)} a_{22}^{(2)} \cdots a_{nn}^{(n)} \neq 0.$$

由于假设方程组的系数矩阵 A 非奇异,若 $a_{11}^{(1)}=0$ 时, A 的第1列一定有非零元素,例如 $a_{i_1,1} \neq 0$. 于是可交换两行元素($r_1 \leftrightarrow r_{i_1}$),将 $a_{i_1,1}$ 调到(1,1)位置,然后进行消元计算,这时 $A^{(2)}$ 右下角矩阵($n-1$)亦为非奇异矩阵,继续这过程,Gauss 消去法照样可进行计算.

定理 6.8 如果 A 为 n 阶非奇异矩阵,则可通过 Gauss 消去法(及交换两行的初等变换)将方程组(6.2)化为三角形方程组(6.4).

矩阵 A 在什么条件下才能保证 $a_{kk}^{(k)} \neq 0$ ($k=1, 2, \dots, n$)呢? 下面的定理给出了这个条件.

引理 约化的主元素 $a_{ii}^{(i)} \neq 0$ ($i=1, 2, \dots, k$)的充要条件是矩阵 A 的顺序主子式即

$$D_i \neq 0 (i=1, 2, \dots, k),$$

$$D_1 = a_{11} \neq 0,$$

$$D_i = \begin{vmatrix} a_{11} & \cdots & a_{1i} \\ \vdots & & \vdots \\ a_{i-1} & \cdots & a_{ii} \end{vmatrix} \neq 0, \quad i = 2, 3, \cdots, k.$$

证 首先利用归纳法证明引理的充分性. 显然, 当 $k=1$ 时引理的充分性是成立的, 现假设引理对 $(k-1)$ 是成立的, 求证引理对 k 亦成立. 由归纳法设有 $a_{ii}^{(i)} \neq 0$ ($i=1, 2, 3, k-1$), 于是可用 Gauss 消去法将 $A^{(1)}=A$ 约化到 $A^{(k)}$, 即

$$A^{(1)} \rightarrow A^{(k)} = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ & & \ddots & \vdots \\ & & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & & \vdots & & \vdots \\ & & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{bmatrix},$$

且有

$$D_2 = \begin{vmatrix} a_{11}^{(1)} & a_{12}^{(1)} \\ 0 & a_{22}^{(2)} \end{vmatrix} = a_{11}^{(1)} a_{22}^{(2)}, \quad D_3 = a_{11}^{(1)} a_{22}^{(2)} a_{33}^{(3)},$$

$$D_k = \begin{vmatrix} a_{11}^{(1)} & \cdots & a_{1k}^{(1)} \\ & \ddots & \vdots \\ & & a_{kk}^{(k)} \end{vmatrix} = a_{11}^{(1)} a_{22}^{(2)} \cdots a_{kk}^{(k)}. \quad (6.8)$$

由设 $D_i \neq 0$ ($i=1, 2, \cdots, k$) 及式 (6.8), 有 $a_{kk}^{(k)} \neq 0$, 即引理的充分性对 k 成立.

必要性成立是显然的, 由假设 $a_{ii}^{(i)} \neq 0$ ($i=1, 2, \cdots, k$), 利用式 (6.6) 亦可推出 $D_i \neq 0$ ($i=1, 2, \cdots, k$).

推论 如果 A 的顺序主子式 $D_k \neq 0$ ($k=1, 2, \cdots, n-1$), 则

$$\begin{cases} a_{11}^{(1)} = D_1, \\ a_{kk}^{(k)} = D_k / D_{k-1}. \end{cases}$$

定理 6.9 如果 n 阶矩阵 A 的所有顺序主子式均不为 0, 即 $D_i \neq 0$ ($i=1, 2, \cdots, n$), 则可通过 Gauss 消去法(不进行交换两行的初等变换), 将方程组 (6.2) 约化为三角形方程组 (6.4).

当用 Gauss 消去法解方程组 (6.2) 时, 其回代过程求解 (6.4) 需乘除 $N_1 = 1 + 2 + 3 + \cdots + n = \frac{1}{2}n(n+1)$ 次. 消元过程在第 k 步的计算中, 计算每个 $l_{ik}, a_{ij}^{(k+1)}, b_i^{(k+1)}$ 均只需乘或除 1 次, 这些数共有 $(n-k)(n-k+2)$ 个, 而消元过程进行 $(n-1)$ 步, 所以消元过程的全部乘除次数是

$$N_2 = \sum_{k=1}^{n-1} (n-k)(n-k+2).$$

最后得 Gauss 消去法的乘除总次数为

$$\begin{aligned}
 N_1 + N_2 &= \frac{1}{2}n(n+1) + \sum_{k=1}^{n-1} (n-k)(n-k+2) \\
 &= \frac{1}{3}n^3 + n^2 - \frac{1}{3}n.
 \end{aligned}$$

化简时利用恒等式 $\sum_{i=1}^m i^2 = m(m+1)(2m+1)/6$.

6.3 Gauss 主元素消去法

在 Gauss 消去法中, $a_{kk}^{(k)}$ 叫主元, 欲使消元过程第 k 步能进行, 必须主元 $a_{kk}^{(k)}$ 不为零, 是否只要主元不为零, 就不会有什么问题呢? 下面先看一实例, 再作分析.

例 6.4 用 3 位尾数的十进浮点数系解方程组

$$\begin{cases} \frac{1}{10000}x_1 + x_2 = 1, \\ x_1 + x_2 = 2. \end{cases} \quad \begin{matrix} (6.9) \\ (6.10) \end{matrix}$$

解 算法一 保留方程(6.9)(即消元第 1 步以 $10^{-3} \times 0.100$ 为主元), 消去式(6.10)中的 x_1 , 得方程组

$$\begin{cases} 10^{-3} \times 0.100x_1 + 10^1 \times 0.100x_2 = 10^1 \times 0.100, \\ -10^5 \times 0.100x_2 = -10^5 \times 0.100, \end{cases}$$

由此得近似解

$$\bar{x}_2 = 10^1 \times 0.100, \quad \bar{x}_1 = 0.$$

算法二 保留方程(6.10)(即消元中以 1 为主元), 消去式(6.9)中的 x_1 , 得方程组

$$\begin{cases} 10^1 \times 0.100x_1 + 10^1 \times 0.100x_2 = 10^1 \times 0.200, \\ 10^1 \times 0.100x_2 = 10^1 \times 0.100, \end{cases}$$

由此得近似解

$$\tilde{x}_2 = 10^1 \times 0.100, \quad \tilde{x}_1 = 10^1 \times 0.100.$$

方程组的精确解是 $x_1^* = \frac{10000}{9999}, \quad x_2^* = \frac{9998}{9999}.$

显然算法一得到的近似解误差是很大的, 为了分析其原因, 用 x_1, x_2 表示某个算法得到的计算解(当所用数系、字长、舍入规则确定之后, 仅与算法有关), 并令

$$\epsilon_1 = x_1 - x_1^*, \quad \epsilon_2 = x_2 - x_2^*.$$

在这里对于两种算法 ϵ_2 相同, ϵ_1 却不同.

当用算法一时, 有

$$\begin{aligned}
 10^{-3} \times 0.100x_1^* + 10^1 \times 0.100x_2^* &= 10^1 \times 0.100, \\
 10^{-3} \times 0.100x_1 + 10^1 \times 0.100x_2 &\approx 10^1 \times 0.100,
 \end{aligned}$$

相减得

$$10^{-3} \times 0.100\epsilon_1 + 10^1 \times 0.100\epsilon_2 \approx 0,$$

所以

$$|\epsilon_1| \approx 10^4 |\epsilon_2|.$$

对于算法二,同理可知 $|\epsilon_1| \approx |\epsilon_2|$.

由此例可知,当用 Gauss 消去法解方程组时,主元的绝对值大,算法的稳定性好.因此实用的 Gauss 消元法应该是在消元过程的每一步都要使主元的绝对值尽量大.

下面介绍列主元素消去法,总假定方程组(6.2)的系数矩阵 A 非奇异.

设方程组(6.2)的增广矩阵为

$$B = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right].$$

首先在 A 的第一列中选取绝对值最大的元素作为主元素,如

$$|a_{i_1 1}| = \max_{1 \leq i \leq n} |a_{i1}| \neq 0,$$

然后交换 B 的第 1 行与第 i_1 行,经第 1 次消元计算得

$$[A | b] \rightarrow [A^{(2)} | b^{(2)}].$$

重复上述过程,设已完成第 $(k-1)$ 步的选主元素、交换两行及消元计算, $[A | b]$ 约化为

$$[A^{(k)} | b^{(k)}] = \left[\begin{array}{cccccc|c} a_{11} & a_{12} & \cdots & a_{1k} & \cdots & a_{1n} & b_1 \\ & a_{22} & \cdots & a_{2k} & \cdots & a_{2n} & b_2 \\ & & \ddots & \vdots & & \vdots & \\ & & & a_{kk} & \cdots & a_{kn} & b_k \\ & & & \vdots & & \vdots & \\ & & & a_{nk} & \cdots & a_{nn} & b_n \end{array} \right],$$

其中, $A^{(k)}$ 的元素仍记为 a_{ij} , $b^{(k)}$ 的元素仍记为 b_i .

第 k 步选主元素(在 $A^{(k)}$ 右下角方阵的第 1 列内选),即确定 i_k ,使

$$|a_{i_k k}| = \max_{k \leq i \leq n} |a_{ik}| \neq 0,$$

交换 $[A^{(k)} | b^{(k)}]$ 第 k 行与 i_k 行的元素,再进行消元计算,最后将原方程组化为

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{array} \right] \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix},$$

回代求解

$$\begin{cases} x_n = b_n / a_{nn}, \\ x_i = (b_i - \sum_{j=i+1}^n a_{ij} x_j) / a_{ii}, \quad i = n-1, \cdots, 2, 1. \end{cases}$$

例 6.5 应用 Gauss 列主元消去法, 用准确算术运算解方程组

$$\begin{bmatrix} 1 & -1 & 1 \\ -3 & 1 & -2 \\ 3 & 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 6 \\ 12 \end{bmatrix}. \quad (6.11)$$

解 消元过程的第 1 步, 选取增广矩阵

$$[A : b] = \left[\begin{array}{ccc|c} 1 & -1 & 1 & 2 \\ -3 & 1 & -2 & 6 \\ 3 & 1 & -1 & 12 \end{array} \right]$$

的第 1 列元素 1, -3, 3 中首先遇到的绝对值最大的 -3 作主元, 即选取第 2 行为主行, 交换 $[A, b]$ 的第 1 行与第 2 行:

$$[A : b] \xrightarrow{r_1 \leftrightarrow r_2} \left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 1 & -1 & 1 & 2 \\ 3 & 1 & -1 & 12 \end{array} \right].$$

计算得乘子

$$l_{21} = \frac{1}{-3} = -\frac{1}{3}, \quad l_{31} = \frac{3}{-3} = -1,$$

进行第 1 步消元:

$$\left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 1 & -1 & 1 & 2 \\ 3 & 1 & -1 & 12 \end{array} \right] \xrightarrow[r_3 - l_{31}r_1]{r_2 - l_{21}r_1} \left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 0 & -\frac{2}{3} & \frac{1}{3} & 4 \\ 0 & 2 & -3 & 18 \end{array} \right].$$

第 2 步, 选取第 1 步消元后得到的矩阵的第 2 列中元素 $-\frac{2}{3}$ 和 2 的绝对值最大元素 2 为主元, 交换该矩阵的第 2 行与第 3 行:

$$\left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 0 & -\frac{2}{3} & \frac{1}{3} & 4 \\ 0 & 2 & -3 & 18 \end{array} \right] \xrightarrow{r_2 \leftrightarrow r_3} \left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 0 & 2 & -3 & 18 \\ 0 & -\frac{2}{3} & \frac{1}{3} & 4 \end{array} \right].$$

计算得乘子

$$l_{32} = \frac{-\frac{2}{3}}{2} = -\frac{1}{3},$$

进行第 2 步消元:

$$\left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 0 & 2 & -3 & 18 \\ 0 & -\frac{2}{3} & \frac{1}{3} & 4 \end{array} \right] \xrightarrow{r_3 - l_{32}r_2} \left[\begin{array}{ccc|c} -3 & 1 & -2 & 6 \\ 0 & 2 & -3 & 18 \\ 0 & 0 & -\frac{2}{3} & 10 \end{array} \right].$$

由回代过程得方程组(6.11)的解为

$$x_3 = 10 / \left(-\frac{2}{3}\right) = -15,$$

$$x_2 = (18 + 3 \times (-15)) / 2 = -\frac{27}{2},$$

$$x_1 = \left(6 - 1 \times \left(-\frac{27}{2}\right) + 2 \times (-15)\right) / (-3) = \frac{7}{2}.$$

Gauss 列主元素法首先在增广矩阵 $[A^{(1)} : b^{(1)}]$ 的第 1 列 n 个元素中选取绝对值最大的一个作为主元素,并把此主元素所在的行与第 1 行交换,然后通过初等行变换把第 1 列后的 $(n-1)$ 个元素消为 0,得增广矩阵 $[A^{(2)} : b^{(2)}]$;其次,在矩阵 $[A^{(2)} : b^{(2)}]$ 的第 2 列后的 $(n-1)$ 个元素中选取绝对值最大的一个作为主元素,并把它所在行与第 2 行交换,然后通过初等行变换把第 2 行后的 $(n-2)$ 个元素消为 0,得增广矩阵 $[A^{(3)} : b^{(3)}]$,按此方法做下去只要 $\det A \neq 0$,消元过程就能进行到底,最后能得到一个与原方程组同解的上三角方程组.

例 6.4 中的算法二则是用列主元素消去法计算的.

有些具有特殊系数矩阵的方程组,按顺序 Gauss 消去法,若能保证主元按绝对值最大,或者舍入误差能得到控制,就不必选主元,这类矩阵有对称的严格对角占优阵、对称正定阵等.

6.4 矩阵分解及其在解方程组中的应用

6.4.1 Gauss 消去法的矩阵形式

设已知方程组 $AX=b$,其增广矩阵为

$$[A : b] = \left[\begin{array}{ccc|c} a_{11}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ a_{21}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ \vdots & & \vdots & \vdots \\ a_{n1}^{(1)} & \cdots & a_{nn}^{(1)} & b_n^{(1)} \end{array} \right],$$

设 $a_{11}^{(1)} \neq 0$,令 $l_{i1} = a_{i1}^{(1)} / a_{11}^{(1)} \quad (i=2,3,\cdots,n)$,

$$L_1 = \left[\begin{array}{cccc|c} 1 & & & & \\ -l_{21} & 1 & & & \\ -l_{31} & & \ddots & & \\ \vdots & & & \ddots & \\ -l_{n1} & \cdots & \cdots & \cdots & 1 \end{array} \right],$$

则 Gauss 消去法消元第 1 步可表示成

$$L_1[A : b] = \left[\begin{array}{cccc|c} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & a_{2n}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{array} \right].$$

设 $a_{22}^{(2)} \neq 0$, 令 $l_{i2} = a_{i2}^{(2)} / a_{22}^{(2)}$ ($i=3, 4, \dots, n$),

$$L_2 = \left[\begin{array}{cccc|c} 1 & & & & \\ 0 & 1 & & & \\ 0 & -l_{32} & 1 & & \\ \vdots & \vdots & & \ddots & \\ 0 & -l_{n2} & \cdots & & 1 \end{array} \right],$$

则 Gauss 消去法消元第 2 步可表示成

$$L_2 L_1[A : b] = \left[\begin{array}{cccc|c} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ 0 & 0 & a_{33}^{(3)} & \cdots & a_{3n}^{(3)} & b_3^{(3)} \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & a_{n3}^{(3)} & \cdots & a_{nn}^{(3)} & b_n^{(3)} \end{array} \right].$$

如此下去, 第 $(n-1)$ 步消元为

$$L_{n-1} \cdots L_1[A : b] = \left[\begin{array}{cccc|c} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ & & \ddots & \vdots & \vdots \\ & & & a_{nn}^{(n)} & b_n^{(n)} \end{array} \right]. \quad (6.12)$$

在这里每个 L_k 都是可逆的, 且容易看出

$$L_k^{-1} = \left[\begin{array}{cccc|c} 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ & & & 1 & \\ & & & l_{k+1,k} & 1 \\ & & & \vdots & \\ & & & l_{n,k} & \cdots & 1 \end{array} \right].$$

记

$$U = \left[\begin{array}{cccc} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ & & \ddots & \vdots \\ & & & a_{nn}^{(n)} \end{array} \right],$$

$$L^{-1} = L_{n-1} L_{n-2} \cdots L_1, \quad Y = (b_1^{(1)}, b_2^{(2)}, \dots, b_n^{(n)})^T,$$

则式(6.12)可表示成

$$L^{-1}[A \ b] = [U \ Y],$$

或

$$L^{-1}A = U, \quad L^{-1}b = Y,$$

从而

$$A = LU, \quad b = LY.$$

不难验证,其中

$$L = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{bmatrix}$$

是单位下三角矩阵.

把一个矩阵分解为一个下三角矩阵和上三角矩阵的相乘的形式叫矩阵的三角分解.

我们知道 Gauss 消去法实质上产生了一个将 A 分解为两个三角形矩阵相乘的形式. 于是得到如下重要定理,它在解方程组的直接法中起着重要作用.

定理 6.10 (矩阵的 LU 分解) 设 A 为 n 阶矩阵,如果 A 的顺序主子式 $D_i \neq 0$ ($i = 1, 2, \dots, n-1$),则 A 可分解为一个单位下三角矩阵 L 和一个上三角矩阵 U 的乘积,且这种分解是唯一的.

证 根据前述 Gauss 消去法的矩阵分析, $A = LU$ 的存在性已经得到证明,现证唯一性.

设有 A 的两种三角分解:

$$A = LU = L_1 U_1. \quad (6.13)$$

当 A 非奇异时,由上式有

$$L_1^{-1}L = U_1 U^{-1},$$

由于 $L_1^{-1}L$ 是单位下三角矩阵, $U_1 U^{-1}$ 是上三角矩阵,所以,它们只能是单位矩阵,即

$$L_1^{-1}L = U_1 U^{-1} = I,$$

由此可知 $L = L_1, U = U_1$.

当 A 奇异时,把等式(6.10)写成分块形式

$$\begin{bmatrix} L_{n-1} & 0 \\ r^T & 1 \end{bmatrix} \begin{bmatrix} U_{n-1} & S \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} L_{1,n-1} & 0 \\ r_1^T & 1 \end{bmatrix} \begin{bmatrix} U_{1,n-1} & S_1 \\ 0 & 0 \end{bmatrix},$$

$$\begin{bmatrix} L_{n-1} U_{n-1} & L_{n-1} S \\ r^T U_{n-1} & r^T S \end{bmatrix} = \begin{bmatrix} L_{1,n-1} U_{1,n-1} & L_{1,n-1} S_1 \\ r_1^T U_{1,n-1} & r_1^T S_1 \end{bmatrix}.$$

其中, $L_{n-1}, U_{n-1}, L_{1,n-1}, U_{1,n-1}$ 皆非奇异,类似前面的推理,可知

$$L_{n-1} = L_{1,n-1} U_{n-1} = U_{1,n-1}, \quad r = r_1, \quad S = S_1,$$

因而 $L = L_1, U = U_1$.

例 6.6 用 Gauss 消去法求下列方程组

$$\begin{cases} 2x_1 + x_2 + x_3 = 4, \\ x_1 + 3x_2 + 2x_3 = 6, \\ x_1 + 2x_2 + 2x_3 = 5 \end{cases}$$

的解, 写出系数矩阵 A 的三角分解式, 并计算 $\det A$.

解 方程组的增广矩阵为

$$\begin{bmatrix} 2 & 1 & 1 & 4 \\ 1 & 3 & 2 & 6 \\ 1 & 2 & 2 & 5 \end{bmatrix},$$

第一次消元, 消元因子 $l_{21} = \frac{1}{2}, l_{31} = \frac{1}{2}$, 进行消元, 得

$$\begin{bmatrix} 2 & 1 & 1 & 4 \\ 0 & \frac{5}{2} & \frac{3}{2} & 4 \\ 0 & \frac{3}{2} & \frac{3}{2} & 3 \end{bmatrix}.$$

第二次消元, 消元因子 $l_{32} = \frac{3}{5}$, 进行消元, 得

$$\begin{bmatrix} 2 & 1 & 1 & 4 \\ 0 & \frac{5}{2} & \frac{3}{2} & 4 \\ 0 & 0 & \frac{3}{5} & \frac{3}{5} \end{bmatrix}.$$

回代得 $x_3 = 1, x_2 = (4 - \frac{3}{2}) / (\frac{5}{2}) = 1, x_1 = (4 - 1 - 1) / 2 = 1$.

易知

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{2} & \frac{3}{5} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{5}{2} & \frac{3}{2} \\ 0 & 0 & \frac{3}{5} \end{bmatrix}.$$

易验证 $A = LU, \det A = 2 \times \frac{5}{2} \times \frac{3}{5} = 3$.

6.4.2 直接三角分解法

现在假定已经有了矩阵的三角分解式 $A = LU$, 那么对方程组 $AX = b$ 即 $LUX = b$, 只需令 $UX = Y$ 便可化为等价的两个系数矩阵为三角形矩阵的方程组

$$\begin{cases} LY = b, \\ UX = Y. \end{cases}$$

如果有若干个系数矩阵相同而右端不同的方程组那么对 A 作 LU 分解,再用代入过程分别求解将是很方便的,这就完全避开了 Gauss 消去法的 m 次重复过程,节省了机时.

1. 不选主元的三角分解法

设 A 为非奇异矩阵,且有分解式

$$A = LU,$$

其中, L 为单位下三角矩阵, U 为上三角矩阵,即

$$A = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & \ddots & & \\ \vdots & \vdots & \ddots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & \cdots & & u_{1n} \\ & u_{22} & \cdots & & u_{2n} \\ & & \ddots & & \vdots \\ & & & \ddots & \vdots \\ & & & & u_{nn} \end{bmatrix}.$$

下面仅以 4 阶阵来说明如何计算出 L, U 的各元素. 设有

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} 1 & & & \\ l_{21} & 1 & & \\ l_{31} & l_{32} & 1 & \\ l_{41} & l_{42} & l_{43} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ & u_{22} & u_{23} & u_{24} \\ & & u_{33} & u_{34} \\ & & & u_{44} \end{bmatrix}$$

$$= \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ l_{21}u_{11} & l_{21}u_{12} + u_{22} & l_{21}u_{13} + u_{23} & l_{21}u_{14} + u_{24} \\ l_{31}u_{11} & l_{31}u_{12} + l_{32}u_{22} & l_{31}u_{13} + l_{32}u_{23} + u_{33} & l_{31}u_{14} + l_{32}u_{24} + u_{34} \\ l_{41}u_{11} & l_{41}u_{12} + l_{42}u_{22} & l_{41}u_{13} + l_{42}u_{23} + l_{43}u_{33} & l_{41}u_{14} + l_{42}u_{24} + l_{43}u_{34} + u_{44} \end{bmatrix}.$$

第 1 步,求 U 的第 1 行及 L 的第 1 列各元素.

U 矩阵的第 1 行元素为: $a_{11}, a_{12}, a_{13}, a_{14}$,

L 矩阵的第 1 列元素为: $1, a_{21}/a_{11}, a_{31}/a_{11}, a_{41}/a_{11}$.

这是因为 $a_{21} = l_{21}u_{11}, a_{31} = l_{31}u_{11}, a_{41} = l_{41}u_{11}$, 而 $u_{11} = a_{11}$, 所以

$$l_{21} = a_{21}/a_{11}, l_{31} = a_{31}/a_{11}, l_{41} = a_{41}/a_{11}.$$

第 2 步,求 U 的第 2 行及 L 的第 2 列各元素.

因为

$$a_{2i} = \sum_{k=1}^4 l_{2k}u_{ki} \quad (i = 2, 3, 4),$$

而

$$l_{22} = 1, \quad l_{23} = 0, \quad l_{24} = 0,$$

故

$$a_{2i} = \sum_{k=1}^{2-1} l_{2k}u_{ki} + u_{2i},$$

即
$$u_{2i} = a_{2i} - \sum_{k=1}^{2-1} l_{2k} u_{ki} \quad (i = 2, 3, 4).$$

这就是 U 的第 2 行各元素的计算公式, 公式中只用到 A 的第 2 行元素及已求出的 U 的第 1 行与 L 的第 1 列元素.

又因为
$$a_{i2} = \sum_{k=1}^4 l_{ik} u_{k2} \quad (i = 2, 3, 4),$$

而
$$u_{32} = u_{42} = 0,$$

故
$$a_{i2} = \sum_{k=1}^{2-1} l_{ik} u_{k2} + l_{i2} u_{22},$$

即
$$l_{i2} = (a_{i2} - \sum_{k=1}^{2-1} l_{ik} u_{k2}) / u_{22}, \quad (i = 2, 3, 4).$$

这就是计算 L 的第 2 列各元素的公式.

第 3 步, 求 U 的第 3 行及 L 的第 3 列各元素.

类似地, 可导出
$$u_{3i} = a_{3i} - \sum_{k=1}^{3-1} l_{3k} u_{ki} \quad (i = 3, 4);$$

$$l_{i3} = (a_{i3} - \sum_{k=1}^{3-1} l_{ik} u_{k3}) / u_{33} \quad (i = 3, 4).$$

最后, 求 U 的第 4 行及 L 的第 4 列各元素.

$$u_{44} = a_{44} - \sum_{k=1}^{4-1} l_{4k} u_{k4}, \quad l_{44} = 1.$$

对于 A 的 LU 分解, 在整个计算中, 只需要 A 的存贮单元即可, 不用另设 L 及 U 的单元, 每步依次求出的 U 及 L 的各行和各列元素, 依次可存在 A 的对应位置, 即为

$$\begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ l_{21} & u_{22} & u_{23} & u_{24} \\ l_{31} & l_{32} & u_{33} & u_{34} \\ l_{41} & l_{42} & l_{43} & u_{44} \end{bmatrix}.$$

推广到 n 阶矩阵的分解, 第 r 步计算 U 的第 r 行及 L 的第 r 列各元素的计算公式为

$$\begin{cases} u_{ri} = a_{ri} - \sum_{k=1}^{r-1} l_{rk} u_{ki}, & i = r, r+1, \dots, n, \end{cases} \quad (6.14)$$

$$\begin{cases} l_{ir} = (a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr}) / u_{rr}, & i = r, r+1, \dots, n. \end{cases} \quad (6.15)$$

把矩阵 A 分解成 LU 后, 求解 $LY=b$ 的前推公式为

$$y_i = b_i - \sum_{k=1}^{i-1} l_{ik} y_k, \quad i = 1, 2, \dots, n.$$

求解 $UX=Y$ 的回代过程为

$$x_i = (y_i - \sum_{k=i+1}^n u_{ik} x_k) / u_{ii}, \quad i = n, n-1, \dots, 2, 1.$$

例 6.7 用直接三角分解法解

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 2 \\ 3 & 1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 14 \\ 18 \\ 20 \end{bmatrix}.$$

解 用分解公式(6.14)、式(6.15)计算得

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & -5 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & -4 \\ 0 & 0 & -24 \end{bmatrix} = LU.$$

先解

$$LY = (14, 18, 20)^T,$$

得

$$Y = (14, -10, -72)^T;$$

再解

$$UX = (14, -10, -72)^T,$$

得

$$X = (1, 2, 3)^T.$$

由公式(6.15)知,在求 l_r 时, u_r 是作为除数的,因而 u_r 的绝对值不能很小;前面讲的三角分解过程中, u_r 是按原有顺序算的,难免在算到某一步时 $|u_r|$ 很小,因而,类似于选主元 Gauss 消去法一样,每一步的 u_r 也有一个选择问题.

2. 选主元的三角分解法解方程组

定理 6.11 若 A 非奇异,则只要适当地对 A 作行交换,就有分解式 $PA=LU$,这里 P 为某个交换阵.

由 Gauss 消去法与 LU 分解的关系可知,在 LU 分解过程中得出的 u_r 就是 Gauss 消去法消元过程第 r 步的主元,所以 $|u_r|$ 愈大,算法的数值稳定性愈好.与列主元消去法类似,可通过适当地交换方程组中各方程的位置,使每个 $|u_r|$ 尽量大.

设对 $A=(a_{ij})_{n \times n}$ 的分解已完成 $(r-1)$ 步,即 L 的前 $(r-1)$ 列, U 的前 $(r-1)$ 行已求得,将分解结果以紧凑格式写出就是矩阵

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & \cdots & & & u_{1n} \\ l_{21} & u_{22} & & & & & \vdots \\ \cdots & \cdots & & & & & \vdots \\ & & & u_{r-1,r-1} & \cdots & & \vdots \\ \vdots & & & l_{r,r-1} & a_{rr} & \cdots & a_{rn} \\ & & & \vdots & \vdots & & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{n,r-1} & a_{nr} & \cdots & a_{nn} \end{bmatrix},$$

第 r 步分解需用到式(6.15),为了避免用小的数 u_r 作除数,引进量 $s_i = a_{ir} -$

$\sum_{k=1}^{r-1} l_{ik} u_{kr}$ ($i = r, r+1, \cdots, n$),与式(6.14)比较便知,假若将 A 的第 i 行($i = r, \cdots, n$)放在第 r 行,那么 s_i 就是由式(6.14)算得的 u_r ,所以若 $s_i = \max_{r \leq i \leq n} |s_i|$,那么在第 r 步

分解开始时就应该将第 r 行与第 i_r 行交换, 这样便得到第 r 步的主元 s_{i_r} .

例 6.8 利用列主元三角分解法求方程组 $AX=b$ 的解, 已知增广矩阵

$$[A^{(1)} : b^{(1)}] = \left[\begin{array}{cccc|c} 1.00 & 0.333 & 1.50 & -0.333 & 3.00 \\ -2.01 & 1.45 & 0.500 & 2.95 & 5.40 \\ 4.32 & -1.95 & 0.000 & 2.08 & 0.130 \\ 5.11 & -4.00 & 3.33 & -1.11 & 3.77 \end{array} \right].$$

解 令

$$s_i = a_{i1}^{(1)} = a_{i1} \quad (i=1, 2, 3, 4),$$

$$s_1 = 1.00, \quad s_2 = -2.01, \quad s_3 = 4.32, \quad s_4 = 5.11.$$

主元为 $s_4 = 5.11, i_1 = 4$, 交换 $[A^{(1)} : b^{(1)}]$ 的第 1 行与第 4 行, 即方程组交换为 $E_{14}AX = E_{14}b$. 再计算 U 的第 1 行与 L 的第 1 列, 得

$$[A^{(2)} : b^{(2)}] = \left[\begin{array}{cccc|c} 5.11 & -4.00 & 3.33 & -1.11 & 3.77 \\ -0.393 & 1.45 & 0.500 & 2.95 & 5.40 \\ 0.845 & -1.95 & 0.000 & 2.08 & 0.130 \\ 0.196 & 0.333 & 1.50 & -0.333 & 3.00 \end{array} \right],$$

$$s_i = a_{i2}^{(2)} - l_{i1}u_{12} \quad (i=2, 3, 4),$$

$$s_2 = -0.12, \quad s_3 = 1.43, \quad s_4 = 1.12.$$

主元为 $s_3 = 1.43, i_2 = 3$, 交换 $[A^{(2)} : b^{(2)}]$ 的第 2 行与第 3 行, 方程组相应地交换为 $E_{23}E_{14}AX = E_{23}E_{14}b$, 再计算 U 的第 2 行与 L 的第 2 列, 得

$$[A^{(3)} : b^{(3)}] = \left[\begin{array}{cccc|c} 5.11 & -4.00 & 3.33 & -1.11 & 3.77 \\ 0.845 & 1.43 & -2.81 & 3.02 & 0.130 \\ -0.393 & -0.0839 & 0.500 & 2.95 & 5.40 \\ 0.196 & 0.783 & 1.50 & -0.333 & 3.00 \end{array} \right],$$

$r=3$:

$$s_3 = a_{33}^{(3)} - l_{31}u_{13} - l_{32}u_{23} = 1.57,$$

$$s_4 = a_{43}^{(3)} - l_{41}u_{13} - l_{42}u_{23} = 3.05.$$

主元为 $s_4 = 3.05, i_3 = 4$, 交换 $[A^{(3)} : b^{(3)}]$ 的第 3 行与第 4 行, 方程组相应地交换为 $E_{34}E_{23}E_{14}AX = E_{34}E_{23}E_{14}b$, 再算 U 的第 3 行与 L 的第 3 列, 得

$$[A^{(4)} : b^{(4)}] = \left[\begin{array}{cccc|c} 5.11 & -4.00 & 3.33 & -1.11 & 3.77 \\ 0.845 & 1.43 & -2.81 & 3.02 & 0.130 \\ 0.196 & 0.783 & 3.05 & -2.48 & 3.00 \\ -0.393 & -0.0839 & 0.515 & 2.95 & 5.40 \end{array} \right],$$

$$r=4: u_{44} = a_{44}^{(4)} - l_{41}u_{14} - l_{42}u_{24} - l_{43}u_{34} = 4.04.$$

至此, 分解完毕, $PA=LU$, 其中

$$L = \left[\begin{array}{cccc} 1 & & & \\ 0.845 & 1 & & \\ 0.196 & 0.783 & 1 & \\ -0.393 & -0.0839 & 0.515 & 1 \end{array} \right],$$

$$U = \begin{bmatrix} 5.11 & -4.00 & 3.33 & -1.11 \\ & 1.43 & -2.81 & 3.02 \\ & & 3.05 & -2.48 \\ & & & 4.04 \end{bmatrix},$$

排列矩阵为

$$P = E_{34} E_{23} E_{14} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

容易验证 $b^{(4)} = Pb$, 最后, 解 $LY = Pb$, 得

$$Y = (3.77, -3.06, 4.65, 4.23)^T$$

再解 $UX = Y$, 得所求的解为(有舍入误差)

$$X = (-0.334, 0.318, 2.38, 1.05)^T.$$

6.4.3 Cholesky(平方根)法

应用有限元法解结构力学问题时, 最后归结为求解线性方程组, 系数矩阵大多具有对称正定性质. 所谓平方根法, 就是利用对称正定矩阵的三角分解而得到的求解对称正定方程组的一种有效方法, 目前在计算机上广泛应用平方根法解此类方程组.

1. 平方根分解法

设 A 的一切顺序主子式不为 0, 由定理 6.10 知有唯一的分解式 $A = LU$, 其中

$$L = \begin{bmatrix} 1 & & & \\ l_{21} & 1 & & \\ l_{31} & l_{32} & 1 & \\ \vdots & \vdots & \ddots & \ddots \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{bmatrix}.$$

若 A 对称且各阶顺序主子式不为 0, 则由定理 6.8 的推论知

$$u_{11} = \det D_1 \neq 0, u_{ii} = \det D_i / \det D_{i-1} \neq 0 \quad (i = 2, 3, \cdots, n).$$

进一步令 $U = DU_0$, 其中

$$D = \begin{bmatrix} u_{11} & & & \\ & u_{22} & & \\ & & \ddots & \\ & & & u_{nn} \end{bmatrix}, \quad U_0 = \begin{bmatrix} 1 & \frac{u_{12}}{u_{11}} & \cdots & \frac{u_{1n}}{u_{11}} \\ & 1 & \cdots & \frac{u_{2n}}{u_{22}} \\ & & \ddots & \vdots \\ & & & 1 \end{bmatrix},$$

便有

$$A = LDU_0.$$

可以证明 $U_0 = L^T$, 事实上 $A = A^T = (LDU_0)^T = U_0^T D L^T$, 显然 U_0^T 是单位下三角矩阵, $D L^T$ 是上三角矩阵, 由分解唯一性(定理 6.10)知必有 $U_0^T = L$, 即 $U_0 = L^T$. 这样一来可得

$$A = LDL^T$$

综上所述, 可得如下定理.

定理 6.12 (对称阵的三角分解定理) 设 A 为 n 阶对称阵, 且 A 的所有顺序主子式均不为零, 则 A 可唯一分解为 $A = LDL^T$. 其中, L 为单位下三角矩阵, D 为对角阵.

现设 A 对称、正定, 因 $A = LDL^T$, 而 D 的元素均大于 0, 故可将 D 分解成 $D = \tilde{D}\tilde{D}^T$, 其中

$$\tilde{D} = \begin{bmatrix} \sqrt{u_{11}} & & & \\ & \sqrt{u_{22}} & & \\ & & \ddots & \\ & & & \sqrt{u_{nn}} \end{bmatrix},$$

并令 $\tilde{L} = L\tilde{D}$, 最后得

$$A = \tilde{L}\tilde{L}^T.$$

在这里 $\tilde{L}$ 是一般的下三角阵, 由推理过程不难看出分解的在一定条件下才具有唯一性, 于是有以下结论.

定理 6.13 (对称正定矩阵的三角分解式) 如果 A 为 n 阶对称正定矩阵, 则存在一个实的非奇异下三角阵 L 使 $A = LL^T$, 当限定 L 的对角元素为正时, 这样分解是唯一的.

关于 $\tilde{L}$ 的元素可用 LU 分解的同样方法得出以下结论:

$$(1) \tilde{l}_{jj} = (a_{jj} - \sum_{k=1}^{j-1} \tilde{l}_{jk}^2)^{1/2}; \quad (6.16)$$

$$(2) \tilde{l}_{ij} = (a_{ij} - \sum_{k=1}^{j-1} \tilde{l}_{ik} \tilde{l}_{jk}) / \tilde{l}_{jj}, \quad i = j+1, \dots, n; j = 1, 2, \dots, n.$$

在完成 $A = \tilde{L}\tilde{L}^T$ 分解之后, 方程 $AX = b$, 即 $\tilde{L}\tilde{L}^T X = b$ 可化为

$$\begin{cases} \tilde{L}Y = b, \\ \tilde{L}^T X = Y. \end{cases} \quad (6.17)$$

由式(6.16)不难得出以下结论:

$$(3) y_i = (b_i - \sum_{k=1}^{i-1} \tilde{l}_{ik} y_k) / \tilde{l}_{ii}, \quad i = 1, 2, \dots, n;$$

$$(4) x_i = (y_i - \sum_{k=i+1}^n \tilde{l}_{ki} x_k) / \tilde{l}_{ii}, \quad i = n, n-1, \dots, 1.$$

由计算公式(6.16)知

$$a_{jj} = \sum_{k=1}^j \tilde{l}_{jk}^2, \quad j = 1, 2, \dots, n,$$

所以

$$\tilde{l}_{jk}^2 \leq a_{jj} \leq \max_{1 \leq j \leq n} \{a_{jj}\},$$

于是

$$\max_{j,k} \{\tilde{l}_{jk}^2\} \leq \max_{1 \leq j \leq n} \{a_{jj}\}.$$

上面分析说明,分解过程中元素 $\tilde{l}_{jk}$ 的数量级不会增大,且对角元素 $\tilde{l}_{jj}$ 恒为正数,于是不选主元素的平方根法是一个数值稳定的方法.

这种方法叫平方根法,也叫 Cholesky 分解,由式(6.16)可以算出 Cholesky 分解的乘除次数为 $\frac{1}{6}n^3 + \frac{1}{2}n^2 - \frac{1}{3}n$ 次,这个数字大约是 LU 分解乘除次数之半,但这里要有 n 次开平方运算.

例 6.9 求矩阵

$$A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 4 & 1 \\ 1 & 1 & 4 \end{bmatrix}$$

的 Cholesky 分解 $A=LL^T$.

解 第 1 步,计算 L 的第一列元素:

$$l_{11} = \sqrt{a_{11}} = \sqrt{4} = 2,$$

$$l_{21} = \frac{a_{21}}{l_{11}} = \frac{2}{2} = 1, \quad l_{31} = \frac{a_{31}}{l_{11}} = \frac{1}{2},$$

因此

$$A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 4 & 1 \\ 1 & 1 & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 2 & 2 & 1 \\ 1 & 4 & 1 \\ \frac{1}{2} & 1 & 4 \end{bmatrix}.$$

第 2 步,计算 L 的第 2 列元素:

$$l_{22} = (a_{22} - l_{21}^2)^{\frac{1}{2}} = (4 - 1^2)^{\frac{1}{2}} = \sqrt{3},$$

$$l_{32} = (a_{32} - l_{31}l_{21})/l_{22} = \left(1 - \frac{1}{2} \times 1\right) / \sqrt{3} = \frac{\sqrt{3}}{6},$$

从而

$$\begin{bmatrix} 2 & 2 & 1 \\ 1 & 4 & 1 \\ \frac{1}{2} & 1 & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 2 & 2 & 1 \\ 1 & \sqrt{3} & 1 \\ \frac{1}{2} & \frac{\sqrt{3}}{6} & 4 \end{bmatrix}.$$

第3步,计算 L 的第3列元素:

$$l_{33} = (a_{33} - l_{31}^2 - l_{32}^2)^{\frac{1}{2}} = \left(4 - \left(\frac{1}{2}\right)^2 - \left(\frac{\sqrt{3}}{6}\right)^2\right)^{\frac{1}{2}} = \frac{\sqrt{33}}{3},$$

于是

$$\begin{bmatrix} 2 & 2 & 1 \\ 1 & \sqrt{3} & 1 \\ \frac{1}{2} & \frac{\sqrt{3}}{6} & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 2 & 2 & 1 \\ 1 & \sqrt{3} & 1 \\ \frac{1}{2} & \frac{\sqrt{3}}{6} & \frac{\sqrt{33}}{3} \end{bmatrix}.$$

这样,我们得到 Cholesky 分解 $A=LL^T$, 其中

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & \sqrt{3} & 0 \\ \frac{1}{2} & \frac{\sqrt{3}}{6} & \frac{\sqrt{33}}{3} \end{bmatrix}.$$

2. 改进的平方根法

为了在计算中避免开方运算,再回到 LDL^T 分解,设

$$A = LDL^T$$

$$= (LD)L^T = \begin{bmatrix} d_1 & & & & \\ l_{21}d_1 & d_2 & & & \\ l_{31}d_1 & l_{32}d_2 & d_3 & & \\ \vdots & \vdots & \ddots & \ddots & \\ l_{n1}d_1 & l_{n2}d_2 & \cdots & l_{nn-1}d_{n-1} & d_n \end{bmatrix} \begin{bmatrix} 1 & l_{21} & l_{31} & \cdots & l_{n1} \\ & 1 & l_{32} & \cdots & l_{n2} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & l_{nn-1} \\ & & & & 1 \end{bmatrix},$$

由矩阵相乘可得

$$\begin{cases} a_{11} = d_1, \\ a_{21} = l_{21}d_1, \\ a_{22} = l_{21}d_1l_{21} + d_2, \\ \begin{cases} l_{21} = a_{21}/d_1, \\ d_2 = a_{22} - l_{21}^2d_1. \end{cases} \end{cases}$$

即

设 L, D 的前 $(i-1)$ 行元素已求得, 则

$$a_{ij} = \sum_{k=1}^{j-1} l_{ik}d_kl_{jk} + l_{ij}d_j, \quad j = 1, 2, \cdots, i,$$

故得

$$\begin{cases} l_{ij} = (a_{ij} - \sum_{k=1}^{j-1} l_{ik}d_kl_{jk})/d_j, & j = 1, 2, \cdots, i-1, \\ d_i = a_{ii} - \sum_{k=1}^{i-1} l_{ik}d_kl_{ik}, & i = 1, 2, \cdots, n. \end{cases} \quad (6.18)$$

为了减少重复相乘的计算,令 $t_{ij} = l_{ij}d_j$,最后可得

$$\begin{cases} t_{ij} = a_{ij} - \sum_{k=1}^{j-1} t_{ik}l_{jk}, & j = 1, 2, \dots, i-1, \\ d_i = a_{ii} - \sum_{k=1}^{i-1} t_{ik}l_{ik}, \\ l_{ij} = t_{ij}/d_j, & j = 1, 2, \dots, i-1. \end{cases} \quad (6.19)$$

按 $i=1, 2, \dots, n$ 的顺序进行计算.

编程序计算时,为了节省存储单元,按式(6.19)分解计算时先把算得的 t_{ij} ($j=1, 2, \dots, i-1$) 存放在一组工作单元中,把 d_i 存放在 a_{ii} 的单元中,再把算得的 l_{ij} 存放在 a_{ij} 的单元中.

A 的 LDL^T 分解完成后, $AX=b$, 即 $LDL^TX=b$ 可化为

$$\begin{cases} LY = b, \\ DL^TX = Y, \end{cases} \quad (6.20)$$

于是

$$\begin{cases} y_i = b_i - \sum_{k=1}^{i-1} l_{ik}y_k, & i = 1, 2, \dots, n, \\ x_i = y_i/d_i - \sum_{k=i+1}^n l_{ki}x_k, & i = n, n-1, \dots, 1. \end{cases} \quad (6.21)$$

例 6.10 用平方根法解方程组

$$\begin{cases} 4x_1 - x_2 + x_3 = 6, \\ -x_1 + 4.25x_2 + 2.75x_3 = -0.5, \\ x_1 + 2.75x_2 + 3.5x_3 = 1.25. \end{cases}$$

解 因系数矩阵对称,且顺序主子式 $\Delta_1=4>0$, $\Delta_2=16>0$, $\Delta_3=16>0$,故方程组为正定方程组,利用算法公式(6.16),有

$$\begin{aligned} j=1: l_{11} &= a_{11}^{1/2} = \sqrt{4} = 2, \\ l_{21} &= a_{21}/l_{11} = -1/2 = -0.5, \\ l_{31} &= a_{31}/l_{11} = 1/2 = 0.5; \end{aligned}$$

$$\begin{aligned} j=2: l_{22} &= (a_{22} - l_{21}^2)^{1/2} = (4.25 - (-0.5)^2)^{1/2} = 2, \\ l_{32} &= (a_{32} - l_{31}l_{21})/l_{22} = (2.75 - 0.5 \times (-0.5))/2 = 1.5; \\ j=3: l_{33} &= (a_{33} - l_{31}^2 - l_{32}^2)^{1/2} = (3.5 - (0.5)^2 - (1.5)^2)^{1/2} = 1. \end{aligned}$$

解

$$LY = b,$$

得

$$\begin{cases} y_1 = b_1/l_{11} = 6/2 = 3, \\ y_2 = (b_2 - l_{21}y_1)/l_{22} = (-0.5 - (-0.5) \times 3)/2 = 0.5, \\ y_3 = (b_3 - l_{31}y_1 - l_{32}y_2)/l_{33} = (1.25 - 0.5 \times 3 - 1.5 \times 0.5)/1 = -1. \end{cases}$$

解

$$L^T X = Y,$$

得

$$\begin{cases} x_3 = y_3/l_{33} = -1/1 = -1, \\ x_2 = (y_2 - l_{32}x_3)/l_{22} = (0.5 - 1.5 \times (-1))/2 = 1, \\ x_1 = (y_1 - l_{21}x_2 - l_{31}x_3)/l_{11} \\ \quad = (3 - (-0.5) \times 1 - 0.5 \times (-1))/2 = 2. \end{cases}$$

因此,系数矩阵的 LL^T 分解为

$$\begin{bmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{bmatrix} = \begin{bmatrix} 2 & & \\ -0.5 & 2 & \\ 0.5 & 1.5 & 1 \end{bmatrix} \begin{bmatrix} 2 & -0.5 & 0.5 \\ & 2 & 1.5 \\ & & 1 \end{bmatrix},$$

方程组的解为

$$X = (2, 1, -1)^T.$$

再用改进的平方根法解上例,用算法公式(6.19),有

$$i = 1: d_1 = a_{11} = 4;$$

$$i = 2: t_{21} = a_{21} = -1, l_{21} = t_{21}/d_1 = -1/4 = -0.25,$$

$$d_2 = a_{22} - t_{21}l_{21} = 4.25 - (-1) \times (-0.25) = 4;$$

$$i = 3: t_{31} = a_{31} = 1,$$

$$t_{32} = a_{32} - t_{31}l_{21} = 2.75 - 1 \times (-0.25) = 3,$$

$$l_{31} = t_{31}/d_1 = 1/4 = 0.25, l_{32} = t_{32}/d_2 = 3/4 = 0.75,$$

$$d_3 = 3.5 - 1 \times 0.25 - 3 \times 0.75 = 1 \quad (d_3 = a_{33} - t_{31}l_{31} - t_{32}l_{32}).$$

因此

$$L = \begin{bmatrix} 1 & & \\ -0.25 & 1 & \\ 0.25 & 0.75 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 4 & & \\ & 4 & \\ & & 1 \end{bmatrix},$$

解

$$LY = b,$$

得

$$\begin{cases} y_1 = b_1 = 6, \\ y_2 = b_2 - l_{21}y_1 = -0.5 - (-0.25) \times 6 = 1, \\ y_3 = 1.25 - 0.25 \times 6 - 0.75 \times 1 = -1. \end{cases}$$

解

$$DL^T X = Y,$$

得

$$\begin{cases} x_3 = y_3/d_3 = -1/1 = -1, \\ x_2 = y_2/d_2 - l_{32}x_3 = \frac{1}{4} - 0.75 \times (-1) = 1, \\ x_1 = y_1/d_1 - l_{21}x_2 - l_{31}x_3 = \frac{6}{4} - (-0.25) \times 1 - 0.25 \times (-1) = 2. \end{cases}$$

LDL^T 分解法与 LL^T 分解法相比,计算量相近,不需做开方运算,在实际计算中,求解系数矩阵正定的线性代数方程组,广泛使用平方根法及改进的平方根法。

3. 追赶法

现先对矩阵对角占优的概念做一介绍. 设 $A=(a_{ij})_{n \times n}$, 若对于 $i=1, 2, \dots, n$, 不等式 $|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ 均成立, 且至少对某个 i 有

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|,$$

则称矩阵 A 按行对角占优. 若对于 $i=1, 2, \dots, n$ 不等式 $|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ 均成立, 则称 A 按行严格对角占优. 类似地, 也有按列对角占优和按列严格对角占优的概念, 按行或按列对角占优简称对角占优, 按行或按列严格对角占优简称强对角占优.

在用差分方法解二阶常微分方程的边值问题, 热传导及三次样条插值函数的求解等问题中会遇到一种特殊的线性方程组, 其系数矩阵为对角占优的三对角矩阵, 且阶数一般较高, 这个问题适合于用 LU 分解法求解的典型问题之一, 为了能大量节省存贮单元, 在这里并不用现成的 LU 分解法, 而是根据这里的特殊条件导出一套递推公式.

设方程组 $AX=F$, 即

$$\begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_i & b_i & c_i \\ & \ddots & \ddots & \ddots & \\ & & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ \vdots \\ \vdots \\ f_n \end{bmatrix}, \quad (6.22)$$

其中, $|b_1| > |c_1| > 0$, $|b_i| \geq |a_i| + |c_i|$
 $(a_i, c_i \neq 0, i=2, 3, \dots, n-1)$, $|b_n| > |a_n| > 0$.

由 LU 分解公式(6.14)和式(6.15)可以看出这里的 L 和 U 是特殊的三角矩阵.

$$L = \begin{bmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & l_3 & \ddots & & \\ & & \ddots & 1 & \\ & & & l_n & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_1 & d_1 & & & \\ & u_2 & d_2 & & \\ & & \ddots & \ddots & \\ & & & u_{n-1} & d_{n-1} \\ & & & & u_n \end{bmatrix},$$

其中, L 和 U 非零元素极少且分布很有规则. 这种情况不宜采用费时的满矩阵的 Doolittle(杜里特尔)分解法, 宜采用一种节省计算量的方法: 直接利用 L 乘 U 得到 A 的乘法规则, 求得矩阵 L 和 U 的诸元素. 计算公式如下:

$$\begin{cases} d_i = c_i, & i = 1, 2, \dots, n-1, \\ u_1 = b_1, \\ l_i = a_i/u_{i-1}, & i = 2, 3, \dots, n, \\ u_i = b_i - l_i c_{i-1}, & i = 2, 3, \dots, n, \end{cases} \quad (6.23)$$

计算次序是 $u_1, l_2, u_2, l_3, u_3, \dots, l_n, u_n$.

原方程组 $AX=F$ 是通过下述两个具有两条对角线元素的三角形方程组实现的.

$$\begin{cases} LY = F, \\ UX = Y. \end{cases}$$

计算公式是

$$\begin{cases} y_1 = f_1, \\ y_i = f_i - l_i y_{i-1}, & i = 2, 3, \dots, n; \end{cases} \quad (6.24)$$

$$\begin{cases} x_n = y_n/u_n, \\ x_i = (y_i - c_i x_{i+1})/u_i, & i = n-1, n-2, \dots, 1. \end{cases} \quad (6.25)$$

称式(6.23)、式(6.24)和式(6.25)为求解三角形方程组的追赶法.

当矩阵 A 是严格对角占优或对称正定时, 追赶法一定可以实现. 下面给出一个使追赶法可行的充分条件.

定理 6.14 设具有式(6.22)形式的三对角阵 A 满足条件

- (1) $|b_1| > |c_1| > 0$;
- (2) $|b_n| > |a_n| > 0$;
- (3) $|b_i| \geq |a_i| + |c_i|, a_i c_i \neq 0, i = 2, 3, \dots, n-1$,

则方程组 $AX=F$ 的解存在唯一, 且追赶法通常是数值稳定的.

追赶法公式简单, 计算量和存储量都小. 整个求解过程仅需 $(5n-4)$ 次乘除和 $3(n-1)$ 次加减运算, 总共 $(8n-7)$ 次运算. 仅需 4 个一维数组存储向量 a, b, c 和 f , 其中 d_i, l_i, u_i 和 x_i 分别存在数组 c, a, b 和 f 中.

例 6.11 用追赶法求解三对角方程组

$$\begin{bmatrix} 2 & 1 & & \\ 1 & 3 & 1 & \\ & 1 & 1 & 1 \\ & & 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 2 \\ 0 \end{bmatrix}.$$

解 设

$$\begin{bmatrix} 2 & 1 & & \\ 1 & 3 & 1 & \\ & 1 & 1 & 1 \\ & & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & & & \\ l_2 & 1 & & \\ & l_3 & 1 & \\ & & l_4 & 1 \end{bmatrix} \begin{bmatrix} u_1 & d_1 & & \\ & u_2 & d_2 & \\ & & u_3 & d_3 \\ & & & u_4 \end{bmatrix}$$

由分解公式(6.23)计算得

$$d_1 = 1, \quad d_2 = 1, \quad d_3 = 1;$$

$$u_1 = 2, \quad l_2 = \frac{1}{2}, \quad u_2 = \frac{5}{2}, \quad l_3 = \frac{2}{5};$$

$$u_3 = \frac{3}{5}, \quad l_4 = \frac{10}{3}, \quad u_4 = -\frac{7}{3}.$$

求解

$$\begin{bmatrix} 1 & & & \\ \frac{1}{2} & 1 & & \\ & \frac{2}{5} & 1 & \\ & & \frac{10}{3} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 2 \\ 0 \end{bmatrix},$$

利用计算公式(6.24)得

$$y_1 = 1, \quad y_2 = \frac{3}{2}, \quad y_3 = \frac{7}{5}, \quad y_4 = -\frac{14}{3}.$$

利用计算公式(6.25)再求解

$$\begin{bmatrix} 2 & 1 & & \\ & \frac{5}{2} & 1 & \\ & & \frac{3}{5} & 1 \\ & & & -\frac{7}{3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{3}{2} \\ \frac{7}{5} \\ -\frac{14}{3} \end{bmatrix},$$

得

$$x_4 = 2, \quad x_3 = -1, \quad x_2 = 1, \quad x_1 = 0.$$

6.5 误差分析

6.5.1 矩阵的条件数

在求解线性代数方程组时,由于实际提供的或经过计算而得到的方程组的系数矩阵和常数项的元素总有一定误差,在将各种原始数据输入计算机并进行进制转换时会带来误差,在计算机内做每次运算还会产生舍入误差,因此,利用各种直接方法都只能求得方程组的近似解,这就需要去估计近似解的误差,或者说估计近似解的精确度,许多例子说明,这个问题是不容忽视的.

例 6.12 对比方程组

$$\begin{bmatrix} 2 & 6 \\ 2 & 6.00001 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 8 \\ 8.00001 \end{bmatrix} \quad (6.26)$$

和
$$\begin{bmatrix} 2 & 6 \\ 2 & 5.99999 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 8 \\ 8.00002 \end{bmatrix}, \quad (6.27)$$

前者解为 $(x_1, x_2) = (1, 1)$, 后者解为 $(x_1, x_2) = (10, -2)$.

这个简单的例子表明, 即使两个线性代数方程组的系数及常数项极其相近, 解的差别却可能很大; 也可解释为当方程组的系数或常数项产生微小扰动(误差)时, 可能使解产生很大误差, 甚至失去近似的意义. 因此, 分析方程组原始数据扰动对解的影响, 是要解决的第一个问题, 基本线索是通过估计扰动方程组解的误差界, 得出原始数据扰动对解的影响程度的一种量度——矩阵的条件数.

要解决的第二个问题是分析算法的数值稳定性, 即计算过程舍入误差对方程组的影响.

对于计算机求得的方程组的近似解, 原始数据扰动及运算舍入误差的影响不是孤立的. 要完全搞清近似解的误差问题不是一件容易的事. 从理论上作出的误差界估计, 常常比实际误差大得多, 因此有时通过一些间接的简便方法来估计近似解的精确度, 尽管这种估计不是对所有情形都是可靠的.

向量范数和矩阵范数是估计方程组近似解误差的基本工具.

下面就一般方程 $AX=b$ 进行讨论, 首先考察右端项 b 的扰动对解的影响.

令 δb 表示右端项 b 的扰动, 相应的解 X 的扰动记为 δX , 即

$$A(X + \delta X) = b + \delta b,$$

从中消去 $AX=b$, 得 $A\delta X = \delta b$, 故有

$$\|\delta X\| = \|A^{-1}\delta b\| \leq \|A^{-1}\| \|\delta b\|.$$

另一方面, 因为

$$\|X\| \geq \frac{\|AX\|}{\|A\|} = \frac{\|b\|}{\|A\|},$$

故有

$$\frac{\|\delta X\|}{\|X\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}.$$

于是得到以下结论.

定理 6.15 设 A 是非奇异阵, $AX=b \neq 0$, 且 $A(X+\delta X)=b+\delta b$, 则

$$\frac{\|\delta X\|}{\|X\|} \leq \|A^{-1}\| \|A\| \frac{\|\delta b\|}{\|b\|}. \quad (6.28)$$

上式给出了解的相对误差的上界, 常数项 b 的相对误差在解中可能放大 $\|A^{-1}\| \|A\|$ 倍.

再考察系数矩阵 A 的扰动对解的影响, 令 δA 表示 A 的扰动, 相应的解 X 的扰动仍记为 δX , 即

$$(A + \delta A)(X + \delta X) = b. \quad (6.29)$$

再从中消去 $AX=b$, 得

$$A\delta X - \delta A(X + \delta X) = 0,$$

故有 $\|\delta X\| = \|A^{-1}\delta A(X + \delta X)\| \leq \|A^{-1}\| \|\delta A\| (\|X\| + \|\delta X\|).$

假设 $\|\delta A\|$ 足够小, 使 $\|A^{-1}\| \|\delta A\| < 1$ 成立, 则由上式得

$$\frac{\|\delta X\|}{\|X\|} \leq \frac{\|A^{-1}\| \|\delta A\|}{1 - \|A^{-1}\| \|\delta A\|},$$

或者

$$\frac{\|\delta X\|}{\|X\|} \leq \frac{\|A^{-1}\| \|A\| \frac{\|\delta A\|}{\|A\|}}{1 - \|A^{-1}\| \|A\| \frac{\|\delta A\|}{\|A\|}}, \quad (6.30)$$

于是又得到以下结论.

定理 6.16 设 A 为非奇异矩阵, $AX=b \neq 0$, 且 $(A+\delta A)(X+\delta X)=b$, 如果 $\|A^{-1}\| \|\delta A\| < 1$, 则式(6.30)成立.

如果 $\|\delta A\|$ 充分小, 且在 $\|A^{-1}\| \|\delta A\| < 1$ 条件下, 那么式(6.30)说明矩阵 A 的相对误差 $\frac{\|\delta A\|}{\|A\|}$ 在解中可放大 $\|A^{-1}\| \|A\|$ 倍.

定义 6.7 设 A 为非奇异阵, 称数 $\text{cond}(A)_p = \|A^{-1}\|_p \|A\|_p$ ($p=1, 2$ 或 ∞) 为矩阵 A 的条件数.

那么式(6.28)、式(6.30)又可分别表示

$$\frac{\|\delta X\|}{\|X\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|},$$

$$\frac{\|\delta X\|}{\|X\|} \leq \text{cond}(A) \frac{\|\delta A\|}{\|A\|} / (1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}).$$

这两个式子说明, 右端项 b 和系数矩阵 A 的扰动对解的影响与条件数 $\text{cond}(A)$ 的大小有关, $\text{cond}(A)$ 愈大, 扰动对解的影响愈大, 因而条件数 $\text{cond}(A)$ 的值刻画了方程组“病态”的程度, 当然方程组病态的程度是相对的.

矩阵的条件数与所用的范数有关, 通常使用的条件数有下列一些:

(1) $\text{cond}(A)_\infty = \|A^{-1}\|_\infty \|A\|_\infty$;

(2) A 的谱条件数

$$\text{cond}(A)_2 = \|A\|_2 \|A^{-1}\|_2 = \sqrt{\frac{\lambda_{\max}(A^T A)}{\lambda_{\min}(A^T A)}},$$

当 A 为对称矩阵时, 有

$$\text{cond}(A)_2 = \frac{|\lambda_1|}{|\lambda_n|},$$

其中, λ_1, λ_n 为 A 的绝对值最大和绝对值最小的特征值.

条件数的性质有如下几点.

(1) 对任何非奇异矩阵 A , 都有 $\text{cond}(A)_p \geq 1$, 事实上, 有

$$\text{cond}(A)_p = \|A^{-1}\|_p \|A\|_p \geq \|A^{-1}A\|_p = 1;$$

(2) 设 A 为非奇异矩阵且 $C \neq 0$ (常数), 则有

$$\text{cond}(CA)_p = \text{cond}(A)_p;$$

(3) 如果 A 为正交矩阵, 则 $\text{cond}(A)_2 = 1$, 如果 A 为非奇异矩阵, R 为正交矩阵, 则有

$$\text{cond}(RA)_2 = \text{cond}(AR)_2 = \text{cond}(A)_2.$$

6.5.2 精度分析

求得方程组 $AX=b$ 的一个近似解 $\tilde{X}$ 以后, 自然希望判断其精度, 检验精度的一个简单办法是, 将近似解 $\tilde{X}$ 再回代到原方程组去求出余量 R , 有

$$R = b - A\tilde{X},$$

如果 $\|R\|$ 很小, 一般情况下可认为解 $\tilde{X}$ 是相当准确的.

定理 6.17 设 $\tilde{X}$ 是方程组 $AX=b$ 的一个近似解, 其精确解记为 X^* , R 为 $\tilde{X}$ 的余量, 则有

$$\frac{\|X^* - \tilde{X}\|}{\|X^*\|} \leq \text{cond}(A) \frac{\|R\|}{\|b\|}. \quad (6.31)$$

证 由于

$$AX^* = b, A(X^* - \tilde{X}) = R,$$

故有

$$\|b\| = \|AX^*\| \leq \|A\| \|X^*\|,$$

$$\|X^* - \tilde{X}\| = \|A^{-1}R\| \leq \|A^{-1}\| \|R\|.$$

由此易得式(6.31).

估计式(6.31)表明, 用余量大小来检验近似解精度的办法对于病态方程组是不可靠的, 事实上, 由例 6.12, 若将方程组(6.27)的解 $x_1=10, x_2=-2$ 作为方程(6.26)的一个近似解, 所得余量为

$$R = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix} = \begin{bmatrix} 2 & 6 \\ 2 & 6.00001 \end{bmatrix} \begin{bmatrix} 10 \\ -2 \end{bmatrix} - \begin{bmatrix} 8 \\ 8.00001 \end{bmatrix} = \begin{bmatrix} 0 \\ -0.00003 \end{bmatrix}.$$

$\|R\|_\infty = 0.00003$ 虽然很小, 但这个近似解却与精确解 $x_1=x_2=1$ 差别很大, 解的误差向量为 $e = \begin{bmatrix} -9 \\ 3 \end{bmatrix}$, $\|e\|_\infty = 9$, 它是 $\|R\|_\infty$ 的 30000 倍!

6.6 线性代数方程组的迭代解法

这一节讨论线性代数方程组(6.32)的迭代解法.

$$AX = b,$$

(6.32)

其中

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

如果将矩阵 A 分解为

$$A = M - N,$$

且 M 非奇异, 则 $AX=b$ 可写成等价形式

$$MX = NX + b,$$

或

$$X = BX + F, \quad (6.33)$$

其中 $B=M^{-1}N, F=M^{-1}b$, 选定 $X^{(0)} \in \mathbb{R}^n$, 由式(6.33)可以构造出迭代方程

$$X^{(k+1)} = BX^{(k)} + F, \quad k = 0, 1, \dots. \quad (6.34)$$

如果由式(6.34)确定的向量序列 $\{X^{(k)}\}$ 在 $k \rightarrow \infty$ 时有极限 X^* , 显然 X^* 是式(6.33)和式(6.32)的解, 式(6.34)形式的计算方法称为解式(6.32)简单形式的迭代法, 矩阵 B 称为迭代法的迭代矩阵.

这里需要讨论的问题是: 如何建立迭代格式? 向量序列收敛的条件是什么? 还有收敛速度、误差估计等等, 下面将一一讨论.

6.6.1 迭代公式的建立

1. Jacobi(雅可比)迭代

例 6.13 求解方程组

$$\begin{cases} 10x_1 - x_2 - 2x_3 = 7.2, \\ -x_1 + 10x_2 - 2x_3 = 8.3, \\ -x_1 - x_2 + 5x_3 = 4.2. \end{cases} \quad (6.35)$$

解 分别从方程组(6.35)的三个方程中分离出 x_1, x_2 和 x_3 , 得

$$\begin{cases} x_1 = 0.1x_2 + 0.2x_3 + 0.72, \\ x_2 = 0.1x_1 + 0.2x_3 + 0.83, \\ x_3 = 0.2x_1 + 0.2x_2 + 0.84. \end{cases} \quad (6.36)$$

据此可建立迭代公式

$$\begin{cases} x_1^{(k+1)} = 0.1x_2^{(k)} + 0.2x_3^{(k)} + 0.72, \\ x_2^{(k+1)} = 0.1x_1^{(k)} + 0.2x_3^{(k)} + 0.83, \\ x_3^{(k+1)} = 0.2x_1^{(k)} + 0.2x_2^{(k)} + 0.84. \end{cases} \quad (6.37)$$

设取迭代初值 $x_1^{(0)} = x_2^{(0)} = x_3^{(0)} = 0$, 表 6.1 记录了式(6.37)的迭代结果, 可以看到, 当迭代次数 k 增大时, 迭代值 $x_1^{(k)}, x_2^{(k)}, x_3^{(k)}$ 会越来越逼近方程组(6.35)的精确解 $x_1^* = 1.1, x_2^* = 1.2, x_3^* = 1.3$.

表 6.1

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0.00000	0.000000	0.00000
1	0.72000	0.83000	0.84000
2	0.97100	1.07000	1.15000
3	1.05700	1.15710	1.24820
4	1.08535	1.18534	1.28282
5	1.09510	1.19510	1.29414
6	1.09834	1.19834	1.29804
7	1.09944	1.19944	1.29933
8	1.09981	1.19981	1.29978
9	1.09994	1.19994	1.29992

这个简单的例子告诉我们,解线性方程组的迭代法,其基本思想是将联立方程组的求解归结为重复计算一组彼此独立的线性表达式,这就使问题得到了简化.

考察一般形式的方程组

$$\sum_{j=1}^n a_{ij}x_j = b_i, \quad i = 1, 2, \dots, n.$$

设从上述方程组的第 i 个方程中分离出变量 x_i , 若 $a_{ii} \neq 0$, ($i=1, 2, \dots, n$) 而将它改写成

$$x_i = \frac{1}{a_{ii}}(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j), \quad i = 1, 2, \dots, n, \quad (6.38)$$

据此建立的迭代公式

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j^{(k)}), \quad i = 1, 2, \dots, n, \quad (6.39)$$

称为解方程组(6.2)的 Jacobi 迭代分量形式.

为了将此迭代格式用矩阵表出, 设 $A = D + L + U$, 其中

$$D = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & \ddots & \\ & & & a_{nn} \end{bmatrix}, \quad L = \begin{bmatrix} 0 & & & \\ a_{21} & 0 & & \\ \vdots & & \ddots & \\ a_{n1} & \dots & \dots & a_{n,n-1} & 0 \end{bmatrix},$$

$$U = \begin{bmatrix} 0 & a_{12} & \dots & a_{1n} \\ & \ddots & \ddots & \vdots \\ & & \ddots & a_{n-1,n} \\ & & & 0 \end{bmatrix}, \quad D^{-1} = \begin{bmatrix} \frac{1}{a_{11}} & & & \\ & \frac{1}{a_{22}} & & \\ & & \ddots & \\ & & & \frac{1}{a_{nn}} \end{bmatrix}.$$

于是,式(6.38)可写成如下形式

$$X = -D^{-1}(L+U)X + F \quad (F = D^{-1}b). \quad (6.40)$$

实际上,这是将方程组(6.32)转化为等价方程组 $X = BX + F$, 此处, $B = -D^{-1}(L+U)$.

根据式(6.40),可建立迭代公式

$$X^{(k+1)} = -D^{-1}(L+U)X^{(k)} + F, \quad k = 0, 1, 2, \dots$$

这个迭代公式称为 Jacobi 迭代的矩阵形式, $B_J = -D^{-1}(L+U)$ 为其迭代矩阵.

2. Gauss-Seidel 迭代法

式(6.39)的计算一般按 $x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_n^{(k+1)}$ 的次序进行,注意到计算 $x_i^{(k+1)}$ 时, $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ 已经算好,而 Jacobi 迭代法(6.39)并不利用这些最新的近似值计算,仍用 $x_1^{(k)}, \dots, x_{i-1}^{(k)}$, 这启发我们对式(6.39)进行修改. 如何修改呢?

再考察前面的例 6.13, 设已得到近似值 $(x_1^{(k)}, x_2^{(k)}, x_3^{(k)})$, 由方程(6.37)的第 1 式得

$$x_1^{(k+1)} = 0.1x_2^{(k)} + 0.2x_3^{(k)} + 0.72.$$

对于收敛的迭代过程,所求出的“新”值 $x_1^{(k+1)}$ 常比“老”值 $x_1^{(k)}$ 更准确些,因此可用它替代“老”值 $x_1^{(k)}$ 做一步的计算,将 $x_1^{(k+1)}$ 与 $x_3^{(k)}$ 一起代入方程(6.37)的第 2 式得

$$x_2^{(k+1)} = 0.1x_1^{(k+1)} + 0.2x_3^{(k)} + 0.83.$$

出于同样的考虑,再用“新”值 $x_2^{(k+1)}$ 替代“老”值 $x_2^{(k)}$, 由方程(6.37)的第 3 个方程得

$$x_3^{(k+1)} = 0.1x_1^{(k+1)} + 0.2x_2^{(k+1)} + 0.84.$$

这样充分利用“新”值建立起来的迭代公式

$$\begin{cases} x_1^{(k+1)} = 0.1x_2^{(k)} + 0.2x_3^{(k)} + 0.72, \\ x_2^{(k+1)} = 0.1x_1^{(k+1)} + 0.2x_3^{(k)} + 0.83, \\ x_3^{(k+1)} = 0.1x_1^{(k+1)} + 0.2x_2^{(k+1)} + 0.84 \end{cases} \quad (6.41)$$

称为解方程组(6.32)的 Gauss-Seidel 公式.

仍取初值 $x_1^{(0)} = x_2^{(0)} = x_3^{(0)} = 0$, 按式(6.41)的计算结果见下表,与表 6.2 的计算结果相比较可以明显地看出,迭代公式(6.41)的效果比公式(6.37)好.

表 6.2

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0.00000	0.00000	0.00000
1	0.72000	0.90200	1.16400
2	1.04308	1.16719	1.28205
3	1.09313	1.19572	1.29777
4	1.09913	1.19947	1.29972
5	1.09989	1.19993	1.29996
6	1.09999	1.19999	1.30000

对于一般形式的方程组(6.32),其 Gauss-Seidel 迭代公式为

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)}), \quad i = 1, 2, \dots, n. \quad (6.42)$$

这个迭代公式称为 Gauss-Seidel 迭代分量形式.

Gauss-Seidel 公式(6.42)的特点是,一旦求出变元 x_i 的某个“新”值 $x_i^{(k+1)}$ 后,将用“新”值 $x_i^{(k+1)}$ 替代“老”值 $x_i^{(k)}$ 进行这一步剩下的计算,因此可将“新”值 $x_i^{(k+1)}$ 存放在“老”值 $x_i^{(k)}$ 所占用的单元内,这时,可将公式(6.42)表为下列动态形式

$$(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j)/a_{ii} \Rightarrow x_i, \quad i = 1, 2, \dots, n.$$

用偏差 $e = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}|$ 来刻划精度,若精度为 ϵ ,则当 $e < \epsilon$ 时,迭代终止.

Gauss-Seidel 迭代公式的矩阵形式导出如下:

$$AX = b \Rightarrow (D + L + U)X = b,$$

于是,有

$$DX = b - (L + U)X = b - LX - UX,$$

因而得

$$DX^{(k+1)} = b - LX^{(k+1)} - UX^{(k)}.$$

设 $(D+L)^{-1}$ 存在,则

$$X^{(k+1)} = -(D+L)^{-1}UX^{(k)} + (D+L)^{-1}b = BX^{(k)} + F.$$

其中, $B = -(D+L)^{-1}U$, $F = (D+L)^{-1}b$. 这就是 Gauss-Seidel 方法的矩阵形式,其迭代矩阵记为 $B_G = -(D+L)^{-1}U$.

以上介绍了求解线性方程组的两种迭代法,一般地说, Gauss-Seidel 迭代法要比 Jacobi 迭代法好,但情况并不总是这样,有时 Gauss-Seidel 迭代比 Jacobi 迭代收敛得慢,甚至可以举出 Jacobi 迭代收敛,但 Gauss-Seidel 迭代反而发散的例子.

6.6.2 迭代法的收敛性

定义 6.8 设有矩阵序列 $A_k = (a_{ij}^{(k)})_{n \times n}$ ($k=1, 2, \dots$) 及 $A = (a_{ij})_{n \times n}$, 如果

$$\lim_{k \rightarrow \infty} a_{ij}^{(k)} = a_{ij}, \quad i, j = 1, 2, \dots, n$$

成立,则称 $\{A_k\}$ 收敛于 A , 记为 $\lim_{k \rightarrow \infty} A_k = A$.

例 6.14 矩阵序列

$$A = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}, \quad A^2 = \begin{bmatrix} \lambda^2 & 2\lambda \\ 0 & \lambda^2 \end{bmatrix}, \quad \dots, \quad A^k = \begin{bmatrix} \lambda^k & k\lambda^{k-1} \\ 0 & \lambda^k \end{bmatrix} \quad \dots$$

当 $|\lambda| < 1$ 时, $A^k \rightarrow \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ (当 $k \rightarrow \infty$ 时).

矩阵序列极限的概念可以用任何矩阵范数来描述.

定理 6.18 $\lim_{k \rightarrow \infty} A_k = A \Leftrightarrow \|A_k - A\| \rightarrow 0$ ($k \rightarrow \infty$).

对于一般简单形式的迭代法

$$X^{(k+1)} = BX^{(k)} + F, \quad (6.43)$$

当 $k \rightarrow \infty$ 时, $X^{(k)}$ 是否有极限的问题, 就是迭代法的收敛问题, 如果设 $X = BX + F$ 的准确解是 X^* , 则由式(6.43)减去 $X^* = BX^* + F$, 便得到

$$X^{(k+1)} - X^* = B(X^{(k)} - X^*).$$

利用此式递推, 得

$$X^{(k)} - X^* = B(X^{(k-1)} - X^*) = \cdots = B^k(X^{(0)} - X^*).$$

所以, 当 $k \rightarrow \infty$ 时是否有 $X^{(k)} - X^* \rightarrow 0$ 的问题便由 B^k 的极限性质决定. 那么 B^k 的极限如何描述的呢?

下面再建立 $\lim_{k \rightarrow \infty} B^k = 0$ 的充要条件.

定理 6.19 设 $B \in \mathbb{R}^{n \times n}$, 则 $\lim_{k \rightarrow \infty} B^k = 0$ 的充分必要条件是 $\rho(B) < 1$, 其中 $\rho(B)$ 为 B 的谱半径.

证 B 的 Jordan 标准型为

$$J = \begin{bmatrix} J_1 & & \\ & J_2 & \\ & & \ddots \\ & & & J_s \end{bmatrix},$$

用对角阵的记号简记为 $J = \text{diag}(J_1, J_2, \dots, J_s)$, 其中的 Jordan 块 J_i 为 r_i 阶方阵

$$J_i = \begin{bmatrix} \lambda_i & 1 & & \\ & \lambda_i & 1 & \\ & & \ddots & \ddots \\ & & & \lambda_i & 1 \\ & & & & \lambda_i \end{bmatrix}_{r_i \times r_i}, \quad i = 1, 2, \dots, s.$$

λ_i 为 B 的特征值, $r_1 + r_2 + \cdots + r_s = n$, 因此存在非奇异矩阵 P , 使 $B = P^{-1}JP$, 则

$$B^k = P^{-1}J^kP.$$

所以, $\lim_{k \rightarrow \infty} B^k = 0$ 的充要条件为 $\lim_{k \rightarrow \infty} J^k = 0$, 因为 J 是块对角阵, 所以 $J^k = \text{diag}(J_1^k, J_2^k, \dots, J_s^k)$. $\lim_{k \rightarrow \infty} J^k = 0$ 的充要条件是 $\lim_{k \rightarrow \infty} J_i^k = 0$ ($i = 1, 2, \dots, s$). 而 J_i^k 可直接计算. 引入记号

$$E_i = \begin{bmatrix} 0 & 1 & & \\ & \ddots & \ddots & \\ & & \ddots & 1 \\ & & & 0 \end{bmatrix}_{r_i \times r_i}, \quad E_i^2 = \begin{bmatrix} 0 & 0 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & 1 \\ & & & \ddots & 0 \\ & & & & 0 \end{bmatrix},$$

$$E_i^3 = \begin{bmatrix} 0 & 0 & 0 & 1 & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & \ddots & 1 \\ & & & \ddots & \ddots & 0 \\ & & & & \ddots & 0 \\ & & & & & 0 \end{bmatrix},$$

则 $J_i^k = (\lambda_i I + E_i)^k = \lambda_i^k I + k\lambda_i^{k-1}E_i + C_k^2\lambda_i^{k-2}E_i^2 + \cdots + E_i^k$.

当 $k > r$ 时,有

$$J_i^k = \begin{bmatrix} \lambda_i^k & k\lambda_i^{k-1} & C_k^2\lambda_i^{k-2} & \cdots & C_k^{r+1}\lambda_i^{k-r-1} \\ & \lambda_i^k & k\lambda_i^{k-1} & \ddots & \vdots \\ & & \ddots & \ddots & C_k^2\lambda_i^{k-2} \\ & & & \ddots & k\lambda_i^{k-1} \\ & & & & \lambda_i^k \end{bmatrix}, \quad i = 1, 2, \dots, s.$$

当 $\rho(B) < 1$ 时,有 $|\lambda_i| < 1 (i=1, 2, \dots, s)$, 从而 $\lim_{k \rightarrow \infty} \lambda_i^k = 0$, 且 $C_k^j \lambda_i^{k-j} \rightarrow 0 (j=1, 2, \dots, i-1)$, 故有 $\lim_{k \rightarrow \infty} J_i^k = 0 (i=1, 2, \dots, s)$; 反之, 若 $\lim_{k \rightarrow \infty} J_i^k = 0$, 必有 $\lambda_i^k \rightarrow 0$, 从而 $|\lambda_i| < 1 (i=1, 2, \dots, s)$, $\rho(B) < 1$.

定理 6.20 (迭代法基本定理) 设有方程组

$$X = BX + F,$$

对于任意初始向量 $X^{(0)}$ 及任意 F , 解此方程组的迭代法 (即 $X^{(k+1)} = BX^{(k)} + F$) 收敛的充要条件是 $\rho(B) < 1$.

证 先证明若 $\rho(B) < 1$, 则方程组 $(I - B)X = F$ 存在唯一解. 因 $\rho(B) < 1$, 即 B 的特征值 $|\lambda_i| < 1 (i=1, 2, \dots, n)$, 则根据矩阵多项式特征值的知识, 矩阵 $I - B$ 的特征值 $\mu_i = 1 - \lambda_i \neq 0, (i=1, 2, \dots, n)$. $\det(I - B) = \prod_{i=1}^n (1 - \lambda_i) \neq 0$, 即 $I - B$ 非奇异, 因此方程组 $X = BX + F$ 有唯一解 X^* .

再证充分性. 因为 $\rho(B) < 1$, 根据定理 6.19, 有 $\lim_{k \rightarrow \infty} B^k = 0$. 而

$$X^{(k)} - X^* = B^k(X^{(0)} - X^*),$$

所以有 $\lim_{k \rightarrow \infty} X^{(k)} = X^*$, 即迭代对任意 F 及 $X^{(0)}$ 收敛.

最后证必要性. 设对任意初始向量 $X^{(0)}$ 和右端项 F , 均有 $\lim_{k \rightarrow \infty} X^{(k)} = X^*$ 且 $X^* = BX^* + F$, 由

$$X^{(k)} - X^* = B^k(X^{(0)} - X^*),$$

对任意 $X^{(0)}$ 有 $\lim_{k \rightarrow \infty} B^k(X^{(0)} - X^*) = 0$, 由此推出 $\lim_{k \rightarrow \infty} B^k = 0$. 由定理 6.18 知 $\rho(B) < 1$.

例 6.15 考察用 Jacobi 迭代法解方程组

$$\begin{cases} 8x_1 - 3x_2 + 2x_3 = 20, \\ 4x_1 + 11x_2 - x_3 = 33, \\ 6x_1 + 3x_2 + 12x_3 = 36 \end{cases} \quad (6.44)$$

的收敛性.

解 迭代矩阵 B_J 的特征方程为

$$\det(\lambda I - B_J) = \lambda^3 + 0.034090909\lambda + 0.039772727 = 0,$$

$$\begin{cases} \lambda_1 = -0.3082, \\ \lambda_2 = 0.1541 + i0.3245, \\ \lambda_3 = 0.1541 - i0.3245, \end{cases}$$

解得

可见 $|\lambda_2| = |\lambda_3| = 0.3592 < 1, \quad |\lambda_1| < 1,$

即 $\rho(B_J) < 1$. 所以用 Jacobi 迭代法解方程组(6.44)是收敛的.

例 6.16 考察用迭代法 $X^{(k+1)} = BX^{(k)} + f (k=0, 1, 2, \dots)$ 解下述方程组的收敛性:

$$X = BX + f,$$

其中

$$B = \begin{bmatrix} 0 & 2 \\ 3 & 0 \end{bmatrix}, \quad f = \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

解 矩阵特征方程为 $\det(\lambda I - B) = \lambda^2 - 6 = 0,$

解得特征根 $\lambda_{1,2} = \pm\sqrt{6}$, 即 $\rho(B) > 1$, 这说明用迭代法解此方程组不收敛.

例 6.17 讨论 Jacobi 迭代法和 Gauss-Seidel 迭代法解方程组 $AX=b$ 的收敛性, 其中 $b \neq 0$,

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 2 & 2 & 2 \\ -1 & -1 & 2 \end{bmatrix}.$$

解 经计算得 $\det A = 18 \neq 0$, 因此方程组 $AX=b (b \neq 0)$ 有唯一解. Jacobi 迭代法的迭代矩阵为

$$B_J = -D^{-1}(L+U) = -\begin{bmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{2} \end{bmatrix} \begin{bmatrix} 0 & -1 & 1 \\ 2 & 0 & 2 \\ -1 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{2} & -\frac{1}{2} \\ -1 & 0 & -1 \\ \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix}.$$

容易计算得 B 的特征值 $0, \pm\frac{\sqrt{5}}{2}i$, 因而 $\rho(B_J) = \frac{\sqrt{5}}{2} > 1$, 根据定理 6.20, Jacobi 迭代法不收敛.

Gauss-Seidel 迭代法的迭代矩阵为

$$\begin{aligned}
 B_G &= -(D+L)^{-1}U = - \begin{bmatrix} 2 & 0 & 0 \\ 2 & 2 & 0 \\ -1 & -1 & 2 \end{bmatrix}^{-1} \begin{bmatrix} 0 & -1 & 1 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} \frac{1}{2} & 0 & 0 \\ -\frac{1}{2} & \frac{1}{2} & 0 \\ 0 & -\frac{1}{4} & -\frac{1}{2} \end{bmatrix} \begin{bmatrix} 0 & -1 & 1 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & 0 & -\frac{1}{2} \end{bmatrix}
 \end{aligned}$$

它的特征值为 $0, -\frac{1}{2}, -\frac{1}{2}$, 因而 $\rho(B_G) = \frac{1}{2} < 1$, 故由定理 6.20 知 Gauss-Seidel 迭代法收敛.

我们也可以举出 Jacobi 迭代法收敛, 而 Gauss-Seidel 迭代法不收敛的例子.

例 6.18 讨论 Jacobi 迭代法和 Gauss-Seidel 迭代法解方程组 $AX=b$ 的收敛性, 其中 $b=[1, 3, 5]^T$,

$$A = \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix}.$$

解 经计算得 $\det A = 1 \neq 0$, 因此 A 非奇异. Jacobi 迭代法的迭代矩阵为

$$B_J = -D^{-1}(L+U) = - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 & 2 & -2 \\ 1 & 0 & 1 \\ 2 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{bmatrix}.$$

容易计算得 B_J 的特征值全为零, 从而 $\rho(B) = 0 < 1$, 故 Jacobi 迭代法收敛.

Gauss-Seidel 迭代法的迭代矩阵为

$$\begin{aligned}
 B_G &= -(D+L)^{-1}U = - \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 2 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 & 2 & -2 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \\
 &= - \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \begin{bmatrix} 0 & 2 & -2 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -2 & 2 \\ 0 & 2 & -3 \\ 0 & 0 & 2 \end{bmatrix}
 \end{aligned}$$

它的特征值为 $0, 2, 2$, 因而 $\rho(B_G) = 2 > 1$, 故 Gauss-Seidel 迭代法不收敛.

从定理 6.19 的证明中可以看到, 若 $\rho(B) < 1$, 且 $\rho(B)$ 越小, 则迭代收敛越快.

因为

$$X^{(k)} - X^* = B^k(X^{(0)} - X^*),$$

故

$$\|X^{(k)} - X^*\|_p \leq \|B\|_p^k \|X^{(0)} - X^*\|_p,$$

将上式可近似看成

$$\|X^{(k)} - X^*\|_p \approx (\rho(B))^k \|X^{(0)} - X^*\|_p,$$

这样 $(\rho(B))^k$ 就表示 k 次迭代后误差的乘数,若要求 k 次迭代后误差不超过初始误差的 10^{-t} 倍,即

$$(\rho(B))^k \leq 10^{-t},$$

即可确定迭代次数要满足

$$k \geq t \ln 10 / (-\ln \rho(B)).$$

有时用 $-\ln \rho(B)$ 表示迭代法 $X^{(k+1)} = BX^{(k)} + F$ 的渐近收敛率,当迭代收敛时, $-\ln \rho(B)$ 是一个正数,它越大则收敛越快.

定义 6.9 称 $R(B) = -\ln \rho(B)$ 为迭代法的收敛速度.

由于一般当 n 较大时,矩阵特征值计算比较困难,基本定理的条件比较难验证,所以最好建立与矩阵元素直接有关的条件来判别迭代法的收敛性.我们已经建立了矩阵范数与谱半径之间的关系,所以,若 $\|B\| < 1$,必有 $\rho(B) < 1$,这就得到如下的定理.

定理 6.21 (迭代法收敛的充分条件) 如果方程组 $X = BX + F$ 的迭代公式为 $X^{(k+1)} = BX^{(k)} + F$,且迭代矩阵的某一种范数 $\|B\|_p = q < 1$,则有

(1) 迭代法收敛,即对任取 $X^{(0)}$,有 $\lim_{k \rightarrow \infty} X^{(k)} = X^*$,且 $X^* = BX^* + F$;

(2) $\|X^* - X^{(k)}\|_p \leq \frac{q}{1-q} \|X^{(k)} - X^{(k-1)}\|_p$;

(3) $\|X^* - X^{(k)}\|_p \leq \frac{q^k}{1-q} \|X^{(1)} - X^{(0)}\|_p$.

证 因 $\rho(B) \leq q < 1$,故由定理 6.20 知,迭代收敛.再证(2)、(3).

因为有 $X^* - X^{(k+1)} = B(X^* - X^{(k)})$ 及 $X^{(k+1)} - X^{(k)} = B(X^{(k)} - X^{(k-1)})$

于是有 $\|X^{(k+1)} - X^{(k)}\|_p \leq q \|X^{(k)} - X^{(k-1)}\|_p \quad (k=1,2,\dots),$ (6.45)

$\|X^* - X^{(k+1)}\|_p \leq q \|X^* - X^{(k)}\|_p \quad (k=1,2,\dots),$ (6.46)

于是 $\|X^{(k+1)} - X^{(k)}\|_p = \|X^* - X^{(k)} - (X^* - X^{(k+1)})\|_p$
 $\geq \|X^* - X^{(k)}\|_p - \|X^* - X^{(k+1)}\|_p$
 $\geq (1-q) \|X^* - X^{(k)}\|_p,$

即 $\|X^* - X^{(k)}\|_p \leq \frac{1}{1-q} \|X^{(k+1)} - X^{(k)}\|_p$
 $\leq \frac{q}{1-q} \|X^{(k)} - X^{(k-1)}\|_p, \quad k=1,2,\dots.$

反复利用式(6.46)即得(3).

例 6.19 仍然考察例 6.15, Jacobi 方法迭代矩阵 B_J 的 ∞ -范数为

$$\|B_J\|_\infty = \max\left\{\frac{5}{8}, \frac{5}{11}, \frac{3}{4}\right\} = \frac{3}{4} < 1,$$

所以,对例 6.15 应用 Jacobi 迭代方法是收敛的.

例 6.20 设有方程组

$$X = BX + f,$$

其中
$$B = \begin{bmatrix} 0.9 & 0 \\ 0.3 & 0.8 \end{bmatrix}, \quad f = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

解 $\|B\|_{\infty} = 1.1$, $\|B\|_1 = 1.2$, $\|B\|_2 = 1.021$, $\|B\|_F = \sqrt{1.54}$. 虽然 B 的这些范数都大于 1, 但 B 的特征值为 $\lambda_1 = 0.9$, $\lambda_2 = 0.8$, 由定理 6.19, 对此方程组应用迭代法还是收敛的.

由定理 6.21 知, $\|B\| = q < 1$ 愈小, 迭代法收敛愈快.

当 B 的某一种范数 $\|B\| < 1$ 时, 若相邻两次迭代 $\|X^{(k)} - X^{(k-1)}\| < \epsilon_0$ (ϵ_0 为给定的精度要求), 则 $\|X^* - X^{(k)}\| \leq \frac{q}{1-q} \epsilon_0$, 所以在电算时通常利用 $\|X^{(k)} - X^{(k-1)}\| \leq \epsilon_0$

来作为控制迭代的终止条件. 不过要注意, 当 $q \approx 1$ 时, $\frac{q}{1-q}$ 较大, 尽管 $\|X^{(k)} - X^{(k-1)}\|$ 已非常小, 但误差向量的模 $\|\epsilon^{(k)}\| = \|X^* - X^{(k)}\|$ 则可能较大, 迭代法收敛将是缓慢的. 定理 6.21 的估计还可以用来事先确定需要迭代多少次才能保证 $\|\epsilon^{(k)}\| < \epsilon$.

若事先给出误差精度 ϵ , 由定理 6.21 中的(3)可得到迭代次数的估计

$$k > \left\lceil \ln \frac{\epsilon(1-q)}{\|X^{(1)} - X^{(0)}\|} / \ln q \right\rceil.$$

例 6.21 用 Jacobi 迭代法解方程组

$$\begin{cases} 20x_1 + 2x_2 + 3x_3 = 24, \\ x_1 + 8x_2 + x_3 = 12, \\ 2x_1 - 3x_2 + 15x_3 = 30. \end{cases}$$

取 $X^{(0)} = (0, 0, 0)^T$, 问 Jacobi 迭代法是否收敛? 若收敛, 需要迭代多少次, 才能保证各分量的误差绝对值小于 10^{-6} ?

解 Jacobi 迭代式为

$$\begin{cases} x_1^{(k+1)} = \frac{1}{20}(-2x_2^{(k)} - 3x_3^{(k)} + 24), \\ x_2^{(k+1)} = \frac{1}{8}(-x_1^{(k)} - x_3^{(k)} + 12), \\ x_3^{(k+1)} = \frac{1}{15}(-2x_1^{(k)} + 3x_2^{(k)} + 30). \end{cases}$$

迭代阵
$$B_J = \begin{bmatrix} 0 & -\frac{1}{10} & -\frac{3}{20} \\ -\frac{1}{8} & 0 & -\frac{1}{8} \\ -\frac{2}{15} & \frac{1}{5} & 0 \end{bmatrix}, \quad \|B_J\|_{\infty} = \frac{1}{3} < 1.$$

所以, 迭代法一定收敛.

经一次迭代得 $x_1^{(1)} = \frac{6}{5}, x_2^{(1)} = \frac{3}{2}, x_3^{(1)} = 2, \|X^{(1)} - X^{(0)}\|_{\infty} = 2,$

$$k > \left\lceil \ln \frac{10^{-6} \times \left(1 - \frac{1}{3}\right)}{2} / \ln \frac{1}{3} \right\rceil = 13.$$

所以,要保证各分量误差绝对值小于 10^{-6} ,需要迭代 14 次.

定义 6.10 (可约与不可约矩阵) 设 $A = (a_{ij})_n \in \mathbf{R}^{n \times n}$ (或 $\mathbf{C}^{n \times n}$), 当 $n \geq 2$ 时, 如果存在 n 阶排列阵 P 使

$$P^T A P = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix} \quad (6.47)$$

成立, 其中 A_{11} 为 r 阶子矩阵, A_{22} 为 $(n-r)$ 阶子矩阵 ($1 \leq r \leq n$), 则称矩阵 A 是可约的, 如果不存在排列阵 P 使式 (6.47) 成立, 则称 A 为不可约矩阵.

A 若是可约的, 显然 $AX=b$ 可以化为低阶的方程组求解.

事实上, 由 $AX=b$ 可化为

$$P^T A P (P^T X) = P^T b,$$

且记 $Y = P^T X = \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix}, P^T b = \begin{bmatrix} d_1 \\ d_2 \end{bmatrix}$, 其中 Y_1, d_1 为 r 维向量. 于是, 求解 $AX=b$ 化为求解方程组

$$\begin{cases} A_{11}Y_1 + A_{12}Y_2 = d_1, \\ A_{22}Y_2 = d_2. \end{cases}$$

由上式第 2 个方程组求出 Y_2 , 再代入第 1 个方程组求出 Y_1 .

显然, 如果 A 所有元素都非零, 则 A 为不可约阵.

易知, A 是可约矩阵, 当且仅当存在一个下标的非空子集 $J \subset \{1, 2, \dots, n\}$, 使 $a_{kj} = 0, k \in J, j \notin J$.

例 6.22 矩阵 $B = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix}$ 是可约矩阵, 即存在排列阵 $P = I_{13}$ 使 $P^T B P =$

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}. \text{ 等价地, 可以取 } J = \{1, 2\}.$$

定理 6.22 (对角占优定理) 如果 $A = (a_{ij})_n \in \mathbf{R}^{n \times n}$ (或 $\mathbf{C}^{n \times n}$) 为严格对角优势阵或为不可约弱对角优势阵, 则 A 是非奇异矩阵.

证 只对 A 为严格对角优势阵证明此定理, 采用反证法. 若 $\det(A) = 0$, 则 $AX=0$ 有非零解, 记 $X = (x_1, x_2, \dots, x_n)^T$, 又记 $|x_k| = \max_{1 \leq i \leq n} |x_i| \neq 0$, 由齐次方程组的第 k 个方程

$$\sum_{j=1}^n a_{kj} x_j = 0,$$

得 $|a_{kk} x_k| = \left| \sum_{\substack{j=1 \\ j \neq k}}^n a_{kj} x_j \right| \leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| |x_j| \leq |x_k| \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|,$

即 $|a_{kk}| \leq \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}|.$

这与假设矛盾,故 $\det(A) \neq 0$, 即 A 非奇异.

定理 6.23 如果 $A \in \mathbb{R}^{n \times n}$ 为严格对角优势矩阵或为不可约弱对角优势矩阵,则对任意的 $X^{(0)}$, 解方程组 $AX=b$ 的 Jacobi 迭代法, Gauss-Seidel 迭代法均收敛.

证 设 A 是严格对角占优阵, 则 Jacobi 迭代收敛(其他情形自己证).

对于 Jacobi 迭代法来说, 迭代矩阵 $B_J = -D^{-1}(L+U)$ 的 ∞ -范数是

$$\|B_J\|_{\infty} = \max_{1 \leq i \leq n} \sum_{\substack{j=1 \\ j \neq i}}^n \frac{|a_{ij}|}{|a_{ii}|} = \max_{1 \leq i \leq n} \left\{ \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| / |a_{ii}| \right\}.$$

由 A 的严格对角占优的定义知 $\|B_J\|_{\infty} < 1$, 所以, Jacobi 迭代法收敛.

例 6.23 对下列方程组建立收敛的迭代公式:

$$\begin{cases} 11x_1 - 3x_2 - 33x_3 = 1, \\ -22x_1 + 11x_2 + x_3 = 0, \\ x_1 - 4x_2 + 2x_3 = 1. \end{cases}$$

解 上述方程组经行交换后得

$$\begin{cases} -22x_1 + 11x_2 + x_3 = 0, \\ x_1 - 4x_2 + 2x_3 = 1, \\ 11x_1 - 3x_2 - 33x_3 = 1. \end{cases}$$

这时, 方程组的系数矩阵是严格对角占优的, 故对此时的方程组建立的 Jacobi 迭代法或 Gauss-Seidel 迭代法是收敛的.

6.6.3 逐次超松弛迭代法(SOR 方法)

使用迭代法的困难之处是计算量难以估计, 有时迭代过程虽然收敛, 但由于收敛速度缓慢, 使计算量变得很大而失去使用价值, 因此, 迭代过程的加速具有重要的意义.

所谓松弛法, 实质上是 Gauss-Seidel 迭代的一种加速方法, 这种方法将前一步的结果 $x_i^{(k)}$ 与 Gauss-Seidel 方法的迭代值 $x_i^{(k+1)}$ 适当加权平均, 期望获得更好的近似值 $x_i^{(k+1)}$.

为了加快迭代的收敛速度, 将 Gauss-Seidel 迭代式改写为

$$x_i^{(k+1)} = x_i^{(k)} + \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, 3, \dots, n.$$

并记
$$r_i^{(k+1)} = b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)}, \quad i = 1, 2, \dots, n.$$

其中, $r_i^{(k+1)}$ 称为 $k+1$ 步第 i 个分量的残量, 当迭代式收敛时, 所有残量 $r_i^{(k+1)} \rightarrow 0$ ($i=1, 2, \dots, n$), Gauss-Seidel 迭代法的第 $(k+1)$ 步, 相当于在第 k 步的基础上每个分量增加一个修正量 $\frac{1}{a_{ii}} r_i^{(k+1)}$, 为了获得更快的收敛效果, 在修正量前乘以一个参数 ω , 即

$$x_i^{(k+1)} = x_i^{(k)} + \frac{\omega}{a_{ii}} (b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)}), \quad i = 1, 2, \dots, n, \quad (6.48)$$

适当选取 ω 的值, 可望比 Gauss-Seidel 迭代收敛更快 ($\omega=1$, 就是 Gauss-Seidel 迭代), 这就是逐次超松弛迭代法, 简称为 SOR 方法. ω 称为松弛因子 (当 $\omega < 1$ 时, 称迭代 (6.44) 为低松弛法, 当 $\omega > 1$ 时, 称式 (6.44) 为超松弛法, 不管是超松弛还是低松弛 ($\omega < 1$, 或 $\omega > 1$), 一概都称为超松弛法).

式 (6.48) 常以如下形式进行计算.

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}} (b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)}), \quad i = 1, 2, \dots, n, \quad (6.49)$$

或

$$\begin{cases} \tilde{x}_i^{(k+1)} = \frac{1}{a_{ii}} (b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)}), \\ x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \omega\tilde{x}_i^{(k+1)}, \quad i = 1, 2, \dots, n. \end{cases}$$

迭代式 (6.49) 的矩阵形式为

$$X^{(k+1)} = (1-\omega)X^{(k)} + \omega D^{-1}(b - LX^{(k+1)} - UX^{(k)}),$$

或

$$X^{(k+1)} = (D + \omega L)^{-1}[(1-\omega)D - \omega U]X^{(k)} + (D + \omega L)^{-1}\omega b.$$

所以, 逐次超松弛迭代法的迭代阵是

$$L = (D + \omega L)^{-1}[(1-\omega)D - \omega U],$$

$$F = \omega(D + \omega L)^{-1}b.$$

由定理 6.20 知, SOR 方法收敛的充要条件是

$$\rho(L_\omega) < 1.$$

定理 6.24 SOR 迭代法收敛的必要条件是

$$0 < \omega < 2.$$

证 由 L_ω 的特征多项式根与系数的关系知

$$\prod_{i=1}^n \lambda_i = \det L_\omega = \det(D + \omega L)^{-1} \cdot \det[(1-\omega)D - \omega U] = (1-\omega)^n,$$

所以

$$\rho(L_\omega) \geq |1-\omega|.$$

若逐次超松弛迭代法收敛, 则

$$|1-\omega| \leq \rho(L_\omega) < 1,$$

即

$$0 < \omega < 2.$$

下面仍从 A 的特点, 给出收敛的判别条件.

定理 6.25 设有线性代数方程组 $AX=b$, 若

- (1) A 为对称正定阵, 则当 $0 < \omega < 2$ 时, 逐次超松弛迭代法收敛;
- (2) A 为严格对角占优阵, 则当 $0 < \omega \leq 1$ 时逐次超松弛迭代收敛.

如何选取 ω 使收敛速度最快? 这自然是逐次超松弛法讨论的重要内容. 目前, 只有少数特殊类型的矩阵, 才有确定的最佳松弛因子的理论公式, 但实际使用时也有一定困难. 通常的办法是选不同的 ω 进行试算, 以确定最佳 ω 的近似值, 或者先取一个 ω ($0 < \omega < 2$), 然后根据迭代过程收敛的快慢, 不断修改 ω , 这样逐步寻找最佳 ω , 直到满意后再固定下来继续迭代, 以达到加速的目的.

例 6.24 分别用 G-S 迭代法, SOR 迭代法 ($\omega=1.25$) 解方程组

$$\begin{cases} 4x_1 + 3x_2 = 24, \\ 3x_1 + 4x_2 + x_3 = 20, \\ -x_2 + 4x_3 = -24. \end{cases}$$

解 取初始向量为 $(1, 1, 1)^T$, 两种迭代计算公式分别为

$$\begin{cases} x_1^{(k+1)} = -0.75x_2^{(k)} + 6, \\ x_2^{(k+1)} = -0.75x_1^{(k+1)} - 0.25x_3^{(k)} + 5, \\ x_3^{(k+1)} = 0.25x_2^{(k+1)} - 6; \end{cases}$$

$$\begin{cases} x_1^{(k+1)} = -0.25x_1^{(k)} - 0.9375x_2^{(k)} + 7.5, \\ x_2^{(k+1)} = -0.9375x_1^{(k+1)} - 0.25x_2^{(k)} - 0.3125x_3^{(k)} + 6.25, \\ x_3^{(k+1)} = 0.3125x_2^{(k+1)} - 0.25x_3^{(k)} - 7.5. \end{cases}$$

本题的准确解是 $(3, 4, -5)^T$, 取 $X^{(0)} = (1, 1, 1)^T$, G-S 迭代 $X^{(7)} = (3.0747070, 3.9501953, -5.0124512)^T$, 而 SOR 方法 ($\omega=1.25$) 7 次迭代后 $X^{(7)} = (2.9998422, 3.9992730, -5.0006739)^T$.

可见本题 SOR 方法 ($\omega=1.25$) 要比 G-S 方法收敛快, 事实上, 继续计算下去, 要达到 7 位小数的精确值, G-S 要迭代 27 次, 而 SOR 方法 ($\omega=1.25$) 只要用 15 次迭代.

定理 6.26 设 A 是正定对称的三对角矩阵, 则 $\rho(B_G) = [\rho(B_I)]^2 < 1$, 且 SOR 方法中松弛因子 ω 的最优选择为

$$\omega_{\text{opt}} = 2 / (1 + \sqrt{1 - [\rho(B_I)]^2}),$$

这时有 $\rho(L_{\omega_{\text{opt}}}) = \omega_{\text{opt}} - 1$.

例 6.25 设 $A = \begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix}$, 不难验证 A 是正定对称的三对角矩阵. 由

$$B_I = -D^{-1}(L+U) = \begin{bmatrix} 0 & -0.75 & 0 \\ -0.75 & 0 & 0.25 \\ 0 & 0.25 & 0 \end{bmatrix},$$

$$\det(\lambda I - B_J) = \lambda(\lambda^2 - 0.625),$$

得到 $\rho(B_J) = \sqrt{0.625} \approx 0.791$, 据定理(6.26), 有 $\rho(B_G) = 0.625$, SOR 方法最优的松弛因子为

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - 0.625}} \approx 1.24,$$

而 $\rho(L_{\omega_{\text{opt}}}) \approx 0.24$, 可见采用最优松弛因子的 SOR 方法收敛速度比 Jacobi 方法和 G-S 方法快得多.

习 题 6

1. 设 $A = \begin{bmatrix} 0.6 & 0.5 \\ 0.1 & 0.3 \end{bmatrix}$, 计算 A 的行范数、列范数、2-范数及 F -范数.

2. 证明:

$$(1) \|X\|_{\infty} \leq \|X\|_1 \leq n \|X\|_{\infty},$$

$$(2) \frac{1}{\sqrt{n}} \|A\|_F \leq \|A\|_2 \leq \|A\|_F.$$

3. 设 $P \in \mathbb{R}^{n \times n}$, 且为非奇异矩阵, 又设 $\|X\|$ 为 $\mathbb{R}^n$ 上一向量范数. 定义

$$\|X\|_P = \|PX\|.$$

证明: $\|X\|_P$ 是 $\mathbb{R}^n$ 上向量的一种范数.

4. 证明: 对任何矩阵算子范数有 $\|I\| = 1$ (其中 I 是单位矩阵), $\|A\| \|A^{-1}\| \geq 1$.

5. 设 $\|A\|_i, \|A\|_j$ 为任意两种 $\mathbb{R}^{n \times n}$ 上矩阵算子范数, 证明存在常数 $c_1, c_2 > 0$, 使

$$c_1 \|A\|_i \leq \|A\|_j \leq c_2 \|A\|_i, \quad (\text{对一切 } A \in \mathbb{R}^{n \times n}).$$

6. 用 Gauss 消去法解方程组

$$\begin{cases} 2x_1 + x_2 + x_3 = 4, \\ 3x_1 + x_2 + 2x_3 = 6, \\ x_1 + 2x_2 + 2x_3 = 5. \end{cases}$$

7. 设方程组

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 1, \\ 5x_1 + 4x_2 + 10x_3 = 0, \\ 3x_1 - 0.1x_2 + x_3 = 2, \end{cases}$$

试用 Gauss 列主元消去法求解.

8. (1) 设 A 是对称矩阵且 $a_{11} \neq 0$, 经过 Gauss 消去法一步后, A 约化为

$$\begin{bmatrix} a_{11} & \alpha_1^T \\ 0 & A_2 \end{bmatrix},$$

证明 A_2 是对称矩阵.

(2) 用 Gauss 消去法解对称方程组

$$\begin{cases} 0.6428x_1 + 0.3475x_2 - 0.8468x_3 = 0.4127, \\ 0.3475x_1 + 1.8423x_2 + 0.4759x_3 = 1.7321, \\ -0.8468x_1 + 0.4759x_2 + 1.2147x_3 = -0.86. \end{cases}$$

9. 设 A 为 n 阶非奇异矩阵且有分解式 $A=LU$, 其中 L 为单位下三角阵, U 为上三角阵, 证明 A 的所有顺序主子式均不为零.

10. 设 A 为 n 阶矩阵, 若 $|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ ($i=1, 2, \dots, n$), 则称 A 为对角优势矩阵.

证明: 设 A 是对角优势矩阵, 又设经过 Gauss 消去法一步后, A 具有形式 $\begin{bmatrix} a_{11} & \alpha_1^T \\ 0 & A_2 \end{bmatrix}$,

则 A_2 是对角优势矩阵, 且由此推断, 对于对称的对角优势矩阵, 用 Gauss 消去法和部分(列)主元 Gauss 消去法可得到同样的结论.

11. 用平方根法(Cholesky 分解)解方程组

$$\begin{bmatrix} 3 & 2 & 3 \\ 2 & 2 & 0 \\ 3 & 0 & 12 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \\ 7 \end{bmatrix}.$$

12. 用追赶法解三对角方程组 $AX=b$, 其中

$$A = \begin{bmatrix} -2 & 1 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & 1 & -2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \end{bmatrix}.$$

13. (1) 如果 A 是对角优势矩阵, 即 $|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$ ($i=1, 2, \dots, n$), 证明: $a_{ii} \neq 0$ ($i=1, 2, \dots, n$).

(2) 设 A 为对角优势矩阵, 使 $A=DB$, 其中 $D=\text{diag}(a_{ii})$, 证明: $B=I-C$, 其中 $\|C\|_{\infty} < 1$, 因此由定理(6.7), A 是非奇异阵.

(3) 证明: 如果应用 Gauss 消去法解对角优势方程组, 则所有元素 $a_{kk}^{(k)} \neq 0$.

14. 设 $A = \begin{bmatrix} 100 & 99 \\ 99 & 98 \end{bmatrix}$, 计算 A 的条件数 $\text{cond}(A)_p$ ($p=2, \infty$).

15. 设 $A, B \in \mathbb{R}^{n \times n}$, 且 $\|\cdot\|$ 为 $\mathbb{R}^{n \times n}$ 上矩阵的算子范数, 证明:

$$\text{cond}(AB) \leq \text{cond}(A) \cdot \text{cond}(B).$$

16. 已知 Hilbert 矩阵

$$H_3 = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}.$$

(1) 计算 H_3 的条件数 $\text{cond}(H)_\infty$.

(2) 解方程 $H_3 X = \left(\frac{11}{6}, \frac{13}{12}, \frac{47}{60}\right)^T = b$ 时, 若 H_3 及 b 有微小误差(取 3 位有效数字), 估计解 X 的误差 $\frac{\|\delta X\|_\infty}{\|X\|_\infty}$.

17. 设方程组

$$\begin{cases} 5x_1 + 2x_2 + x_3 = -12, \\ -x_1 + 4x_2 + 2x_3 = 20, \\ 2x_1 + 3x_2 + 10x_3 = 3. \end{cases}$$

(1) 考察用 Jacobi 迭代法、G-S 迭代法解此方程组的收敛性.

(2) 用 Jacobi 迭代法及 G-S 迭代法解此方程组, 要求当 $\|X^{(k+1)} - X^{(k)}\|_\infty < 10^{-4}$ 时迭代终止.

18. 设方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 = b_1, \\ a_{21}x_1 + a_{22}x_2 = b_2, \end{cases} \quad (a_{11}, a_{22} \neq 0)$$

的迭代公式为

$$\begin{cases} x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)}), \\ x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)}), \quad k = 0, 1, 2, \dots, n. \end{cases}$$

证明: 由上述迭代公式产生的向量序列 $\{X^{(k)}\}$ 收敛的充要条件是 $r = \left| \frac{a_{12}a_{21}}{a_{11}a_{22}} \right| < 1$.

19. 设有下列方程组

$$(1) \begin{cases} x_1 + 0.4x_2 + 0.4x_3 = 1, \\ 0.4x_1 + x_2 + 0.8x_3 = 2, \\ 0.4x_1 + 0.8x_2 + x_3 = 3; \end{cases} \quad (2) \begin{cases} x_1 + 2x_2 - 2x_3 = 1, \\ x_1 + x_2 + x_3 = 1, \\ 2x_1 + 2x_2 + x_3 = 1. \end{cases}$$

试考察解此方程的 Jacobi 迭代法及 G-S 迭代法的收敛性.

20. 设 $X = BX + f$, 其中 $B = \begin{bmatrix} 0.9 & 0 \\ 0.3 & 0.8 \end{bmatrix}$, $f = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, 证明: 虽然 $\|B\| > 1$, 但迭代法

$X^{(k+1)} = BX^{(k)} + f$ 仍是收敛的.

21. 设 $AX = b$, 其中 A 对称正定, 问解此方程组的 Jacobi 迭代是否一定收敛? 试考

察第19题(1)方程组.

22. 设 $A = \begin{bmatrix} 10 & a & 0 \\ b & 10 & b \\ 0 & a & 5 \end{bmatrix}$, $\det A \neq 0$, 用 a, b 表示解方程组 $AX = f$ 的 Jacobi 迭代法

及 G-S 迭代法收敛的充分必要条件.

23. 用 SOR 方法解方程组(分别取松弛因子 $\omega=1.03, \omega=1, \omega=1.1$)

$$\begin{cases} x_1 + x_2 = 1, \\ -x_1 - 4x_2 - x_3 = 4, \\ -x_2 + 4x_3 = -3. \end{cases}$$

精确解 $X^* = \left(\frac{1}{2}, 1, -\frac{1}{2}\right)^T$, 要求当 $\|X^* - X^{(k)}\|_\infty < 5 \times 10^{-6}$ 时迭代终止, 并且对每一个 ω 值确定迭代次数.

24. 设有方程组 $AX=b$, 其中 A 为对称正定阵, 且有迭代公式

$$X^{(k+1)} = X^{(k)} + \omega(b - AX^{(k)}), \quad k = 0, 1, 2, \dots$$

讨论使迭代序列收敛的 ω 的取值范围.

25. 设

$$A = \begin{bmatrix} 5 & 1 & 2 & 3 \\ 0 & 2 & 0 & 4 \\ 3 & -1 & 2 & -1 \\ 0 & 3 & 0 & 7 \end{bmatrix}.$$

证明: A 为可约矩阵.

26. 给定迭代过程 $X^{(k+1)} = CX^{(k)} + g$, 其中 $C \in \mathbb{R}^{n \times n}$ ($k=0, 1, 2, \dots$), 证明: 如果 C 的特征值 $\lambda_i(C) = 0$ ($i=1, 2, \dots, n$), 则此迭代过程最多迭代 n 次收敛于方程组的解.

27. 求证 $\lim_{k \rightarrow \infty} A_k = A$ 的充要条件是对任何的向量 X 都有 $\lim_{k \rightarrow \infty} A_k X = AX$.

28. 设 $B \in \mathbb{R}^{n \times n}$, $S^{(k)} = I + B + \dots + B^k$. 证明: 若 $\rho(B) < 1$, 则 $(I-B)$ 非奇异, 且 $\{S^{(k)}\}$ 收敛, $\lim_{k \rightarrow \infty} S^{(k)} = (I-B)^{-1}$; 反之, 若 $\{S^{(k)}\}$ 收敛, 则 $\rho(B) < 1$.

数值实验题 6

实验 6.1 主元的选取与算法的稳定性

问题提出: Gauss 消去法是我们在线性代数中已经熟悉的. 但由于计算机的数值运算是在一个有限的浮点数集合上进行的, 如何才能确保 Gauss 消去法作为数值算法的

稳定性呢? Gauss 消去法从理论算法到数值算法,其关键是主元的选择. 主元的选择从数学理论上看起来平凡,它却是数值分析中十分典型的问题.

实验内容:考虑线性方程组

$$AX = b \quad (A \in \mathbb{R}^{n \times n}, b \in \mathbb{R}^n),$$

编制一个能自动选取主元,又能手动选取主元的求解线性代数方程组的 Gauss 消去过程.

实验要求:

$$(1) \text{ 取矩阵 } A = \begin{pmatrix} 6 & 1 & & & \\ 8 & 6 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 8 & 6 & 1 \\ & & & 8 & 6 \end{pmatrix}, \quad b = \begin{pmatrix} 7 \\ 15 \\ \vdots \\ 15 \\ 14 \end{pmatrix}, \text{ 则方程有解 } X^* = (1, 1, \dots, 1)^T.$$

取 $n=10$ 计算矩阵的条件数. 让程序自动选取主元,结果如何?

(2) 现选择程序中手动选取主元的功能. 每步消去过程总选取按模最小或按模尽可能小的元素作为主元,观察并记录计算结果. 若每步消去过程总选取按模最大的元素作为主元,结果又如何? 分析实验的结果.

(3) 取矩阵阶数 $n=20$ 或者更大,重复上述实验过程,观察记录并分析不同的问题及消去过程中选择不同的主元时计算结果的差异,说明主元素的选取在消去过程中的作用.

(4) 选取其他你感兴趣的问题或者随机生成矩阵,计算其条件数. 重复上述实验,观察记录并分析实验的结果.

实验 6.2 线性代数方程组的性态与条件数的估计

问题提出:理论上,线性代数方程组 $AX=b$ 的摄动满足

$$\frac{\|\Delta X\|}{\|X\|} \leq \frac{\text{cond}(A)}{1 - \|A^{-1}\| \|\Delta A\|} \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right)$$

矩阵的条件数确实是对矩阵病态性的刻画,但在实际应用中直接计算它显然不现实,因为计算 $\|A^{-1}\|$ 通常要比求解方程 $AX=b$ 还困难.

实验内容:Matlab 中提供有函数“condest”可以用来估计矩阵的条件数,它给出的是按 1—范数的条件数. 首先构造非奇异矩阵 A 和右端,使得方程组是可以精确求解的. 再人为地引进系数矩阵和右端的扰动 ΔA 和 Δb ,使得 $\|\Delta A\|$ 和 $\|\Delta b\|$ 充分小.

实验要求:

(1) 假设方程 $AX=b$ 的解为 X ,求解方程 $(A+\Delta A)\hat{X}=b+\Delta b$,以 1—范数,给出 $\frac{\|\Delta X\|}{\|X\|} = \frac{\|\hat{X}-X\|}{\|X\|}$ 的计算结果.

(2) 选择一系列维数递增的矩阵(可以是随机生成的), 比较函数“condest”所需机器时间的差别. 考虑若干逆是已知的矩阵, 借助函数“eig”很容易给出 $\text{cond}_2(A)$ 的数值. 将它与函数“cond(A, 2)”所得到的结果比较.

(3) 利用“condest”给出矩阵 A 条件数的估计, 针对实验要求(1)中的结果给出 $\frac{\|\Delta X\|}{\|X\|}$ 的理论估计, 并将它与实验要求(1)给出的计算结果进行比较, 分析所得结果. 注意, 如果给出了 $\text{cond}(A)$ 和 $\|A\|$ 的估计, 马上就可以给出 $\|A^{-1}\|$ 的估计.

(4) 估计著名的 Hilbert 矩阵的条件数

$$H = (H_{i,j})_{n \times n}, \quad h_{i,j} = \frac{1}{i+j-1}, \quad i, j = 1, 2, \dots, n.$$

实验 6.3 病态的线性方程组的求解

问题提出: 理论的分析表明, 求解病态的线性方程组是困难的. 实际情况是否如此, 会出现怎样的现象呢?

实验内容: 考虑方程组 $Hx=b$ 的求解, 其中系数矩阵 H 为 Hilbert 矩阵,

$$H = (H_{i,j})_{n \times n}, \quad h_{i,j} = \frac{1}{i+j-1}, \quad i, j = 1, 2, \dots, n.$$

这是一个著名的病态问题. 通过首先给定解(例如取为各个分量均为 1)再计算出右端的办法给出确定的问题.

实验要求:

(1) 选择问题的维数为 6, 分别用 Gauss 消去法(即 LU 分解)、 J 迭代方法、G-S 迭代和 SOR 迭代求解方程组, 其各自的结果如何? 将计算结果与问题的解比较, 结论如何.

(2) 逐步增大问题的维数, 仍然用上述的方法来解它们, 计算的结果如何? 计算的结果说明了什么?

(3) 讨论病态问题求解的算法.

第 7 章 非线性方程和方程组的解法

设有非线性方程

$$f(x) = 0, \quad (7.1)$$

其中, $f(x)$ 是一元非线性函数. 若 $f(x)$ 是 n 次代数多项式, 则称方程 (7.1) 为代数方程或 n 次多项式方程; 若 $f(x)$ 是超越函数, 则称方程 (7.1) 为超越方程; 若有常数 x^* , 使 $f(x^*) = 0$, 则称 x^* 为方程 (7.1) 的根, 又称为函数 $f(x)$ 的零点; 若 $f(x)$ 能分解为

$$f(x) = (x - x^*)^m \varphi(x),$$

其中, $\varphi(x^*) \neq 0$, 则称 x^* 是方程 (7.1) 的 m 重根或 $f(x)$ 的 m 重零点, 当 $m=1$ 时, x^* 称为方程 (7.1) 的单根或 $f(x)$ 的单零点.

只有很少类型的非线性方程能解出根的解析表达式. 对于大多数非线性方程, 只能用数值方法求出根的近似值. 本章将要介绍几种常用的有效的数值求根方法, 由于它们都属于迭代法, 因而还要讨论这些方法的收敛性和收敛速度.

7.1 二分法

在方程 (7.1) 中, 设 $f(x)$ 为 $[a, b]$ 上的连续函数, 且 $f(a)f(b) < 0$, 由连续函数性质知, 方程 (7.1) 在 $[a, b]$ 内至少有一实根, 为简单起见, 设有唯一实根 x^* , 可用二分法来求实根 x^* 的近似值.

二分法的基本思想就是逐步将含有根 x^* 的区间二分, 通过判别函数值的符号, 进一步探索有根区间, 将有根区间缩小到充分小, 从而求出满足给定精度的根 x^* 的近似值. 如图 7.1 所示, 其具体做法如下: 记 $a_1 = a, b_1 = b$, 先将 $[a_1, b_1]$ 二分, 计算中点 $x_1 = \frac{a_1 + b_1}{2}$ 及 $f(x_1)$, 如果 $f(x_1) = 0$, 则 $x_1 = x^*$; 否则, 如果 $f(x_1)f(a_1) < 0$, 则根一定在区

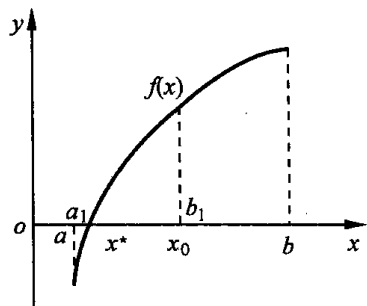


图 7.1

间 $[a, x_1] = [a_2, b_2]$ 内, 这样, 就得到长度缩小一半的有根区间 $[a_2, b_2]$, 即

$$f(a_2)f(b_2) < 0, \text{ 且 } b_2 - a_2 = \frac{1}{2}(b - a).$$

对有根区间 $[a_2, b_2]$ 施行同样手续, 即二分求中点, 判别函数值符号, 则可得到长度又缩小一半的有根区间 $[a_3, b_3]$.

重复上述过程,到第 k 步就得到根 x^* 的近似值序列 $\{x_k\}$ 及包含根 x^* 的区间套,且有

$$(1) [a_1, b_1] \supset [a_2, b_2] \supset \cdots \supset [a_k, b_k] \supset \cdots;$$

$$(2) f(a_k)f(b_k) < 0, \quad x^* \in [a_k, b_k];$$

$$(3) b_k - a_k = \frac{1}{2^{k-1}}(b - a);$$

$$(4) x_k = \frac{a_k + b_k}{2}, \text{ 且 } |x^* - x_k| \leq \frac{b - a}{2^k}, k = 1, 2, \dots.$$

由(4)显然有 $\lim_{k \rightarrow \infty} x_k = x^*$, 且 x_k 以等比数列的收敛速度收敛于 x^* , 因此, 可用二分法求 $f(x) = 0$ 的实根 x^* 达到任意指定精度. 事实上, 对于 $\varepsilon > 0$ (为给定精度要求), 由

$$|x_k - x^*| \leq \frac{b - a}{2^k} < \varepsilon,$$

得

$$2^{-k}(b - a) < \varepsilon \text{ 或 } 2^{-k} < \frac{\varepsilon}{b - a},$$

两边取对数, 即得

$$k > \frac{\ln(b - a) - \ln \varepsilon}{\ln 2},$$

这样就得到区间二分次数 k .

二分法方法可靠、程序简单, 且对函数 $f(x)$ 性质要求不高, 只要连续即可, 但要求得到较精确的结果所费的时间长, 一般不单独使用, 求方程根的主要方法是迭代法.

注 方程 $f(x) = 0$ 的根往往不止一个, 这里所讲的求近似根, 是指求某一特定的根, 因而事先要给出所求根的大体范围, 确定根的范围, 称为根的隔离. 除代数方程有较完整的理论外, 一般用试探办法大体划分出根的范围, 然后再用近似求根法加以精确化.

7.2 简单迭代法

7.2.1 迭代法的基本思想

通常, 用迭代法求 $f(x) = 0$ 的近似根时, 需要将它转化为等价方程

$$x = \varphi(x). \quad (7.2)$$

由于方程(7.2)是隐式方程, 其右端含有未知的 x , 因而不能直接求解, 如果给出根的某个猜测值 x_0 , 将它代入方程(7.2)的右端, 即转化为显式的计算公式

$$x_1 = \varphi(x_0),$$

再取 x_1 作为新的猜测值, 又有

$$x_2 = \varphi(x_1),$$

如此反复计算,迭代公式为

$$x_{k+1} = \varphi(x_k), \quad k = 0, 1, 2, \dots, \quad (7.3)$$

如果迭代值 x_k 有极限,则称迭代收敛;若 $\varphi(x)$ 是连续函数,这时极限值 $x^* = \lim_{k \rightarrow \infty} x_k$ 显然就是方程(7.2)的根.

称式(7.3)为求解方程 $x = \varphi(x)$ 的简单迭代法,其中 $\varphi(x)$ 为迭代函数,因 $x^* = \varphi(x^*)$,故 x^* 是迭代函数的不动点.简单迭代法(7.3)又称为不动点迭代法.

我们看到,迭代法的基本思想是,将隐式方程 $x = \varphi(x)$ 的求根问题归结为计算一组显式 $x_{k+1} = \varphi(x_k)$,也就是说,迭代过程实质上是一个逐步显式化的过程.

例 7.1 设 $f(x) = x^3 + 4x^2 - 10 = 0$ (此方程在 $[1, 2]$ 中有唯一根),用不同的方法将它变换成等价的方程.

解 (1) $x = \varphi_1(x) = x - x^3 - 4x^2 + 10$;

$$(2) \quad x = \varphi_2(x) = \left(\frac{10}{x} - 4x\right)^{\frac{1}{2}};$$

$$(3) \quad x = \varphi_3(x) = \frac{1}{2}(10 - x^3)^{\frac{1}{2}};$$

$$(4) \quad x = \varphi_4(x) = \left(\frac{10}{4+x}\right)^{\frac{1}{2}};$$

$$(5) \quad x = \varphi_5(x) = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}.$$

对所选取的 $\varphi_i(x)$ ($i=1, 2, 3, 4, 5$) 取初始近似值 $x_0 = 1.5$,迭代法计算结果列入表 7.1 中.不难看出,表中迭代过程(3),(4),(5)都得到较好的结果.若用二分法达到同样精确度,则需迭代 27 次.同时,由迭代过程(1)中出现发散和迭代过程(2)中出现负数开方,可以看出迭代函数 $\varphi(x)$ 选得不同,相应的 $\{x_k\}$ 的收敛情况也不一样.

表 7.1

k	(1)	(2)	(3)	(4)	(5)
0	1.5	1.5	1.5	1.5	1.5
1	-0.875	0.8165	1.28695377	1.34839972	1.37333333
2	6.732	2.9969	1.40254080	1.36737637	1.36526201
3	-469.7	$(-8.65)^{1/2}$	1.34545837	1.36495702	1.36523001
4	1.03×10^8		1.37517025	1.36526475	
5			1.36009419	1.36522559	
6			1.36784697	1.36523058	
7			1.36388700	1.36522994	
8			1.36591673	1.36523002	
9			1.36487822	1.36523001	
10			1.36541006		
11			1.36513782		
12			1.36527721		

续表

k	(1)	(2)	(3)	(4)	(5)
13			1.36520585		
14			1.36524238		
15			1.36522368		
16			1.36523326		
17			1.36522835		
18			1.36523086		
19			1.36522958		
20			1.36523024		
21			1.36522990		
22			1.36523007		
23			1.36522998		
24			1.36523003		
25			1.36523001		
26			1.36523002		
27			1.36523001		

综上所述,用迭代法求方程 $f(x)=0$ 的近似解,需要讨论如下问题.

- (1) 如何选取合适的迭代函数 $\varphi(x)$?
- (2) 迭代函数 $\varphi(x)$ 应满足什么条件,序列 $\{x_k\}$ 收敛($x_{k+1}=\varphi(x_k)$)?
- (3) 怎样加速序列 $\{x_k\}$ 的收敛?

下面将讨论这些问题.

7.2.2 迭代过程的收敛性

为使迭代法有效,必须保证它的收敛性,一个发散(即不收敛)的迭代过程,纵使进行千百次迭代,其结果也是毫无价值的.

考察图 7.2 对研究迭代法的收敛性是有启发的.

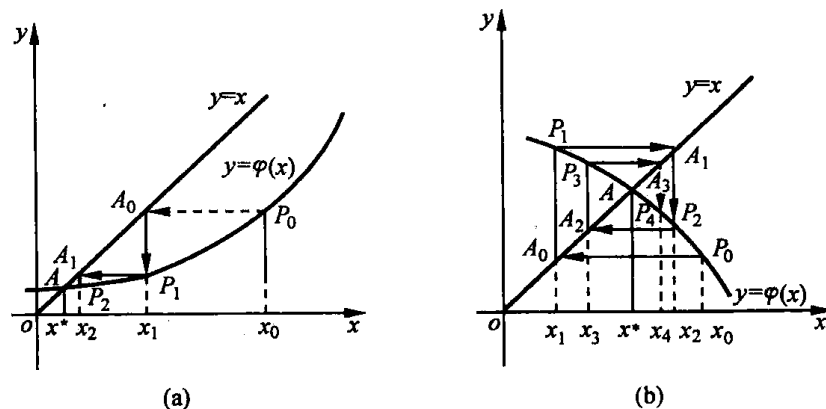
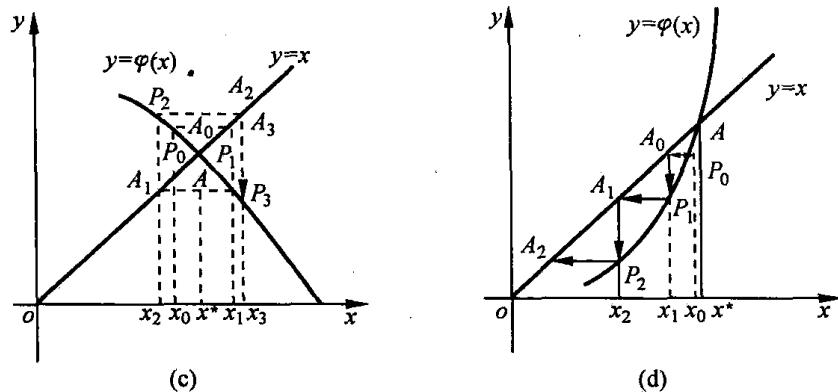


图 7.2



续图 7.2

由以上几何图形可以明显地看出,这时保证迭代 $x_{k+1} = \varphi(x_k)$ 收敛的条件是 $|\varphi'(x)| = |k| < 1$.

进而研究一般情形.

定理 7.1 设 $\varphi(x)$ 在 $[a, b]$ 上满足

(1) 对任意 $x \in [a, b]$, 总有 $\varphi(x) \in [a, b]$;

(2) 存在 $0 \leq L < 1$, 使对任意 $x \in [a, b]$, $|\varphi'(x)| \leq L$ 成立, 则 $x = \varphi(x)$ 在 (a, b) 上有唯一解 x^* , 且迭代过程 $x_{k+1} = \varphi(x_k)$ 对任意初值 $x_0 \in [a, b]$ 均收敛于方程 $x = \varphi(x)$ 的根, 其误差估计式为

$$|x^* - x_k| \leq \frac{1}{1-L} |x_{k+1} - x_k|; \quad (7.4)$$

$$|x^* - x_k| \leq \frac{L^k}{1-L} |x_1 - x_0|. \quad (7.5)$$

证 先证 x^* 的存在唯一性. 由于 $\varphi'(x)$ 在 $[a, b]$ 上存在, 故 $\varphi(x)$ 在 $[a, b]$ 上连续, 作 $f(x) = x - \varphi(x)$, 显然 $f(x)$ 在 $[a, b]$ 上连续, 由条件(1)知, $f(a) = a - \varphi(a) \leq 0$, $f(b) = b - \varphi(b) \geq 0$, 故存在 $x^* \in [a, b]$ 使 $f(x^*) = 0$, 即 $x^* = \varphi(x^*)$.

若设另有 $\bar{x}^*$ 使 $\bar{x}^* = \varphi(\bar{x}^*)$, 则

$$\begin{aligned} |x^* - \bar{x}^*| &= |\varphi(x^*) - \varphi(\bar{x}^*)| = |\varphi'(\xi_1)(x^* - \bar{x}^*)| \\ &\leq L |x^* - \bar{x}^*|, \quad (\xi_1 \text{ 介于 } x^* \text{ 与 } \bar{x}^* \text{ 之间}), \end{aligned}$$

即 $L \geq 1$, 与条件(2)矛盾, 故 $x^* - \bar{x}^* = 0$, 即 $x^* = \bar{x}^*$.

再来证收敛性, 根据条件(2), 有

$$\begin{aligned} |x_{k+1} - x^*| &= |\varphi(x_k) - \varphi(x^*)| = |\varphi'(\xi_2)(x_k - x^*)| \\ &\leq L |x_k - x^*|, \quad (\xi_2 \text{ 介于 } x_k \text{ 与 } x^* \text{ 之间}). \end{aligned}$$

反复利用上式, 得

$$|x_k - x^*| \leq L^k |x_0 - x^*|,$$

当 $k \rightarrow \infty$ 时, $L^k \rightarrow 0$, 故 $|x_k - x^*| \rightarrow 0$, 即

$$\lim_{k \rightarrow \infty} x_{k+1} = x^*.$$

再证误差估计式. 由

$$|x_{k+1} - x_k| \leq L |x_k - x_{k-1}| \leq \cdots \leq L^k |x_1 - x_0|,$$

于是, 对于固定的 k 和任意正整数 p , 有

$$\begin{aligned} |x_{k+p} - x_k| &\leq |x_{k+p} - x_{k+p-1}| + |x_{k+p-1} - x_{k+p-2}| + \cdots + |x_{k+1} - x_k| \\ &\leq (L^{k+p-1} + L^{k+p-2} + \cdots + L^k) |x_1 - x_0| \\ &= L^k (L^{p-1} + L^{p-2} + \cdots + L + 1) |x_1 - x_0|, \end{aligned}$$

而当 $p \rightarrow \infty$ 时, $(L^{p-1} + L^{p-2} + \cdots + L + 1) \rightarrow \frac{1}{1-L}$, $\lim_{p \rightarrow \infty} x_{k+p} = x^*$, 所以

$$|x^* - x_k| \leq \frac{L^k}{1-L} |x_1 - x_0|,$$

对任意正整数 p , 有

$$|x_{k+p} - x_k| \leq (L^{p-1} + L^{p-2} + \cdots + 1) |x_{k+1} - x_k| \leq \frac{1}{1-L} |x_{k+1} - x_k|,$$

在上式中令 $p \rightarrow \infty$, 知

$$|x^* - x_k| \leq \frac{1}{1-L} |x_{k+1} - x_k|.$$

式(7.4)表明, 在正常收敛的情况下, 即 L 不太接近于 1, 只要相邻两次迭代值 x_k , x_{k+1} 的偏差充分小, 就能保证迭代 x_k (或 x_{k+1}) 足够准确, 因此可用 $|x_{k+1} - x_k|$ 来控制迭代过程应否结束. 从证明中显见当 $|\varphi'(x)| > 1$ 时, 迭代发散.

例 7.2 求方程 $f(x) = x^3 - x - 1 = 0$ 在 $x = 1.5$ 附近的根.

解 若将方程改写为 $x = x^3 - 1$, 建立迭代公式

$$x_{k+1} = x_k^3 - 1$$

是发散的, 这是因为

$$\varphi'(x) = (x^3 - 1)' = 3x^2.$$

当 $x > 0.7$ 时, 均有

$$|\varphi'(x)| > 1.$$

例 7.3 对方程 $f(x) = e^x - x - 2 = 0$, 用迭代法 $x_{k+1} = e^{x_k} - 2$, $k = 0, 1, 2, \cdots$ 求根是否收敛? 若不收敛, 试构造收敛的迭代公式.

解 因 $f(1) = e - 3 < 0$, $f(2) = e^2 - 4 > 0$, 而 $f(x) \in C[1, 2]$, 故在 $(1, 2)$ 上存在方程 $f(x) = 0$ 的根. 迭代函数 $\varphi(x) = e^x - 2$, $\varphi'(x) = e^x$, 当 $x \in [1, 2]$ 时 $|\varphi'(x)| = e^x > 1$, 故迭代公式

$$x_{k+1} = e^{x_k} - 2, \quad k = 0, 1, 2, \cdots,$$

产生的序列 $\{x_k\}$ 发散.

但若将原方程化为 $x = \ln(2 + x)$, 这时迭代函数取为 $\psi(x) = \ln(2 + x)$, 而 $\psi'(x) =$

$\frac{1}{2+x}$, 则当 $x \in [1, 2]$ 时, 有 $|\psi'(x)| = \frac{1}{2+x} < 1$, 故迭代公式

$$x_{k+1} = \ln(2+x_k), \quad k = 0, 1, 2, \dots,$$

产生的序列 $\{x_k\}$ 是收敛的.

例 7.4 考察例 7.1 中迭代过程(3)和(4)的收敛性, 当 $|x_k - x^*| < 10^{-5}$ 时, 确定(4)中迭代次数 k .

解 对于迭代过程(4), 迭代函数 $\varphi_4(x) = (\frac{10}{4+x})^{\frac{1}{2}}$, 于是

$$|\varphi'_4(x)| = \left| \frac{-5}{\sqrt{10}(4+x)^{3/2}} \right| \leq \frac{5}{\sqrt{10}(5)^{3/2}} < 0.15.$$

因此, 迭代函数 $\varphi_4(x)$ 在 $[1, 2]$ 上满足定理条件, 故迭代过程(4)收敛. 由

$$|x^* - x_k| \leq \frac{L^k}{1-L} |x_1 - x_0| < \varepsilon \quad (\varepsilon \text{ 为给定精度要求}),$$

可知迭代次数 k 应取
$$k > \frac{\lg \varepsilon - \lg \frac{|x_1 - x_0|}{1-L}}{\lg L}.$$

由要求 $|x_k - x^*| < 10^{-5}$, 应用上述公式, 其中 $x_1 = 1.3439972$, $L_4 = 0.15$, $x_0 = 1.5$, 则有

$$k > \frac{\lg 10^{-5} - \lg \frac{1.5 - 1.3439972}{0.85}}{\lg 0.15} = 6.97.$$

于是, 推得所要求迭代次数 $k=7$.

对于迭代过程(3), 迭代函数 $\varphi_3(x) = \frac{1}{2}(10-x^3)^{1/2}$, 于是

$$\varphi'_3(x) = -\frac{3}{4} \frac{x^2}{\sqrt{10-x^3}} < 0, \quad x \in [1, 1.5],$$

且有

$$|\varphi'_3(x)| \leq |\varphi'_3(1.5)| < 0.66 = L_3.$$

同时, 注意到 $|\varphi'_3(2)| \approx 2.12$, 所以在 $[1, 2]$ 上, 定理 7.1 中条件(2)不满足. 而迭代函数 $\varphi_3(x)$ 在 $[1, 1.5]$ 上满足定理 7.1 的条件, 故迭代过程当初值限制在 $[1, 1.5]$ 上时, 迭代过程收敛.

注 此题中 $L_4 < L_3$, 可知迭代过程(4)比迭代过程(3)收敛快.

7.2.3 局部收敛性与收敛阶

在应用中通常已知根的存在, 希望用迭代法求出足够精确的近似值, 因此, 在实际使用迭代法时, 总是在根 x^* 邻近范围内考虑, 同时, 定理中的条件 $|\varphi'(x)| \leq L < 1$, 对于较大范围的含根区间可能不满足, 而在根的邻近是成立的, 即局部收敛性.

定义 7.1 如果存在 x^* 的某个邻域 $R: |x-x^*| \leq \delta$, 使迭代过程 $x_{k+1} = \varphi(x_k)$ 对于任意初值 $x_0 \in R$ 均收敛, 则称迭代过程 $x_{k+1} = \varphi(x_k)$ 在根 x^* 邻近具有局部收敛性.

定理 7.2 设 x^* 为方程 $x = \varphi(x)$ 的根, $\varphi'(x)$ 在 x^* 的邻近连续, 且 $|\varphi'(x^*)| < 1$, 则迭代过程 $x_{k+1} = \varphi(x_k)$ 在 x^* 邻近具有局部收敛性.

例 7.5 设已知方程 $x = e^{-x}$ 在 $x = 0.5$ 附近有一个根, 考察用迭代法 $x_{k+1} = e^{-x_k}$ 解此方程的收敛性, 并求出在 $x = 0.5$ 附近的一个根.

解 易知方程在 $[0.5, 0.6]$ 上有一个根, 现考察迭代过程 $x_{k+1} = e^{-x_k} = \varphi(x_k)$ 的收敛性, 显然

$$|\varphi'(x)| = e^{-x} \leq e^{-0.5} \approx 0.61 < 1, \quad x \in [0.5, 0.6],$$

故由定理 7.2 知, 当取 $x_0 = 0.5$ 时, 迭代法收敛; 计算得 $x_{18} = 0.56714$, 得到的精度为 10^{-5} .

一种迭代法具有实用价值, 不但需要肯定它收敛, 还要求它收敛得比较快, 所谓迭代过程的收敛速度, 是指在接近收敛时迭代误差的下降速度. 定理 7.1 中若 $L \ll 1$, 则收敛快; 若 L 接近 1, 则收敛很慢. 下面介绍收敛阶的概念, 它是判断迭代公式优劣的一个重要标准.

定义 7.2 设 $\{x_k\}$ 为收敛于 x^* 的序列, 记迭代误差 $e_k = x_k - x^*$, 若存在实数 $p \geq 1$ 及正常数 c , 使

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^p} = c \neq 0$$

成立, 则称序列 $\{x_k\}$ 是 p 阶收敛的. 当 $p=1$ 且 $0 < c < 1$ 时, 称为线性收敛; $p > 1$ 时, 称为超线性收敛; 当 $p=2$ 时, 称为平方收敛; p 越大收敛越快.

定理 7.3 给定方程 $x = \varphi(x)$, 若满足

- (1) x^* 为 $x = \varphi(x)$ 的根, 迭代过程为 $x_{k+1} = \varphi(x_k)$;
- (2) $\varphi(x)$ 在 x^* 的某个邻域内 p 次连续可微 ($p \geq 2$);
- (3) $\varphi'(x^*) = \varphi''(x^*) = \cdots = \varphi^{(p-1)}(x^*) = 0$, 而 $\varphi^{(p)}(x^*) \neq 0$,

则当初值 x_0 充分接近 x^* 时, 迭代过程 $x_{k+1} = \varphi(x_k)$ 是 p 阶收敛的, 且

$$\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k^p} = \frac{\varphi^{(p)}(x^*)}{p!} \neq 0.$$

证 由于 $\varphi'(x^*) = 0$, 据定理 7.2 立即可以断定迭代过程 $x_{k+1} = \varphi(x_k)$ 具有局部收敛性.

再将 $\varphi(x_k)$ 在根 x^* 处做泰勒展开, 利用定理 7.3 条件(3), 则有

$$\varphi(x_k) = \varphi(x^*) + \frac{\varphi^{(p)}(\xi)}{p!} (x_k - x^*)^p \quad (\xi \text{ 在 } x_k \text{ 与 } x^* \text{ 之间}).$$

注意到 $\varphi(x_k) = x_{k+1}$, $\varphi(x^*) = x^*$, 由上式得

$$x_{k+1} - x^* = \frac{\varphi^{(p)}(\xi)}{p!} (x_k - x^*)^p,$$

因此迭代误差,当 $k \rightarrow \infty$ 时有

$$\frac{e_{k+1}}{e_k^p} \rightarrow \frac{\varphi^{(p)}(x^*)}{p!},$$

这表明迭代过程 $x_{k+1} = \varphi(x_k)$ 确实为 p 阶收敛. 证毕.

上述定理告诉我们,迭代过程的收敛速度依赖于迭代函数 $\varphi(x)$ 的选取. 如果当 $x \in [a, b]$ 时 $\varphi'(x) \neq 0$, 则该迭代过程只可能是线性收敛.

例 7.6 用简单迭代法求方程

$$x^3 - x - 1 = 0$$

的正实根 x^* .

解 算法一 将方程改写成

$$x = \sqrt[3]{x+1},$$

得迭代函数 $\varphi(x) = \sqrt[3]{x+1}$. 因 $\varphi'(x) = \frac{1}{3(x+1)^{2/3}}$ 在 x^* 的某邻域内连续, 且 $|\varphi'(x^*)| < 1$, 但 $\varphi'(x^*) \neq 0$, 所以, 当 x_0 充分接近 x^* 时, 迭代公式

$$x_{k+1} = \sqrt[3]{x_k + 1}, \quad k = 0, 1, 2, \dots,$$

产生的序列 $\{x_k\}$ 收敛于 x^* , 但只有线性收敛速度.

算法二 将原方程改写成

$$x = \frac{2x^3 + 1}{3x^2 - 1},$$

得迭代函数

$$\varphi(x) = \frac{2x^3 + 1}{3x^2 - 1},$$

这时,

$$\varphi'(x) = \frac{6x(x^3 - x - 1)}{(3x^2 - 1)^2}$$

在 x^* 的某邻域内连续, 且 $\varphi'(x^*) = 0$, 所以, 只要 x_0 充分接近 x^* , 由迭代公式

$$x_{k+1} = \frac{2x_k^3 + 1}{3x_k^2 - 1}, \quad k = 0, 1, 2, \dots,$$

产生的序列 $\{x_k\}$ 以至少是二阶的收敛速度收敛于 x^* .

例 7.7 用迭代法 $x_{k+1} = \varphi(x_k)$ 求函数 $f(x)$ 的单重零点, 其中 $\varphi(x) = x - r_1(x)f(x) - r_2(x)f^2(x)$, 确定未知函数 $r_1(x)$ 、 $r_2(x)$, 使这个迭代法至少是三阶局部收敛的.

解 $\varphi'(x) = 1 - r_1'(x)f(x) - r_1(x)f'(x) - r_2'(x)f^2(x) - 2r_2(x)f(x)f'(x)$, 由 $\varphi'(x^*) = 0$ 和 $f(x^*) = 0, f'(x^*) \neq 0$, 得

$$1 - r_1(x^*)f'(x^*) = 0,$$

故取

$$r_1(x) = \frac{1}{f'(x)}.$$

这时, $\varphi'(x) = -r'_1(x)f(x) - r'_2(x)f^2(x) - 2r_2(x)f(x)f'(x),$

$$\begin{aligned} \varphi''(x) = & -r''_1(x)f(x) - r'_1(x)f'(x) - r''_2(x)f^2(x) - 4r'_2(x)f(x)f'(x) \\ & - 2r_2(x)[f'(x)]^2 - 2r_2(x)f(x)f''(x), \end{aligned}$$

其中

$$r'_1(x) = -\frac{f''(x)}{[f'(x)]^2},$$

从而由 $\varphi''(x^*)=0$ 和 $f(x^*)=0, f'(x^*) \neq 0$, 得

$$\frac{f''(x^*)}{f'(x^*)} - 2r_2(x^*)[f'(x^*)]^2 = 0,$$

故取

$$r_2(x) = \frac{f''(x)}{2[f'(x)]^3},$$

则迭代公式

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} - \frac{f^2(x_k)[f''(x_k)]}{2[f'(x_k)]^3}$$

至少是三阶局部收敛的.

7.3 迭代过程的加速

对于收敛的迭代过程, 只要迭代足够多次, 就可以使结果达到任意的精度, 但有时迭代过程收敛缓慢, 从而使计算量变得很大, 因此迭代过程的加速是个重要课题.

7.3.1 Aitken(埃特金)算法

设 $\{x_k\}$ 是一个线性收敛序列, 其极限为 x^* , 误差 $e_k = x^* - x_k$, 即

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|} = c, \quad (0 < c < 1).$$

因此, 当 k 充分大时有

$$\frac{x^* - x_{k+2}}{x^* - x_{k+1}} \approx \frac{x^* - x_{k+1}}{x^* - x_k},$$

由此得

$$x^* \approx x_{k+2} - \frac{(x_{k+2} - x_{k+1})^2}{x_{k+2} - 2x_{k+1} + x_k} = x_k - \frac{(x_{k+1} - x_k)^2}{x_k - 2x_{k+1} + x_{k+2}}.$$

以上式右端得出的结果作为新的改进值, 记

$$\bar{x}_{k+1} = x_k - \frac{(x_{k+1} - x_k)^2}{x_k - 2x_{k+1} + x_{k+2}} = x_k - (\Delta x_k)^2 / \Delta^2 x_k, \quad (7.6)$$

可望得到的新序列 $\{\bar{x}_k\}$ 较原序列 $\{x_k\}$ 更快的收敛到 x^* .

式(7.6)称为 Aitken Δ^2 加速方法.

可以证明 $\lim_{k \rightarrow \infty} \frac{\bar{x}_k - x^*}{x_k - x^*} = 0$, 它表明序列 $\{\bar{x}_k\}$ 的收敛速度比 $\{x_k\}$ 的收敛速度快.

7.3.2 Steffensen(斯蒂芬森)迭代法

埃特金方法不管原序列 $\{x_k\}$ 是怎样产生的, 对 $\{x_k\}$ 进行加速计算, 得到序列 $\{\bar{x}_k\}$, 如果把埃特金加速技巧与不动点迭代结合, 则可得到如下的迭代法:

$$\begin{aligned} y_k &= \varphi(x_k), \quad z_k = \varphi(y_k), \\ x_{k+1} &= x_k - \frac{(y_k - x_k)^2}{z_k - 2y_k + x_k}, \quad k = 0, 1, 2, \dots, \end{aligned} \quad (7.7)$$

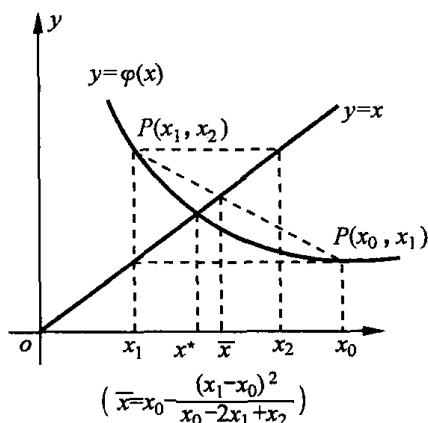


图 7.3

称为 Steffensen(斯蒂芬森)迭代法. 其几何解释如图 7.3 所示, 它可以这样理解, 我们要求 $x = \varphi(x)$ 的根 x^* , 令 $\varepsilon(x) = \varphi(x) - x$, $\varepsilon(x^*) = \varphi(x^*) - x^* = 0$, 已知 x^* 的近似值 x_k 及 y_k , 其误差分别为

$$\varepsilon(x_k) = \varphi(x_k) - x_k = y_k - x_k,$$

$$\varepsilon(y_k) = \varphi(y_k) - y_k = z_k - y_k,$$

把误差 $\varepsilon(x)$ “外推到零”, 即过 $(x_k, \varepsilon(x_k))$ 及 $(y_k, \varepsilon(y_k))$ 两点做线性插值函数, 它与 x 轴的交点就是式(7.7)中的 x_{k+1} , 即方程

$$\varepsilon(x_k) + \frac{\varepsilon(y_k) - \varepsilon(x_k)}{y_k - x_k}(x - x_k) = 0$$

的解

$$x = x_k - \frac{\varepsilon(x_k)}{\varepsilon(y_k) - \varepsilon(x_k)}(y_k - x_k) = x_k - \frac{(y_k - x_k)^2}{z_k - 2y_k + x_k} = x_{k+1}.$$

实际上式(7.7)是将不动点迭代法(7.3)计算两步合并成一步得到的, 可将它写成另一种不动点迭代

$$x_{k+1} = \psi(x_k), \quad k = 0, 1, 2, \dots, \quad (7.8)$$

其中

$$\psi(x) = x - \frac{[\varphi(x) - x]^2}{\varphi(\varphi(x)) - 2\varphi(x) + x}. \quad (7.9)$$

对不动点迭代(7.8)有以下局部收敛性定理.

定理 7.4 若 x^* 为式(7.9)定义的迭代函数 $\psi(x)$ 的不动点, 则 x^* 为 $\varphi(x)$ 的不动点; 反之, 若 x^* 为 $\varphi(x)$ 的不动点, 设 $\varphi''(x)$ 存在, $\varphi'(x^*) \neq 1$, 则 x^* 是 $\psi(x)$ 的不动点, 且 Steffensen 迭代法(7.7)是 2 阶收敛的.

例 7.8 用 Steffensen 迭代法求解方程 $x^3 - x - 1 = 0$.

解 例 7.2 中已指出迭代 $x_{k+1} = x_k^3 - 1$ 是发散的, 现用式(7.7)计算, 取 $\varphi(x) = x^3 - 1$, 计算结果列入表 7.2 中.

表 7.2

k	x_k	y_k	z_k
0	1.5	2.37500	12.3965
1	1.41629	1.84092	5.23887
2	1.35565	1.49140	2.31727
3	1.32895	1.34706	1.44435
4	1.32480	1.32517	1.32712
5	1.32472		

计算表明它是收敛的,这说明即使迭代法(7.3)不收敛,用 Steffensen 迭代法(7.7)仍可能收敛.至于原来已收敛的迭代法(7.3),由定理 7.4 可知它可达到 2 阶收敛.更进一步还可知若迭代法(7.3)为 p 阶收敛,则迭代法(7.7)为 $(p+1)$ 阶收敛.

例 7.9 求方程 $3x^2 - e^x = 0$ 在 $[3, 4]$ 中的解.

解 由方程得 $e^x = 3x^2$, 取对数得

$$x = \ln 3x^2 = 2\ln x + \ln 3 = \varphi(x).$$

若构造迭代法

$$x_{k+1} = 2\ln x_k + \ln 3,$$

由于 $\varphi'(x) = \frac{2}{x}$, $|\max_{3 \leq x \leq 4} \varphi'(x)| \leq \frac{2}{3} < 1$, 且当 $x \in [3, 4]$ 时, $\varphi(x) \in [3, 4]$, 根据定理 7.1

此迭代法是收敛的;若取 $x_0 = 3.5$ 迭代 18 次得 $x_{18} = 3.73308$, 有 6 位有效数字.

若用 Steffensen 迭代法(7.7)进行加速,计算结果列入表 7.3 中.

表 7.3

k	x_k	y_k	z_k
0	3.5	3.60414	3.66278
1	3.73835	3.73590	3.73459
2	3.73308		

这里计算 2 步(相当于(7.3)迭代 4 步)结果与 x_{18} 相同,说明用迭代法(7.7)的收敛速度比迭代法(7.3)快得多.

7.4 Newton 迭代法

解非线性方程 $f(x) = 0$ 的牛顿法,是一种将非线性方程线性化方法.它是解代数方程和超越方程的有效方法之一.牛顿方法在单根附近具有较高的收敛速度,而且牛顿方法不仅可以用来求 $f(x) = 0$ 的实根,还可用来求代数方程的复根,同时还可推广用

来解非线性方程组.

7.4.1 Newton 法及其收敛性

用简单迭代法求方程(7.1)的根 x^* , 十分重要的问题是构造迭代函数 $\varphi(x)$. 为了使收敛速度的阶高一些, 由定理 7.3 知, 应尽可能使 $\varphi(x)$ 在 $x=x^*$ 处有直到更高阶导数等于 0.

现在令 $\varphi(x)=x+h(x)f(x)$, $h(x)$ 为待定函数, 但 $h(x^*)\neq 0$, 则方程(7.1)与方程 $x=x+h(x)f(x)$ 有共同的根 x^* . 今用条件 $\varphi'(x^*)=0$ 确定 $h(x)$, 由

$$\begin{aligned}\varphi'(x^*) &= 1 + h'(x^*)f(x^*) + h(x^*)f'(x^*) \\ &= 1 + h(x^*)f'(x^*) = 0\end{aligned}$$

知, $h(x)$ 必需满足 $h(x^*) = \frac{-1}{f'(x^*)}$, 显然, 取 $h(x) = \frac{-1}{f'(x)}$ 就具备这个条件, 并且也满足 $h(x^*)\neq 0$, 于是确定

$$\varphi(x) = x - f(x)/f'(x),$$

它满足 $\varphi'(x^*)=0$, 由此得出下面的特殊的简单迭代法

$$x_{k+1} = x_k - f(x_k)/f'(x_k), \quad k = 0, 1, 2, \dots, \quad (7.10)$$

式(7.10)所表示的迭代法称为 Newton 法. 如果 Newton 法(7.10)收敛于方程(7.1)的根, 则它至少有二阶的收敛速度, Newton 法可求方程(7.1)的实数根和复数根.

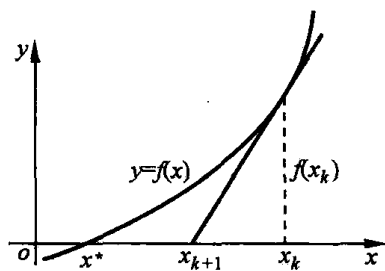


图 7.4

当求实数根时, Newton 法有明显的几何意义. 当获得 x_k 之后, 过曲线 $y=f(x)$ 上的点 $(x_k, f(x_k))$ 作该曲线的切线, 此切线与 x 轴相交所得的交点的横坐标就是 Newton 法迭代序列的第 $(k+1)$ 个元素 x_{k+1} , 如图 7.4 所示. 事实上, 该切线的方程为

$$y = f'(x_k)(x - x_k) + f(x_k),$$

令 $y=0$, 就得

$$x = x_k - \frac{f(x_k)}{f'(x_k)} = x_{k+1}.$$

因而 Newton 法(7.10)又称为切线法.

定理 7.5 给定 $f(x)=0$, 若满足条件

- (1) $f(x)$ 在根 x^* 邻近二阶连续可微;
- (2) $f'(x^*)\neq 0$,

则存在 x^* 的一个邻域 $R: \{x \mid |x-x^*| \leq \delta\}$ 使对于任意初值 $x_0 \in R$, Newton 迭代序列 $\{x_k\}$ 收敛于 x^* , 并有

$$\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k^2} = \frac{f''(x^*)}{2f'(x^*)}.$$

其中 $e_k = x_k - x^*$ 为迭代误差, 从而 Newton 法至少二阶收敛.

证 称 $\varphi(x) = x - f(x)/f'(x)$ 为 Newton 迭代函数. 当 $f'(x^*) \neq 0$ 时, 显然有 $x^* = \varphi(x^*)$, 即 x^* 是 $\varphi(x)$ 的不动点. 进而容易求出

$$\varphi'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}.$$

因为 $f'(x^*) \neq 0$, 所以 $\varphi'(x^*) = 0$. 于是, 由定理 7.3 得知 Newton 迭代法 (7.10) 至少具有二阶局部收敛性. 进而, 在点 x_k 处作 Taylor 展开

$$0 = f(x^*) = f(x_k) + f'(x_k)(x^* - x_k) + \frac{f''(\xi_k)}{2}(x^* - x_k)^2,$$

其中, ξ_k 在 x_k 与 x^* 之间. Newton 迭代法 (7.10) 等价于

$$f(x_k) - f'(x_k)x_k = -f'(x_k)x_{k+1},$$

将它代入上式, 得到

$$0 = f'(x_k)(x^* - x_{k+1}) + \frac{f''(\xi_k)}{2}(x^* - x_k)^2,$$

即

$$-\frac{e_{k+1}}{e_k^2} = \frac{f''(\xi_k)}{2f'(x_k)}.$$

令 $k \rightarrow \infty$, 由局部收敛性得知 $x_k \rightarrow x^*$, 同时 $\xi_k \rightarrow x^*$, 于是有

$$\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k^2} = \frac{f''(x^*)}{2f'(x^*)}.$$

例 7.10 用 Newton 法解方程 $xe^x - 1 = 0$.

解 将方程 $xe^x - 1 = 0$ 化为 $x - e^{-x} = 0$, 则牛顿公式为

$$x_{k+1} = x_k - \frac{x_k - e^{-x_k}}{1 + x_k},$$

取迭代值 $x_0 = 0.5$, 迭代结果列于表 7.4 中.

由例 7.10 知, 若用不动点迭代到同一精度要迭代 17 次, 可见 Newton 法的收敛速度是很快的.

例 7.11 用 Newton 迭代法求 $\sqrt{2}$ 的近似值.

解 $\sqrt{2}$ 可视为 $x^2 - 2 = 0$ 的正根, 这里 $f(x) = x^2 - 2$. $f'(x) = 2x$, 则 Newton 迭代公式为

$$x_{k+1} = x_k - \frac{x_k^2 - 2}{2x_k} = \frac{1}{2}x_k + \frac{1}{x_k}, \quad k = 0, 1, 2.$$

取初始值 $x_0 = 1$, 则有

$$x_1 = 1.5000000000, \quad x_2 = 1.416666667,$$

$$x_3 = 1.414215686, \quad x_4 = 1.414213562.$$

x_4 具有 10 位有效数字, 故取

$$\sqrt{2} \approx 1.414213562.$$

表 7.4

k	x_k
0	0.5
1	0.57102
2	0.56716
3	0.56714

7.4.2 Newton 法的改进

Newton 法收敛很快(对单根),算法简单,是迭代法中较好者,但缺点是:对重根收敛较慢;初值 x_0 不能偏离 x^* 太大,否则可能不收敛;需要计算导数值.针对这几改进如下.

1. Newton 下山法

由 Newton 迭代法的收敛性定理知(局部收敛性),Newton 迭代法对初值 x_0 的要求是很苛刻的,在实际应用中,往往很难给出较好的初值 x_0 . Newton 下山法就是在事先没有给出较好的初值情况下,求 $f(x)=0$ 根的一种修正的 Newton 法.

为此,将 Newton 法的计算结果 $\bar{x}_{k+1}=x_k-\frac{f(x_k)}{f'(x_k)}$ 与前一步的近似值 x_k 做适当加权平均作为新的近似值

$$\begin{aligned} x_{k+1} &= \lambda \bar{x}_{k+1} + (1-\lambda)x_k = \lambda x_k - \lambda \frac{f(x_k)}{f'(x_k)} + (1-\lambda)x_k \\ &= x_k - \lambda \frac{f(x_k)}{f'(x_k)}, \quad k=0,1,2,\dots \end{aligned} \quad (7.11)$$

上式即是在 Newton 迭代公式中引入了因子 λ ,对 Newton 公式做了修改,引入因子的作用是,适当调整 λ 可使计算过程中得到的数列 $\{|f(x_k)|\}$ 是单调递减的,这样就容易保证迭代的收敛性.

其中 x_0 为初值, λ 称为下山因子(一个可选择的参数),且满足 $0 < \epsilon_\lambda \leq \lambda \leq 1$, ϵ_λ 为下山因子下界.

选择因子 λ (即选取 $\lambda=1, \frac{1}{2}, \frac{1}{2^2}, \dots, \lambda \geq \epsilon_\lambda$)使

$$|f(x_{k+1})| < |f(x_k)|,$$

称为“下山”,将下山法和 Newton 法结合起来使用的方法,称为 Newton 下山法.

下面给出 Newton 下山法的算法.

计算开始前先给出根的绝对误差限 ϵ ,下山因子下界 ϵ_λ , $|f(x)|$ 的下界 ϵ_f ,适当的数 δ 及最大迭代次数 N .

- (1) 给出初值 x_0 ;
- (2) 令 $\lambda=1$;
- (3) 利用式(7.11)由 x_k 计算 x_{k+1} 并计算 $f(x_{k+1})$;
- (4) 若 $k > N$ 转(9),否则执行(5);
- (5) 若 $|f(x_{k+1})| < |f(x_k)|$ 执行(6),否则执行(7);
- (6) 若 $|x_{k+1} - x_k| < \epsilon$,迭代结束,取 $x^* \approx x_{k+1}$,否则, k 增加 1,转(3);

- (7) 若 $\lambda > \varepsilon_\lambda$, 则以 $\frac{1}{2}\lambda$ 代替 λ , k 增加 1, 转(3), 否则转(8);
- (8) 若 $|f(x_{k+1})| < \varepsilon_f$, 则 $x^* \approx x_k$, 迭代结束, 否则以 $(x_{k+1} + \delta)$ 代替 x_k , k 增加 1, 转(2);
- (9) 已迭代 N 次仍未达到要求, 下山失败, 应另选初值 x_0 .

例 7.12 求方程 $x^3 - x - 1 = 0$ 在 $x = 1.5$ 附近的一个根 x^* .

解 (1) 用 Newton 法计算.

$$\text{Newton 公式 } x_{k+1} = x_k - \frac{x_k^3 - x_k - 1}{3x_k^2 - 1},$$

显然, 当取初值 $x_0 = 1.5$ 时, 计算结果列于表 7.5 中.

表 7.5

k	0	1	2	3
x_k	1.5	1.34783	1.32520	1.32472

表中 x_3 的每一位数字均为有效数字.

当取初值 $x_0 = 0.6$ 时, 按 Newton 公式计算有 $x_1 = 17.9, x_2 = 11.9, \dots$ 影响收敛速度. $x_1 = 17.9$, 这个结果反而比 x_0 更偏离了所求的根 x^* .

(2) 用 Newton 下山法计算.

按 Newton 公式求得的迭代值 $\bar{x}_1 = 17.9$, 设取下山因子 $\lambda = \frac{1}{32}$, 由式(7.11)可求得 $x_1 = \frac{1}{32}\bar{x}_1 + \frac{31}{32}x_0 = 1.140625$ 这个结果纠正了 $\bar{x}_1$ 的严重偏差.

2. m 重根的处理

在上一段的讨论中, 都是假定了在方程 $f(x) = 0$ 的根 x^* 的某邻域内 $f'(x) \neq 0$, 即 x^* 是方程 $f(x) = 0$ 的单根. 这一段要讨论方程 $f(x) = 0$ 有多重根时 Newton 法的收敛情况.

设 x^* 是方程 $f(x) = 0$ 的 m 重根 ($m \geq 2$), $f(x)$ 在 x^* 的某邻域内有 m 阶连续导数, 这时

$$f(x^*) = f'(x^*) = \dots = f^{(m-1)}(x^*) = 0, \quad f^{(m)}(x^*) \neq 0,$$

由 Taylor 公式, 得

$$f(x) = \frac{f^{(m)}(\xi_1)}{m!}(x - x^*)^m, \quad f'(x) = \frac{f^{(m)}(\xi_2)}{(m-1)!}(x - x^*)^{m-1},$$

$$f''(x) = \frac{f^{(m)}(\xi_3)}{(m-2)!}(x - x^*)^{m-2},$$

其中, ξ_1, ξ_2, ξ_3 都在 x 与 x^* 之间. 由 Newton 法的迭代函数 $\varphi(x) = x - f(x)/f'(x)$, 可得

$$\begin{aligned}\varphi'(x^*) &= \lim_{x \rightarrow x^*} \varphi'(x) = \lim_{x \rightarrow x^*} \frac{f(x)f''(x)}{[f'(x)]^2} \\ &= \lim_{x \rightarrow x^*} \frac{(m-1)f^{(m)}(\xi_1)f^{(m)}(\xi_3)}{m[f^{(m)}(\xi_2)]^2} = 1 - \frac{1}{m}.\end{aligned}$$

由此可见, 方程 $f(x)=0$ 的 m 重根 x^* 仍然是其等价方程 $x=\varphi(x)$ 的根, 而且由于 $0 < \varphi'(x^*) < 1$, 所以, 只要 x_0 充分靠近 x^* , 由 Newton 法产生的序列 $\{x_k\}$ 仍收敛于 x^* , 但是只有线性的收敛速度.

若把迭代函数修改为

$$\tilde{\varphi}(x) = x - \frac{mf(x)}{f'(x)},$$

则有

$$\begin{aligned}\tilde{\varphi}(x^*) &= \lim_{x \rightarrow x^*} \tilde{\varphi}(x) = \lim_{x \rightarrow x^*} \left[x - \frac{(x-x^*)f^{(m)}(\xi_1)}{f^{(m)}(\xi_2)} \right] = x^*, \\ \tilde{\varphi}'(x^*) &= \lim_{x \rightarrow x^*} \tilde{\varphi}'(x) = \lim_{x \rightarrow x^*} \left[1 - m + \frac{mf(x)f''(x)}{[f'(x)]^2} \right] \\ &= 1 - m + m\left(1 - \frac{1}{m}\right) = 0.\end{aligned}$$

这两个等式说明, 当 x^* 是方程 $f(x)=0$ 的 m 重根时, 变形的 Newton 法

$$x_{k+1} = x_k - \frac{mf(x_k)}{f'(x_k)}, \quad k = 0, 1, 2, \dots, \quad (7.12)$$

不仅可以收敛于 x^* , 而且仍具有 2 阶的收敛速度.

在重根的情况下, 一般重数 m 是不知道的, 因此, 使用式(7.12)就有困难, 为此, 考虑函数

$$u(x) = f(x)/f'(x),$$

因为 x^* 是 $f(x)$ 的 m 重零点, 故 x^* 是 $u(x)$ 的单零点. 求解方程 $u(x)=0$ 的 Newton 法迭代函数为

$$g(x) = x - \frac{u(x)}{u'(x)} = x - \frac{f(x)f'(x)}{[f'(x)]^2 - f(x)f''(x)},$$

于是, 迭代公式

$$x_{k+1} = x_k - \frac{f(x_k)f'(x_k)}{[f'(x_k)]^2 - f(x_k)f''(x_k)}, \quad k = 0, 1, 2, \dots, \quad (7.13)$$

产生的序列 $\{x_k\}$ 如果收敛于方程 $f(x)=0$ 的 m 重根 x^* , 就至少是二阶收敛的. 它的缺点是要计算 $f''(x)$, 运算量稍大一些.

例 7.13 已知方程

$$f(x) = x^4 - 1.4x^3 - 0.48x^2 + 1.408x - 0.512 = 0$$

有一个三重根 $x^* = 0.8$, 这里

$$f'(x) = 4x^3 - 4.2x^2 - 0.96x + 1.408,$$

$$f''(x) = 12x^2 - 8.4x - 0.96.$$

如果使用 Newton 法 $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ 进行迭代, 并取初始值 $x_0 = 1$, 则有

$$\begin{aligned} x_1 &= 0.935483871, & x_2 &= 0.891352317, & x_3 &= 0.861384034, \\ x_4 &= 0.841145166, & x_5 &= 0.827531521, & x_6 &= 0.818400202, \\ x_7 &= 0.812287419, & x_8 &= 0.808200849, & x_9 &= 0.805471359, \\ x_{10} &= 0.803649413. \end{aligned}$$

可以看出, 越往后收敛越慢. 如果使用迭代公式(7.13), 也取初值 $x_0 = 1$, 则有

$$x_1 = 0.794019934, \quad x_2 = 0.799993333, \quad x_3 = 0.799997493,$$

收敛得非常快.

3. 简化的 Newton 法

Newton 法突出的优点是收敛速度快(在单根附近), 但它需要提供导数值 $f'(x)$, 如果函数 $f(x)$ 求导有困难, 那么使用 Newton 法是不方便的.

假设 $f'(x)$ 在所考察的范围内改变不大, 即近似地等于某个定数 M , 有

$$f'(x) \approx M.$$

将 Newton 法中的导数值 $f'(x_k)$ 用定值 M 近似地替代, 即得简化的 Newton 法

$$x_{k+1} = x_k - \frac{f(x_k)}{M}, \quad k = 0, 1, 2, \dots \quad (7.14)$$

在图形上(见图 7.5)简化的 Newton 法用斜率为 M 的一族平行线与 x 轴的交点逼近所求的根, 因此这种方法亦称平行线法.

简化的 Newton 法由于避开了导数求值, 因而其算法简单且使用方便, 但它为此所付出的代价是减缓了收敛速度. 事实上, 由于迭代函数 $\varphi(x) = x - f(x)/M$, 而 $\varphi'(x) = 1 - \frac{f'(x)}{M}$, 故当 $M \neq f'(x^*)$ 时, $\varphi'(x^*) \neq 0$, 可见简化的

Newton 法通常不可能达到二阶, 它甚至可能不收敛.

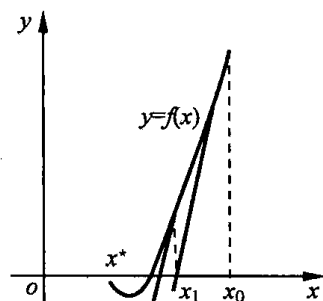


图 7.5

7.5 弦截法与抛物线法

Newton 法是二阶方法, 但每步要计算 $f(x_k)$ 和 $f'(x_k)$, 在很多情况下计算导数值比较困难, 为解决此问题, 上段介绍了简化的 Newton 法, 但我们知道, 简化的 Newton 法可能只具 1 阶收敛性.

7.5.1 弦截法

为避免导数的计算而又适当保持收敛的高速度,改用差商 $\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$ 替换 Newton 法中的导数 $f'(x_k)$ 而考察下列迭代公式

$$x_{k+1} = x_k - \frac{f(x_k)}{f(x_k) - f(x_{k-1})}(x_k - x_{k-1}), \quad k = 1, 2, \dots \quad (7.15)$$

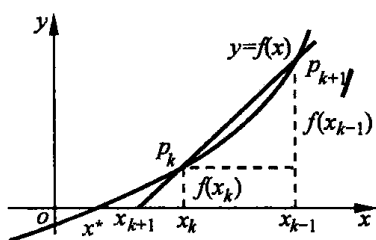


图 7.6

这个公式称为弦截法. 它的几何意义如图 7.6 所示. 弦截法不是用切线, 而是用通过点 $(x_{k-1}, f(x_{k-1}))$ 、 $(x_k, f(x_k))$ 的割线近似 $y=f(x)$, 割线与 x 轴交点的横坐标就作为 x_{k+1} (差商即为弦 $\overline{p_{k-1}p_k}$ 的斜率).

注 显然弦 $\overline{p_{k-1}p_k}$ 就为曲线 $y=f(x)$ 在 x_{k-1}, x_k 处的线性插值函数.

Newton 法和弦截法都是先将 $f(x)$ 线性化, 然后求根, 但线性化的方式不同: 从分析的角度说, 牛顿法是在根 x^* 的邻近点用 Taylor 展开法得出 $f(x)$ 的线性近似函数, 即用 x^* 邻近点处的切线函数作为 $f(x)$ 的近似; 而弦截法是在 x^* 邻近用 $f(x)$ 的一次插值函数作为 $f(x)$ 的近似函数.

两种方法都是线性化方法, 但两者有本质的区别, Newton 法在计算 x_{k+1} 时只用到前一步的值 x_k , 而弦截法需要用两个猜测值 x_{k-1}, x_k , 因此使用这种方法必须先给出两个开始值 x_0, x_1 .

例 7.14 用弦截法解方程 $f(x) = xe^x - 1 = 0$.

解 设取 $x_0 = 0.5, x_1 = 0.6$ 作为开始值, 用弦截法求得的结果见表 7.6, 比较例 7.10 Newton 法的计算结果可以看出, 弦截法的收敛速度也是相当快的.

弦截法的收敛速度为多少呢? 它具有超线性收敛性, 利用函数线性插值的一个性质

$$x_{k+1} - x^* = (x_k - x^*)(x_{k-1} - x^*) \frac{f''(\eta)}{2f'(\xi)}.$$

容易证得下述定理.

定理 7.6 假设 $f(x)$ 在根 x^* 的邻域 $R: |x - x^*| \leq \delta$ 内具有 2 阶连续导数, 且对任意 $x \in R$ 有 $f'(x) \neq 0$, 又初值

$x_0, x_1 \in R$, 那么当邻域 R 充分小时, 弦截法将按阶 $p = \frac{1+\sqrt{5}}{2} \approx 1.618$ 收敛到根 x^* .

表 7.6

k	x_k
0	0.5
1	0.6
2	0.56532
3	0.56709
4	0.56714

7.5.2 Muller(抛物线)法

弦截法是通过两点作直线与 x 轴交点求出 x_{k+1} , 进一步考虑通过 $(x_{k-2}, f(x_{k-2}))$ 、

$(x_{k-1}, f(x_{k-1}))$ 与 $(x_k, f(x_k))$ 三点作抛物线,与 x 轴交点作为 x_{k+1} ,这就是 Muller 方法,又称抛物线法.

通过 $(x_i, f(x_i)) (i=k-2, k-1, k)$ 的二次多项式为

$$p(x) = f(x_k) + (x - x_k)f(x_k, x_{k-1}) \\ + (x - x_k)(x - x_{k-1})f(x_k, x_{k-1}, x_{k-2}),$$

把它改写为

$$p(x) = f(x_k) + \omega(x - x_k) + f(x_k, x_{k-1}, x_{k-2})(x - x_k)^2,$$

其中, $\omega = f(x_k, x_{k-1}) + (x_k - x_{k-1})f(x_k, x_{k-1}, x_{k-2})$,这样 $p(x)=0$ 就是 $(x-x_k)$ 的二次方程,可解出

$$x - x_k = \frac{-\omega \pm [\omega^2 - 4f(x_k)f(x_k, x_{k-1}, x_{k-2})]^{1/2}}{2f(x_k, x_{k-1}, x_{k-2})}.$$

这里有两个解,应选择正负号使 $(x-x_k)$ 尽量小,即以接近 x_k 的 x 为 x_{k+1} ,并且为避免有效数字的消失,可把分子有理化,并把其中正负号选与 ω 同号,则有

$$x_{k+1} = x_k - \frac{2f(x_k)}{\omega + \text{sign}(\omega)[\omega^2 - 4f(x_k)f(x_k, x_{k-1}, x_{k-2})]^{1/2}}. \quad (7.16)$$

可以验证,若式(7.16)产生的序列收敛,必收敛到 f 的零点 x^* ,若 f 在 x^* 邻域有3阶连续可导性且 $f'(x^*) \neq 0$,则有

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1} - x^*|}{|x_k - x^*|^p} = \left| \frac{f'''(x^*)}{6f'(x^*)} \right|^{(p-1)/2}$$

其中, $p=1.839$ 是方程 $\lambda^3 - \lambda^2 - \lambda - 1=0$ 的根.

抛物线法较弦截法收敛得快.

从式(7.16)看到,即使 x_{k-2}, x_{k-1}, x_k 均为实数, x_{k+1} 也可以是复数,所以 Muller 方法可适用多项式方程的实根和复根.

7.6 解非线性方程组的 Newton 迭代法

考虑方程组

$$\begin{cases} f_1(x_1, \dots, x_n) = 0, \\ \dots \\ f_n(x_1, \dots, x_n) = 0, \end{cases} \quad (7.17)$$

其中, $f_1, \dots, f_n$ 均为 $(x_1, \dots, x_n)$ 的多元函数.若用向量记号记 $X=(x_1, \dots, x_n)^T \in \mathbb{R}^n$, $F=(f_1, \dots, f_n)^T$,方程组(7.17)就可写成

$$F(X) = 0. \quad (7.18)$$

当 $n \geq 2$,且 $f_i (i=1, 2, \dots, n)$ 中至少有一个是自变量 $x_i (i=1, 2, \dots, n)$ 的非线性函数时,则称方程组(7.17)为非线性方程组.非线性方程组求根问题是前面介绍的方程(即

$n=1$)求根的直接推广,实际上只要把前面介绍的单变量函数 $f(x)$ 看成向量函数 $F(X)$ 则可将单变量方程求根方法推广到方程组(7.18). 若已给出方程组(7.18)的一个近似根 $X^{(k)} = (X_1^{(k)}, \dots, X_n^{(k)})^T$, 将函数 $F(X)$ 的分量 $f_i(X) (i=1, 2, \dots, n)$ 在 $X^{(k)}$ 用多元函数 Taylor 展开, 并取其线性部分, 则可表示为

$$F(X) \approx F(X^{(k)}) + F'(X^{(k)})(X - X^{(k)}).$$

令上式右端为零, 得到线性方程组

$$F'(X^{(k)})(X - X^{(k)}) = -F(X^{(k)}), \quad (7.19)$$

其中

$$F'(X) = \begin{bmatrix} \frac{\partial f_1(x)}{\partial x_1} & \frac{\partial f_1(x)}{\partial x_2} & \dots & \frac{\partial f_1(x)}{\partial x_n} \\ \frac{\partial f_2(x)}{\partial x_1} & \frac{\partial f_2(x)}{\partial x_2} & \dots & \frac{\partial f_2(x)}{\partial x_n} \\ \vdots & \vdots & & \vdots \\ \frac{\partial f_n(x)}{\partial x_1} & \frac{\partial f_n(x)}{\partial x_2} & \dots & \frac{\partial f_n(x)}{\partial x_n} \end{bmatrix},$$

称为 $F(X)$ 的 Jacobi 矩阵. 求解线性方程组(7.19), 并记解为 $X^{(k+1)}$, 则得

$$X^{(k+1)} = X^{(k)} - F'(X^{(k)})^{-1} F(X^{(k)}), \quad k = 0, 1, 2, \dots. \quad (7.20)$$

这就是解非线性方程(7.18)的 Newton 迭代法.

例 7.15 求解方程组

$$\begin{cases} f_1(x_1, x_2) = x_1 + 2x_2 - 3 = 0, \\ f_2(x_1, x_2) = 2x_1^2 + x_2^2 - 5 = 0, \end{cases}$$

给定初值 $X^{(0)} = (1.5, 1.0)^T$, 用 Newton 法求解.

解 先求 Jacobi 矩阵

$$F'(X) = \begin{pmatrix} 1 & 2 \\ 4x_1 & 2x_2 \end{pmatrix},$$

$$F'(X)^{-1} = \frac{1}{2x_2 - 8x_1} \begin{bmatrix} 2x_2 & -2 \\ -4x_1 & 1 \end{bmatrix}.$$

由 Newton 法(7.20)得

$$X^{(k+1)} = X^{(k)} - \frac{1}{2x_2^{(k)} - 8x_1^{(k)}} \begin{bmatrix} 2x_2^{(k)} & -2 \\ -4x_1^{(k)} & 1 \end{bmatrix} \begin{bmatrix} x_1^{(k)} + 2x_2^{(k)} - 3 \\ 2(x_1^{(k)})^2 + (x_2^{(k)})^2 - 5 \end{bmatrix},$$

即

$$\begin{cases} x_1^{(k+1)} = x_1^{(k)} - \frac{(x_2^{(k)})^2 - 2(x_1^{(k)})^2 + x_1^{(k)}x_2^{(k)} - 3x_2^{(k)} + 5}{x_2^{(k)} - 4x_1^{(k)}}, \\ x_2^{(k+1)} = x_2^{(k)} - \frac{(x_2^{(k)})^2 - 2(x_1^{(k)})^2 - 8x_1^{(k)}x_2^{(k)} + 12x_1^{(k)} - 5}{2x_2^{(k)} - 8x_1^{(k)}}, \end{cases} \quad k=0, 1, 2, \dots.$$

由 $X^{(0)} = (1.5, 1.0)^T$ 逐次迭代得到

$$X^{(1)} = (1.5, 0.75)^T, \quad X^{(2)} = (1.488095, 0.755952)^T,$$

$$X^{(3)} = (1.488034, 0.755983)^T,$$

$X^{(3)}$ 的每一位都是有效数字.

习 题 7

1. 证明: $1 - x - \sin x = 0$ 在 $[0, 1]$ 内有一个根. 使用二分法求误差不大于 $\frac{1}{2} \times 10^{-4}$ 的根要迭代多少次?

2. 比较以下两方法求 $e^x + 10x - 2 = 0$ 的根到 3 位小数所需的计算量.

(1) 在区间 $[0, 1]$ 内用二分法;

(2) 用迭代法 $x_{k+1} = (2 - e^{x_k})/10$, 取初值 $x_0 = 0$.

3. 当 R 取适当值时, 曲线 $y = x^2$ 与 $y^2 + (x - 8)^2 = R^2$ 相切, 试用迭代法求切点横坐标的近似值, 要求不少于 4 位有效数字, 也不求 R .

4. 对 $\varphi(x) = x + x^3$, $x = 0$ 为 $\varphi(x)$ 的一个不动点, 验证用 $x_{k+1} = \varphi(x_k)$ 迭代对 $x_0 \neq 0$ 不收敛, 但改用 Steffensen 方法却是收敛的.

5. 设有解方程 $12 - 3x + 2\cos x = 0$ 的迭代法

$$x_{n+1} = 4 + \frac{2}{3} \cos x_n.$$

(1) 证明: $\forall x_0 \in R$, 均有 $\lim_{n \rightarrow \infty} x_n = x^*$ (x^* 为方程的根).

(2) 取 $x_0 = 4$, 用此迭代法求方程根的近似值, 误差不超过 10^{-3} .

(3) 此迭代法的收敛阶是多少? 证明你的结论.

6. 证明: 迭代公式

$$x_{k+1} = \frac{x_k(x_k^2 + 3a)}{3x_k^2 + a}$$

是计算 $\sqrt[3]{a}$ 的三阶方法; 假定初值 x_0 充分靠近根, 求 $\lim_{k \rightarrow \infty} \frac{\sqrt[3]{a} - x_{k+1}}{(\sqrt[3]{a} - x_k)^3}$.

7. 试确定常数 p, q, r , 使迭代公式

$$x_{k+1} = px_k + q \frac{a}{x_k^2} + r \frac{a^2}{x_k^5}$$

产生的序列 $\{x_k\}$ 收敛到 $\sqrt[3]{a}$, 并使其收敛阶尽可能高.

8. 用简单迭代法(不动点迭代)求方程 $x^2 + 10x - 18 = 0$ 在 $(1, 2)$ 内的根, 取初始值

$x_0 = 1.5$, 要求

- (1) 任意构造迭代格式, 并验证所得格式是收敛的;
 - (2) 用 Aitken 迭代过程加速一次, 以求得更精确的近似值;
 - (3) 计算过程和结果均保留 4 位有效数字.
9. 用 Newton 法求解 Leonardo 方程 $x^3 + 2x^2 + 10x - 20 = 0$, 要求 $|x_{k+1} - x_k| < 10^{-6}$.
10. 应用 Newton 法于方程 $x^3 - a = 0$, 求立方根 $\sqrt[3]{a}$ 的迭代公式, 并讨论其收敛性.
11. 设 x^* 是 $f(x) = 0$ 的三重根, $f(x)$ 在 x^* 的某邻域内有三阶连续导
- (1) 试证明对 $f(x) = 0$ 产生的 Newton 迭代法在 x^* 附近是线性收敛的;
 - (2) 试将 Newton 公式变形, 使之在 x^* 附近具有二阶收敛性并加以证明.
12. 用弦截法求 $f(x) = x^3 + 2x^2 + 10x - 20 = 0$ 的根, 要求 $|x_{k+1} - x_k| < 10^{-6}$.

数值实验题 7

实验 7.1 迭代法、初始值与收敛性

实验目的: 初步认识非线性问题的迭代法与线性问题迭代法的差别, 探讨迭代法及初始值与迭代收敛性的关系.

问题提出: 迭代法是求解非线性方程及方程组的基本思想方法, 与线性方程的情况一样, 其构造方法可以有多种多样, 但关键是怎样才能使迭代收敛且有较快的收敛速度.

实验内容: 考虑一个简单的代数方程

$$x^2 - x - 1 = 0.$$

针对上述方程, 可以构造多种迭代法, 如

$$x_{n+1} = x_n^2 - 1, \quad (\text{E. 7. 1})$$

$$x_{n+1} = 1 + \frac{1}{x_n}, \quad (\text{E. 7. 2})$$

$$x_{n+1} = \sqrt{x_n + 1}. \quad (\text{E. 7. 3})$$

在实轴上取初始值 x_0 , 请读者分别用迭代(E. 7. 1)~(E. 7. 3)做实验, 记录各算法的迭代过程.

实验要求:

(1) 取定某个初始值, 分别计算(E. 7. 1)~(E. 7. 3)迭代结果, 它们的收敛性如何? 重复选取不同的初始值, 反复实验. 请读者自行设计一种比较形象的记录方式(如利用 Matlab 的图形功能), 分析三种迭代法的收敛性与初值选取的关系.

(2) 对三个迭代法中的某个,取不同的初始值进行迭代,结果如何?试分析迭代法对不同的初值是否有差异?

(3) 线性方程组迭代法的收敛性是不依赖初始值选取的.比较线性与非线性问题迭代的差异,有何结论和问题.

实验 7.2 算法设计与比较

问题提出:非线性方程组的数值求解方法很多,基本的思想是线性化.不同的方法效果如何,要靠计算的实践来分析、比较.

实验内容:考虑算法

(1) Newton 法.

(2) 简化 Newton 法.

分别编写它们的 Matlab 程序.

实验要求:

(1) 用上述各种方法,分别计算下面的两个例子.在达到精度相同的前提下,比较其迭代次数、浮点运算次数和 CPU 时间等.

$$\textcircled{1} \begin{cases} 12x_1 - x_2^2 - 4x_3 - 7 = 0, \\ x_1^2 + 10x_2 - x_3 - 11 = 0, \\ x_2^3 + 10x_3 - 8 = 0, \end{cases} \quad \text{取 } \mathbf{x}^{(0)} = (1, 1, 1)^T. \quad (\text{E. 7.4})$$

$$\textcircled{2} \begin{cases} 3x_1 - \cos(x_2 x_3) - \frac{1}{2} = 0, \\ x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06 = 0, \\ e - x_1 x_2 + 20x_3 + \frac{1}{3}(10\pi - 3) = 0, \end{cases} \quad \text{取 } \mathbf{x}^{(0)} = (0, 0, 0)^T. \quad (\text{E. 7.5})$$

(2) 取其他初值 $\mathbf{x}^{(0)}$, 结果又如何? 反复选取不同的初值, 比较其结果.

第 8 章 矩阵特征值与特征向量的计算

设 A 是 $n \times n$ 矩阵, 如果数 λ 和 n 维非零向量 x 满足

$$Ax = \lambda x,$$

则称 λ 为矩阵 A 的一个特征值, x 称为矩阵 A 的属于 λ 的特征向量.

在工程技术中, 计算矩阵的特征值和特征向量主要使用数值解法, 本章将阐述其中几种, 并且只限于讨论实矩阵的情形.

8.1 幂法和反幂法

8.1.1 幂法

在有些问题中, 只要求出矩阵的主特征值(模最大的特征值), 幂法就是求矩阵主特征值及相应的特征向量的一种基本方法.

设 $n \times n$ 实矩阵 A 具有 n 个线性无关的特征向量 $x_1, x_2, \dots, x_n$, 其相应的特征值 $\lambda_1, \lambda_2, \dots, \lambda_n$ 满足不等式

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|, \quad (8.1)$$

其中, $Ax_i = \lambda_i x_i, i=1, 2, \dots, n$. 现在要求出 λ_1 和相应的特征向量.

任取一 n 维非零向量 u_0 , 从 u_0 出发, 按照如下的递推

$$u_k = Au_{k-1}, \quad k = 1, 2, \dots, \quad (8.2)$$

可产生一个向量序列 $\{u_k\}$, 分析这一序列的收敛情况, 可从中找出计算 λ_1 和相应的特征向量的方法.

因 n 维向量组 $x_1, x_2, \dots, x_n$ 线性无关, 故对于向量 u_0 , 必存在唯一的不全为零的数组 $a_1, a_2, \dots, a_n$, 使得

$$u_0 = a_1 x_1 + a_2 x_2 + \dots + a_n x_n.$$

由式(8.2)可得

$$\begin{aligned} u_k &= Au_{k-1} = A^2 u_{k-2} = \dots = A^k u_0 \\ &= a_1 A^k x_1 + a_2 A^k x_2 + \dots + a_n A^k x_n \\ &= a_1 \lambda_1^k x_1 + a_2 \lambda_2^k x_2 + \dots + a_n \lambda_n^k x_n \\ &= \lambda_1^k \left[a_1 x_1 + a_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k x_2 + \dots + a_n \left(\frac{\lambda_n}{\lambda_1} \right)^k x_n \right]. \end{aligned} \quad (8.3)$$

设 $a_1 \neq 0$, 由于有式(8.1)成立, 故从式(8.3)可看出, 当 k 充分大时, 有

$$u_k \approx \lambda_1^k a_1 x_1,$$

因为,矩阵的特征向量与任何一个非零常数相乘之后仍是该矩阵的属于同一个特征值的特征向量,所以,当 k 充分大时,由迭代公式(8.2)产生的 u_k 可近似地作为矩阵 A 的属于 λ_1 的特征向量.迭代公式(8.2)实质上是

$$u_k = A^k u_0,$$

故称这种迭代法为幂法.如果所选取的 u_0 使得 $a_1=0$,那么由于计算过程有舍入误差的影响,必然会在迭代的某一步产生这样的 u_k ,它在 x_1 方向上的分量不为零.这时,相当于以 u_k 为初始向量重新开始迭代.

在实际计算时,为了避免迭代向量 u_k 的模过大(当 $|\lambda_1|>1$)或过小(当 $|\lambda_1|<1$),通常每迭代一次都对 u_k 进行归一化,使其范数($\|\cdot\|_2$ 或 $\|\cdot\|_\infty$)等于1.因此,实际使用的迭代公式是

$$\begin{cases} y_{k-1} = \frac{u_{k-1}}{\|u_{k-1}\|}, \\ u_k = A y_{k-1}, \quad k = 1, 2, \dots, \end{cases} \quad (8.4)$$

由迭代公式(8.4)可知

$$u_k = \frac{A u_{k-1}}{\|u_{k-1}\|} = \frac{A^2 u_{k-2}}{\|A u_{k-2}\|} = \dots = \frac{A^k u_0}{\|A^k u_0\|},$$

因而

$$\begin{aligned} y_k &= \frac{u}{\|u_k\|} = \frac{A^k u_0}{\|A^k u_0\|} \\ &= \left(\frac{\lambda_1}{|\lambda_1|} \right)^k = \frac{a_1 x_1 + a_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k x_2 + \dots + a_n \left(\frac{\lambda_n}{\lambda_1} \right)^k x_n}{\left\| a_1 x_1 + a_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k x_2 + \dots + a_n \left(\frac{\lambda_n}{\lambda_1} \right)^k x_m \right\|}. \end{aligned} \quad (8.5)$$

由于式(8.1)成立,故当 $k \rightarrow \infty$ 时,若 $\lambda_1 > 0$,就有 $y_k \rightarrow \frac{a_1 x_1}{\|a_1 x_1\|}$,即 y_k 有确定的极限;若 $\lambda_1 < 0$,则 y_k 的各个分量之模有确定的极限,而 y_k 各个分量的正负号每迭代一次就改变一次.无论是哪一种情况都说明,当 k 充分大量,由式(8.4)得到的 y_{k-1} 可以近似地作为 A 的属于 λ_1 的特征向量.

关于 λ_1 的计算,有两种方法可供选择.第一种方法是,在式(8.4)中使用范数 $\|\cdot\|_2$,并且令

$$\beta_k = y_{k-1}^T u_k, \quad (8.6)$$

那么,由于 $u_k = A y_{k-1}$ 并根据式(8.5)可得

$$\lim_{k \rightarrow \infty} \beta_k = \frac{a_1 x_1^T}{\|a_1 x_1\|_2} A \frac{x_1 x_1}{\|a_1 x_1\|_2} = \frac{a_1^2 x_1^T x_1}{\|a_1 x_1\|_2^2} \lambda_1 = \lambda_1.$$

把式(8.4)和式(8.6)结合在一起,得到第一种幂法迭代格式:

$$\begin{cases} \text{任取非零向量 } u_0 \in \mathbb{R}^n, \\ \eta_{k-1} = \sqrt{u_{k-1}^T u_{k-1}}, \\ y_{k-1} = u_{k-1} / \eta_{k-1}, \\ \beta_k = y_{k-1}^T u_k, \end{cases} \quad k=1, 2, \dots. \quad (8.7)$$

当 $|\beta_k - \beta_{k-1}| < \varepsilon$ (允许误差) 时, 迭代终止, 以当前的 β_k 作为 λ_1 的近似值, 以 y_{k-1} 作为 A 的属于 λ_1 的特征向量.

第二种方法是, 在式(8.4)中使用范数 $\|\cdot\|_\infty$, 并且令

$$\beta_k = \frac{e_r^T u_k}{e_r^T y_{k-1}}, \quad (8.8)$$

把式(8.4)和式(8.8)结合在一起, 得到第二种幂法迭代格式:

$$\begin{cases} \text{任取非零向量 } u_0 = (h_1^{(0)}, \dots, h_n^{(0)})^T, \\ |h_r^{(k-1)}| = \max_{1 \leq j \leq n} |h_j^{(k-1)}|, \\ y_{k-1} = u_{k-1} / |h_r^{(k-1)}|, \\ u_k = Ay_{k-1} = (h_1^{(k)}, \dots, h_n^{(k)})^T, \\ \beta_k = \text{sign}(h_r^{(k-1)}) h_r^k, \end{cases} \quad k=1, 2, \dots. \quad (8.9)$$

终止迭代的控制也用 $|\beta_k - \beta_{k-1}| < \varepsilon$, 当前的 β_k 和 y_{k-1} 即分别作为 λ_1 和相应的特征向量. 在迭代格式(8.9)中, $|h_r^{(k-1)}| = \|u_{k-1}\|_\infty$, $\text{sign}(h_r^{(k-1)}) = e_r^T y_{(k-1)}$, $h_r^{(k)} = e_r^T u_k$.

两种迭代格式(8.7)和式(8.9)相比较, 格式(8.7)编制程序容易, 迭代一次所需时间较少; 格式(8.9)每迭代一次都要判断 u_{k-1} 的第几个分量的模最大, 因而所需时间较多, 但它在计算过程中舍入误差的影响比格式(8.7)小.

设矩阵 A 的特征值 $\lambda_i (i=1, 2, \dots, n)$ 满足条件

$$\lambda_1 = \lambda_2 = \dots = \lambda_m, \quad |\lambda_1| > |\lambda_{m+1}| \geq |\lambda_{m+2}| \geq \dots \geq |\lambda_n|, \quad (8.10)$$

但 A 仍有 n 个线性无关的特征向量. 这时, 由式(8.5)可看出, 只要 $a_1, a_2, \dots, a_m$ 不全为零, 则当 $\lambda_1 > 0$ 时, 有

$$\lim_{k \rightarrow \infty} y_k = \frac{a_1 x_1 + a_2 x_2 + \dots + a_m x_m}{\|a_1 x_1 + a_2 x_2 + \dots + a_m x_m\|}.$$

当 $\lambda_1 < 0$, 并且 k 足够大时, 有

$$y_k \approx \left(\frac{\lambda_1}{|\lambda_1|} \right)^k = \frac{a_1 x_1 + a_2 x_2 + \dots + a_m x_m}{\|a_1 x_1 + a_2 x_2 + \dots + a_m x_m\|}.$$

由于向量 $a_1 x_1 + a_2 x_2 + \dots + a_m x_m$ 仍是 A 的属于 λ_1 的特征向量, 故使用幂法迭代格式(8.7)和式(8.9). 当 k 足够大时 y_k 都可以近似地作为 A 的属于 λ_1 的特征向量, 而两种迭代格式中的 β_k 都收敛于 λ_1 .

例 8.1 求矩阵

$$A = \begin{bmatrix} 6 & -12 & 6 \\ -21 & -3 & 24 \\ -12 & -12 & 51 \end{bmatrix},$$

按模最大的特征值 λ_1 和属于 λ_1 的特征向量, 要求 $|\beta_k - \beta_{k-1}| < 0.001$.

解 采用幂法迭代格式(8.9)进行计算, 计算结果见表 8.1, 表中的数字只写到小数点后第 4 位.

表 8.1

k	u_k^T			y_k^T			β_k
0	1	0	0	1	0	0	
1	6	-21	-12	0.2857	-1	-0.5714	6
2	10.2857	-16.7143	-20.5714	0.5000	-0.8125	-1	16.7143
3	6.7500	-32.0625	-47.2500	0.1429	-0.6786	-1	47.2500
4	3.000	-24.9643	-44.5714	0.0673	-0.5601	-1	44.5714
5	1.1250	-23.7332	-45.0865	0.0250	-0.5264	-1	45.0865
6	0.4664	-22.9448	-44.9827	0.0104	-0.5101	-1	44.9827
7	0.1832	-22.6875	-45.0035	0.0041	-0.5041	-1	45.0035
8	0.0740	-22.5731	-44.9993	0.0016	-0.5016	-1	44.9993
9	0.0294	-22.5296	-45.0001				45.0001

因 $|\beta_9 - \beta_8| < 0.001$, 故 A 的模为最大的特征值是 $\lambda_1 \approx 45.0001$, 相应的特征向量为

$$x_1 \approx (0.0016, -0.5016, -1)^T,$$

精确结果是: $\lambda_1 = 45$, $x_1 = (0, -0.5, -1)^T$.

对幂法这里只限于讨论特征值满足式(8.1)或式(8.10)的两种情况. 如果在幂法(8.7)和式(8.9)的迭代过程中, 当 k 逐渐增大时, y_k 总是波动不定, 这说明矩阵 A 的特征值不满足式(8.1)和式(8.10). 这时最好改用其他方法计算 A 的特征值和特征向量.

幂法的收敛速度与比值 $\left| \frac{\lambda_2}{\lambda_1} \right|$ (式(5.1)的情况)或 $\left| \frac{\lambda_{m+1}}{\lambda_1} \right|$ (式(5.10)的情况)有关, 比值越小, 收敛速度越快. 此外, 当矩阵 A 没有 n 个线性无关的特征向量, 幂法仍然可以使用, 但收敛速度特别慢, 应改用其他方法.

8.1.2 带原点平移的幂法

为了加速收敛, 可采用带原点平移的幂法.

用幂法求 λ_1 时, 迭代过程收敛的快慢主要取决于 $\left| \frac{\lambda_2}{\lambda_1} \right| = r$, 当 r 愈接近 1, 收敛愈慢, 由于 $(A - pI)$ 的特征值 $u_i = \lambda_i - p$, 相应的特征向量不变, 适当地选取平移量 p 可使 $\left| \frac{\lambda_2}{\lambda_1} \right| < \left| \frac{\lambda_2}{\lambda_1} \right|$, 而且 $|\lambda_1 - p| > |\lambda_j - p|$ ($j = 2, 3, \dots, n$), 那么对 $(A - pI)$ 应用乘幂法就能较快地收敛, 求出 $u_1 = \lambda_1 - p$ 后, 易得 $\lambda_1 = u_1 + p$, 这种方法叫原点平移的幂法.

由于

$$\begin{aligned} u_k &= (A - pI)^k u_0 \\ &= (\lambda_1 - p)^k \left[a_1 x_1 + \left(\frac{\lambda_2 - p}{\lambda_1 - p} \right)^k a_2 x_2 + \cdots + \left(\frac{\lambda_n - p}{\lambda_1 - p} \right)^k a_n x_n \right], \end{aligned}$$

因此为了加速收敛,选取 p 时,应使 $\left| \frac{\lambda_2 - p}{\lambda_1 - p} \right|$ 较 $\left| \frac{\lambda_2}{\lambda_1} \right|$ 为小. 例如,如果 $\lambda_1 = 14, \lambda_2 = 13$, 那么 $\frac{\lambda_2}{\lambda_1} \approx 1$, 用原幂法迭代就收敛很慢;若选 $p = 12$, 且 $(\lambda_1 - 12)$ 又是 $(A - 12I)$ 的按模最大的特征值,那么, $(A - 12I)$ 代替 A 进行幂法迭代,由于 $\left| \frac{\lambda_2 - 12}{\lambda_1 - 12} \right| = 0.5$ 而使得收敛加快.

容易验证,只要选

$$p = \frac{1}{2}(\lambda_2 + \lambda_n),$$

则 $(\lambda_1 - p)$ 必是 $(A - pI)$ 的按模最大的特征值,并且有

$$0 < \frac{\lambda_2 - p}{\lambda_1 - p} < \frac{\lambda_2}{\lambda_1}.$$

因而用带原点平移的幂法求 λ_1 能加速收敛.

如果选

$$p = \frac{1}{2}(\lambda_1 + \lambda_{n-1}),$$

那么,矩阵 $(A - pI)$ 的按模最大的特征值是 $(\lambda_n - p)$. 所以,用矩阵 $(A - pI)$ 进行幂法迭代将获得 A 的按模最小的特征值 λ_n 以及相应的特征向量.

设矩阵 A 的特征值分布为式(8.1),这时要想用带原点平移的幂法加速计算 λ_1 ,选 p 必须使得 $(\lambda_1 - p)$ 是 $(A - pI)$ 的按模最大的特征值,同时要满足条件

$$\left| \frac{\lambda_j - p}{\lambda_1 - p} \right| < \left| \frac{\lambda_2}{\lambda_1} \right|, \quad j = 2, 3, \dots, n,$$

选择这样的 p 并没有一般公式.

由于矩阵的特征值分布情况预先不知道,因而实际使用带原点平移的幂法会有困难,但它使用简便,因而有时也使用. 通常使用者对特征值分布要有一个大概的了解,以便粗略地估计一个 p 值. 也可以在迭代过程中修改 p 值,直到所用的 p 值能明显地加速收敛为止.

例 8.2 用带原点平移的幂法求矩阵

$$A = \begin{bmatrix} 2 & 3 & 2 \\ 10 & 3 & 4 \\ 3 & 6 & 1 \end{bmatrix}$$

的按模最大的特征值 λ_1 和属于 λ_1 的特征向量.

解 选 $p = -2$, 对矩阵

$$A - pI = \begin{bmatrix} 4 & 3 & 2 \\ 10 & 5 & 4 \\ 3 & 6 & 3 \end{bmatrix}$$

使用幂法迭代格式(8.9), 迭代至 $|\beta_k - \beta_{k-1}| < 10^{-3}$ 为止. 计算结果见表 8.2, 表中的数字只写到小数点后第 4 位.

表 8.2

k	u_k^T			y_k^T			β_k
0	1	0	0	1	0	0	
1	4	10	3	0.4	1	0.3	4
2	5.2	10.2	8.1	0.5098	1	0.7941	10.2
3	6.6275	13.2745	9.9118	0.4993	1	0.7467	13.2745
4	6.4904	12.9793	9.7378	0.5001	1	0.7503	12.9793
5	6.5007	13.0016	9.7509	0.5000	1	0.7500	13.0016
6	6.4999	12.9999	9.7499	0.5000	1	0.7500	12.9999
7	6.5000	13.0000	9.7500				13.0000

从表中可看出, β_7 已满足要求, 所求的特征值为 $\lambda_1 \approx \beta_7 - 2 = 11.0000$, 相应的特征向量为 $x_1 = (0.50007, 1, 0.7500)^T$.

8.1.3 反幂法

反幂法用来计算矩阵按模最小的特征值及其特征向量, 还用来计算对应于一个给定近似特征值的特征向量.

设 $n \times n$ 实矩阵 A 非奇异, 且具有 n 个线性无关的特征向量 $x_1, x_2, \dots, x_n$, 其特征值满足

$$|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_{n-1}| > |\lambda_n|,$$

其中, $Ax_i = \lambda_i x_i$, $i = 1, 2, \dots, n$, 现在要计算 A 的按模最小的特征值 λ_n 以及相应的特征值.

因 A 非奇异, 故 $\lambda_i \neq 0$, $i = 1, 2, \dots, n$. 由

$$Ax_i = \lambda_i x_i,$$

得

$$A^{-1}x_i = \frac{1}{\lambda_i}x_i, \quad i = 1, 2, \dots, n,$$

所以, $\frac{1}{\lambda_n}$ 是矩阵 A^{-1} 的按模最大的特征值, x_n 是 A^{-1} 的属于 $\frac{1}{\lambda_n}$ 的特征向量. 于是, 对矩阵 A^{-1} 使用幂法(8.7)或式(8.9)就可求出 $\frac{1}{\lambda_n}$ (从而求出 λ_n) 以及相应的特征向量. 这里使用幂法迭代格式(8.7), 得出如下的反幂法迭代格式:

$$\begin{cases} \text{任取非零向量 } u_0 \in \mathbb{R}^n, \\ \eta_{k-1} = \sqrt{u_{k-1}^T u_{k-1}}, \\ y_{k-1} = u_{k-1} / \eta_{k-1}, \quad k = 1, 2, \dots, \\ Au_k = y_{k-1}, \\ \beta_k = y_{k-1}^T u_k, \end{cases} \quad (8.11)$$

每迭代一次都要求解一次线性方程组 $Au_k = y_{k-1}$. 当 k 足够大时, $\lambda_n \approx \frac{1}{\beta_k}$, y_{k-1} 可近似地作为矩阵 A 的属于 λ_n 的特征向量. 比值 $\left| \frac{\lambda_n}{\lambda_{n-1}} \right|$ 越小, 收敛得越快.

例 8.3 用反幂法求矩阵

$$A = \begin{bmatrix} 6 & -12 & 6 \\ -21 & -3 & 24 \\ -12 & -12 & 51 \end{bmatrix}$$

的按模最小的特征值和相应的特征向量, 要求 $|\beta_k - \beta_{k-1}| < 10^{-3}$.

解 使用反幂法迭代公式(8.11), 计算结果见表 8.3, 表中的数字只写到小数点后第 4 位.

表 8.3

k	u_k^T			y_k^T			β_k
0	1	1	1	0.5774	0.5774	0.5774	
1	-0.0321	-0.0706	-0.0128	-0.4082	-0.8981	-0.1633	-0.0667
2	0.0680	0.0844	0.0327	0.6011	0.7453	0.2885	-0.1089
3	-0.0557	-0.0925	-0.0292	-0.4976	-0.8272	-0.2611	-0.1108
4	0.0608	0.0867	0.0296	0.5532	0.7884	0.2691	-0.1097
5	-0.0587	-0.0903	-0.0298	-0.5250	-0.8083	-0.2665	-0.1117
6	0.0597	0.0884	0.0296	0.5393	0.7985	0.2676	-0.1107
7	-0.0592	-0.0894	-0.0297				-0.1113

从表中可看出, β_7 已满足精度要求. 所以, A 的按模最小的特征值为 $\lambda_3 \approx \frac{1}{\beta_7} = -8.9847$, 相应的特征向量为 $x_3 \approx (0.5393, 0.7985, 0.2676)^T$.

在实际计算中, 常采用带原点平移的反幂法求矩阵 A 的某个特征值 λ_s , 而且平移量是已求得的 λ_s 的近似值. 设已知数 μ 是 A 的某个特征值 λ_s 的近似值, 并且与 A 的其余特征值 $\lambda_i (i \neq s)$ 相比较, μ 最靠近 λ_s , 即

$$0 < |\lambda_s - \mu| < |\lambda_i - \mu|, \quad 1 \leq i \leq n, \quad i \neq s.$$

于是对矩阵 $(A - \mu I)$ 实行反幂法迭代, 就可求出 A 的特征值 λ_s 及其相应的特征向量. 具体说, 就是把反幂法迭代格式(8.11)中的 $Au_k = y_{k-1}$ 改为

$$(A - \mu I)u_k = y_{k-1},$$

当 k 足够大时, 取 $\lambda_s \approx \beta_k + \mu$, 其相应的特征向量近似地等于 y_{k-1} .

由于幂法和反幂法的迭代是否收敛, 依赖于特征值的分布情况, 因此实际使用时很不方便, 特别是不适合于自动计算. 只是在矩阵阶数非常高, 无法利用其他更有效的算法时, 才用幂法计算按模最大的特征值和相应的特征向量, 而用反幂法计算按模最小的特征值和相应的特征向量.

8.2 Jacobi(雅可比)方法

Jacobi 方法是用于计算实对称矩阵的全部特征值及对应特征向量的一种变换方法.

Jacobi 方法的基本思想是, 通过一组平面旋转变换(正交相似变换)将对称矩阵 A 化为对角矩阵, 及其全部特征值. 由代数学知识, 对于一个实对称矩阵 $A = (a_{ij})_{n \times n}$, 一定存在正交矩阵 U , 使

$$U^T A U = D,$$

其中, D 是对角矩阵, 其主对角线元素 $\lambda_j (j=1, 2, \dots, n)$ 是矩阵 A 的特征值, 而正交矩阵 U 的第 j 列就是 A 的属于 λ_j 的特征向量 ($j=1, 2, \dots, n$). 于是求实对称矩阵 A 的特征值问题等于寻找正交矩阵 U , 使 $U^T A U = D$ 为对角阵, 而这个问题的主要困难在于如何构造 U .

设有实对称矩阵 $A = (a_{ij})_{n \times n}$, 它的一对非主对角线元素 $a_{pq} = a_{qp} (p \neq q)$ 不为零. 取 $n \times n$ 的正交矩阵

$$U_{pq} = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & \cos \varphi & & -\sin \varphi \\ & & & & 1 & \\ & & & & & \ddots \\ & & & & & & 1 \\ & & & & & & & \ddots \\ & & & & \sin \varphi & & \cos \varphi & & \\ & & & & & & & 1 & \\ & & & & & & & & \ddots \\ & & & & & & & & & 1 \end{bmatrix} \quad \begin{array}{l} \text{第 } p \text{ 行} \\ \\ \\ \text{第 } q \text{ 行} \end{array} \quad (8.12)$$

第 p 列 第 q 列

U_{pq} 的主对角线元素中, $a_{pp} = a_{qq} = \cos \varphi$, 其余为 1; U_{pq} 的非主对角线元素中, $a_{pq} = -\sin \varphi$, $a_{qp} = \sin \varphi$, 其余为 0. U_{pq} 是 n 维空间中的二维坐标旋转变换矩阵. 设 $x \in \mathbb{R}^n$, 则 $U_{pq}x$ 相当于将坐标轴 ox_p 和 ox_q 在 x_p, x_q 所在平面旋转了一个角度 φ , 其他坐标轴保持不变, 故称式

(8.12)的 U_{pq} 为平面旋转矩阵. 用 U_{pq} 对 A 作正交相似变换, 得到矩阵

$$A_1 = U_{pq}^T A U_{pq} = (a_{ij}^{(1)})_{n \times n}.$$

显然, A_1 仍是实对称矩阵, A_1 与 A 的特征值完全相同. 通过直接计算, 知

$$\begin{cases} a_{pp}^{(1)} = a_{pp} \cos^2 \varphi + a_{qq} \sin^2 \varphi + 2a_{pq} \cos \varphi \sin \varphi, \\ a_{qq}^{(1)} = a_{pp} \sin^2 \varphi + a_{qq} \cos^2 \varphi + 2a_{pq} \cos \varphi \sin \varphi, \\ a_{pi}^{(1)} = a_{ip}^{(1)} = a_{pi} \cos \varphi + a_{qi} \sin \varphi, \\ a_{qi}^{(1)} = a_{iq}^{(1)} = -a_{pi} \sin \varphi + a_{qi} \cos \varphi, & (i, j \neq p, q) \\ a_{ij}^{(1)} = a_{ji}^{(1)} = a_{ij}, \\ a_{pq}^{(1)} = a_{qp}^{(1)} = \frac{1}{2}(a_{qq} - a_{pp}) \sin 2\varphi + a_{pq} \cos 2\varphi. \end{cases} \quad (8.13)$$

可见, 变换的结果: 矩阵 A_1 的第 p 行、第 p 列和第 q 行、第 q 列的元素发生了变化, 其余元素不变. 特别地, 当取 φ 满足关系式

$$\cot 2\varphi = \frac{a_{pp} - a_{qq}}{2a_{pq}} \quad (8.14)$$

时, 可以得到 $a_{pq}^{(1)} = a_{qp}^{(1)} = 0$, 也就是说, 用平面旋转变换矩阵 U_{pq} 对 A 进行正交相似变换, 可以将 A 的两个非主对角线元素 a_{pq} 和 a_{qp} 化为零.

求实对称矩阵 A 的特征值和特征向量是一个迭代过程, 其迭代步骤如下.

(1) 在 A 的非主对角线元素中, 找出按模最大的元素 a_{pq} ;

(2) 由等式(8.14)计算 $\cot(2\varphi)$, 并由此求出 $\sin \varphi, \cos \varphi$ 以及相应的平面旋转矩阵 U_{pq} ;

(3) 按照公式(8.13)计算矩阵 A_1 的元素 $a_{ij}^{(1)}$;

(4) 若 $\max_{i < j} |a_{ij}^{(1)}| < \epsilon$ (允许误差), 则停止计算, 所求特征值为 $\lambda_i \approx a_{ii}^{(1)}, i = 1, 2, \dots, n$; 否则, 令 $A = A_1$, 重复执行上述各步骤.

当条件 $\max_{i < j} |a_{ij}^{(1)}| < \epsilon$ 满足时, A_1 的所有非主对角线元素在所给精度要求下近似等于零, A_1 几乎是一个对角矩阵. 因此, 可取 A_1 的主对角线元素作为 A 的特征值的近似值, 即

$$\lambda_i \approx a_{ii}^{(1)}, \quad i = 1, 2, \dots, n.$$

设经过 N 次迭代, 上述条件得到满足, 又记第 k 次迭代所得的平面旋转矩阵为 $U_{p_k q_k}$, 那么, 经过 N 次迭代所得的矩阵 A_1 为

$$A_1 = U_{p_N q_N}^T \cdots U_{p_2 q_2}^T U_{p_1 q_1}^T A U_{p_1 q_1} U_{p_2 q_2} \cdots U_{p_N q_N}.$$

记

$$U = U_{p_1 q_1} U_{p_2 q_2} \cdots U_{p_N q_N},$$

则 U 是正交矩阵, 并且有

$$A_1 = U^T A U,$$

因为 A_1 被看作是对角矩阵, 所以矩阵 U 的第 j 列就是 A 的属于特征值 $\lambda_j \approx a_{jj}^{(1)}$ 的近似特征向量, 并且所有的特征向量都是正交规范化的.

在旋转变换中可以逐步形成 U , 而不必保存每一次的变换矩阵 $U_{p_k q_k}$. 记

$$R_0 = I,$$

令

$$R_k = R_{k-1} U_{p_k q_k}, \quad k = 1, 2, \dots, N, \quad (8.15)$$

则

$$U = R_N.$$

根据式(8.15), 计算矩阵 R_k 的元素 $r_{ij}^{(k)}$ 的公式为

$$\begin{cases} r_{ip_k}^{(k)} = r_{ip_k}^{(k-1)} \cos \varphi + r_{iq_k}^{(k-1)} \sin \varphi, \\ r_{iq_k}^{(k)} = -r_{ip_k}^{(k-1)} \sin \varphi + r_{iq_k}^{(k-1)} \cos \varphi, \quad (j = 1, 2, \dots, n, \text{ 且 } j \neq p_k, q_k). \\ r_{ij}^{(k)} = r_{ij}^{(k-1)} \end{cases} \quad (8.16)$$

用 Jacobi 方法计算 $n \times n$ 矩阵 A 的特征值和特征向量, 每迭代一次都要按公式(8.13)、式(8.14)和式(8.16)计算一次. 计算量主要是乘法的次数, 约为 $8n$ 次.

关于 Jacobi 方法的收敛性, 有以下的定理.

定理 8.1 设 $A = (a_{ij})_{n \times n}$ 是实对称矩阵, 由 Jacobi 方法的第 k 次迭代得到的矩阵记为 $A_k = (a_{ij}^{(k)})_{n \times n}$, 又记

$$\eta_k = \sum_{\substack{i, j=1 \\ i \neq j}}^n (a_{ij}^{(k)})^2,$$

则有 $\lim_{k \rightarrow \infty} \eta_k = 0$ 成立.

证 令

$$\delta_k = \sum_{i=1}^n (a_{ii}^{(k)})^2,$$

由式(8.13), 经过计算, 并注意到

$$\frac{1}{2} (a_{qq}^{(k)} - a_{pp}^{(k)}) \sin 2\varphi + a_{pq}^{(k)} \cos 2\varphi = 0,$$

可得

$$(a_{pp}^{(k+1)} + a_{qq}^{(k+1)})^2 = (a_{pp}^{(k)})^2 + (a_{qq}^{(k)})^2 + 2(a_{pq}^{(k)})^2,$$

又由于

$$a_{ii}^{(k+1)} = a_{ii}^{(k)}, \quad (i \neq p, q),$$

所以

$$\delta_{k+1} = \delta_k + 2(a_{pq}^{(k)})^2.$$

因实对称矩阵经过正交相似变换后, 其元素的平方和不变, 即

$$\eta_{k+1} + \delta_{k+1} = \eta_k + \delta_k,$$

故有

$$\eta_{k+1} = \eta_k - (\delta_{k+1} - \delta_k) = \eta_k - 2(a_{pq}^{(k)})^2,$$

又因

$$|a_{pq}^{(k)}| = \max_{i < j} |a_{ij}^{(k)}|,$$

故得

$$\eta_k \leq n(n-1)(a_{pq}^{(k)})^2,$$

从而有

$$\eta_{k+1} = \eta_k - 2(a_{pq}^{(k)})^2 \leq \eta_k - \frac{2}{n(n-1)}\eta_k = \left[1 - \frac{2}{n(n-1)}\right]\eta_k, \quad (8.17)$$

反复使用式(8.17),得

$$\eta_{k+1} \leq \left[1 - \frac{2}{n(n-1)}\right]^{k+1} \eta_0,$$

其中, η_0 是矩阵 A 的非主对角线元素平方之和.

由于

$$0 \leq 1 - \frac{2}{n(n-1)} < 1,$$

所以有 $\lim_{k \rightarrow \infty} \eta_k = 0$. 证毕.

前面叙述的 Jacobi 方法又称为经典的 Jacobi 方法,它每次迭代都是把按模最大的非主对角线元素 a_{pq} 作为消元对象. 不论实对称矩阵 A 的特征值如何分布,经典的 Jacobi 方法总是收敛的,而且当 A 的阶数不太高时,收敛速度还比较快. 此外,这个方法具有较强的数值稳定性,求得的结果精度一般都比较高的,特别是求得特征向量正交性很好,这是其他方法所不如的. 经典的 Jacobi 方法的缺点是,不能有效地利用矩阵的各种特殊形状(例如环状或稀疏等)以节省工作量,这是因为它在迭代过程中一般都会破坏原矩阵的特殊形状. 还有其他一些缺点,例如,绝对值较小的特征值精度差一些;由于每迭代一次都要在非主对角线元素中寻找模数最大的元素,因而比较费时间.

8.3 QR 方法

QR 方法是求一般矩阵全部特征值的最有效并广泛应用的方法之一,在实际应用中,经常把一般的矩阵经过正交相似变换化成上 Hessenberg 矩阵,再应用 QR 方法求其特征值和特征向量.

8.3.1 基本 QR 方法

QR 方法是求一般 $n \times n$ 矩阵全部特征值和特征向量的一种迭代方法,但这里只限于讨论实矩阵的情形. 设 A 是 $n \times n$ 非奇异实矩阵.

令 $A_1 = A$, 对 A_1 作 QR 分解, 得

$$A_1 = Q_1 R_1,$$

让上式右端逆序相乘, 令

$$A_2 = R_1 Q_1,$$

又对 A_2 作 QR 分解, 得

$$A_3 = Q_2 R_2,$$

反复进行这种运算, 就得到一个矩阵序列 $\{A_k\}$, 这个序列由迭代公式

$$\begin{cases} A_1 = A, \\ A_k = Q_k R_k, \\ A_{k+1} = R_k Q_k, \quad k = 1, 2, \dots \end{cases} \quad (8.18)$$

产生, 其中 $R_k Q_k$ 是 A_k 的 QR 分解, Q_k 是正交矩阵, R_k 是上三角矩阵, 由于

$$R_k = Q_k^T A_k,$$

故有

$$A_{k+1} = Q_k^T A_k Q_k, \quad k = 1, 2, \dots$$

因而 A_{k+1} 与 A_k ($k=1, 2, \dots$) 相似. 所有 A_k 都与 A 相似, 它们都有共同的特征值. 在一定的条件下, 迭代公式(8.18)产生的序列 $\{A_k\}$ 收敛于上三角(或分块上三角)矩阵, 其主对角线元素(或子块)有确定的极限. 如果收敛于上三角矩阵, 则这个上三角矩阵的主对角线元素就是矩阵 A 的特征值; 如果收敛于分块上三角矩阵, 则主对角线上各个子块的特征值就是矩阵 A 的特征值. 迭代公式(8.18)称为基本 QR 方法. 这里给出两个关于基本 QR 方法的收敛性定理, 但不予以证明.

定理 8.2 设 A 为 $n \times n$ 实矩阵, 它的特征值 λ_i ($i=1, 2, \dots, n$) 满足

$$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0,$$

则由基本 QR 方法(8.18)产生的矩阵序列 $\{A_k\}$ 收敛于一个上三角矩阵(其主对角线以上的元素可以不收敛). 若 A 是对称矩阵, 则序列 $\{A_k\}$ 收敛于一个对角矩阵.

定理 8.3 对于任何 $n \times n$ 实矩阵 A , 由基本 QR 方法(8.18)产生的矩阵序列 $\{A_k\}$ 收敛于分块上三角矩阵(其主对角线子块以上的元素可以不收敛), 并且每个主对角线子块有等模的特征值.

从公式(8.18)可知, 要实现一步 QR 方法的迭代, 就需要做一次 QR 分解, 再做一次矩阵相乘, 当 A 是一般矩阵时, 计算量是很大的. 为了减少计算量, 在实际计算时, 总是先将原矩阵 A 经过相似变换, 化为上 Hessenberg 矩阵 $A_1 = (a_{ij}^{(1)})_{n \times n}$, 其中, 当 $i > j + 1$ 时, $a_{ij}^{(1)} = 0$. 然后对使用 QR 方法(8.18). 容易看出, 从上 Hessenberg 矩阵 A_1 出发, 经过公式(8.18)的迭代所得的 A_k ($k=2, 3, \dots$) 全都是上 Hessenberg 矩阵. 事实上, 设 A_k 是上 Hessenberg 矩阵, 则由于 R_k^{-1} 是上 Hessenberg 矩阵, 故

$$Q_k = A_k R_k^{-1}$$

是上 Hessenberg 矩阵, 因而

$$A_{k+1} = R_k Q_k$$

也是上 Hessenberg 矩阵.

8.3.2 化矩阵为上 Hessenberg 阵

对实矩阵 $A = (a_{ij})_{n \times n}$ 作相似变换化为上 Hessenberg 矩阵, 其变换矩阵可用 Householder 矩阵.

设 $a_{i1} (i=3, 4, \dots, n)$ 不全为零, 令

$$\begin{aligned} s_1 &= (0, a_{21}, \dots, a_{n1})^T, \\ a_1 &= -\operatorname{sign}(a_{21}) \|s_1\|_2, \\ u_1 &= s_1 - a_1 e_2, \end{aligned}$$

(若 $a_{21}=0$, 则取 $a_1 = \|s_1\|_2$), 构成 Householder 矩阵

$$H_1 = I - 2u_1 u_1^T / (u_1^T u_1) = \begin{bmatrix} 1 & \mathbf{0} \\ \mathbf{0} & W_1 \end{bmatrix},$$

其中, W_1 是 $(n-1) \times (n-1)$ 矩阵. 根据引理 2.1, 有

$$H_1 s_1 = a_1 e_2 = (0, a_1, 0, \dots, 0)^T,$$

因而得

$$A^{(2)} = H_1 A H_1 = \begin{bmatrix} a_{11} & a_{12}^{(2)} & \cdots & a_{1n}^{(2)} \\ a_1 & & & \\ 0 & \vdots & & \vdots \\ \vdots & & & \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} \end{bmatrix}.$$

若 $a_{i1} (i=3, 4, \dots, n)$ 全为零, 则取 $H_1 = I$, 并且

$$A^{(2)} = H_1 A H_1 = A.$$

设 $a_{i2}^{(2)} (i=4, 5, \dots, n)$ 不全为零, 令

$$\begin{aligned} s_2 &= (0, 0, a_{32}^{(2)}, \dots, a_{n2}^{(2)})^T, \\ a_2 &= -\operatorname{sign}(a_{32}^{(2)}) \|s_2\|_2, \\ u_2 &= s_2 - a_2 e_3, \end{aligned}$$

(若 $a_{32}^{(2)}=0$, 则取 $a_2 = \|s_2\|_2$), 构成 Householder 矩阵

$$H_2 = I - 2u_2 u_2^T / (u_2^T u_2) = \begin{bmatrix} I_2 & \mathbf{0} \\ \mathbf{0} & W_2 \end{bmatrix},$$

其中, I_2 是 2×2 单位矩阵, W_2 是 $(n-2) \times (n-2)$ 矩阵. 可得

$$H_2 s_2 = a_2 e_3 = (0, 0, a_2, 0, \dots, 0)^T,$$

$$A^{(3)} = H_2 A^{(2)} H_2 = \begin{bmatrix} a_{11} & a_{12}^{(2)} & a_{13}^{(2)} & \cdots & a_{1n}^{(2)} \\ a_1 & a_{22}^{(2)} & & & \\ & a_2 & & & \\ & & & & \vdots \\ & & a_{n3}^{(3)} & & a_{nn}^{(3)} \end{bmatrix}.$$

若 $a_{i2}^{(2)} (i=4, 5, \dots, n)$ 全为零, 则取 $H_2 = I$, 并且

$$A^{(3)} = H_2 A^{(2)} H_2 = A^{(2)}.$$

一般地, 设按照上述方法已得到矩阵 $A^{(r)} (r \geq 2)$, 又设 $a_{ir}^{(r)} (i=r+2, r+3, \dots, n)$ 全为零, 则令

$$s_r = (0, \dots, 0, a_{r+1,r}^{(r)}, \dots, a_{nr}^{(r)})^T,$$

$$a_r = -\text{sign}(a_{r+1,r}^{(r)}) \|s_r\|_2,$$

$$u_r = s_r - a_r e_{r+1},$$

(若 $a_{r+1,r}^{(r)} = 0$, 则取 $a_r = \|s_r\|_2$), 构成 Householder 矩阵

$$H_r = I - 2u_r u_r^T / (u_r^T u_r) = \begin{bmatrix} I_r & 0 \\ 0 & W_r \end{bmatrix},$$

其中, I_r 是 $r \times r$ 单位矩阵, W_r 是 $(n-r) \times (n-r)$ 矩阵, 可得

$$H_r s_r = a_r e_{r+1} = (0, \dots, 0, a_r, 0, \dots, 0)^T,$$

$$A^{(r+1)} = H_r A^{(r)} H_r = \begin{bmatrix} a_{11} & \cdots & a_{1r}^{(r)} & a_{1,r+1}^{(r+1)} & \cdots & a_{1n}^{(r+1)} \\ a_1 & & & & & \\ & & \vdots & \vdots & & \vdots \\ & & \ddots & a_{rr}^{(r)} & & \\ & & & a_r & & \\ & & & & a_{n,r+1}^{(r+1)} & \cdots & a_{nn}^{(r+1)} \end{bmatrix}.$$

若 $a_{ir}^{(r)} (i=r+2, r+3, \dots, n)$ 全为零, 则取 $H_r = I$, 并且有

$$A^{(r+1)} = H_r A^{(r)} H_r = A^{(r)}.$$

当 $r=n-2$ 时, 就得拟上三角矩阵

$$\begin{aligned} A^{(n-1)} &= H_{n-2} \cdots H_2 H_1 A H_1 H_2 \cdots H_{n-2} \\ &= \begin{bmatrix} a_{11} & \cdots & a_{1,n-2}^{(n-2)} & a_{1,n-1}^{(n-1)} & a_{1n}^{(n-1)} \\ a_1 & & & & \\ & \ddots & \vdots & \vdots & \vdots \\ & & a_{n-2,n-2}^{(n-2)} & & \\ & & a_{n-2} & & \\ & & & a_{n,n-1}^{(n-1)} & a_{nn}^{(n-1)} \end{bmatrix}. \end{aligned} \quad (8.19)$$

因 $H_r (r=1, 2, \dots, n-2)$ 是对称正交矩阵, 故

$$P = H_1 H_2 \cdots H_{n-2}$$

是正交矩阵, 并且

$$P^T = (H_1 H_2 \cdots H_{n-2})^T = H_{n-2} \cdots H_2 H_1,$$

所以有

$$A^{(n-1)} = P^T A P,$$

因而 $A^{(n-1)}$ 与 A 相似.

特别是当 A 为实对称矩阵时, $A^{(n-1)}$ 也是实对称矩阵, 因而 $A^{(n-1)}$ 是对称三角矩阵:

$$A^{(n-1)} = \begin{bmatrix} a_{11} & a_1 & & & \\ a_1 & a_{22}^{(2)} & a_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-2} & a_{n-1, n-1}^{(n-1)} & a_{n, n-1}^{(n-1)} \\ & & & a_{n, n-1}^{(n-1)} & a_{nn}^{(n-1)} \end{bmatrix}.$$

8.3.3 带原点平移的 QR 方法

理论上已经证明, 基本 QR 方法(8.18)的收敛速度一般是线性收敛的. 为了加速收敛, 引进带原点平移的 QR 方法.

设 A 是 $n \times n$ 非奇异实矩阵, 令 $A_1 = A$, 并取一适当的数 u_1 , 对矩阵 $(A_1 - u_1 I)$ 作 QR 分解

$$A - u_1 I = Q_1 R_1,$$

然后令

$$A_2 = R_1 Q_1 + u_1 I,$$

又取一适当的数 u_2 , 对矩阵 $(A_2 - u_2 I)$ 作 QR 分解

$$A_2 - u_2 I = Q_2 R_2.$$

令

$$A_3 = R_2 Q_2 + u_2 I,$$

反复进行这种运算, 得到迭代公式

$$\begin{cases} A_1 = A, \\ A_k - u_k I = Q_k R_k, \quad k = 1, 2, \dots, \\ A_{k+1} = R_k Q_k + u_k I, \end{cases} \quad (8.20)$$

其中, $Q_k R_k$ 是矩阵 $(A_k - u_k I)$ 的 QR 分解, Q_k 是正交矩阵, R_k 是上三角矩阵. 迭代公式(8.20)称为带原点平移的 QR 方法, 称 u_k 为第 k 次迭代中的原点平移量, 这样产生的序列有下述性质.

(1) A_{k+1} 与 A_k 相似;

(2) 当 A_k 为上 Hessenberg 阵时, A_{k+1} 也是上 Hessenberg 阵, 迭代若干次后, 若

$|a_{n,n-1}^{(k+1)}|$ 足够小, 就可以认为 $a_{n,n-1}^{(k+1)}$ 是特征值的近似值, 然后将 A_{k+1} 的第 n 行第 n 列去掉 (仍是上 Hessenberg 阵).

如果在式 (8.20) 中对 $(A_k - u_k I)$ 作 QR 分解时利用 Householder 矩阵, 则有

$$R_k = H_{n-1}^{(k)} H_{n-2}^{(k)} \cdots H_1^{(k)} (A_k - u_k I), \quad (8.21)$$

$$Q_k = H_1^{(k)} H_2^{(k)} \cdots H_{n-1}^{(k)},$$

因而

$$A_{k+1} = H_{n-1}^{(k)} \cdots H_{n-2}^{(k)} H_1^{(k)} A_k H_1^{(k)} H_2^{(k)} \cdots H_{n-1}^{(k)}, \quad k = 1, 2, \dots, \quad (8.22)$$

其中, $H_i^{(k)} (i=1, 2, \dots, n-1)$ 是式 (8.21) 中的 Householder 矩阵.

对压缩后的矩阵重复上述分解过程, 恰当的选用平移量 u_k 可加速分解过程收敛, u_k 的选取一般有两种:

(1) 令 $u_k = a_{nn}^{(k)}$;

(2) 设 A_1 已经是上 Hessenberg 矩阵, 则 A_k 的形式为

$$A_k = \begin{bmatrix} a_{11}^{(k)} & a_{12}^{(k)} & \cdots & \cdots & \cdots & \cdots & \cdots & a_{1n}^{(k)} \\ a_{21}^{(k)} & a_{22}^{(k)} & & \ddots & & & & \vdots \\ & \ddots & \ddots & \ddots & \ddots & & & \vdots \\ & & \ddots & \ddots & \ddots & \ddots & & \vdots \\ & & & \ddots & \ddots & \ddots & \ddots & \vdots \\ & & & & \ddots & \ddots & \ddots & \vdots \\ & & & & & \ddots & \ddots & \vdots \\ & & & & & & a_{n,n-1}^{(k)} & a_{nn}^{(k)} \end{bmatrix}.$$

先求出 A_k 最右下角的二阶子矩阵

$$D_k = \begin{bmatrix} a_{n-1,n-1}^{(k)} & a_{n-1,n}^{(k)} \\ a_{n,n-1}^{(k)} & a_{nn}^{(k)} \end{bmatrix}$$

的两个特征值, 然后取其中按模最小的一个作为平移量 u_k . 按照这种平移量的选取方法进行式 (8.22) 的迭代, 可以断言, 下列极限成立:

$$\lim_{k \rightarrow \infty} a_{n-1,n-2}^{(k)} a_{n,n-1}^{(k)} = 0.$$

在迭代过程中, 如果 $a_{n,n-1}^{(k+1)}$ 已接近于零, 就取 $a_{nn}^{(k+1)}$ 作为矩阵 A 的特征值 λ_n 的近似值, 并对 A_{k+1} 左上角的 $(n-1)$ 阶主子矩阵进行式 (8.22) 的迭代; 如果在迭代过程中 $a_{n-1,n-2}^{(k+1)}$ 变得接近于零, 则取矩阵 D_{k+1} 的两个特征值作为矩阵 A 的特征值 λ_{n-1} 和 λ_n 的近似值, 然后对 A_{k+1} 左上角的 $(n-2)$ 阶主子矩阵进行式 (8.22) 的迭代. 可见, 每求得 A 的一个或两个特征值之后就降阶迭代, 直至余下一个 2×2 的矩阵为止.

习 题 8

1. 设 A 为 n 阶实对称方阵, λ_1 为其最大特征值, 证明:

$$\lambda_1 = \max_{x \neq 0} \frac{X^T A X}{X^T X}$$

2. 用幂法计算下列矩阵的主特征值及对应的特征向量:

$$(1) A_1 = \begin{bmatrix} 7 & 3 & -2 \\ 3 & 4 & -1 \\ -2 & -1 & 3 \end{bmatrix}, \quad (2) A_2 = \begin{bmatrix} 3 & -4 & 3 \\ -4 & 6 & 3 \\ 3 & 3 & 1 \end{bmatrix}.$$

当特征值有 3 位小数稳定时, 迭代终止.

3. 利用反幂法求矩阵

$$\begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

的最接近于 6 的特征值及对应的特征向量.

4. 在用幂法求

$$A = \begin{bmatrix} -3 & 8 & 9 \\ 8 & -2 & 4 \\ 9 & 4 & 2 \end{bmatrix}$$

的模最大的特征值和相应的特征向量时, 为了加速收敛, 采用平移原点的方法, 对本题, 试用 5 加于各主对角元以改善其收敛速度, 为此取

$$B = A + 5I = \begin{bmatrix} 2 & 8 & 9 \\ 8 & 3 & 4 \\ 9 & 4 & 7 \end{bmatrix},$$

试以 $H^{(0)} = (1, 1, 1)^T$ 为初始向量, 通过求 B 的模最大的特征值去求 A 的模最大的特征值.

5. 设方阵 A 的特征值都是实数, 且满足

$$\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_n,$$

作原点平移 $(\lambda_i - p)$, $i=1, 2, \dots, n$, 证明: 当 $p = \frac{1}{2}(\lambda_2 + \lambda_n)$ 时, 幂法收敛最快.

6. 用 Jacobi 方法计算

$$A = \begin{bmatrix} 1.0 & 1.0 & 0.5 \\ 1.0 & 1.0 & 0.25 \\ 0.5 & 0.25 & 2.0 \end{bmatrix}$$

的全部特征值及特征向量.

7. 利用 Householder 矩阵将

$$A = \begin{bmatrix} 1 & 3 & 4 \\ 3 & 1 & 2 \\ 4 & 2 & 1 \end{bmatrix}$$

正交相似约化为对称三对角阵.

8. 用带位移的 QR 方法计算下列矩阵的全部特征值:

$$(1) A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & -1 & 1 \\ 0 & 1 & 3 \end{bmatrix}, \quad (2) B = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

9. 试用 Householder 矩阵将 A 分解为 QR , 其中 Q 为正交阵, R 为上三角阵, $A =$

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & -1 & -1 \\ 2 & -4 & 5 \end{bmatrix}.$$

数值实验题 8

实验 8.1 矩阵特征值问题条件数的估计

问题提出:第 6 章讨论的矩阵条件数,反映的是作为线性代数方程组 $AX=b$ 的求解,问题是否病态,矩阵特征值问题是否病态,则是完全不同的问题,从理论上严格讨论矩阵特征值问题的条件是困难的,简单的数值实验可以给出感性的认识.

实验内容:Matlab 中提供有函数“condeig”可以用来估计矩阵特征值问题的条件数,我们来尝试它.

实验要求:

(1) 选择一系列维数递增的矩阵(例如可以是随机生成的),用“condeig”估计它们对矩阵特征值问题的条件数.

(2) Hilbert 矩阵

$$H = (h_{i,j})_{n \times n}, \quad h_{i,j} = \frac{1}{i+j-1}, \quad i, j = 1, 2, \dots, n.$$

对线性代数方程组问题是著名的病态问题,用“condeig”估计它作为矩阵特征值问题的条件数.

实验 8.2 QR 算法的模拟及实现

问题提出:QR 算法是矩阵特征值计算中实用而有效的方法,原理型的算法很容易实现,实用型的算法则需要程序技巧.

实验内容:Matlab 中提供有函数“eig”可以用来计算矩阵的特征值,其核心算法就是 QR 算法,Matlab 中还提供有函数“qr”,可以实现矩阵的 QR 分解.

实验要求:

(1) 利用函数“qr”,通过矩阵分解实现原理型的 QR 算法.

(2) 分别利用原理型的 QR 算法和 Matlab 中提供的函数“eig”计算下面矩阵的特征值并比较结果.

$$H = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 \\ 4 & 4 & 5 & 6 & 7 \\ 0 & 3 & 6 & 7 & 8 \\ 0 & 0 & 2 & 8 & 9 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

答案与提示

习 题 1

1. $e(\ln x) \approx \frac{1}{x}(x-x^*) = e(x) = \delta$; 2. $0.02n$; 3. $5, 2, 4, 5, 1$; 4. (1) $\epsilon^* = 1.05 \times 10^{-3}$,
 (2) $\epsilon^* = 0.215$, (3) $\epsilon^* = 10^{-5}$; 5. $e_r^*(v) = \frac{1}{300}$; 6. $\epsilon = \frac{1}{2} \times 10^{-3}$; 7. $x_1 = 55.982, x_2 = 0.01786$;
 8. $\arctan \frac{1}{1+N(N+1)}$; 9. $\epsilon(x^*) \leq 0.005(\text{cm})$; 11. 误差限 $\epsilon(s^*) = 0.0744(m^2)$, 相对误差限 $\epsilon_r(s^*) = 0.55\%$; 12. $|y_{10} - y_{10}^*| \leq 10^{10}\delta$, 计算过程不稳定; 13. (1) 0.0052328 , (2) 0.008 , (3) 0.0051253 .
 (4) 1; 以 $\frac{1}{(3+2\sqrt{2})^3}$ 计算得到的结果最好; 14. $\epsilon(f^*) \approx \frac{|y^* - y|}{|y^*|} \leq 4.37 \times 10^{-6}$.

习 题 2

1. $L_2(x) = \frac{5}{6}x^2 + \frac{3}{2}x - \frac{7}{3}$; 2. $\sin(0.33) \approx 0.324027$; 3. 提示: 利用插值余项;
 4. $R_2 = 1.06 \times 10^{-8} + \frac{1}{2} \times 10^{-5} \approx 5.0106 \times 10^{-6}$; 5. $h \leq 0.006$; 6. 提示: $\Delta f(x) = f(x+h) - f(x) = f'(x)h + \frac{f''(x)}{2!}h^2 + \dots$; 7. 提示: 由定义直接可得; 8. 提示: 利用差商的性质 1;
 9. $1, 0$; 11. $P(x) = \frac{1}{4}x^2(x-3)^2$ $R_4(x) = \frac{f^{(5)}(\xi)}{5!}x^2(x-1)^2(x-2)$, $\xi \in (0, 2)$; 12. $x \approx 0.255189$;
 13. $|R_1(x)| \leq \frac{3}{4}h^2$; 14. $I_h(x) = \sum_{i=0}^n x_i^4 \alpha_i(x) + 4x_i^3 \beta_i(x)$, $|R(x)| \leq \frac{h^4}{384} \max_{a \leq x \leq b} |f^{(4)}(x)| \leq \frac{h^4}{16}$;
 15. (1) $S(x) = \begin{cases} \frac{1}{90}x(1-x)(19x-26), & x \in [0, 1], \\ \frac{1}{90}(x-1)(x-2)(5x-12), & x \in [1, 2], \\ \frac{1}{90}(3-x)(x-2)(x-4), & x \in [2, 3], \end{cases}$
 (2) $S(x) = \begin{cases} \frac{1}{15}x(x-1)(11x-15), & x \in [0, 1], \\ \frac{1}{15}(x-1)(x-2)(7-3x), & x \in [1, 2], \\ \frac{1}{15}(x-3)^3(x-2), & x \in [2, 3]. \end{cases}$
 16. (1) $3, 0$, (2) $0, x^5 + 2x^4 + x^3 + 1$, (3) 0 , 17. 8125 ; (4) $b = -2, c = 3$.

习 题 3

1. (1) 否, (2) 是; 2. $S_1(x) = x - \frac{1}{6}$, $S_2(x) = 375.24253x^{100} - 375.14824x^{101}$;

3. $s(x) = 0.1171875 + 1.640625x^2 - 0.823125x^4$;

4. $\int_0^1 T_n^*(x) T_m^*(x) \frac{1}{\sqrt{x-x^2}} dx = \int_0^1 T_n(2x-1) T_m(2x-1) \frac{1}{\sqrt{x-x^2}} dx \xrightarrow{\text{令 } t=2x-1}$

$$\int_{-1}^1 T_n(t) T_m(t) \frac{1}{\sqrt{1-t^2}} dt = \begin{cases} 0, & m \neq n; \\ \pi, & m = n = 0, \\ \frac{\pi}{2}, & m = n \neq 0, \end{cases}$$

5. 提示: 利用 $\sin[(n+2)\theta] = 2\sin[(n+1)\theta] \cdot \cos\theta - \sin n\theta$;

6. $s_3^*(x) = a_1^* P_1(x) + a_3^* P_3(x) = 1.5531913x - 0.5622283x^3$, $\|\delta_3\|_2^2 = 0.000154$;

7. $s(x) = 0.9726046 + 0.0500351x^2$, $\|\delta\|^2 = 0.015$;

8. $s(x) = 22.25376t - 7.8550478$; 9. $y = F(t) = 5.2151034e^{-7.4961634\frac{1}{t}}$;

10. $P_1(x) = \frac{1}{2}[e - (e-1)\ln(e-1)] + (e-1)x$; 11. $r = -\frac{1}{2}$ 且唯一;

12. $P(x) = 5x^3 - \frac{5}{4}x^2 + \frac{1}{4}x - \frac{129}{128}$; 13. $P_3(x) = -0.208972306x^3 + 0.985674118x$;

14. $P_3(x) = 0.9773958x - 0.15625x^3$;

15. (1) $n, x_k = \cos \frac{2k+1}{2n}\pi, k=0, 1, \dots, n-1$; (2) $0, x - \frac{2}{3}$; (3) $\begin{cases} 0, & m \neq n, \\ \frac{2}{2n+1}, & n=m, \end{cases}$ (4) Legendre.

习 题 4

1. (1) $\int_{-h}^h f(x) dx \approx \frac{h}{3}f(-h) + \frac{4}{3}hf(0) + \frac{h}{3}f(h)$ 具有三次代数精度,

(2) $\int_{-1}^1 f(x) dx \approx \frac{1}{3}[f(-1) + 2f(-0.28990) + 3f(0.52660)]$ 具有二次代数精度;

2. $T = 1.8591409$, $|R(f)| \leq \frac{1}{12}e$, $S = 1.7188612$, $|R(f)| \leq \frac{e}{2880}$, $C = 1.718282688$, $|R(f)| \leq \frac{2e}{945 \times 4^6}$; 3. 提示: 将 $f(x)$ 分别在 $x = a, b, \frac{a+b}{2}$ 处 Taylor 展开, 再积分即可得到三种矩阵公式;

4. (1) $\int_0^1 f(x) dx \approx \frac{1}{3}[2f(\frac{1}{4}) - f(\frac{1}{2}) + 2f(\frac{3}{4})]$, (2) 3 次, (3) $\frac{1}{3}$;

5. (1) 取 $h = \frac{1}{8}$, 则 $I \approx T_8 = 0.9456911$, (2) 取 $h = \frac{1}{4}$, $|R_n(f)| \leq 0.271 \times 10^{-6}$;

(3) 取 7 个节点处的函数值;

6. $v(80) \approx 3087.03333 \text{ m/s}$; 7. $T_{16} \approx 3.140941612$; $S_4 = 3.141592503$; 8. 0.71327 ;

9. (1) 1.098630 , (2) 三点, 1.098039283 , 五点, 1.098609289 ;

(3) 复化, 1.098537573 , 真值 $I = 1.098612289$;

10. $\int_0^1 \frac{1}{\sqrt{x}} f(x) dx \approx (1 + \frac{1}{3}\sqrt{\frac{5}{6}})f(\frac{3}{7} - \frac{2}{7}\sqrt{\frac{6}{5}}) + (1 - \frac{1}{3}\sqrt{\frac{5}{6}})f(\frac{3}{7} + \frac{2}{7}\sqrt{\frac{6}{5}})$;

11. 0.2842485 ; 13. $\frac{\pi}{\sqrt{3}}$;

15. $\pi \approx 3.141580$; 16. $f'(1.0) \approx -0.25000$, $|R(1.0)| \approx 0.00250$, $f'(1.1) \approx -0.21694$,
 $|R(1.1)| \approx 0.00125$, $f'(1.2) \approx -0.18596$, $|R(1.2)| \approx 0.00250$;
 17. 利用 $f'(2) = \frac{1}{\sqrt{2+h} + \sqrt{2-h}}$ 重新计算; 18. (1) $n=75$; (2) n ; (3) $b-a$; (4) 0.

习 题 5

1. $y_{n+1} = 0.8y_n - 0.2x_n y_n^2$, $y(0.2) \approx y_1 = 0.8$, $y(0.4) \approx y_2 = 0.5888$, $y(0.6) \approx y_3 = 0.3895$;
 2. $y(1.1) \approx 0.523835$, $y(1.2) \approx 0.545499$, $y(1.3) \approx 0.565276$, $y(1.4) \approx 0.583403$, $y(1.5) \approx$
 0.600078; 3. $\begin{cases} y_{n+1}^{(0)} = 0.9x_n, \\ y_{n+1}^{(k+1)} = 0.95y_n - 0.05y_{n+1}^{(k)}, \end{cases} \quad y(0.2) \approx y_2 = y_2^{(5)} = 0.818594$;
 4. 提示: 考虑局部截断误差的阶; 7. $T_{n+1} = -\frac{1}{3}(b+1)h^3 y''(x_n) + O(h^4)$;
 8. (1) $[0, 0.002]$; (2) 无限制; (3) 无限制;
 9. $y_{n+1} = -4y_n + 5y_{n-1} + h(4f_n + 2f_{n-1})$, $\frac{1}{4!}h^4 y^{(4)}(x_n)$;
 10. $\begin{cases} \text{预估} & p_{n+1} = y_n + \frac{h}{12}(23f_n - 16f_{n-1} + 5f_{n-2}), \\ \text{改进} & m_{n+1} = p_{n+1} - \frac{9}{10}(p_n - c_n), \\ \text{计算} & m'_{n+1} = f(x_{n+1}, m_{n+1}), \\ \text{校正} & c_{n+1} = y_n + \frac{h}{12}(5m'_{n+1} + 8f_n - f_{n-1}), \\ \text{改进} & y_{n+1} = c_{n+1} + \frac{1}{10}(p_{n+1} - c_{n+1}). \end{cases}$

习 题 6

1. $\|A\|_\infty = 1.1$, $\|A\|_1 = 0.8$, $\|A\|_2 = 0.83$, $\|A\|_F = 0.8426$;
 6. $x_3 = x_2 = x_1 = 1$; 7. $x_3 = -1.4$, $x_2 = 2$, $x_1 = 1.2$.
 8. (1) A_2 的元素为 $a_{ij}^{(2)} = a_{ij}^{(1)} - \frac{a_{i1}^{(1)}}{a_{11}^{(1)}} a_{1j}^{(1)} = a_{ij}^{(1)} - \frac{a_{j1}^{(1)}}{a_{11}^{(1)}} a_{i1}^{(1)} = a_{ji}^{(2)}$, 故对称,
 (2) $X^* = (4.586035, -0.6315228, 2.735199)^T$;
 11. $\begin{bmatrix} 3 & 2 & 3 \\ 2 & 2 & 0 \\ 3 & 0 & 12 \end{bmatrix} = \begin{bmatrix} \sqrt{3} & & \\ 2/\sqrt{3} & \sqrt{2/3} & \\ \sqrt{3} & -\sqrt{6} & \sqrt{3} \end{bmatrix} \begin{bmatrix} \sqrt{3} & 2/\sqrt{3} & \sqrt{3} \\ & \sqrt{2/3} & -\sqrt{6} \\ & & \sqrt{3} \end{bmatrix}$, $x_3 = \frac{1}{3}$, $x_2 = \frac{1}{2}$, $x_1 = 1$;
 12. $y_1 = 1$, $y_2 = \frac{3}{2}$, $y_3 = 1$, $y_4 = -\frac{1}{4}$, 得 $x_4 = \frac{1}{5}$, $x_3 = -\frac{3}{5}$, $x_2 = -\frac{7}{5}$, $x_1 = -\frac{6}{5}$;
 14. $\text{cond}(A_2) = 39601$; 16. $\text{cond}(H)_\infty = 748$, $\frac{\|\delta X\|_\infty}{\|X\|_\infty} \approx 51.2\%$;

17. (1) 因系数矩阵是强对角占优阵, 故两种迭代均收敛, (2) $X^{(8)} = (-4.000036, 2.999985, 2.00003)^T$;

19. (1) Jacobi 迭代发散, G-S 迭代收敛; (2) Jacobi 迭代收敛, G-S 迭代发散.

20. 因 $\rho(B) = 0.9 < 1$, 故迭代收敛; 21. 不一定;

22. $|ab| < \frac{100}{3}$; 23. $\omega = 1.03, X^{(5)} = (0.5000043, 0.1000001, -0.4999999)^T$,

$\omega = 1, X^{(6)} = (0.5000038, 0.1000002, -0.4999995)^T$,

$\omega = 1.1, X^{(6)} = (0.5000035, 0.0999989, -0.5000003)^T$;

24. $0 < \omega < \frac{2}{\rho(A)}$; 25. 排列阵 $P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$.

习 题 7

1. $k=14$; 2. (1) 二分 10 次, (2) 迭代 5 次; 3. 两切点横坐标为 1.483;

5. (1) $|\varphi'(x)| \leq \frac{2}{3} < 1$, (2) $x_5 = 3.347$, (3) 线性收敛;

7. $p=q=\frac{5}{9}, r=-\frac{1}{9}$; 8. 1.557; 9. $x^* \approx x_5 = 1.368808108$;

10. $x_{k+1} = \frac{2}{3}x_k + \frac{a}{3x_k^2}, k=0, 1, \dots, \varphi'(\sqrt[3]{a})=0, \varphi''(\sqrt[3]{a}) \neq 0$, 故二阶收敛; 12. $x_5 = 1.368808108$.

习 题 8

2. (1) 取 $u_0 = (1, 1, 1)^T, \lambda_1 \approx 9.6058, X_1 \approx (1, 0.6056, -0.3945)^T$,

(2) 取 $u_0 = (1, 1, 1)^T, \lambda_1 \approx 8.86951, X_1 \approx (-0.60422, 1, 0.15094)^T$;

3. 特征值为 $\lambda_1 \approx 2.5365, \lambda_2 \approx -0.016647, \lambda_3 \approx 1.4802, R = P_1^T P_2^T P_3^T P_4^T P_5^T$ 的列向量为 A 的特征向量 (近似),

$$R \approx \begin{bmatrix} 0.53163 & -0.72110 & -0.44429 \\ 0.46133 & 0.68645 & -0.56211 \\ 0.71031 & 0.093870 & 0.69759 \end{bmatrix};$$

6. B 的模最大的特征值是 18.2576, 对应特征向量 $u_1 = (0.91995, 0.74452, 1)^T$, 从而 A 的按模最大的特征值是 13.2576, 对应的特征向量同 u_1 .

附录 数值实验程序

第 1 章

```
function t_charpt1_1
%数值试验 1.1 病态问题
%输入:[0 20]之间的扰动项及小的扰动常数
%输出:加扰动后得到的全部根
result=inputdlg('请输入扰动项:在[0 20]之间的整数:','charpt 1_1',1,{ '19' });
Numb=str2num(char(result));
if((Numb>20)|(Numb<0))
    errordlg('请输入正确的扰动项:[0 20]之间的整数!');
    return;
end
result=inputdlg('请输入(0 1)之间的扰动常数:','charpt 1_1',1,{ '0.00001' });
ess=str2num(char(result));
ve=zeros(1,21);
ve(21-Numb)=ess;
root=roots(poly(1:20)+ve);
disp(['对扰动项',num2str(Numb),'加扰动',num2str(ess),'得到的全部根为:']);
disp(num2str(root));
```

```
function t_charpt1_2
%数值试验 1.2:误差传播与算法稳定性
%输入:递推公式选择与递推步数
%输出:各步递推值及误差结果,以及递推值和误差与递推步数的关系图

prompts={'请选择递推关系式,若选 E. 1. 7,请输入 1,否则输入 2: '};
result=inputdlg(prompts,'charpt 1_2',1,{ '1' });
Nb=str2num(char(result));
if((Nb~=1)&(Nb~=2))
    errordlg('请选择递推关系式,若选 E. 1. 7,请输入 1,否则输入 2!');
    return;
end
```

```
result=inputdlg({'请输入递推步数 n:','charpt 1_2',1,'10'});
steps=str2num(char(result));
if(steps<1)
    errordlg('递推步数错误!');
    return;
end
result=inputdlg({'请输入计算中所采用的有效数字位数:','charpt 1_2',1,'5'});
Sd=str2num(char(result));
format long %设置显示精度
result=zeros(1,steps);%存储计算结果
err=result;%存储计算的绝对误差
func=result;%存储用库函数 quadl 计算出的积分的近似值

%用库函数 quadl 计算积分的近似值
for n=1:steps
    fun=@(x) x.^n.* exp(x-1);
    func(n)=quadl(fun,0,1);
end

if(Nb==1)
    %用算法 E. 1. 7 计算
    digits(Sd);%控制有效数字位数
    result(1)=subs(vpa(1/exp(1)));
    for n=2:steps
        result(n)=subs(vpa(1-n * result(n-1)));
    end
    err=abs(result-func);
elseif(Nb==2)
    %用算法 E. 1. 8 计算
    digits(Sd);%控制有效数字位数
    result(steps)=0;
    for n=steps:-1:2
        result(n-1)=subs(vpa((1-result(n))/n));
    end
    err=abs(result-func);
```



```

end

clf;%清除当前图像窗口
disp('递推值:');
disp(sprintf('%e ',result));
disp('误差:');
disp(sprintf('%e ',err));
plot([1:steps],result,'-');
grid on
hold on;
plot([1:steps],err,'r--');
xlabel('n');
ylabel('En-and ERR n-');
text(2,err(2),'\uparrow err(n)');
text(4,result(4),'\downarrow En');

```

第 2 章

```

function t_charpt2
%数值试验二:含“试验 2.1:多项式插值的震荡现象”和“试验 2.2:样条插值的收敛”
%输入:试验选择,函数式选择,插值节点数
%输出:拟合函数及原函数的图形
result=inputdlg({'请选择试验,若选 2.1,请输入 1,否则输入 2:','charpt_2',1,{'1'});
Nb=str2num(char(result));
if(Nb~=1)&(Nb~=2) errordlg('试验选择错误!');
    return;
end
prompts={'请选择试验函数,若选 f(x),请输入 f,若选 h(x),请输入 h,若选 g(x),请输入 g:'};
result=inputdlg(prompts,'charpt 2',1,{'f'});
Nb_f=char(result);
if(Nb_f~='f'&Nb_f~='h'&Nb_f~='g')
    errordlg('试验函数选择错误!');
    return;
end

```

```

result=inputdlg('请输入插值多项式的次数 N:', 'charpt_2', 1, {'10'});
Nd=str2num(char(result));
if(Nd<1)
    errordlg('插值多项式的次数输入错误!');
    return;
end
switch Nb_f
    case 'f'
        f=inline('1./(1+25*x.^2)'); a=-1; b=1;
    case 'h'
        f=inline('x./(1+x.^4)'); a=-5; b=5;
    case 'g'
        f=inline('atan(x)'); a=-5; b=5;
end
if(Nb==1)
    x0=linspace(a,b,Nd+1); y0=feval(f,x0);
    x=a:0.1:b; y=Lagrange(x0,y0,x);
    clf;
    fplot(f,[a b], 'co');
    hold on;
    plot(x,y, 'b-');
    xlabel('x'); ylabel('y=f(x) o and y=Ln(x)-');
elseif(Nb==2)
    x0=linspace(a,b,Nd+1); y0=feval(f,x0);
    x=a:0.1:b;
    cs=spline(x0,y0); y=ppval(cs,x);
    clf;
    fplot(f,[a b], 'co');
    hold on;
    plot(x,y, 'k-');
    xlabel('x'); ylabel('y=f(x) o and y=Spline(x)-');
end
%-----
function y=Lagrange(x0,y0,x)
%Lagrange 插值

```

```

n=length(x0);
m=length(x);
for i=1:m
    z=x(i);
    s=0.0;
    for k=1:n
        p=1.0;
        for j=1:n
            if(j~=k)
                p=p*(z-x0(j))/(x0(k)-x0(j));
            end
        end
        s=s+p*y0(k);
    end
    y(i)=s;
end
End

```

%

实验 2.2 样条插值的收敛——实验要求(2)

%用 3 次样条插值设计车门的曲线

```

x0=0:10;
y0=[0.8,0.0,0.79,1.53,2.19,2.71,3.03,3.27,2.89,3.06,3.19,3.29,0.2];
x=0:0.1:10;
cs=spline(x0,y0);
y=ppval(cs,x);
plot(x0,y0(2:12),'o');
hold on;
plot(x,y,'k-');
xlabel('x');
ylabel('y=f(x) o and y=Spline(x) -');

```

第 3 章

function charpt3

%数值实验三:含“实验 3.1”和“实验 3.2”

%子函数调用:dlsa

```

%输入:实验选择
%输出:原函数及求得的相应插值多项式的函数的图像以及参数 alph 和误差 r
result=inputdlg('请选择实验,若选 3.1,请输入 1,否则输入 2:','charpt_3',1,{'1'});
Nb=str2num(char(result));
if(Nb~=1)&(Nb~=2) error('实验选择错误!');
    return;
end
x0=-1:0.5:2;
y0=[-4.447 -0.452 0.551 0.048 -0.447 0.549 4.552];
n=3;%n 为拟合阶次
if(Nb==1)
    alph=polyfit(x0,y0,n);
    y=polyval(alph,x0);
    r=(y0-y)*(y0-y)';%平方误差
    x=-1:0.01:2;
    y=polyval(alph,x);
    plot(x,y,'k-');
    xlabel('x');ylabel('y0 * and polyfit. y-');
    hold on;
    plot(x0,y0,'*');
    title('离散数据的多项式拟合');
    grid on;
else
    result=inputdlg('请输入权向量 w:','charpt_3',1,{'[1 1 1 1 1 1 1]'});
    w=str2num(char(result));
    [a,b,c,alph,r]=dlsa(x0,y0,w,n);
end
disp(['平方误差:',sprintf('%g',r)]);
disp(['参数 alph:',sprintf('%g\t',alph)])
%-----
function [a,b,c,alph,r]=dlsa(x,y,w,n)
%功能:用正交化方法对离散数据作多项式最小二乘拟合。
%输入:m+1 个离散点(x,y,w),x,y,w 分别用行向量给出。
%    拟合多项式的次数 n,0<n<m.

```

```

%输出:三项递推公式的参数 a,b,拟合多项式 s(x)的系数 c 和 alph,
%    平方误差 r=(y-s,y-s),并作离散点列和拟合曲线的图形
m=length(x)-1;
if(n<1|n>=m)
    error('错误:n<1 或者 n>=m!');
    return;
end
%求三项递推公式的参数 a,b,拟合多项式 s(x)的系数 c,其中 d(k)=(y,sk);
s1=0; s2=ones(1,m+1); v2=sum(w);
d(1)=y * w'; c(1)=d(1)/v2;
for k=1:n
    xs=x. * s2.^2 * w'; a(k)=xs/v2;
    if(k==1)
        b(k)=0;
    else
        b(k)=v2/v1;
    end
    s3=(x-a(k)). * s2-b(k) * s1;
    v3=s3.^2 * w';
    d(k+1)=y. * s3 * w'; c(k+1)=d(k+1)/v3;
    s1=s2; s2=s3; v1=v2; v2=v3;
end
%求平方误差 r
r=y. * y * w'-c * d';
%,求拟合多项式 s(x)的降幂系数 alph
alph=zeros(1,n+1); T=zeros(n+1,n+2);
T(:,2)=ones(n+1,1); T(2,3)=-a(1);
if(n>=2)
    for k=3:n+1
        for i=3:k+1
            T(k,i)=T(k-1,i)-a(k-1) * T(k-1,i-1)-b(k-1) * T(k-2,i-2);
        end
    end
end
for i=1:n+1

```

```

    for k=i:n+1
        alph(n+2-i)=alph(n+2-i)+c(k)*T(k,k+2-i);
    end
end
%用秦九韶方法计算 s(t)的输出序列(t,s)
xmin=min(x); xmax=max(x); dx=(xmax-xmin)/(25*m);
t=(xmin-dx):dx:(xmax+dx);
s=alph(1);
for k=2:n+1
    s=s.*t+alph(k);
end
%输出点列 x-y 和拟合曲线 t-s 的图形
plot(x,y,'* ',t,s,'-');
title('离散数据的多项式拟合');
xlabel('x'); ylabel('y');
grid on;

```

第 4 章

```
function t_charpt4
```

```

%数值实验四:含“实验 4.1:复化求积公式计算定积分”和“实验 4.2:比较复化
%Simpson方法与变步长 Simpson 方法”

```

```

result=inputdlg({'请选择实验,若选 4.1,请输入 1,否则输入 2:','charpt 4',1,{'1'}));

```

```

Nt=str2num(char(result));

```

```

if(Nt~=1)&(Nt~=2)

```

```

    errordlg('实验选择错误!');

```

```

    return;

```

```

end

```

```

if(Nt==1)

```

```

    prompts={'请选择积分公式,若用复化梯形,输入 T,用复化 Simpson,输入 S,用复化 Gauss_Legendre,输入 GL: '};

```

```

    result=inputdlg(prompts,'charpt 4',1,{'T'});

```

```

    Nb=char(result);

```

```

    if(Nb~='T' & Nb~='S' & Nb~='GL')

```

```
        errordlg('积分公式选择错误!');
        return;
    end
    result=inputdlg({'请输入积分式题号 1-4:','实验 4.1',1,{'1'}});
    Nb_f=str2num(char(result));
    if(Nb_f<1 | Nb_f>4)
        errordlg('没有该积分式!');
        return;
    end
    switch Nb_f
    case 1
        fun=inline('-2./(x.^2-1)');a=2;b=3;
    case 2
        fun=inline('4./(x.^2+1)');a=0;b=1;
    case 3
        fun=inline('3.^x');a=0;b=1;
    case 4
        fun=inline('x.*exp(x)');a=1;b=2;
    end
    tol=0.5e-7;
    if(Nb=='T')%用复化梯形公式
        tic;
        t=(fun(a)+fun(b))*(b-a)/2;
        k=1;t0=0;
        while (abs(t-t0)>=tol*3)
            t0=t; h=(b-a)/2^k;
            t=t0/2+h*sum(fun(a+h;2*h;b-h));
            k=k+1;
        end
        time=toc;
    elseif(Nb=='S')%用复化 Simpson 公式
        %先根据复化 Simpson 公式的余项估计步长
        prompts={'请输入用复化 Simpson 公式应取的步长:'};
        result=inputdlg(prompts,'实验 4.2',1,{'0.01'});
        h=str2num(char(result));
```

```

    if(h<=0)
        errordlg('请输入正确的步长!');
        return;
    end
    tic;
    N=floor((b-a)/h);
    detsum=0;
    for i=1:N-1
        xk=a+i*h;
        detsum=detsum+fun(xk);
    end
    t=h*(fun(a)+fun(b)+2*detsum)/2;
    time=toc;
elseif(Nb=='GL')%用复化 Gauss_Legendre I
    %先根据复化 Gauss_Legendre I 公式的余项估计步长
    prompts={'请输入用复化 Gauss_Legendre I 公式应取的步长:'};
    result=inputdlg(prompts,'实验 4.2',1,{ '0.01' });
    h=str2num(char(result));
    if(h<=0)
        errordlg('请输入正确的步长!');
        return;
    end
    tic;
    N=floor((b-a)/h); t=0;
    for k=0:N-1
        xk=a+k*h+h/2;
        t=t+fun(xk-h/(2*sqrt(3)))+fun(xk+h/(2*sqrt(3)));
    end
    t=t*h/2;
    time=toc;
end
disp(['实验 4.1 题(',num2str(Nb_f),')的计算结果:',num2str(t)]);
switch Nb_f
case 1
    disp('精确解:ln2-ln3=-0.4054651081')

```



```

disp(['绝对误差:', num2str(abs(t+0.4054651081))]);
disp(['运行时间:', num2str(time)]);
case 2
disp('精确解:  $\pi=3.14159265358979$ ')
disp(['绝对误差:', num2str(abs(t-pi))]);
disp(['运行时间:', num2str(time)]);
case 3
disp('精确解:  $2/\ln 3=1.82047845325368$ ')
disp(['绝对误差:', num2str(abs(t-1.82047845325368))]);
disp(['运行时间:', num2str(time)]);
case 4
disp('精确解:  $e^2=7.38905609893065$ ')
disp(['绝对误差:', num2str(abs(t-7.38905609893065))]);
disp(['运行时间:', num2str(time)]);
end
elseif(Nt==2)
prompts={'请选择积分公式,若用复化 Simpson,输入 S,用变步长 Simpson,输入 VS: '};
result=inputdlg(prompts, 'charpt 4', 1, {'T'});
Nb=char(result);
if(Nb~='S' & Nb~='VS')
errordlg('积分公式选择错误!');
return;
end
result=inputdlg({'请输入积分式题号 1-3: '}, '实验 4.2', 1, {'1'});
Nb_f=str2num(char(result));
if(Nb_f<1 | Nb_f>3)
errordlg('没有该积分式!');
return;
end
switch Nb_f
case 1
fun=inline('x.^6/10-x^2+x'); a=0; b=2;
case 2
fun=inline('x.*sqrt(x)'); a=0; b=1;

```

```
case 3
    fun=inline('1./sqrt(x)'); a=5;b=200;
end
tol=0.5e-7;
if(Nb=='S')%用复化 Simpson 公式
    %先根据复化 Simpson 公式的余项估计步长
    prompts={'请输入用复化 Simpson 公式应取的步长:'};
    result=inputdlg(prompts,'实验 4.2',1,{ '0.01' });
    h=str2num(char(result));
    if(h<=0)
        errordlg('请输入正确的步长!');
        return;
    end
    tic;
    N=floor((b-a)/h);
    detsum=0;
    for i=1:N-1
        xk=a+i*h;
        detsum=detsum+fun(xk);
    end
    t=h*(fun(a)+fun(b)+2*detsum)/2;
    n=2*N+1;%所需节点数
    time=toc;
elseif(Nb=='VS')%用变步长 Simpson
    tic;
    [t,iter]=quad(fun,a,b,tol);
    n=2*iter+1;%所需节点数
    time=toc;
end
disp(['实验 4.2 题(' ,num2str(Nb_f),')的计算结果 :']);
if(Nb=='S')
    disp('用复化 Simpson 公式计算:');
    disp(['所需的节点数:',num2str(n)]);
    disp(['求得的近似值:',num2str(t)]);
    disp(['运行时间:',num2str(time)]);
```

```

elseif(Nb=='VS')
    disp('用变步长 Simpson 公式计算:');
    disp(['所需的节点数:',num2str(n)]);
    disp(['求得的近似值:',num2str(t)]);
    disp(['运行时间:',num2str(time)]);
end
end

```

第 5 章

```

function charpt5_1
%数值实验 5.1:常微分方程性态和 R-K 法稳定性实验
%输入:参数 a,步长 h
%输出:精确解和数值解图形对比
clf;
result=inputdlg({'请输入[-50 50]间的参数 a:'},'实验 5.1',1,{'-40'});
a=str2num(char(result));
if((a<-50)|(a>50))
    errordlg('请输入正确的参数 a!');
    return;
end
result=inputdlg({'请输入(0 1)之间的步长:'},'实验 5.1',1,{'0.01'});
h=str2num(char(result));
if((h>=1)|(h<=0))
    errordlg('请输入正确的(0 1)间的步长!');
    return;
end
x=0:h:1;
y=x;
N=length(x);
y(1)=1;
func=inline('1+(y-x). * a');
for n=1:N-1
    k1=func(a,x(n),y(n));
    k2=func(a,x(n)+h/2,y(n)+k1 * h/2);
    k3=func(a,x(n)+h/2,y(n)+k2 * h/2);

```

```

    k4=func(a,x(n)+h,y(n)+k3*h);
    y(n+1)=y(n)+h*(k1+2*k2+2*k3+k4)/6;
end
y0=exp(a*x)+x;
plot(x,y0,'g+');
hold on;
plot(x,y,'b--');
xlabel('x');
ylabel('y0=exp(ax)+x: + and R-K(x)--');

```

```
function charpt5_2
```

```
%数值实验 5.2 及 5.3: 比较 4 阶 Adams 预测-校正法与经典 R-K 法的计算精度及
%耗费时间
```

```
%子函数调用: R_K44(经典 R-K 法)
```

```
%输入: 步长 h、方程解法选择
```

```
%输出: 精确解和数值解图形对比
```

```
result=inputdlg({'请输入(0 1]之间的步长:'},'charpt5_2',1,{ '0.2'});
```

```
h=str2num(char(result));
```

```
if((h>1)|(h<=0))
```

```
    errordlg('请输入正确的(0 1]间的步长!');
```

```
    return;
```

```
end
```

```
result=inputdlg({'若选 R-K 法,请输入 1,否则输入 2:'},'charpt5_2',1,{ '1'});
```

```
Nb=str2num(char(result));
```

```
if((Nb~=1)&(Nb~=2))
```

```
    errordlg('选择错误!');
```

```
    return;
```

```
end
```

```
func=inline('-y+cos(2*x)-2*sin(2*x)+2*x.*exp(-x)');
```

```
tic;
```

```
[x,y1]=R_K44(func,0,2,1,h);    %经典 R-K 法计算
```

```
time=toc;
```

```
N=length(x);
```

```
if(Nb==2)
```

```

%4 阶 Adams 预测-校正法计算
tic;
y=x;
f=x;
y(1)=y1(1);y(2)=y1(2);
y(3)=y1(3);y(4)=y1(4);
ynp=0;
ync=0;
f(1)=func(x(1),y(1));
f(2)=func(x(2),y(2));
f(3)=func(x(3),y(3));
for n=4:N-1
    f(n)=func(x(n),y(n));
    ynlp=y(n)+h/24*(55*f(n)-59*f(n-1)+37*f(n-2)-9*f(n-3));
    ynlpm=ynlp+251/270*(ync-ynp);
    ynlc=y(n)+h/24*(9*func(x(n+1),ynlpm)+19*f(n)-5*f(n-1)+
f(n-2));
    y(n+1)=ynlc-19/270*(ynlc-ynlp);
    ync=ynlc;
    ynp=ynlp;
end
time=toc;
else
    x=x;
    y=y1;
end
y0=x.^2.*exp(-x)+cos(2*x);
plot(x,y0,'g+');
hold on;
plot(x,y,'b-');
xlabel('x');
ylabel('y0=x.^2.*exp(-x)+cos(2*x);+ and Adams(x)——');
d=floor(length(x)/10);
disp(['十个等距节点:',num2str(x(1:d:N))]);
disp(['对应的计算值:',num2str(y(1:d:N))]);

```

```

disp(['对应的精确值:',num2str(y0(1;d:N))]);
disp(['CPU 时间:',num2str(time)]);
%-----
function [x,y]=R_K44(func,bt,up,y0,h)
%经典 R-K 法
%输入:func 为 f(x),up、bt 分别为 x 取值上限、下限,y0 为初值 y(0),h 为步长
%输出:x,y 分别为 x,y=f(x)的计算值
x=bt:h:up;
y=zeros(size(x));
y(1)=y0;
for i=1:length(x)-1
    k1=func(x(i),y(i));
    k2=func(x(i)+h/2,y(i)+h/2*k1);
    k3=func(x(i)+h/2,y(i)+h/2*k2);
    k4=func(x(i)+h,y(i)+h*k3);
    y(i+1)=y(i)+h*(k1+2*(k2+k3)+k4)/6;
end

```

第 6 章

```

function t_charpt6
%数值实验六:含“主元选取与算法的稳定性”和“线性代数方程组的性态估计与条件数
%估计”
%输入与输出:图形界面交互式输入输出
%子函数调用:func_on_ok,on_approach,matr_Callback,clear_Callback
%分别为:“ok”按钮响应函数、算法选择响应函数、获取矩阵输入、‘clear’按钮响应函
%数
global h0 h_matr h_msize h_vect h_appr h_main_e h_err h_sw h_x0 h_close h_clear h
_result
h0=figure(1);
set(h0,'resize','off');
uicontrol(h0,'style','text','string','矩阵:', 'position',[30 360 60 30], 'FontSize',
10);
h_matr=uicontrol(h0,'style','popupmenu','string',{'矩阵 A','矩阵 H','其他'},
'position',[90 360 90 30], 'FontSize',10);
set(h_matr,'BackgroundColor',[1 1 1], 'callback','matr_Callback');

```

```
uicontrol(h0,'style','text','string','矩阵大小:', 'position',[250 360 100 30], 'FontSize',10);
h_msize=uicontrol(h0,'style','edit','string','10', 'position',[350 360 100 30], 'FontSize',10, 'BackgroundColor',[1 1 1]);
uicontrol(h0,'style','text','string','向量 b:', 'position',[30 310 60 30], 'FontSize',10);
h_vect=uicontrol(h0,'style','edit','string','[7 15 15 15 15 15 15 15 15 14]', 'position',[90 310 360 30]);
set(h_vect,'FontSize',8, 'BackgroundColor',[1 1 1]);
uicontrol(h0,'style','text','string','解法:', 'position',[30 260 60 30], 'FontSize',10);
h_appr=uicontrol(h0,'style','popupmenu','string',{'Gauss 消元法','J 迭代法','GS 迭代法','SOR 迭代法'}, 'position',[90 260 110 30], 'FontSize',10, 'BackgroundColor',[1 1 1], 'callback','on_approach');
uicontrol(h0,'style','text','string','主元选择:', 'position',[250 260 100 30], 'FontSize',10);
h_main_e=uicontrol(h0,'style','popupmenu','string',{'不选主元','自动选主元','手动选主元'}, 'position',[350 260 100 30], 'FontSize',10, 'BackgroundColor',[1 1 1]);
uicontrol(h0,'style','text','string','误差限:', 'position',[30 210 60 30], 'FontSize',10);
h_err=uicontrol(h0,'style','edit','string','1.0e-6', 'position',[90 210 90 30], 'FontSize',10, 'BackgroundColor',[0.83 0.81 0.78], 'enable','off');
uicontrol(h0,'style','text','string','松弛因子 w:', 'position',[250 210 100 30], 'FontSize',10);
h_sw=uicontrol(h0,'style','edit','string','1.25', 'position',[350 210 100 30], 'FontSize',10, 'BackgroundColor',[0.83 0.81 0.78], 'enable','off');
uicontrol(h0,'style','text','string','初值:', 'position',[30 160 60 30], 'FontSize',10);
h_x0=uicontrol(h0,'style','edit','position',[90 160 360 30], 'FontSize',10, 'BackgroundColor',[0.83 0.81 0.78], 'enable','off');
uicontrol(h0,'style','text','string','方程解:', 'position',[30 110 60 30], 'FontSize',10);
h_result=uicontrol(h0,'style','text','position',[90 50 365 90], 'FontSize',8, 'BackgroundColor',[0.6 0.7 0.6], 'tag','result');
```

```

h_ok=icontrol(h0,'style','pushbutton','position',[110 20 60 30],'FontSize',15,
'string','ok','callback','func_on_ok');
h_clear=icontrol(h0,'style','pushbutton','position',[220 20 60 30],'FontSize',
15,'string','clear','callback','clear_Callback');
h_close=icontrol(h0,'style','pushbutton','position',[340 20 60 30],'FontSize',
15,'string','close','callback','close');
%-----
function clear_Callback
%复位按钮响应函数
global h_result h_main_e h_err h_sw
set(h_result,'string','');
set(h_main_e,'enable','on');
set(h_err,'enable','off');
set(h_sw,'enable','off');
%-----
function on_approach
%算法选择响应函数
global h_main_e h_err h_sw h_x0
N=get(gcf,'value');
if(N==1)
    set(h_main_e,'enable','on');
    set(h_err,'enable','off','backgroundcolor',[0.83 0.81 0.78]);
    set(h_sw,'enable','off','backgroundcolor',[0.83 0.81 0.78]);
    set(h_x0,'enable','off','backgroundcolor',[0.83 0.81 0.78]);
else
    set(h_err,'enable','on','backgroundcolor',[1 1 1]);
    set(h_sw,'enable','on','backgroundcolor',[1 1 1]);
    set(h_x0,'enable','on','backgroundcolor',[1 1 1]);
    set(h_main_e,'enable','off');
end
%-----
function matr_Callback
%获取矩阵输入
N=get(gcf,'value');
global B h_msize h_vect

```



```

if(N==1)
    row=str2num(get(h_msize,'string'));
    b=[7 ones(1,row-2) * 15 14];
    set(h_vect,'string',num2str(b));
elseif(N==3)
    Ans=inputdlg('请输入矩阵 A:', 'Charpt6', 2);
    B=str2num(Ans{1});
end
%-----
function func_on_ok
%“ok”按钮响应函数,总执行函数
%子函数调用:mymax、zhuyuanxy1、zhuyuanxy、jacobi、g_seidel、sor
global h_matr h_msize h_vect h_appr h_main_e h_err h_sw h_x0 h_result
row=str2num(get(h_msize,'string'));
switch get(h_matr,'value')
    case 1
        a1=sparse(1:row,1:row,6);
        a2=sparse(1:row-1,2:row,1,row,row);
        A=a1+a2+8 * a2';
    case 2
        A=hilb(row);
    case 3
        global B;
        A=B;
end
b=str2num(get(h_vect,'string'));
N=get(h_appr,'value');
X=zeros(row,1);
if(N==1)
    Nb=get(h_main_e,'value');
    B=zhuyuanxy([A b'],Nb);
    [row col]=size(B);
    X(row)=B(row,col)/B(row,row);
    for k=(row-1):-1:1
        X(k)=(B(k,col)-B(k,k+1:row) * (X(k+1:row)))/B(k,k);
    end
end

```

```
end
else
    tol=str2num(get(h_err,'string'));
    x0=str2num(get(h_x0,'string'));
    if isempty(x0)
        errordlg('请输入初值 x0!');
        return;
    end
    if(N==2)
        X=jacobi(A,b,x0,tol);
    elseif(N==3)
        X=g_seidel(A,b,x0,tol);
    elseif(N==4)
        w=str2num(get(h_sw,'string'));
        X=sor(A,b,w,x0,tol);
    end
end
[r c]=size(X);
if(r~=1)
    X=X';
end
set(h_result,'string',num2str(X));
%-----
function [m,l]=mymax(x)
%获取数列 x 中绝对值最大的值及下标
m=x(1);
l=1;
n=length(x);
for i=2:n
    if(abs(x(i))>abs(m))
        m=x(i);
        l=i;
    end
end
end
%-----
```

```

function B=zhuyuanxyl(A,Nb)
%主元选取
[row col]=size(A);
if(Nb==2)
    [m,L]=mymax(A(1:row));
elseif(Nb==1)
    m=A(1);
    L=1;
elseif(Nb==3)
    A(1:row);
    Ans=inputdlg('请输入最大主元所在的列:', 'Charpt6', 1);
    L=str2num(char(Ans));
    m=A(L);
end
if(m==0)
    return;
end
B=A;
B(1,:)=A(L,:);
B(L,:)=A(1,:);
for x=2:row
    B(x,:)=B(x,:)-B(1,:)*B(x,1)/m;
end
%-----
function B=zhuyuanxy(A,Nb)
%主元选取变换
[row col]=size(A);
B=zeros(row,col);
for i=1:row-1
    B(i:row,i:col)=zhuyuanxyl(A,Nb);
    A=B(i+1:row,i+1:col);
end
%-----
function y=jacobi(A,b,x0,tol)
%jacobi 迭代法

```

```
[r c]=size(b);
if(r==1)
    b=b';
end
[r c]=size(x0);
if(r==1)
    x0=x0';
end
clear r c
if(isempty(tol)|tol==0)
    tol=1.0e-6;
end
D=diag(diag(A));
U=triu(A,1);
L=triu(A',1)';
B=-D\'(L+U);
f=D\'b;
y=B * x0+f;
n=1;
while norm(y-x0)>=tol
    x0=y;
    y=B * x0+f;
    n=n+1;
end
%-----
function y=g_seidel(A,b,x0,tol)
%gauss-seidel 迭代
[r c]=size(b);
if(r==1)
    b=b';
end
[r c]=size(x0);
if(r==1)
    x0=x0';
end
```

```

clear r c
if(isempty(tol) | tol == 0)
    tol = 1.0e-6;
end
D = diag(diag(A));
U = triu(A, 1);
L = triu(A', 1)';
G = -(D + L) \ U;
f = (D + L) \ b;
y = G * x0 + f;
n = 1;
while norm(y - x0) >= tol
    x0 = y;
    y = G * x0 + f;
    n = n + 1;
end
%-----
function y = sor(A, b, w, x0, tol)
%超松弛迭代
[r c] = size(b);
if(r == 1)
    b = b';
end
[r c] = size(x0);
if(r == 1)
    x0 = x0';
end
clear r c
if(isempty(tol) | tol == 0)
    tol = 1.0e-6;
end
D = diag(diag(A));
U = triu(A, 1);
L = triu(A', 1)';
lw = (D + w * L) \ ((1 - w) * D - w * U);

```

```

f=w*(D+w*L)\b;
y=lw*x0+f;
n=1;
while norm(y-x0)>=tol
    x0=y;
    y=lw*x0+f;
    n=n+1;
end

```

第 7 章

```

function t_charpt7_1
%数值实验 7.1:迭代法、初始值与收敛性
%输入:迭代式、迭代步数、迭代初值
%输出:迭代结果及图形显示
result=inputdlg({'请选择迭代式:1、2、3 分别对应式 E. 7. 1、E. 7. 2、E. 7. 3'},'charpt7_1',1,{ '1' });
Nb=str2num(char(result));
result=inputdlg({'请输入迭代步数:'},'charpt7_1',1,{ '10' });
N=str2num(char(result));
result=inputdlg({'请输入迭代初值:'},'charpt7_1',1,{ '0' });
x0=str2num(char(result));
if isempty(N) | N==0
    errordlg('请输入步数!');
    return;
end
if isempty(x0)
    errordlg('请输入初值 x0!');
    return;
end
format long;
x=iteration(x0,N,Nb);
disp('迭代结果:');
disp(sprintf('%g ',x));
plot([1:N+1],x);
xlabel('迭代步数 n');

```

```

ylabel('迭代值 x(n)');
%-----
function x=iteration(x0,N,Nb)
x=zeros(1,N+1);
x(1)=x0;
switch Nb
    case 1
        for i=1:N
            x(i+1)=x(i)^2-1;
        end
    case 2
        for i=1:N
            x(i+1)=1+1/x(i);
        end
    case 3
        for i=1:N
            x(i+1)=sqrt(x(i)+1);
        end
end

function t_charpt7_2
%数值实验 7.2:算法设计与比较
%输入:选择方程,选择算法,所需精度,迭代初值
%输出:方程组的解,迭代次数,CPU 时间
result=inputdlg({'请选择方程:1 对应 E. 7. 4,2 对应 E. 7. 5'},'charpt7_2',1,{ '1' });
Nb=str2num(char(result));
result=inputdlg({'请选择方法:1、2 分别对应牛顿法和拟牛顿法'},'charpt7_2',1,
{ '1' });
Nt=str2num(char(result));
result=inputdlg({'请输入要求达到的精度:'},'实验 7. 2',1,{ '1e-6' });
tol=str2num(char(result));
result=inputdlg({'请输入迭代初值:'},'实验 7. 2',1,{ '[1;1;1]' });
x0=str2num(char(result));
if(isempty(tol))

```

```

    errordlg('请输入精度!');
    return;
end
if(Nt==1)
    [x,n,T]=newtonpro(tol,Nb,x0);
else
    [x,n,T]=broyden1(tol,Nb,x0);
end
if(Nt==1)
    disp(['采用牛顿法对方程组 E. 7. ',num2str(Nb+3),'得到的实验结果为:']);
else
    disp(['采用拟牛顿法对方程组 E. 7. ',num2str(Nb+3),'得到的实验结果为:']);
end
format long;
disp(['CPU 时间:',num2str(T)]);
disp(['方程组解:',num2str(x)]);
disp(['迭代次数:',num2str(n)]);
disp(['精度:',num2str(tol)]);
%-----
function [x,n,T]=broyden1(tol,Nb,x0)
tic;
x=x0;
B=inv(df(x0,Nb)');
x1=x-B*fc(x,Nb);
s=x1-x;
n=1;
y=fc(x1,Nb)-fc(x,Nb);
B1=B+(s-B*y)*s'*B/(s'*B*y);
while(norm(s)>=tol&(n<100000000))
    x=x1;
    B=B1;
    x1=x-B*fc(x,Nb);
    B1=B+(s-B*y)*s'*B/(s'*B*y);
    s=x1-x;
    y=fc(x1,Nb)-fc(x,Nb);
end

```



```

        n=n+1;
    end
    x=x1';
    T=toc;
    %-----
function [x,n,T]=newtonpro(tol,Nb,x0)
tic;
x1=x0-inv(df(x0,Nb))*fc(x0,Nb);
n=1;
while((norm(x1-x0)>=tol)&(n<100000000))
    x0=x1;
    x1=x0-inv(df(x0,Nb))*fc(x0,Nb);
    n=n+1;
end
x=x1';
T=toc;
%-----
function y=fc(x,Nb)
if(Nb==1)
    y1=12*x(1)-x(2)^2-4*x(3)-7;
    y2=x(1)^2+10*x(2)-x(3)-11;
    y3=x(2)^3+10*x(3)-8;
    y=[y1 y2 y3]';
else
    y1=3*x(1)-cos(x(2)*x(3))-0.5;
    y2=x(1)^2-81*(x(2)+0.1)^2+sin(x(3))+1.06;
    y3=exp(1)-x(1)*x(2)+20*x(3)+(10*pi-3)/3;
    y=[y1 y2 y3]';
end
%-----
function y=df(x,Nb)
if(Nb==1)
    y=[12 -2*x(2) -4;2*x(1) 10 -1;0 3*x(2)^2 10];
else
    y=[3 sin(x(2)*x(3))*x(3) sin(x(2)*x(3))*x(2);...
```

```

2 * x(1) - 2 * 81 * (x(2) + 0.1) cos(x(3)); ...
-x(2) - x(1) 20];

```

```
End
```

第 8 章

```
function charpt8_1
```

```
%数值实验 8.1:矩阵特征值问题条件数的估计
```

```
%用 matlab 中提供的函数“condeig”估计矩阵特征值问题的条件数
```

```
%结果在命令窗口显示
```

```
Nt=inputdlg({'矩阵输入:输入 R,M,H,分别表示随机生成,手动输入,Hilbert 矩  
阵'},'charpt8_1',1,{'R'});
```

```
Nt=char(Nt);
```

```
switch Nt
```

```
case 'R'
```

```
result=inputdlg({'输入矩阵行数或列数'},'charpt8_1',1,{'3'});
```

```
row=str2num(char(result));
```

```
A=rand(row,row);
```

```
case 'M'
```

```
result=inputdlg({'请输入矩阵 A:','charpt8_1',2); %格式如[1 2 3;12 3  
3;3 54 6]
```

```
A=str2num(result{1});
```

```
case 'H'
```

```
result=inputdlg({'输入 Hilbert 矩阵行数或列数'},'charpt8_1',1,{'3'});
```

```
row=str2num(char(result));
```

```
A=hilb(row);
```

```
otherwise
```

```
errordlg('输入错误!');
```

```
return;
```

```
end
```

```
disp('矩阵 A: ');
```

```
disp(num2str(A));
```

```
disp('矩阵 A 的特征值问题条件数: ');
```

```
disp(num2str(condeig(A)));
```

```
function charpt8_2
%数值实验 8.2:QR 算法的模拟及实现
%结果在命令窗口显示
H=[2 3 4 5 6;4 4 5 6 7;0 3 6 7 8;0 0 2 8 9;0 0 0 1 0];
B=[6 2 1;2 3 1;1 1 1];
D=eig(H)';
disp('函数 eig 求得的矩阵 H 的特征值为: ');
disp(num2str(D));
disp('利用函数 qr 实现原理型 QR 算法求得的矩阵 H 的特征值及迭代次数为: ');
[iter,D]=qralg(H);
disp(['迭代步数:',num2str(iter)]);
disp(['特征值:',num2str(D)]);
D=eig(B)';
disp('函数 eig 求得的矩阵 B 的特征值为: ');
disp(num2str(D));
[iter,D]=qralg(B);
disp('利用函数 qr 实现原理型 QR 算法求得的矩阵 B 的特征值及迭代步数为: ');
disp(['迭代步数:',num2str(iter)]);
disp(['特征值:',num2str(D)]);
%-----
function [iter,D]=qralg(A)
%功能:用基本 QR 算法求实方阵的全部特征值
%输入:n 阶实方阵 A
%输出:迭代次数 iter,A 的全部特征值 D

%初始化
ep=0.5e-4;
[n,n]=size(A);
D=zeros(1,n);
i=n;
m=n;
iter=0;
%化矩阵 A 为 Hessenberg 型
A=hessen(A);
%用基本 QR 变换进行迭代
```

```

while(n>0)
    if(m<=2)
        la=eig(A(1:m,1:m));
        D(1:m)=la';
        break;
    end
    iter=iter+1;
    A=qrtran(m,A);
    for k=m-1:-1:1
        if(abs(A(k+1,k))<ep)
            if(m-k<=2)
                la=eig(A(k+1:m,k+1:m));
                j=i-m+k+1;
                D(j:i)=la';
                i=j-1;
                m=k;
                break;
            end
        end
    end
end
end
%-----
function A=hessen(A)
%功能:用 Householder 变换化矩阵 A 为上 Hessenberg 形
%输入:n 阶实方阵 A
%输出:A 的 Hessenberg 形
[n,n]=size(A);
for k=1:n-2
    x=A(k+1:n,k);
    H=househ(x);
    A(k+1:n,k:n)=H * A(k+1:n,k:n);
    A(1:n,k+1:n)=A(1:n,k+1:n) * H;
end
%-----
function H=househ(x)

```

```

%功能:对于向量 x,构造 Householder 变换矩阵 H,
%      使得  $Hx = (*, 0, \dots, 0)'$ , 其中  $|*| = \text{norm}(x, 2)$ 
%输入:列向量 x
%输出:Householder 变换矩阵 H
n=length(x);
I=eye(n);
sn=sign(x(1));
if(sn==0)
    sn=1;
end
z=x(2:n);
if(norm(z,inf)==0)
    H=I;
    return;
end
sigma=-sn * norm(x);
u=x;
u(1)=u(1)-sigma;
lo=sigma * (sigma-x(1));
H=I-u * u' / lo;
%-----
function A=qrtran(m,A)
%功能:对 A 的左上角的 m 阶对角块作 QR 变换:
%      先用 Givens 变换作 QR 分解  $A=QR$ , 再作相似变换  $A:=Q'AQ=RQ$ 
%输入:n 阶 Hessenberg 形矩阵 A, 其中  $A(m+1,m)=0, m>2$ 
%输出:变换后的 Hessenberg 形矩阵 A
Q=eye(m);
for i=1:m-1
    xi=A(i,i);
    xk=A(i+1,i);
    if(xk~=0)
        d=sqrt(xi^2+xk^2);
        c=xi/d;
        s=xk/d;
        J=[c s;-s c];
    end
    A=A(i:i+1,i:i+1)*J;
    Q=Q(i:i+1,i:i+1)*J;
end
A=A(m:m+1,m:m+1)*Q;
Q=Q(m:m+1,m:m+1)*Q;

```

```
A(i:i+1,i;m)=J * A(i:i+1,i;m);  
Q(1;m,i:i+1)=Q(1;m,i:i+1) * J';  
end  
end  
A(1;m,1;m)=A(1;m,1;m) * Q;
```

参 考 文 献

- [1] 李庆扬,王能超.数值分析[M].武汉:华中工学院出版社,1986.
- [2] 施妙根,顾丽珍.科学和工程计算基础[M].北京:清华大学出版社,1999.
- [3] 李红,徐长发.数值分析学习辅导,习题解析[M].武汉:华中科技大学出版社,2001.